# REFINING REFINEMENT TYPES: ENHANCING TYPE-BASED REASONING FOR AUTOMATED VERIFICATION AND TESTING

by

**Zhe Zhou**

**A Dissertation**

*Submitted to the Faculty of Purdue University*

*In Partial Fulfillment of the Requirements for the degree of*

**Doctor of Philosophy**

Department of Computer Science

West Lafayette, Indiana

August 2025

# THE PURDUE UNIVERSITY GRADUATE SCHOOL
# STATEMENT OF COMMITTEE APPROVAL

**Dr. Suresh Jagannathan, Chair**

Department of Computer Science

**Dr. Benjamin Delaware**

Department of Computer Science

**Dr. Tiark Rompf**

Department of Computer Science

**Dr. Xiangyu Zhang**

Department of Computer Science

**Approved by:**

Dr. Voicu S. Popescu

To my parents and my wife, who have always supported me in my academic journey.

# ACKNOWLEDGMENTS

My journey to Purdue began after a period of uncertainty during my undergraduate studies and a year of experience as a software engineer. I made the difficult decision to leave my hometown of Beijing, where I had lived for 20 years, and move to the United States to pursue a PhD. The PhD journey spanned from age 22 to 29, encompassing the majority of my twenties. Throughout my time at Purdue, I have been fortunate to receive support from many people; without their help, I would not have been able to complete my PhD.

First, I would like to express my deep gratitude to my advisor, Dr. Suresh Jagannathan, for his invaluable guidance and steadfast support during my PhD. Before joining Suresh's group, I worked as a software engineer at a face-recognition company, mainly using C++ and Java, which was a very different experience from the research I would later pursue. Suresh taught me to approach computer science, and especially programming languages, from a scientific perspective, helping me move beyond my initial engineering mindset. In my first year, I often asked basic questions and proposed engineering-style ideas, but he was always patient and willing to explain things. His sharp insight into research problems and strong sense of direction have continually inspired me. When research became stuck, Suresh had a unique ability to reframe the problem, often finding a new direction that preserved the core idea. He also sets high standards for his students, which has pushed me to tackle more challenging and interesting problems. I have learned a tremendous amount from him.

I would also like to thank my co-advisor, Dr. Benjamin Delaware. Ben has the rare ability to be both serious and meticulous, while also being kind and maintaining a good sense of humor. During our research, Ben was rigorous about the meaning and definition of every symbol and term, which guided me to think like a programming languages researcher. He consistently exemplifies a research style that strikes a balance between theory and practice, focusing on realistic and familiar problems and then developing theoretically interesting solutions. He provided detailed guidance on writing papers and even helped me work through Coq proofs. Ben is always kind and supportive, especially when students are nervous during presentations or group meetings. He is willing to take extra time to understand students'

ideas, even when they are not fully formed at the beginning. His sense of humor during challenging research periods has also been very helpful to me.

I also thank other faculty members and mentors. My PhD committee members, Tiark Rompf and Xiangyu Zhang, provided me with a lot of valuable feedback on my research and thesis writing. Tiark's work on type systems provided much of the inspiration for my research; I also like his homepage, which features many lovely animations and interesting projects. Xiangyu is curious about my research and asked me many questions about automata-based reasoning. I would like to thank Ankush Desai, my manager and mentor during my AWS internship, for his invaluable support. He is both smart and friendly, and his extensive experience in research and industry was a tremendous help to me. Tianyi Zhang asked me about the actual experience of using Coq in my research, which is a very interesting topic. Guangyu Sun, my undergraduate advisor, also gave me lots of advice during my PhD study. I also appreciate the feedback from other Purdue faculty members: Xiaokang Qiu, Jenna DiVincenzo, Roopsha Samanta, and Jingbo Wang.

I have had the privilege of working with many talented individuals throughout my PhD journey. Rob Dickerson helped me write my first paper and provided invaluable feedback. He also gave me a great deal of confidence—I remember once asking him about my English writing and oral presentation skills, and he told me he respects anyone who completes a PhD in a second language. Ashish Mishra is a great friend, and we often discussed research together; he introduced me to the topic of "symbolic automata" which became a significant part of my thesis. Qianchuan Ye has been a wonderful mentor, assisting me with Coq proofs and offering valuable advice on my research. Yongwei Yuan has a remarkable ability to identify the core ideas of a project and persistently investigate challenging problems. Julia Belyakova is exceptionally insightful, and I greatly admire her research style. Patrick LaFontaine is hardworking and has a strong research sense; I also deeply appreciate his efforts in organizing the PurPL group, which has hosted many engaging research presentations. Minghai Lu, Danning Xie, and Songlin Jia have also contributed to and broadened my research experience.

I have also learned a great deal from other students in my group. Kia Rahmani, Gowtham Kaki, He Zhu, and K.C. Sivaramakrishnan, all former students, generously offered advice on how to work with Suresh. Zikang Xiong, a close friend who joined Purdue and graduated

alongside me, has been a frequent collaborator in discussions about research and life. I am grateful for my conversations with Anmol Sahoo, a good friend and colleague who shares my enthusiasm for OCaml and algebraic effects. Other students, former or current, from Suresh's group, such as Xuankang Lin, Wenxian Zhang, Yi Xu, Yefeng Li, Joseph Eappen, Rachit Kumar, Aadi Rave, and Asha Soroushpoor, have also been both smart and friendly.

Beyond my group, I have benefited from interactions with many colleagues at Purdue, especially those in the PurPL group. Participating in the category theory and homotopy type theory reading group with Qianchuan, Rob, and Raghav Malik was a wonderful experience. Later, I had the opportunity to host a reading group on PL and SE papers with Danning, Minghai, Songlin, Mingwei Zheng, Xuyang Li, Yueyang Tang, Yi Wu, Chengpeng Wang, Guannan Wei, Siyu Chen, and Wen Fan. Guannan has been especially helpful, teaching me many foundational PL concepts—such as "continuation"—and offering support in life as well. Chengpeng Wang and Qingkai Shi are good friends with broad connections across the community. My roommate Maoyuan Song is a good friend and very talented in theoretical computer science. I am also grateful to Wei You, Yikang Chen, Hongao Wang, Zhuo Zhang, Shengwei An, Guanhong Tao, Siyuan He, Le Yu, Yu Shi, Frank Feng, Xiangzhe Xu, Wuqi Zhang, and Fei Wang, whose friendship has made my research life at Purdue far less lonely. I would also like to thank my friends from the Purdue ACG club (PACGA), who have brought much color and joy to my life at Purdue.

I have had the pleasure of meeting many wonderful PL researchers outside Purdue. Ranjit Jhala, who also works on refinement type systems, gave me lots of valuable advice. Naijun Zhang, Jeffrey Foster, Peisen Yao, Chuan Luo, and Umang Mathur shared their experiences in research with me. I met JiaLu Zhang at my first research conference, which was held in Chicago. JiaLu told me that he had five submissions rejected over fifteen times before finally getting one accepted—allowing him to attend the conference with me. I admire his unyielding perseverance, and I am glad to see that his papers were later published one after another and that he successfully secured an academic job. Yizhou Zhang, Loris D'Antoni, Lionel Parreaux, Yilin Lou, Qingxiang Cao, Fei He, Di Wang (also my undergraduate classmate), Zhenjiang Hu, Yingfei Xiong, Xin Zhang, Yuting Wang, Xuyin Deng, Zhiqiang Zuo, Hanru Jiang, Yao Li, and Yingfei Xu all work on interesting research topics and have given me valuable

advice—especially Yizhou, who attended almost every PL conference I have attended, and with whom I often shared meals. I met Guanyan Li in Copenhagen, and we quickly became friends due to our shared interests in research and web novels. I am also grateful to Harrison Goldstein, Yifan Zhao, Qiyuan Zhao, Xing Zhang, Wenyuan Xu, Zhongye Wang, Ke Du, Chenyu Zhou, Ruyi Ji, Guangzhen Lu, Shaohua Li, Xiang Li, Xuanyu Peng, Luyao Ren, Zhixuan Yang, Wenjia Ye, Yiyun Zhen, Ziyang Li, Jiani Huang, Tesla Zhang, Qiyuan Xu, Ruifeng Xie, Cong Ma, Wenhao Tang, Zhuyang Wang, and many other friends. Finally, I thank Marisa Kirisame (who uses the same name as an ACG character) for helping me a lot with PL research; it is a pity that I have never met Marisa in person.

I have had the pleasure of meeting many wonderful people during my internships. I attended the OPLSS 2019 summer school with Shaowei Zhu, where we learned a great deal about programming languages together. Five years later, we reunited at AWS, working in the same office in California during my internship, where Shaowei had become an applied scientist. Deyuan He (he is an excellent violinist) is a close friend who shares my interest in ACG; we completed two internships together at AWS under the mentorship of Ankush. I also enjoyed working with my internship group, including Federico Mora, Zhendong Ang, Ahmad Humayun, and other colleagues at AWS. In California, I reunited with my classmate Ximing Qiao from middle school, high school, and undergraduate years; I have also shared life with other friends: Fenghong Chen, Tingyu Ge, Yuyou Fan, Yifan Zhu, Yifeng Ding, and Yanjun Wang.

Finally, I want to thank my parents and my wife, Yihan Xu, who have always supported me in my academic journey. I am also grateful to my friends in China, with whom I have mostly communicated through social media. I also thank Runnan Wang, Muhan Zeng, Dian Wu, and other friends at Lunatic Works, which is our game workshop focused on visual novels. In my spare time, I have focused on game development, which has provided a source of positive reinforcement throughout my long PhD journey.

# TABLE OF CONTENTS


LIST OF TABLES . . . . . 11

LIST OF FIGURES . . . . . 12

ABSTRACT . . . . . 15

1 INTRODUCTION . . . . . 16
- 1.1 Problem Statement . . . . . 18
- 1.2 Contributions . . . . . 21
  - 1.2.1 Reasoning about clients with hidden library internal states . . . . . 21
  - 1.2.2 Reasoning about clients with missing library specifications . . . . . 22
  - 1.2.3 Reasoning about the coverage of test generators in PBT . . . . . 22
- 1.3 Roadmap . . . . . 23

2 BACKGROUND . . . . . 24
- 2.1 Refinement Types . . . . . 24
- 2.2 Property-Based Testing . . . . . 27
- 2.3 Must-Style Reasoning and Incorrectness Logic . . . . . 29
- 2.4 Linear Temporal Logic on Finite Traces . . . . . 29
- 2.5 Symbolic Finite Automata . . . . . 30

3 AUTOMATED INFERENCE OF LIBRARY SPECIFICATIONS . . . . . 33
- 3.1 Overview and Motivation . . . . . 37
  - 3.1.1 Elrond in Action . . . . . 40
  - 3.1.2 Data-Driven Abduction . . . . . 42
- 3.2 Problem Formulation . . . . . 52
- 3.3 Learning Library Specifications . . . . . 55
  - 3.3.1 Learning Safe and Consistent Verification Interfaces . . . . . 58
  - 3.3.2 Learning Maximal Verification Interfaces . . . . . 62
- 3.4 Algorithm . . . . . 65
  - 3.4.1 Specification Inference . . . . . 66
  - 3.4.2 Weakening . . . . . 69
  - 3.4.3 Multi-Query Algorithm . . . . . 73
- 3.5 Implementation and Evaluation . . . . . 77

3.6 Discussion . . . 82

4 VERIFICATION OF REPRESENTATION INVARIANTS OF ABSTRACT DATATYPE LIBRARIES USING SYMBOLIC FINITE AUTOMATA . . . 87
4.1 Motivation . . . 90
4.2 Language . . . 97
4.3 Type System . . . 100
4.3.1 HATs, By Example . . . 102
4.3.2 Typing Rules . . . 105
4.3.3 Type Soundness . . . 110
4.4 Typing Algorithm . . . 112
4.4.1 SFA Inclusion . . . 113
4.4.2 Bidirectional Type System . . . 116
4.4.3 Soundness and Decidability of Typing Algorithm . . . 120
4.5 Implementation and Evaluation . . . 135

5 POLYMORPHIC COVERAGE TYPES . . . 140
5.1 Overview . . . 146
5.2 Language . . . 152
5.3 Type System . . . 155
5.4 Type Soundness . . . 162
5.5 Typing Algorithm . . . 165
5.5.1 Bidirectional Typing Algorithm . . . 165
5.5.2 Auxiliary Typing Functions . . . 169
5.5.3 Soundness and Completeness of the Algorithmic Typing . . . 174
5.6 Polymorphic Refinements . . . 184
5.6.1 Extended Type Syntax . . . 186
5.6.2 Extended Typing Rules . . . 188
5.6.3 Extended Typing Algorithm . . . 189
5.7 The Coverage Monad and Monadic Generator Combinators . . . 197
5.7.1 Coverage Monad . . . 197
5.7.2 Monadic Combinator Examples . . . 202
5.8 Implementation and Evaluation . . . 205
5.8.1 Completeness of Hand-Written Generators . . . 206
5.8.2 Case Study: Well-Typed STLC Terms . . . 208
5.8.3 Completeness of Synthesized Generators . . . 210

5.8.4 Case Study: Checking Coverage in Practice . . . . . 213

6 RELATED WORK . . . . . 219

7 CONCLUSION . . . . . 225

REFERENCES . . . . . 227

A OMITTED RULES AND EVALUATION DETAILS FOR HAT . . . . . 243
A.1 Basic Typing Rules . . . . . 243
A.2 Symbolic Regular Language . . . . . 244
A.3 Evaluation Details . . . . . 244

B OMITTED RULES FOR COVERAGE TYPE . . . . . 247
B.1 Operational Semantics . . . . . 247
B.2 Basic Typing rules . . . . . 247

# LIST OF TABLES


3.1 Potential negative feature vectors extracted from Cex. . . . . 46

3.2 Positive feature vectors extracted from executing `concat`([a], [b]). . . . . 47

3.3 Disjoint positive and negative feature vectors of $\mathbf{R_{top}}$. . . . . 47

3.4 Feature vectors for $\mathbf{R_{top}}$ extracted from Cex'. . . . . 49

3.5 A weakening feature vector of $\mathbf{R_{push}}$ in Fig. 3.4 . . . . . 51

3.6 Experimental results. . . . . 77

4.1 Experimental results of using Marple to verify representation invariants of ADTs. . . 136

4.2 Details of the properties used in the benchmarks shown in Table 4.1. Representation invariants of client ADTs are expressed in terms of their interactions with the backing stateful library. . . . . 138

5.1 Examples of overapproximate and underapproximate (coverage) typings. . . . . 146

5.2 Example typings for $\boldsymbol{\lambda}^U$ primitives . . . . . 158

5.3 Experimental results on hand-written generators. Existing benchmarks are annotated with their source: QuickChick [23] (*), QuickCheck [52] (°), and bespoke test input generators from [80] (⋆) and [72] (⋄). . . . . 207

5.4 Experimental results from the STLC case study. . . . . 209

5.5 Quantifying the space of safe and complete test input generators constructed by an automated program synthesis tool. . . . . 211

5.6 Experimental results that use monadic combinators from the `Gen` library of the `QCheck` framework. . . . . 213

5.7 Experimental results for test generators of open-source repositories. . . . . 214

A.1 Full table of Table 4.1, first part. . . . . 245

A.2 Full table of Table 4.1, second part. . . . . 246

# LIST OF FIGURES


1.1 A client program manipulating the filesystem. . . . . 18

1.2 Two filesystem library implementations: one using an N-ary tree (left) and another using a key-value store (right). . . . . 20

2.1 An example of property-based testing. . . . . 28

2.2 Strings in finite and infinite domain. . . . . 30

2.3 A finite automaton (left) and a symbolic finite automaton (right). . . . . 31

2.4 Inclusion over symbolic finite automata . . . . . 31

3.1 Elrond pipeline. . . . . 36

3.2 The details of Elrond's Specification Inference component from Fig. 3.1. . . . . 38

3.3 The details of Elrond's Weakening component from Fig. 3.1. . . . . 39

3.4 Candidate verification interface for the **concat** example. . . . . 42

3.5 Elements in the circle labelled $\Sigma \Rightarrow \Phi$ represent positive examples, while the orange $\neg(\Sigma \Rightarrow \Phi)$ circle contains negative examples. . . . . 43

3.6 Maximal specifications for `Stack` operations. The special variable $\nu$ represents the result of the method. . . . . 51

3.7 Run times of the first 1000 weakening iterations in benchmarks. . . . . 81

3.8 The refinement types that encoding maximal specifications shown in Fig. 3.6. . . . . 83

4.1 A file system datatype based on underline key-value store library. . . . . 91

4.2 $\boldsymbol{\lambda}^U$ term syntax. . . . . 97

4.3 Trace syntax. . . . . 98

4.4 Operational Semantics . . . . . 99

4.5 $\boldsymbol{\lambda}^U$ types. . . . . 101

4.6 Type Erasure rules. . . . . 105

4.7 Type Well-formedness rules. $\lfloor\Gamma\rfloor \vdash_{\mathtt{s}} e : \lfloor t\rfloor$ is the standard typing judgment for basic types. . . . . 106

4.8 Subtyping rules. . . . . 107

4.9 Type rules. All typing judgements (i.e., $\Gamma \vdash e : t$ and $\Gamma \vdash e : \tau$) assume the corresponding basic type judgement ($\lfloor\Gamma\rfloor \vdash_{\mathtt{s}} e : \lfloor t\rfloor$ holds). . . . . 108

4.10 Type denotations in $\boldsymbol{\lambda}^U$ . . . . . 110

4.11 Bidirectional Typing Rules (Part 1). . . . . 117

4.12 Bidirectional Typing Rules (Part 2). . . . . 118
4.13 Auxiliary Typing Functions . . . . . 118
5.1 A BST generator. . . . . 141
5.2 A tree generator expressed using `QCheck`. . . . . 144
5.3 An even number generator defined in terms of an integer number generator. . . 148
5.4 This function generates a BST with a supplied lower bound, `low`. . . . . 150
5.5 $\boldsymbol{\lambda}^U$ syntax. . . . . 153
5.6 Auxiliary typing relations . . . . . 156
5.7 Typing rules (Part 1) . . . . . 158
5.8 Typing rules (Part 2) . . . . . 159
5.9 Bidirectional Typing Rules (Part 1) . . . . . 166
5.10 Bidirectional Typing Rules (Part 2) . . . . . 167
5.11 Disjunction and Conjunction . . . . . 170
5.12 Exists and Forall . . . . . 170
5.13 Auxiliary Algorithmic Typing Functions . . . . . 172
5.14 A `union` combinator example. . . . . 184
5.15 Extended $\boldsymbol{\lambda}^U$ syntax. . . . . 186
5.16 Extended typing relations . . . . . 188
5.17 Extended bidirectional Typing Rules . . . . . 189
5.18 The definition of the test generator monad in `QCheck`. . . . . 197
5.19 The coverage monad and the signatures of its operators. . . . . 198
5.20 A generator for powers of 2 less than $2^{10}$. . . . . 199
5.21 Example of safety monad. . . . . 200
5.22 Defining a tree generator using `frequency` and `fix`. . . . . 204
5.23 A 5-node tree that cannot be produced by `tree_gen 5`. . . . . 204
5.24 Datatypes from the STLC case study. . . . . 208
5.25 Three example generators that generate size-bounded lists. . . . . 212
5.26 A tree-like block chain generator. . . . . 216
A.1 Basic Typing Rules . . . . . 243
A.2 Basic Qualifier Typing Rules . . . . . 243

A.3 Symbolic Regular Language. . . . . 244
A.4 Symbolic Regular Language. . . . . 244
B.1 Small Step Operational Semantics . . . . . 247
B.2 Basic Typing Rules . . . . . 248
B.3 Basic Qualifier Typing Rules . . . . . 248

# ABSTRACT

Formal verification often requires making difficult tradeoffs between user effort and the strength of guarantees. Refinement type systems are an established formal reasoning technique that balance these competing objectives by leveraging automated theorem provers to verify semantically rich program properties automatically. While refinement types have been integrated into a number of mainstream programming languages, several practical and technical challenges stand in the way of wider adoption. In real-world scenarios, library specifications are frequently missing or incomplete, making it difficult to reason about clients of those libraries using refinement types. At the same time, existing refinement types struggle to capture the behavior of libraries with hidden internal state and cannot express properties beyond traditional functional correctness, including the completeness of test input generators. This dissertation addresses these challenges by extending refinement type systems to support more expressive specifications and more scalable type inference. Its specific contributions include integrating data-driven and symbolic reasoning techniques to abduce library specifications, developing novel type specifications that leverage advances in automata theory, and establishing new theoretical foundations for type-based reasoning about new categories of program behaviors. It also demonstrates the practical benefits of these theoretical advances on a variety of real-world software systems.

# 1. INTRODUCTION

Functional correctness—ensuring a program's runtime behavior aligns with developer expectations—represents the gold standard in reasoning about programs. Broadly, two approaches dominate efforts to establish such guarantees: program verification and testing. Floyd [1] and Hoare [2] pioneered program verification through *Floyd-Hoare Logic*, a program logic consisting of inference rules inductively defined over program syntax to rigorously and formally prove functional correctness. In recent years, this methodology has matured to the point that teams of proof engineers have successfully established functional correctness for safety-critical software, including compilers [3, 4], operating system kernels [5], distributed systems [6], and cryptographic libraries [7]. Achieving such functional correctness assurance requires considerable human effort, particularly for modern software comprising thousands of lines of code. Verifying the `CompCert` C compiler, for example, required over six person-years and more than 100,000 lines of `Coq` code [4]. To address this challenge, automated techniques have emerged. Automated verification frameworks offer push-button verification by distilling logical formulae that are sufficient for proving functional correctness into *verification conditions* (VCs). These VCs can then be solved by automated theorem provers (e.g., the `Z3` SMT solver [8]), dramatically reducing user burden while preserving principled guarantees.

To make these powerful verification techniques practical for real-world software, they must support modular programming—a paradigm that decomposes large systems into smaller, composable components, and is fundamental to building *scalable* software systems. This aligns well with how programs are structured today in the client-library paradigm. Clients treat libraries opaquely, manipulating internal states solely via *application programming interfaces* (APIs), which are usually annotated with types to provide basic usage safety. On the other hand, library implementers are responsible for ensuring they implement the API correctly with respect to the API types. Logical *specifications* are the cornerstone of compositional verification, providing a means to establish functional correctness guarantees that go beyond basic type safety. These specifications facilitate modular reasoning. On one hand, clients can be verified against the API specifications alone, without needing access to the library's source code. On the other hand, library implementations can be checked for conformance

against the same specifications. This decoupling allows client verification to hold for any library variant that satisfies the specification. Furthermore, it helps constrain the size of the generated verification conditions (VCs), making automated verification feasible within a reasonable time.

In contrast, program testing (e.g., unit testing) provides a more lightweight but less rigorous mechanism for identifying violations of intended behavior. Testing frameworks shift the goal from proof to bug detection, allowing early-stage identification of errors in practical software development workflows [9, 10]. Writing comprehensive unit tests is labor-intensive; in practice, developers often hesitate to write the volume of unit tests that they themselves believe are necessary [11]. To address this, automated testing approaches like property-based testing (PBT) [12] reduce the burden of writing numerous unit tests by relying on test input generators to produce high-quality test cases that reflect deep semantic and structural properties of the program.

A prominent example of an automated and compositional verification framework is provided by refinement type systems [13]. These systems enrich program types with qualifiers from First-Order Logic (FOL) to create a highly expressive, type-based specification language. By building on a language's native type-checking process, this approach has been adapted to a variety of programming languages, including Haskell [13], OCaml [14], TypeScript [15], Rust [16], Ruby [17], and C [18, 19]. This technique has been successfully applied in practice; for example, it has been used to verify a variety of properties across more than 10,000 lines of code in popular Haskell libraries such as `containers`, `hscolour`, `bytestring`, `text`, `vector-algorithms`, and `xmonad` [20]. Furthermore, refinement types can also be used to specify properties to test against in PBT, providing a more rigorous mechanism than specifying properties with arbitrary code [21].

Despite these advances, leveraging refinement types to facilitate automated reasoning in large-scale software systems remains challenging. A fundamental limitation is that library API specifications are frequently missing or incomplete, which renders refinement type systems inapplicable in many practical scenarios. Moreover, the expressiveness of existing refinement type languages is often insufficient to model libraries that maintain private or hidden internal states. On the testing side, while refinement types can specify properties for

```
module type Path = sig
  type t = string
  val parent: t -> t
  val isRoot: t -> bool
  ...
end
```

```
module type FileSystem = sig
  val init: unit -> unit
  val add: Path.t -> Bytes.t -> bool
  val exists: Path.t -> bool
  val initDir: unit -> Bytes.t
  ...
end
```

---

```
open FileSystem
let rec add_path (p: Path.t): unit =
  if Path.isRoot p || exists p then ()
  else
    let _ = add_path (Path.parent p) in
    let _ = add p (initDir ()) in ()
```

**Figure 1.1.** A client program manipulating the filesystem.

PBT, they currently lack a mechanism for precisely capturing the space of inputs probed by test generators, which can lead to systematic gaps in the assurance case they provide.

To address these challenges, this dissertation proposes novel extensions to existing refinement type systems that support more expressive specifications and more scalable type qualifier inference. In pursuit of the first goal, this dissertation enriches the language of type refinements to capture the hidden internal state of library components and to reason about the coverage completeness of test input generators. To mitigate the issue of missing or incomplete specifications, we propose a data-driven technique that automatically infers the most general specifications of library APIs needed to guarantee the safety of a client program.

## 1.1 Problem Statement

To illustrate the limitations of existing approaches, consider the program in Fig. 1.1. The client program (`add_path`) uses two libraries: `Path` (representing paths as strings) and `FileSystem` (managing filesystem operations). Relying on the operations of these libraries (e.g., `FileSystem.add`), `add_path` inserts long paths (e.g., `/a/b/c`) by recursively creating missing parent directories (e.g., `/a`, `/a/b`). The desired functional correctness property of `add_path` can be expressed as the following refinement type:

$\tau_{\texttt{client}}$: *After calling* `add_path` *with a path* `p`*, that path is guaranteed to exist in the filesystem (i.e.,* `FileSystem.exists` *returns true for* `p`*).*

Establishing this top-level property of `add_path` requires auxiliary specifications for the library operations it uses. For example, the following specifications for the `Path.isRoot` and `Path.parent` methods are needed:

$\tau_{\texttt{Path.isRoot}}$: *A path is the root path if and only if it is equal to the string "/".*

$\tau_{\texttt{Path.parent}}$: *The parent of a path is its prefix before the last "/"; the parent of the root path is defined to be itself.*

These specifications are critical for reasoning that the recursive function `add_path` will not loop forever at the root path.

With the inputs introduced above, the refinement type system can verify the correctness of `add_path` by checking the following type judgment:

$$\texttt{Path.isRoot} : \tau_{\texttt{Path.isRoot}}, \ldots \vdash \texttt{add_path} : \tau_{\texttt{client}}$$

The refinement type system can then derive VCs for this type judgment, which can be solved by an off-the-shelf automated theorem prover [22].

However, such library specifications are often missing or incomplete in real-world projects. The refinement type system may be given only the source code of `add_path` and its functional correctness property $\tau_{\texttt{client}}$, but no information about the behavior of the components it uses. In this case, the refinement type checker will simply fail.

Moreover, internal state in libraries is often hidden from the client, rendering automated verification inapplicable or infeasible. To see why, consider the two implementations of the `FileSystem` APIs shown in Fig. 1.2: (1) the first implementation (left sub-figure of Fig. 1.2) implements the filesystem directly using an N-ary tree structure which is hidden from the client, and (2) the second implementation (right sub-figure of Fig. 1.2) relies on the effectful `KVStore` library to maintain filesystem entries as path-file pairs. Verification on the client side expects the following (informal) specifications of the `FileSystem` library:

$\tau_{\texttt{exists}}$: *Returns true if the input path* `p` *exists in the filesystem.*

```
module type FileSystem = struct
  type t = NAryTree.t
  (* an universal file system instance *)
  let fs: ref t = (* omitted *)
  (* APIs implemented by the library *)
  let init: unit -> unit = ...
  let add: Path.t -> File.t -> bool = ...
  let exists: Path.t -> bool = ...
  ...
end
```

```
module type KVStore = sig
  val put: string -> bytes -> unit
  val get: string -> bytes
  ...
end

module type FileSystem = struct
  open KVStore
  (* APIs implemented by the library *)
  let init: unit -> unit = ...
  let add: Path.t -> File.t -> bool = ...
  let exists: Path.t -> bool = ...
  ...
end
```

**Figure 1.2.** Two filesystem library implementations: one using an N-ary tree (left) and another using a key-value store (right).

$\tau_{\texttt{add}}$: *The input path* `p` *is added if it does not exist in the filesystem but its parent path does.*

Mathematically formalizing these informal definitions in an existing refinement type system depends on the *representation* of the filesystem—a property that varies across different `FileSystem` implementations. For instance, the left-hand implementation in Fig. 1.2, which models the filesystem as an N-ary tree, requires the following specification for the `exists` API:

$\tau^{\texttt{Tree}}_{\texttt{exists}}$: *A path* `p` *exists in the filesystem if and only if it corresponds to a valid path in the tree structure.*

In contrast, the right-hand implementation in Fig. 1.2 employs a key-value store for filesystem representation, with the following specification:

$\tau^{\texttt{KVStore}}_{\texttt{exists}}$: *A path* `p` *exists in the filesystem when it appears as a valid key in the store.*

Existing refinement type systems require precise specifications of library APIs, such as $\tau^{\texttt{Tree}}_{\texttt{exists}}$ and $\tau^{\texttt{KVStore}}_{\texttt{exists}}$, for verification to succeed. However, these specifications are inherently tied to the library's implementation, exposing internal details and thereby violating the principle of compositional verification. This dependency precludes the modular verification of the client

program. The fundamental issue is the inability of current refinement type languages to specify properties of a library's hidden state in a representation-agnostic way.

We now turn our attention to reasoning about the correctness of the `FileSystem` library implementation using automated testing. Consider the implementation shown on the left side of Fig. 1.2. Here, all API implementations depend on a filesystem instance, represented as an N-ary tree, which acts as a hidden input. However, not all N-ary trees represent valid file systems. This constraint is formalized as the following refinement type:

$\tau_{\texttt{FS}}$: *The root node of the N-ary tree must store the root path "/", other nodes can only store valid file or directory names.*

For instance, an N-ary tree with a non-root path (e.g., "`a/b/c`") at its root is considered invalid. Property-Based Testing (PBT) [23] can automatically test the API implementation with "valid filesystem instances," thereby enhancing the effectiveness and coverage of the testing process [12]. To do so, PBT employs a *test input generator*—a randomized function—to produce N-ary tree instances that satisfy $\tau_{\texttt{FS}}$. However, this testing style introduces a verification challenge beyond the capabilities of standard refinement type systems. While refinement types can verify the *safety* of generators (i.e., that any generated tree satisfies $\tau_{\texttt{FS}}$), they cannot verify *coverage completeness*, which requires that the generator is capable of producing *all* trees that satisfy $\tau_{\texttt{FS}}$. This limitation means refinement types cannot guarantee that PBT will detect bugs in corner cases. For example, a "constant" generator that always produces a filesystem containing *only* the root path is undesirable because it fails to cover all valid filesystem instances, such as one with a regular file under the root path.

## 1.2 Contributions

Concretely, this dissertation extends the existing refinement type system [22] to address the challenges identified above through the following key contributions.

### 1.2.1 Reasoning about clients with hidden library internal states

To verify client programs that use libraries equipped with opaque internal states, we abstract the hidden library state as a sequence of effectful operations, or *event traces*. To

reason about the properties of this trace-based representation, we introduce a novel variant of refinement types called *Hoare Automata Types* (HATs) [24]. HATs extend the language of type refinements from a decidable fragment of FOL to Linear Temporal Logic on finite sequences ($\text{LTL}_f$) and also leverage recent advances in Symbolic Finite Automata (SFA) from automata theory. In HATs, refinement types are augmented by pre- and post-condition SFAs, which abstract the hidden state before and after a client program is executed.

### 1.2.2 Reasoning about clients with missing library specifications

To enable reasoning in the face of missing or incomplete library specifications (e.g., for `FileSystem.exists`), we introduce a novel data-driven inference procedure [25] that infers this information automatically. Our solution proposes a novel multi-abduction procedure for precise specification inference of datatype libraries, guided by client-side verification tasks. Our technique combines a data-driven learning framework to postulate candidate specifications, uses SMT-provided counterexamples to refine these candidates, and employs a novel symbolic weakening procedure to prevent generating specifications that overfit to sampled tests.

### 1.2.3 Reasoning about the coverage of test generators in PBT

To reason about the coverage completeness of test generators in PBT, we introduce a novel variant of refinement types called *coverage types* [26]. Coverage types feature a novel denotation that embeds "must-style" under-approximation reasoning principles as a fundamental part of the type system. The types associated with expressions now capture the set of values guaranteed to be produced by an expression, rather than the typical formulation in which types represent the set of values an expression may produce. Combined with standard refinement types, coverage types can guarantee that a test generator is both sound and complete. To better support checking the coverage properties of real-world test generators, we extend this type system with type and qualifier polymorphism. These extensions enable our system to statically verify the coverage guarantees of test input generators constructed using the monadic combinators provided by mainstream PBT frameworks.

## 1.3 Roadmap

The remainder of this dissertation is structured as follows: Chapter 2 introduces the key concepts and background material that form the foundation for this work (e.g., refinement types and property-based testing). The details of our library specification inference technique are presented in Chapter 3; this material originally appeared at OOPSLA '21 [25], and this dissertation extends it as a technique for refinement type inference. The details of our type system for library implementations with internal states and temporal specifications are given in Chapter 4; this work was originally published at PLDI '24 [24]. A detailed presentation of the coverage type system and its polymorphic extension is provided in Chapter 5; a portion of this work was originally published at PLDI '23 [26]. Chapter 6 presents a detailed discussion of related work, which is integrated from the related works of the corresponding conference papers [24–26] for Chapter 3–Chapter 5, and Chapter 7 concludes the dissertation.

# 2. BACKGROUND

In this chapter, we provide an overview of the technical background necessary to appreciate the contributions in subsequent chapters, including key concepts and prior approaches.

## 2.1 Refinement Types

Refinement types [13, 22, 27, 28] are a variant of the *dependent types* [29] that express semantically rich correctness properties within types without sacrificing automated verification, which is achieved by reusing the type-checking procedure. Besides system-level verification [30], refinement types have been successfully applied to various application domains, including cryptographic protocols [31], deep learning models [32], and randomized algorithms [33].

A refinement type refines a base type (i.e., a non-function type) with a logical *qualifier* that constrains its set of valid inhabitants. For instance, while the base type `Int` denotes the set of all integers, the refinement type $\{\nu{:}\texttt{int} \mid 0 \leq \nu\}$ restricts this set to the natural numbers, where $\nu$ is a self-reference representing the term. Refinements naturally integrate with dependent function types, where the output type may refer to input parameters. For example, the addition operator can be assigned the following refinement type:

$$\vdash \texttt{add} : x{:}\{\nu{:}\texttt{int} \mid \top\} \to y{:}\{\nu{:}\texttt{int} \mid \top\} \to \{\nu{:}\texttt{int} \mid \nu = x + y\}$$

Here, the parameters $x$ and $y$ are arbitrary integers (denoted by the $\top$ qualifier), and the return type asserts that the result $\nu$ must equal their sum.

Verification in refinement type systems proceeds via a *bidirectional* typing algorithm [34] that consists of type synthesis and type checking phases. The type synthesis phase $\vdash e \Rightarrow \tau_{\mathsf{syn}}$ infers a refinement type $\tau_{\mathsf{syn}}$ that overapproximates the behavior of the program $e$, while the type checking phase $\vdash e \Leftarrow \tau_{\mathsf{chk}}$ confirms that the program $e$ satisfies a refinement type $\tau_{\mathsf{chk}}$ that encodes the desired specification. To bridge these phases, the system uses the subtyping judgment $\vdash \tau_{\mathsf{syn}} <: \tau_{\mathsf{chk}}$. The verification conditions (VCs) generated from such subtyping judgments are sufficient to ensure that the program adheres to the stated

correctness properties. We illustrate this process using the `nat_minus` function defined as follows:

```
let nat_minus (a: int) (b: int) =
    if a < b then 0 else a - b
```

Here, the developer intends for the function to return a natural number whenever both inputs are non-negative, that can be encoded as the following typing judgement:

$$\vdash \texttt{nat_minus} \Leftarrow \tau_{\texttt{chk}} \quad \text{where}$$
$$\tau_{\texttt{chk}} \doteq a{:}\{\nu{:}\texttt{int} \mid 0 \leq \nu\} \rightarrow b{:}\{\nu{:}\texttt{int} \mid 0 \leq \nu\} \rightarrow \{\nu{:}\texttt{int} \mid 0 \leq \nu\}$$

To ensure that `nat_minus` meets this specification, a refinement type checker will first synthesize a type $\tau_{\texttt{syn}}$ for its body that captures the control-flow structure of the program:

$$\vdash \texttt{nat_minus} \Rightarrow \tau_{\texttt{syn}} \quad \text{where}$$
$$\tau_{\texttt{syn}} \doteq a{:}\{\nu{:}\texttt{int} \mid 0 \leq \nu\} \rightarrow b{:}\{\nu{:}\texttt{int} \mid 0 \leq \nu\} \rightarrow \{\nu{:}\texttt{int} \mid (a < b \wedge \nu = 0) \vee (a \geq b \wedge \nu = a - b)\}$$

The parameter types in $\tau_{\texttt{syn}}$ are the same as those in $\tau_{\texttt{chk}}$, while its return type explicitly reflects the semantics of the `if`-expression: returning 0 when $a < b$, and $a - b$ otherwise. The refinement type system then reduces verification to the subtyping check $\vdash \tau_{\texttt{syn}} <: \tau_{\texttt{chk}}$, which yields the following verification condition:

$$\phi_{VC} \doteq \forall a{:}\texttt{int}.\ \underbrace{0 \leq a}_{\texttt{qualifier of parameter } a} \implies \forall b{:}\texttt{int}.\ \underbrace{0 \leq b}_{\texttt{qualifier of parameter } b} \implies \forall \nu{:}\texttt{int}.\ \underbrace{(a < b \wedge \nu = 0) \vee (a \geq b \wedge \nu = a - b)}_{\texttt{qualifier on } \tau_{\texttt{syn}}\texttt{'s return type}} \implies \underbrace{0 \leq \nu}_{\texttt{qualifier on } \tau_{\texttt{chk}}\texttt{'s return type}}$$

This formula can be discharged by a SMT solver (e.g., `Z3` [8]), which confirms that `nat_minus` returns a non-negative integer when applied to non-negative inputs, as required.

A key strength of refinement types lies in their support for efficient and automated verification. Rooted in the Curry-Howard correspondence [35], which interprets propositions as types and proofs as programs, interactive theorem provers (ITPs) such as `Coq` and `Lean` leverage rich dependent type theories to encode and reason about complex specifications. In contrast to these highly expressive systems, refinement types deliberately trade expressiveness for automation and decidability. To ensure decidable type checking, refinement type systems impose several constraints: (1) qualifiers must be written in a decidable fragment of logic; (2) function types directly refined by logical predicates (e.g., $\{\nu{:}\texttt{int} \to \texttt{int} \mid \phi\}$) are disallowed to avoid introducing higher-order terms such as $\nu{:}\texttt{int} \to \texttt{int}$ into the logic; and (3) verification conditions (VCs) derived from subtyping judgments must be encoded as queries amenable to efficient, decidable reasoning.

Most refinement type systems adopt a decidable fragment of first-order logic (FOL) as their qualifier language and discharge VCs by encoding them as queries to SMT solvers such as `Z3` [8]. To handle inductive datatypes, several modern refinement systems [13, 28] extend the logic with limited forms of uninterpreted functions, enabling the specification of shape properties of datatypes. For instance, the uninterpreted function $len : \texttt{'a list} \to \texttt{int}$ can represent the length of a (polymorphic) list (`'a list`). With this, the list concatenation function `concat` can be assigned the following refinement type:

$$l_1{:}\{\nu{:}\texttt{'a list} \mid \top\} \to l_2{:}\{\nu{:}\texttt{'a list} \mid \top\} \to \{\nu{:}\texttt{'a list} \mid len(\nu) = len(l_1) + len(l_2)\}$$

Here, the output is specified to have a length equal to the sum of the input list lengths. To relate such uninterpreted functions to their intended semantics, the underlying theory must supply axioms capturing their basic properties. To preserve decidability, these axioms are also restricted to a decidable fragment of FOL. For example, a standard axiom for lists states:

$$\forall l : \texttt{'a list}.\ 0 \leq len(l)$$

This expresses that the length of any list is non-negative.

While most refinement type systems rely on FOL and SMT solving, this is not the only choice. This dissertation explores an alternative approach: extending the qualifier logic to $\mathrm{LTL}_f$ and employing symbolic finite automata inclusion as its decidable decision procedure. The technical details of this approach are introduced later in this chapter.

## 2.2 Property-Based Testing

Property-based testing (PBT) [12, 36, 37] is an automated testing approach that uses random tests to determine whether a user-defined property is satisfied by the component under test. The PBT methodology rests on two key components: *executable properties* that capture the expected input-output behaviors of the program under test, and *test input generators* that generate random datatype values of the input types needed to validate these behaviors. In contrast to unit tests, which rely on single examples of inputs and outputs, generators are meant to provide a *family* of inputs against which programs can be tested, with the goal of ensuring that the set of generated tests provides good coverage of all expected datatype inputs to the system under test. To prune irrelevant inputs, PBT frameworks allow users to define custom datatype generators that produce data with a shape the developer believes is most likely to trigger interesting (i.e., faulty) behavior. Moreover, PBT frameworks are often equipped with a "shrink" procedure that automatically reduces failing inputs to minimal examples that are reported to users.

We illustrate the key components of PBT using the example in Fig. 2.1, which focuses on validating the behavior of a `merge` function used as part of a merge sort implementation, `merge_sort`. As shown on line 11, the `merge` function is applied to a pair of sorted lists recursively returned by `merge_sort`, each of which is assumed to be sorted. Thus, the executable property under test can be assigned the following refinement type:

$$\tau_{\texttt{merge}} \doteq \texttt{l}_1{:}\{\nu{:}\texttt{int list} \mid \texttt{sorted}(\nu)\} \to \texttt{l}_2{:}\{\nu{:}\texttt{int list} \mid \texttt{sorted}(\nu)\} \to \{\nu{:}\texttt{int list} \mid \texttt{sorted}(\nu)\}$$

This indicates that if both input lists are sorted, then the output list is also sorted. The predicate `sorted`, implemented as an executable function on line 13, defines the sortedness

```
1 let rec merge (l1: int list) (l2: int list): int list =
2     match l1, l2 with
3     | [], _ -> l2
4     | _, [] -> l1
5     | hd1 :: tl1, hd2 :: tl2 -> (* implementation omitted. *)
6
7 let rec merge_sort (l: int list) : int list =
8     if List.length l <= 1 then l
9     else
10        let l1, l2 = split_list l in
11        merge (merge_sort l1) (merge_sort l2)
12
13 let sorted: int list -> bool =
14     (* a function checks sorted property, implementation omitted. *)
15
16 let constant_sorted_list_gen: unit -> int list =
17     fun () -> [1]
18     (* a safe but coverage-incomplete test input generator. *)
```

**Figure 2.1.** An example of property-based testing.

of integer lists. Importantly, the specification $\tau_{\texttt{merge}}$ assumes the input lists are sorted; the `merge` function is not required to satisfy the postcondition otherwise. For instance, although the implementation on lines 1–5 may return the second list directly on line 3, it is correct with respect to $\tau_{\texttt{merge}}$ because the precondition disregards cases where the arguments are not sorted (e.g., `merge` $[]\ [2;1] = [2;1]$ does not violate $\tau_{\texttt{merge}}$).

A *safe* test input generator only produces inputs that satisfy the precondition of the property under test. A safe generator for the `merge` function would only produce sorted lists, thereby avoiding irrelevant inputs that violate the precondition of $\tau_{\texttt{merge}}$. On the other hand, *completeness* is also a critical requirement for generators: a generator should be capable of producing the full range of valid test inputs. The generator `constant_sorted_list_gen` on line 16, for instance, is safe but not coverage-complete: it always returns the constant list [1], which is sorted but fails to generate other valid test inputs such as [] or [2].

## 2.3 Must-Style Reasoning and Incorrectness Logic

Traditional program verification, including the refinement type systems discussed previously, primarily focuses on *safety* properties. This form of reasoning, often called *may-style* analysis, works by over-approximating a program's behavior to prove the *absence* of errors [38]. For example, a safety verifier might prove that a program can *never* divide by zero. While essential, this approach cannot be used to prove the *presence* of desired behaviors, such as the guarantee that a test generator can produce all valid inputs.

To address this limitation, a complementary form of reasoning known as *must-style* analysis has been developed. Must-style analysis works by under-approximating program behavior to prove that certain outcomes are guaranteed to be realized. This is crucial for reasoning about properties like liveness, termination, and coverage completeness.

A prominent example of must-style reasoning is *Incorrectness Logic* [39]. As a dual to Hoare Logic, which proves the absence of incorrect final states, Incorrectness Logic is designed to formally prove the *presence* of bugs. The central judgment in Incorrectness Logic provides a powerful reachability guarantee: it allows one to prove that a program, starting from a given set of initial states, *can* terminate in a specified set of "bad" final states. This ability to reason about the reachability of states is fundamental to providing formal guarantees for bug-finding tools and for ensuring the completeness of systems like test input generators, a topic central to this dissertation.

## 2.4 Linear Temporal Logic on Finite Traces

Linear Temporal Logic (LTL) is a modal logic used for specifying properties of reactive systems, which are typically modeled as infinite computations or traces [40]. LTL extends propositional logic with temporal operators to reason about the evolution of a system over time. Key operators include $\Box\phi$ (Globally, $\phi$ is always true), $\Diamond\phi$ (Eventually, $\phi$ is eventually true), $\bigcirc\phi$ (Next, $\phi$ is true in the next state), and $\phi\,\mathcal{U}\,\psi$ ($\phi$ is true until $\psi$ becomes true).

While LTL is well-suited for non-terminating systems like operating systems, its semantics are defined over infinite traces, making it unsuitable for specifying properties of terminating computations, which are common in many program verification tasks. To address this, LTL on

finite traces ($\text{LTL}_f$) was introduced [41]. $\text{LTL}_f$ adapts the semantics of LTL to be interpreted over finite traces. The key difference lies in how the end of a trace is handled. Since a finite trace has a final state, temporal operators are interpreted accordingly; for example, the $\bigcirc$ operator is false at the last state of a trace, as there is no successor state.

A fundamental contribution of $\text{LTL}_f$ is its deep connection to automata theory. Just as LTL formulas correspond to Büchi automata, which accept infinite strings, $\text{LTL}_f$ formulas are expressively equivalent to standard finite automata (FA) and, consequently, regular expressions [41]. This equivalence is critical for automated reasoning: an $\text{LTL}_f$ formula can be translated into an equivalent finite automaton that accepts all and only the finite traces satisfying the formula. This allows $\text{LTL}_f$ model checking to be reduced to standard automata-theoretic problems, such as checking for language inclusion, which are decidable and can be solved efficiently. This dissertation leverages this connection to reason about programs that operate on finite sequences of events.

## 2.5 Symbolic Finite Automata

$$\begin{aligned}
\text{string in finite alphabet} &\doteq \texttt{a}; \texttt{b}; \texttt{a}; \texttt{a} \\
\text{string in infinite alphabet} &\doteq 3; 2; 1; 5 \\
&\simeq \texttt{odd}; \texttt{even}; \texttt{odd}; \texttt{odd} \quad \text{(after mapping into predicate domain)}
\end{aligned}$$

**Figure 2.2.** Strings in finite and infinite domain.

Unlike standard finite automata, which work over a *finite* domain of input *characters*, symbolic finite automata (SFAs, or simply symbolic automata) [42, 43] work over an *infinite* alphabet. This allows us to represent *strings* (finite sequences of characters) that may appear in realistic programs (e.g., sequences of integers). For example, in the strings shown in Fig. 2.2, the characters in the second string are integer values drawn from an infinite domain, rather than letters like `a`, `b`, or `c`. To deal with infinite alphabets, SFAs have transitions over *predicates* instead of characters. When the user only cares about parity, for example, the predicate domain can be $\{\texttt{odd}, \texttt{even}\}$, which contains two predicates over integers. As

shown in the gray box, using this sort of predicate abstraction, a string in an infinite integer character domain can be abstracted into a string in a finite predicate domain.

**Figure 2.3.** A finite automaton (left) and a symbolic finite automaton (right).

Formally, the symbolic alphabet of a SFA is defined as a finite set of logical predicates over the character domain. A string is accepted by a SFA if its elements satisfy the predicates on the corresponding transitions. Fig. 2.3 shows two automata: a standard FA accepting the string `a`; `b`; `a`; `a` and a SFA that accepts the integer sequence `3`; `2`; `1`; `5`. The distinction lies in the representation—characters versus predicates—while the structural behavior remains identical.

The expressiveness of a SFA is directly tied to its symbolic alphabet. For example, using only the predicates {`odd`, `even`}, a SFA cannot distinguish between characters 1 and 3, since both of them satisfy `odd`. Consequently, such a SFA cannot distinguish between `3`; `2`; `1`; `5` and `1`; `2`; `1`; `1`, accepting or rejecting both together. This abstraction is often desirable, as it enables SFA-based analyses to ignore irrelevant details and focus on properties of interest.

**Figure 2.4.** Inclusion over symbolic finite automata

By supporting infinite alphabets, SFAs are strictly more expressive than FAs. This enhanced expressivity comes at a cost, however, as the standard FA inclusion algorithm cannot be directly applied to SFAs. Thankfully, prior work has shown how to reduce an inclusion check over SFAs to an inclusion check over FAs [42]. The high-level approach is to first construct a finite set of equivalence classes over the SFA's infinite character domain,

defined in terms of a set of maximal satisfiable boolean combinations of logic literals (called *minterms*) in the automaton's symbolic alphabet. An alphabet transformation procedure then replaces characters in the original SFA with their corresponding equivalence classes and replaces the SFA's symbolic alphabet with minterms, thus allowing SFAs to be translated into FAs. Note that only satisfiable minterms are considered, as unsatisfiable ones do not correspond to realizable strings. As illustrated in Fig. 2.4, consider checking whether the left SFA (from Fig. 2.3) is included in the right SFA, which requires that the final character must not be zero. An inclusion check without a satisfiability check may report a counterexample such as $\texttt{odd} \wedge \top; \texttt{odd} \wedge (\nu = 0)$, which represents a string accepted by the left automaton but not the right. However, this string is unrealizable, as no integer can simultaneously be odd and equal to zero. Hence, satisfiability checking is essential to ensure the soundness of the inclusion algorithm. Provided that satisfiability is decidable for the predicate logic in use, inclusion checking for SFAs is likewise decidable [43].

# 3. AUTOMATED INFERENCE OF LIBRARY SPECIFICATIONS

Using a specification of a library's methods in the verification of its clients is a hallmark of modular reasoning. Because these specifications encapsulate the interface between the client and the library, each may be independently verified without access to the other's implementation. This modularity is particularly beneficial when the library function is complex or its source code is unavailable. All too often, though, such specifications are either missing or incomplete, preventing the verification of clients without making (often unwarranted) assumptions about the behavior of the library. This problem is further exacerbated when libraries expose rich datatype functionality, which often leads to specifications that rely on inductive invariants [44, 45] and complex structural relations. One solution to this problem is to automatically *infer* missing specifications. Unfortunately, while significant progress has been made in specification inference over the past several years [46–49], existing techniques have not considered inference in the frequently occurring case of client programs that make use of data structure libraries with unavailable implementations.

To highlight the challenge, consider the following simple program, which concatenates two stacks together using four operations provided by a Stack library: **push**, **top**, **is_empty** and **tail**.

```
1 let rec concat s1 s2 =
2   if Stack.is_empty s1 then s2
3   else Stack.push (Stack.top s1) (concat (Stack.tail s1) s2)
```

To ensure the correctness of this client function, its author may wish to verify that (a) the top element of the output stack is always the top element of one of the input stacks; and, (b) every element of the output stack is also an element of one of the input stacks and vice-versa. In order to express this behavior in a form amenable for automatic verification, we need some mechanism to encode the semantics of stacks in a decidable logic. To do so, we rely on a

pair of *method predicates*, "$a$ is the head of stack $s$", $hd(s, a)$, and "$a$ is a member of stack $s$", $mem(s, a)$, to write our postcondition:

$$\forall u, (hd(\nu, u) \implies hd(\mathtt{s}_1, u) \vee hd(\mathtt{s}_2, u)) \wedge (mem(\nu, u) \iff mem(\mathtt{s}_1, u) \vee mem(\mathtt{s}_2, u)) \qquad (\phi_{\mathtt{concat}})$$

We assume these method predicates are associated with (possibly blackbox) implementations that we can use to check the specifications in which they appear. For example, $hd$ may be defined in terms of the stack operations **top** and **is_empty**, while the implementation of $mem$ might additionally use **tail**. The variable $\nu$ in $\phi_{\mathtt{concat}}$ is used to represent the output stack of `concat`. The above assertion claims that the head of the output stack must be the head of either $\mathtt{s}_1$ or $\mathtt{s}_2$ and that any element found in the output must be a member of either $\mathtt{s}_1$ or $\mathtt{s}_2$.

By treating method predicates ($hd$ and $mem$) and library functions (**push**, **top**, **is_empty**, and **tail**) as uninterpreted function symbols, it is straightforward to generate verification conditions (VCs), e.g. using weakest precondition inference, which can be handed off to an off-the-shelf SMT solver like Z3 to check. However, the counter-examples returned by the theorem prover may be spurious, generated by incorrect assumptions about library method behavior in the absence of any constraints on these behaviors outside the client VCs. For example, the prover might assume the formula $\neg hd(\mathtt{s}_1, \mathbf{top}(\mathtt{s}_1))$ is valid, i.e. that the result of $\mathbf{top}(\mathtt{s}_1)$ is not the head of $\mathtt{s}_1$. This claim is obviously inconsistent with the client's expectation of **top**'s semantics, but it is not disallowed by any constraints in the SMT query. Using this assumption, Z3 may return the following counterexample: $\exists s\ u.hd(s, u) \wedge \neg mem(s, u)$. This counterexample, which is obviously incongruous with the intended semantics of $hd$ and $mem$, occurs because the expected relationship between $hd$ and $mem$ is lost when the predicates are embedded as uninterpreted functions in the SMT query.

To overcome this problem, we need stronger specifications for the library methods, defined in terms of these method predicates, that are sufficient to imply the desired client postcondition. In particular, these specifications should rule out spurious unsafe executions such as the counter-example given above. In the (quite likely) scenario that such library specifications

are not already available, a reasonable fallback is to infer *some* specifications for these functions that are strong enough to ensure the safety of the client. Traditional approaches to specification inference usually adopt a closed-world assumption in which specifications of library methods are discovered in isolation, independent of the client context in which they are being used. Such assumptions are not applicable here since (a) we do not have access to the library's method implementations and (b) the nature of the specifications we need to infer are impacted by the verification demands of the client. In this setting, some form of data-driven inference [47–49] can be beneficial. Such an approach may be tailored to the client context in which the library methods are used, postulating candidate specifications for library methods based on observations of their input-output behavior. Unfortunately, completely blackbox data-driven approaches are susceptible to overfitting on the set of observations used to train them, and can thus discount reasonable and safe behaviors of the underlying library functions.

To address the problem of overfitting, we might instead consider attacking this problem from a purely logical standpoint, treating specification inference as an instance of a multi-abductive inference problem [46] that tries to find formulae $\mathbf{R_{push}}$, $\mathbf{R_{top}}$, $\mathbf{R_{tail}}$ and $\mathbf{R_{is_empty}}$ such that $\bigwedge R_i \not\models \bot$ and yet which are sufficient to prove the desired verification condition. While such problems have been previously solved over linear integer arithmetic constraints [46] using quantifier elimination, these prior techniques cannot be directly applied to formulae with uninterpreted function symbols like the method predicates (e.g., *hd* and *mem*) used to encode library method specifications in our setting.

In this work, we combine aspects of these data-driven and abductive approaches in a way that addresses the limitations each approach has when considered independently. Our technique uses SMT-provided counterexamples to generate infeasible interpretations of these predicates (similar to other abductive inference methods) while using concrete test data to generate feasible interpretations (similar to data-driven inference techniques). This combination yields a novel CEGIS-style inference methodology that allows us to postulate specifications built from method predicates sufficient to prove the postcondition in a purely blackbox setting. The specifications learned by this procedure are guaranteed to be both *consistent* with the observed input-output behavior of the blackbox library implementations

and *safe* with respect to the postcondition of the client program. As there may be many such specifications, we also endeavor to find a *maximal* one that is at least as weak as every other safe and consistent specification, in order to avoid overfitting to observed library behaviors. Our algorithm applies another data-driven weakening procedure to find these maximal specifications.

To demonstrate the effectiveness of our approach, we have implemented a fully automated abductive specification inference pipeline in OCaml called Elrond (see Fig. 3.1). This pipeline takes as input (a) an OCaml client program that may call blackbox library code defined over algebraic datatypes like lists, trees, heaps, etc.; (b) assertions about the behavior of this client program; and, (c) a set of method predicates (e.g., *hd* or *mem*), along with their (possibly blackbox) implementations, that are used to synthesize library method specifications. It

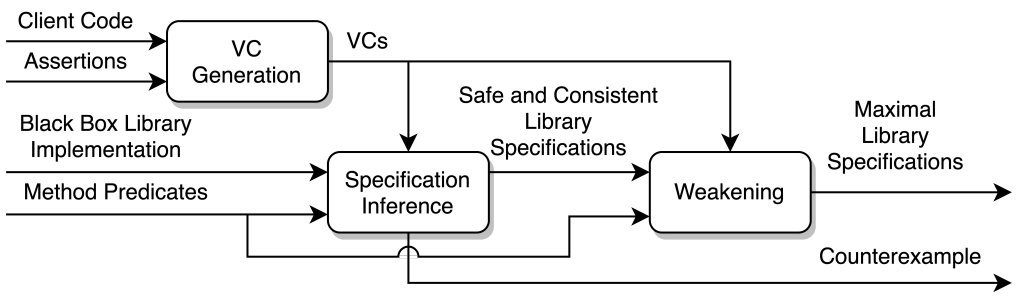


**Figure 3.1.** Elrond pipeline.

combines tests and counterexample-guided refinement techniques to either generate a set of maximal specifications for the library methods used by the client program, or a counterexample that demonstrates a violation of the postcondition. The notion of "weakest" used in our definition of maximal is bounded by the "shape" of specifications (e.g., the number of quantified variables, the set of method predicates, etc.) and a time bound. Our results over a range of sophisticated data-structure manipulating programs, including those drawn from e.g., [50], show that Elrond is able to discover maximally-weak specifications (as determined by an oracle executing without any time constraints) for the vast majority of applications in our benchmark suite within one hour.

Our key contribution is thus a new abductive inference framework that is a fusion of automated data-driven methods and counterexample-guided refinement techniques, tailored

to specification inference for libraries that make use of rich algebraic datatypes. Specifically, we:

1. Frame client-side verification as a multi-abduction inference problem that searches for library method specifications that are both consistent with the method's implementation and sufficient to verify client assertions.

2. Devise a novel specification weakening procedure that yields the weakest specification among the collection of all safe and consistent ones with respect to a given set of quantified variables and method predicates.

3. Evaluate our approach in a tool, Elrond, which we use to analyze a comprehensive set of realistic and challenging functional (OCaml) data structure programs. An artifact containing this tool and our benchmark suite is publicly available [51].

The remainder of the chapter is structured as follows. The next section presents an overview of our approach using a detailed example to motivate its key ideas. A formal characterization of the problem is given in Sec. 3.2. Sec. 3.3 defines how a data-driven learning strategy can be used to perform inference. A detailed presentation of the algorithm used to manifest these ideas in a practical implementation is given in Sec. 3.4. Details of our implementation and evaluation results are explained in Sec. 3.5. We conclude in Sec. 3.6 with a discussion of how our work can be viewed as a technique for automated refinement type inference.

## 3.1 Overview and Motivation

We divide the inference of maximal library specifications into two stages, which are represented as the "Specification Inference" and "Weakening" components in Fig. 3.1. Both stages leverage data-driven learning to overcome the lack of a purely logical abduction procedure for our specification language. The initial inference stage learns a set of safe and consistent specifications from a combination of concrete tests and verifier-provided counterexamples. The next stage then weakens these specifications by iteratively augmenting this data set with additional safe behaviors until a set of maximal specifications are found.

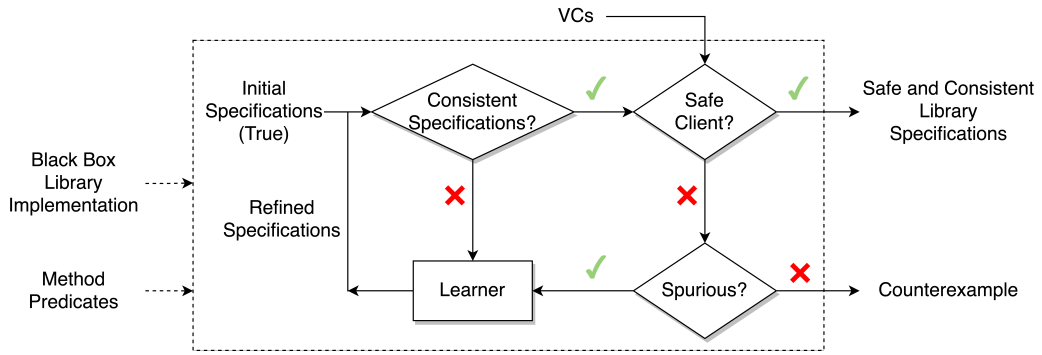


**Figure 3.2.** The details of Elrond's Specification Inference component from Fig. 3.1.

Fig. 3.2 provides a more detailed depiction of the initial specification inference stage in Elrond.[1] Starting from an initial set of maximally permissive specifications, this stage iteratively refines the set of candidate specifications until either a set of safe and consistent solutions or a counterexample witnessing an unsafe execution is found.

Each iteration first uses a property-based sampler, e.g. QuickCheck [52], to look for executions of the blackbox library implementations that are inconsistent with the current set of inferred specifications. The reliance on a generator to provide high-quality tests provides yet more motivation for the subsequent weakening phase, in order to ensure that the final abduced specifications are not overfitted to or otherwise biased by the tests provided by the generator. At the same time, we also observe that the sorts of shape properties (e.g., membership and ordering) used in our specifications and assertions are relatively under-constrained and are thus amenable to property-based random sampling. We do not ask, for example, for QuickCheck to generate inputs satisfying non-structural properties like "*a list whose 116$^{th}$ element is equal to 5.*"

Any tests that are disallowed by the current solution are passed to a learner which uses them to generalize the current specification. If no inconsistencies are detected, Elrond attempts to verify the client against the candidate specifications using a theorem prover. If the inferred specifications are sufficient to prove client safety, the loop exits, returning the discovered

[1]↑The blackbox library implementation and method predicates are used by multiple components; their corresponding arrows within the diagram are omitted for clarity.

solution. If not, the verifier has identified a model that represents a *potential* safety violation. The model is then analyzed in an attempt to extract test inputs that trigger a safety violation. If we are unable to find such a counterexample, the model is most likely incongruous with the semantics of the method predicates and is thus spurious. In this case, the model is passed to the learner so that it can be used to strengthen candidate specifications, preventing this and similar spurious counterexamples from manifesting in subsequent iterations.

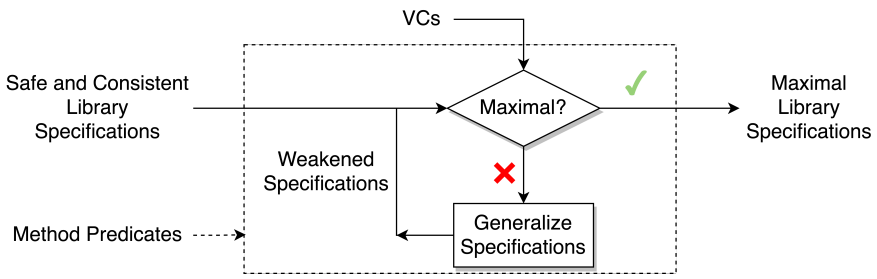


**Figure 3.3.** The details of Elrond's Weakening component from Fig. 3.1.

While the previous loop is guaranteed to return safe and consistent solutions, it may find specifications that are nonetheless *too strong* with respect to the underlying library implementation. This occurs when the property-based sampler fails to find a test that identifies an inconsistent specification, which may happen when the input space of a library function is very large. To combat overfitting specifications to test data, candidate solutions are iteratively weakened using the data-driven counterexample-guided refinement loop depicted in Fig. 3.3[2]. The data in this phase is supplied by the underlying theorem prover rather than a concrete test generator. Each iteration of the refinement loop first attempts to find a safe execution of the client program that is disallowed by the current set of specifications. If no such execution can be found, the specifications are maximal and the loop terminates. Otherwise, the identified execution is passed to a learner, which uses it to generalize the candidate solution so that the execution is permitted before continuing the refinement loop. The learner always generalizes candidate specifications, maintaining the invariant that the current solution is consistent with all previously observed library behaviors.

[2]↑Arrows for method predicates are again omitted for clarity.

### 3.1.1 Elrond in Action

To illustrate our approach in more detail, we apply it to the stack concatenation example from the introduction. Given the postcondition $\phi_{\texttt{concat}}$ and the implementation of `concat` from beginning of this chapter, Elrond generates a formula that can be simplified to the following implication:

$$
\begin{aligned}
&\forall \mathtt{s}_1, \mathtt{s}_2, \nu, \nu_{\mathbf{top}}, \nu_{\mathbf{tail}}, \nu_{\mathbf{concat}}, \nu_{\mathbf{is_empty}}, \\
&(\mathbf{R}_{\mathbf{is_empty}}(\mathtt{s}_1, \nu_{\mathbf{is_empty}}) \wedge \mathbf{R}_{\mathbf{top}}(\mathtt{s}_1, \nu_{\mathbf{top}}) \wedge \mathbf{R}_{\mathbf{tail}}(\mathtt{s}_1, \nu_{\mathbf{tail}}) \wedge \mathbf{R}_{\mathbf{push}}(\nu_{\mathbf{top}}, \nu_{\texttt{concat}}, \nu)) \\
&\quad \Longrightarrow \\
&((\nu_{\mathbf{is_empty}} = \bot \wedge \forall u, (hd(\nu_{\mathbf{concat}}, u) \implies hd(\nu_{\mathbf{tail}}, u) \vee hd(\mathtt{s}_2, u)) \wedge \\
&\qquad (mem(\nu_{\mathbf{concat}}, u) \iff mem(\nu_{\mathbf{tail}}, u) \vee mem(\mathtt{s}_2, u))) \implies \\
&\quad \forall u, (hd(\nu, u) \implies hd(\mathtt{s}_1, u) \vee hd(\mathtt{s}_2, u)) \wedge (mem(\nu, u) \iff mem(\mathtt{s}_1, u) \vee mem(\mathtt{s}_2, u)))
\end{aligned}
$$

The four predicates in the premise of this formula correspond to the four library functions (**push**, **top**, **is_empty** and **tail**) invoked in a recursive call to `concat`. The specification of a blackbox library function $f$ in our assertion logic is represented as a *placeholder predicate*: an uninterpreted predicate that relates the parameters of $f$ to its return value. For a library function $f$, we adopt a naming convention of $\mathbf{R_f}$ and $\nu_f$ for its placeholder predicates and return values, respectively. The predicate $\mathbf{R}_{\mathbf{top}}(\mathtt{s}_1, \nu_{\mathbf{top}})$ in the above formula, for example, says the variable $\nu_{\mathbf{top}}$ holds the return value of the call to `Stack.top s1` in `concat`. The result of the recursive call to `concat` is similarly denoted as $\nu_{\texttt{concat}}$. The conclusion of the formula encodes the expected verification condition for a recursive call to `concat`: namely, that if the result of the call to `Stack.is_empty s1` is false and the recursive call to `concat (Stack.tail s1) s2` satisfies $\phi_{\texttt{concat}}$, then the result of `concat` must also satisfy $\phi_{\texttt{concat}}$. The remainder of this section refers to the premise and conclusion of this implication as $\Sigma_{concat}$ and $\Phi_{\texttt{concat}}$, respectively.

### Method Predicates

From a logical standpoint, the method predicates used in $\Phi_{\texttt{concat}}$ are simply uninterpreted function symbols which have no intrinsic semantics.

This representation allows our specifications to use predicates whose semantics may be difficult to encode directly in the logic. Embedding recursively defined predicates like `mem`, for example, requires particular care [47].

In order to ensure that the specifications inferred by Elrond are tethered to reality, users must also supply Elrond with implementations (possibly blackbox) of these predicates. One possible implementation for *hd* and *mem* is:

```
1  let hd(s, u) =
2     if Stack.is_empty s then false else (Stack.top s) == u
3
4  let rec mem(s, u) =
5     if Stack.is_empty s then false else (Stack.top s = u) ||
6                                         (mem(Stack.tail s, u))
```

where `Stack.is\_empty`, `Stack.top` and `Stack.tail` refer to blackbox implementations of stack library methods.

While it is possible to naïvely include a method predicate for each library method, such an approach may not be useful for verification. Some functions may be irrelevant to client assertions, unnecessarily increasing the set of possible specifications that must be considered. Conversely, the library may not include functions for desirable predicates; e.g., the **Stack** library does not provide a Stack.mem function, although it is quite relevant for verifying our running example.

### Solution Space

Our ultimate goal is to find a mapping from each placeholder predicate in $\Sigma_{concat}$ to an interpretation that entails $\Phi_{\texttt{concat}}$. We refer to such a mapping as a *verification interface*; Fig. 3.4 presents a potential verification interface for our running example. Not every mapping that ensures the safety of `concat` is reasonable, however. At one extreme, interpreting every

$$
\begin{aligned}
&\mathbf{R_{is_empty}}(\mathtt{s},\nu) \mapsto \forall u, (\nu \implies \neg mem(\mathtt{s},u)) \wedge (\neg\nu \wedge hd(\mathtt{s},u) \implies mem(\mathtt{s},u))\\
&\mathbf{R_{top}}(\mathtt{s},\nu) \mapsto \forall u, mem(\mathtt{s},\nu) \wedge (\nu = u \iff hd(\mathtt{s},u))\\
&\mathbf{R_{tail}}(\mathtt{s},\nu) \mapsto \forall u, (mem(\mathtt{s},u) \implies (mem(\nu,u) \vee hd(\mathtt{s},u)))\wedge\\
&\qquad\qquad ((mem(\nu,u) \vee hd(\nu,u)) \implies mem(\mathtt{s},u))\\
&\mathbf{R_{push}}(\mathtt{x},\mathtt{s},\nu) \mapsto \forall u, (mem(\nu,u) \wedge mem(\mathtt{s},u) \implies \neg(\mathtt{x} = u \vee hd(\nu,u)))\wedge\\
&\qquad\qquad (mem(\nu,u) \wedge \neg mem(\mathtt{s},u) \implies (\mathtt{x} = u \wedge hd(\nu,u)))\wedge\\
&\qquad\qquad ((\mathtt{x} = u \vee hd(\nu,u) \vee hd(\mathtt{s},u) \vee mem(\mathtt{s},u)) \implies mem(\nu,u))
\end{aligned}
$$

**Figure 3.4.** Candidate verification interface for the **concat** example.

predicate as $\perp$ ensures the safety of the client, but does not capture the behavior of any sensible stack implementation. Our goal, then, is to find interpretations that are *general* enough to cover a range of possible implementations. From a purely logical perspective, this an instance of a multi-abductive inference problem that tries to find the weakest interpretations of $\mathbf{R_{push}}$, $\mathbf{R_{top}}$, $\mathbf{R_{tail}}$ and $\mathbf{R_{is_empty}}$ in terms of predicates *mem* and *hd* such that the interpretations are self-consistent (i.e. $\bigwedge R_i \not\models \perp$) and which are sufficient to prove the desired verification condition. While solutions to the multi-abduction problem have been developed for domains that admit quantifier elimination, e.g. linear integer arithmetic constraints [46], there is no purely logical solution for formulae involving equalities over uninterpreted functions.

An additional challenge in our setting is that we seek to infer specifications consistent with the library's implementation, a requirement that is absent in [46].

### 3.1.2 Data-Driven Abduction

We overcome these challenges by adopting a data-driven approach to abducing maximal library specifications, framing the problem as one of training a Boolean classifier on a set of *example behaviors* for each function $f$. Under this interpretation, a classifier represents a specification of the "acceptable" behaviors of $f$. Thus, the goal of the specification inference stage of our algorithm is to learn a set of classifiers that recognize the behaviors of each $f$ that a) are consistent with $f$'s underlying implementation and b) preserve the safety of the client program. As discussed at the start of this section, this algorithm uses both an

SMT-based verifier and a property-based test generator as sources of training data for our learner. The former identifies example behaviors consistent with $\neg(\Sigma_{\mathbf{concat}} \implies \Phi_{\mathbf{concat}})$; these are labelled as "negative" examples, so that the learned specifications can help the solver rule out behaviors that are inconsistent with the semantics of the method predicates or that produce unsafe executions (i.e., interpretations that would violate the postcondition). Example behaviors drawn from tests are labelled as "positive", so that our learner is biased towards explanations that are consistent with the (unknown) implementations of library functions. Notably, our algorithm generalizes this data-driven abduction procedure for individual functions to the multi-abduction case, ensuring that discovered interpretations are globally consistent over all library methods.

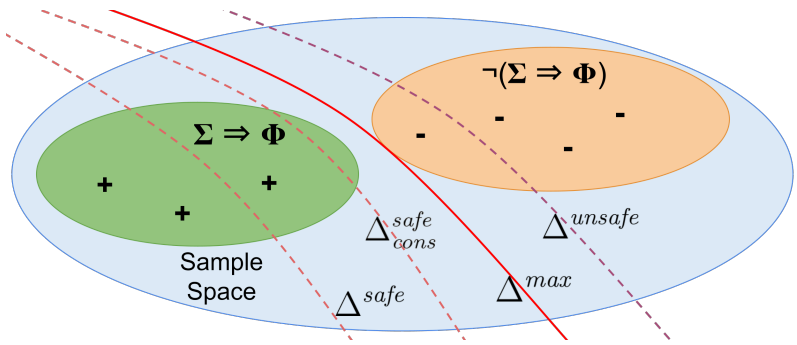


**Figure 3.5.** Elements in the circle labelled $\Sigma \Rightarrow \Phi$ represent positive examples, while the orange $\neg(\Sigma \Rightarrow \Phi)$ circle contains negative examples.

Fig. 3.5 depicts the space of example behaviors for our learner, as well four potential verification interfaces. Each of these represents a potential solution in the *hypothesis space* for this learner, which is tasked with building a classifier that separates negative (-) and positive examples (+) for each library function. Our algorithm generates a mapping $\Delta$, which maps placeholder predicates to specifications, separating the two. The dashed purple line, labelled $\Delta^{unsafe}$, represents an unsafe verification interface that allows a client program to violate the desired postcondition. The remaining red lines represent the range of safe verification interfaces. The two dashed red lines represent the verification interfaces that are sufficient to verify the client, but which are suboptimal. $\Delta^{safe}$ is safe but inconsistent with the observed behaviors of the library implementation, and is thus overly restrictive. $\Delta^{safe}_{cons}$ is safe and consistent, but not maximal, as there exists a weaker verification interface ($\Delta^{max}$) in the

hypothesis space that is still safe and consistent. Intuitively, the goal of our first phase is to identify $\Delta^{safe}_{cons}$, which is then weakened by the second phase to produce $\Delta^{max}$.

### Hypothesis space

Our learner limits the shape of solutions it considers so that inferred specifications are both amenable to automatic verification and strong enough to verify specified postconditions. To enable automated verification of client programs, potential specifications are required to be prenex universally-quantified propositional formulae over datatype values and variables representing arguments to the predicates under consideration. Some possible specifications of the library function **push** in our running example include:

$$\mathbf{push}(x, l) = \nu : \left\{ \begin{array}{ll} \forall u, mem(l, u) \implies mem(\nu, u), & \forall u, \neg mem(\nu, u), \\ \forall u, u = x, & \forall u, u = x \iff hd(\nu, u), \\ \forall u, mem(l, u) \wedge hd(l, u), & \ldots \end{array} \right\}$$

which contains, among other candidates, the desired specification.

All atomic literals in generated formulae are applications of uninterpreted method predicates and equalities over quantified variables, parameters, and return values of functions. The literals in the above formulae are simply applications of $hd$ and $mem$ to $l, \nu$ and $u$, and the equality $u = x$. We automatically discard equalities between terms of different types, e.g. $l = x$. The *feature set* for the predicate $\mathbf{R_{push}}$, i.e. the set of atomic elements used to construct its specification, is thus: $\{hd(l, u),\ mem(l, u),\ hd(\nu', u),\ mem(\nu, u),\ x = u\}$.

### Training Data

We now consider how to represent library behaviors in a form that is useful for learning a solution in our hypothesis space. To illustrate our chosen representation, consider the counterexample produced by an off-the-shelf theorem prover when asked to verify the formula

$\Sigma_{\texttt{concat}} \implies \Phi_{\texttt{concat}}$ from our running stack example, where the set of candidate specifications are initialized to true (i.e., $\forall \mathbf{f}.\, \mathbf{R_f} \mapsto \top$):

$$\forall l\ u, \neg hd(l,u) \wedge \nu_{\texttt{top}} = \texttt{a} \wedge (mem(l,u) \iff ((l = \nu \vee l = \nu_{\textbf{concat}} \vee l = \nu_{\textbf{tail}}) \wedge u = \texttt{a})) \tag{Cex}$$

Intuitively, this counterexample asserts that the stacks $\nu$, $\nu_{\texttt{tail}}$, and $\nu_{\texttt{concat}}$ contain exactly one element, the constant $\texttt{a}$, and the other two stacks, $\texttt{s}_1$ and $\texttt{s}_2$, are empty. This assertion indeed violates the second conjunct of the postcondition, $mem(\nu, u) \iff mem(\texttt{s}_1, u) \vee mem(\texttt{s}_2, u)$, but it is inconsistent with the expected semantics of the library functions, and can thus be safely ignored. The verifier generates this counterexample because the interpretations of the placeholder predicates in $\Sigma_{\texttt{concat}}$ are too permissive. In order for the verifier to rule out this counterexample, the placeholder predicates need to be strengthened to rule out this inconsistent behavior.

Ignoring how we identify this counterexample as spurious for now, note that there are many ways to strengthen these specifications. One approach is to focus on one particular function at a time. For example, we could choose to refine the specification of $\mathbf{R_{top}}$ so that it guarantees that $\nu_{\textbf{top}}$ is a member of $\texttt{s}_1$. Alternatively, we could focus on $\mathbf{R_{tail}}$, ensuring the members of $\nu_{\textbf{tail}}$ are also contained by $\texttt{s}_1$. In general, however, it may be necessary to strengthen multiple specifications at once. Therefore, instead of focusing on one specification at a time, we learn refined specifications simultaneously.

The first step to refining our placeholder specifications is to extract data from Cex in a form that can be used to train a classifier. We do this by using the assignments to the arguments of the placeholder predicates in a counterexample to build *feature vectors* that describe the valuations of method predicates and equalities in the unsafe execution. Table 3.1 presents the feature vectors extracted from Cex. The first column of this table indicates the particular placeholder predicates that can be strengthened to rule out this counterexample. The second column gives the feature vectors for a particular instantiation of the quantified variables (*e.g.*, $\texttt{u}$) of the placeholders. The subsequent columns list applications of method predicates, with the rows underneath listing the valuation of these predicates in the offending

**Table 3.1.** Potential negative feature vectors extracted from CEX.

| $\mathbf{R_{top}}(\mathbf{s_1}, \nu_{\mathbf{top}})$ | $\forall \mathbf{u}$ | $hd(\mathbf{s}_1, \mathbf{u})$ | $mem(\mathbf{s}_1, \mathbf{u})$ | $\nu_{\mathbf{top}} = \mathbf{u}$ | | |
|---|---|---|---|---|---|---|
| | $\mathbf{u} = \mathtt{a}$ | false | false | true | | |
| | $\mathbf{u} \neq \mathtt{a}$ | false | false | false | | |
| $\mathbf{R_{tail}}(\mathbf{s_1}, \nu_{\mathbf{tail}})$ | $\forall \mathbf{u}$ | $hd(\mathbf{s}_1, \mathbf{u})$ | $mem(\mathbf{s}_1, \mathbf{u})$ | $hd(\nu_{\mathbf{tail}}, \mathbf{u})$ | $mem(\nu_{\mathbf{tail}}, \mathbf{u})$ | |
| | $\mathbf{u} = \mathtt{a}$ | false | false | false | true | |
| | $\mathbf{u} \neq \mathtt{a}$ | false | false | false | false | |
| $\mathbf{R_{push}}(\nu_{top}, \nu_{concat}, \nu)$ | $\forall \mathbf{u}$ | $hd(\nu_{\mathbf{concat}}, \mathbf{u})$ | $mem(\nu_{\mathbf{concat}}, \mathbf{u})$ | $hd(\nu, \mathbf{u})$ | $mem(\nu, \mathbf{u})$ | $\nu_{\mathbf{top}} = \mathbf{u}$ |
| | $\mathbf{u} = \mathtt{a}$ | false | true | false | true | true |
| | $\mathbf{u} \neq \mathtt{a}$ | false | false | false | false | false |
| $\mathbf{R_{is_empty}}(\mathbf{s_1}, \nu_{\mathbf{is_empty}})$ | $\forall \mathbf{u}$ | $hd(\mathbf{s}_1, \mathbf{u})$ | $mem(\mathbf{s}_1, \mathbf{u})$ | $\nu_{\mathbf{is_empty}}$ | | |
| | $\mathbf{u} = \mathtt{a}$ | false | false | false | | |
| | $\mathbf{u} \neq \mathtt{a}$ | false | false | false | | |

run. The second row corresponds to the assertion that $\neg hd(\mathtt{s}_1, \mathtt{a}) \wedge \neg mem(\mathtt{s}_1, \mathtt{a}) \wedge \nu_{\mathbf{top}} = \mathtt{a}$, for example. A strengthening of the specifications that disallows any one of these interpretations will also rule out the corresponding unsafe run of the program. Put another way, each row corresponds to a potential *negative feature vector*, and a classifier (i.e., specification) for the corresponding placeholder that disallows this feature will disallow the counterexample.

The designation of these features as *potentially* negative is deliberate, as we only want to disallow features that are inconsistent with the implementation of the library functions. As an example, the first feature vector for $\mathbf{R_{top}}$ (the second row of Table 3.1) states that the result of `top s1` is not the head element of the input stack (since $\nu_{\mathbf{top}} = \mathbf{u}$ is true and $hd(\mathbf{s}_1, \mathbf{u})$ is false), and thus is inconsistent with *any* reasonable implementation of **top**. In contrast, the next feature vector is compatible with an execution of **top** where, e.g. $\mathbf{top}([1]) = 1$ and $\mathbf{u} = 2$.

The second feature vector represents a behavior that is consistent with the underlying library implementation and that should be allowed by the learned specification. The consistency checker generates this positive training data via random testing of the client program. To see how we extract positive feature vectors from training data, consider the execution of

`concat` with the inputs $\mathbf{s}_1 = [\mathtt{a}]$ and $\mathbf{s}_2 = [\mathtt{b}]$, which produces the following assignment to program variables:

$$\nu_{\mathbf{is_empty}} = \bot, \nu_{\mathbf{top}} = \mathtt{a}, \nu_{\mathbf{tail}} = [], \nu_{\mathbf{concat}} = [\mathtt{b}], \nu = [\mathtt{a}; \mathtt{b}]$$

Similar to how we built negative feature vectors from Cex, we can construct feature vectors for each function specification from these assignments. Table 3.2 illustrates the feature vectors corresponding to this assignment. Consider the second row where $\mathbf{u}$ is instantiated with $\mathtt{a}$: under this assignment, $hd(\mathbf{s}_1, \mathbf{u}) \equiv \mathbf{hd}([\mathtt{a}], \mathtt{a})$ is true, as are $mem(\mathbf{s}_1, \mathbf{u}) \equiv \mathbf{mem}([\mathtt{a}], \mathtt{a})$ and $\nu_{\mathbf{top}} = \mathbf{u} \equiv \mathtt{a} = \mathtt{a}$.

**Table 3.2.** Positive feature vectors extracted from executing `concat`([a], [b]).

| $\mathbf{R_{top}}(\mathbf{s_1}, \nu_{\mathbf{top}})$ | $\forall \mathbf{u}$ | $hd(\mathbf{s}_1, \mathbf{u})$ | $mem(\mathbf{s}_1, \mathbf{u})$ | $\nu_{\mathbf{top}} = \mathbf{u}$ | | |
|---|---|---|---|---|---|---|
| | $\mathbf{u} = \mathtt{a}$ | true | true | true | | |
| | $\mathbf{u} = \mathtt{b}$ | false | false | false | | |
| $\mathbf{R_{tail}}(\mathbf{s_1}, \nu_{\mathbf{tail}})$ | $\forall \mathbf{u}$ | $hd(\mathbf{s}_1, \mathbf{u})$ | $mem(\mathbf{s}_1, \mathbf{u})$ | $hd(\nu_{\mathbf{tail}}, \mathbf{u})$ | $mem(\nu_{\mathbf{tail}}, \mathbf{u})$ | |
| | $\mathbf{u} = \mathtt{a}$ | true | true | false | false | |
| | $\mathbf{u} = \mathtt{b}$ | false | false | false | false | |
| $\mathbf{R_{push}}(\nu_{top}, \nu_{concat}, \nu)$ | $\forall \mathbf{u}$ | $hd(\nu_{\mathbf{concat}}, \mathbf{u})$ | $mem(\nu_{\mathbf{concat}}, \mathbf{u})$ | $hd(\nu, \mathbf{u})$ | $mem(\nu, \mathbf{u})$ | $\nu_{\mathbf{top}} = \mathbf{u}$ |
| | $\mathbf{u} = \mathtt{a}$ | false | false | true | true | true |
| | $\mathbf{u} = \mathtt{b}$ | true | true | false | true | false |
| $\mathbf{R_{is_empty}}(\mathbf{s_1}, \nu_{\mathbf{is_empty}})$ | $\forall \mathbf{u}$ | $hd(\mathbf{s}_1, \mathbf{u})$ | $mem(\mathbf{s}_1, \mathbf{u})$ | $\nu_{\mathbf{is_empty}}$ | | |
| | $\mathbf{u} = \mathtt{a}$ | true | true | false | | |
| | $\mathbf{u} = \mathtt{b}$ | false | false | false | | |

**Table 3.3.** Disjoint positive and negative feature vectors of $\mathbf{R_{top}}$.

| $\mathbf{R_{top}}(\mathbf{s_1}, \nu_{\mathbf{top}})$ | $hd(\mathbf{s}_1, \mathbf{u})$ | $mem(\mathbf{s}_1, \mathbf{u})$ | $\nu_{\mathbf{top}} = \mathbf{u}$ |
|---|---|---|---|
| - | false | false | true |
| + | true | true | true |
| + | false | false | false |

### Classification

In order to train a classifier, we need to label the extracted feature vectors as either positive or negative. In other words, we need to identify behaviors that should (and should not) be

allowed by the inferred specification. Assigning labels is not as straightforward as labelling the feature vectors extracted from counterexamples as negative and those extracted from testing as positive, as the two sets can overlap. We observe this in the vectors of $\mathbf{R_{top}}$ from Table 3.1 and Table 3.2: the interpretation $hd(\mathbf{s}_1, \mathbf{u}) \mapsto \mathbf{false}; mem(\mathbf{s}_1, \mathbf{u}) \mapsto \mathbf{false}; \nu_{\mathbf{top}} = \mathbf{u} \mapsto \mathbf{false}$; occurs in both tables. Intuitively, we do not want to strengthen the specification of $\mathbf{R_{top}}$ to rule out this interpretation, as the positive sample is a witness that this execution is consistent with the implementation of $\mathbf{R_{top}}$. Ultimately, therefore, the specification must be relaxed to allow the execution. In cases where a negative feature vector conflicts with a positive feature vector, we identify the potential negative feature vector as positive and remove it from the learner's negative feature vector set. This strategy is similar to the one used by [49] to deal with inductiveness counterexamples.

Thankfully, as long as the counterexample set contains at least one feature vector not known to be consistent with the underlying library implementation, we can strengthen the specifications to disallow it. The first feature vector for $\mathbf{R_{top}}$ in Table 3.1 represents one such infeasible execution. This vector encodes the case where $u$ is the output of **top** but is not a member or head of the input stack. Clearly, no reasonable implementation would support such an interpretation. We use this observation to label as "negative" those feature vectors that are extracted from a counterexample but do not appear in the set drawn from a concrete execution. Table 3.3 shows the partition of positive and negative feature vectors for $\mathbf{R_{top}}$.

Given this labelled set of positive and negative feature vectors, our data-driven learner builds a separator over the training data. One such classifier (formula) for the data in Table 3.3 is $\nu_{\mathbf{top}} = \mathbf{u} \implies hd(\mathbf{s}_1, \mathbf{u})$. Substituting similarly learned specifications for the other library functions in $\Sigma_{\texttt{concat}}$ equips the SMT solver with enough constraints to rule out Cex while maintaining the invariant that the learned specifications are also consistent with the underlying library implementations. Additional iterations of this counterexample-guided refinement loop gather additional positive and negative features, eventually producing the specifications for the library functions presented in Fig. 3.4.

### Identifying Spurious Counterexamples

Thus far, we have only considered spurious counterexamples generated by the safety checker. Counterexamples, however, can also result from an incorrect client assertion. For example, suppose the client (unsoundly) asserts:

$$\forall u, mem(\nu, u) \implies mem(\mathtt{s}_2, u)$$

This assertion is wrong, assuming reasonable implementations of **top** and **push**, as the elements of the result stack $\nu$ can also come from $\mathtt{s}_1$. We distinguish counterexamples corresponding to actual safety violations by first checking if all the feature vectors extracted from the counterexample are included in the set of known positive feature vectors. For example, given this unsound assertion, the verifier may produce the following counterexample:

$$\begin{aligned} \forall l\ u, (hd(l, u) &\iff ((l = \nu \vee l = \mathtt{s}_1) \wedge u = \mathsf{a})) \wedge \nu_{\mathtt{top}} = \mathsf{a} \wedge \\ (mem(l, u) &\iff ((l = \nu \vee l = \mathtt{s}_1) \wedge u = \mathsf{a})) \end{aligned} \tag{Cex'}$$

**Table 3.4.** Feature vectors for $\mathbf{R_{top}}$ extracted from Cex'.

| $\mathbf{R_{top}}(\mathbf{s_1}, \nu_{\mathbf{top}})$ | $\forall \mathbf{u}$ | $hd(\mathbf{s}_1, \mathbf{u})$ | $mem(\mathbf{s}_1, \mathbf{u})$ | $\nu_{\mathbf{top}} = \mathbf{u}$ |
|---|---|---|---|---|
| | $\mathbf{u} = \mathsf{a}$ | true | true | true |
| | $\mathbf{u} \neq \mathsf{a}$ | false | false | false |

Table 3.4 shows all of the feature vectors for $\mathbf{R_{top}}$ that are extracted from this counterexample. Since these are a subset of the positive feature vectors from Table 3.2, there are no feature vectors that can be labeled as negative, and there are thus no new bad behaviors that the learner can use to generate a refined specification mapping that rejects the counterexample. In this scenario, our algorithm tries to discover concrete values of $\mathtt{s}_1$ and $\mathtt{s}_2$ consistent with Cex'; that is, $\mathtt{s}_1$ only contains $\mathsf{a}$ and $\mathtt{s}_2$ is empty. When called with these parameters, **concat**$([\mathsf{a}], [])$ will return $\nu \equiv [\mathsf{a}]$, which Elrond returns as a witness of an unsafe execution.

Note that this situation may also occur when the feature set is not large enough, as the specifications in the corresponding hypothesis space are not expressive enough to identify a spurious counterexample. Thus, if we are not able to find inputs that trigger a safety violation, we grow the feature set by increasing the number of quantified variables (*e.g.* from $u$ to $u, v$) so that Elrond can explore a richer space of specifications.

### Weakening

While the above strategy is guaranteed to find a safe and consistent verification interface when one exists, the solutions it produces may still be suboptimal, as illustrated in Fig. 3.5. For example, the first conjunct of the specification for `push` in Fig. 3.4 states that any existing member of both the input and output stacks should not be the same as the element being added to the stack; that is, `push` always produces a stack with no duplicates. This specification is too restrictive, however, as it disallows reasonable behaviors such as `push([1;2], 1) = [1;1;2]`. However, if our sampler never generates an observation corresponding to this behavior, e.g. $x \equiv 1$, $\mathtt{s} \equiv [1;2]$ and $\nu_{push} \equiv [1;1;2]$, the candidate specification for `top` produced by Elrond's first phase will incorrectly disallow it.

In other words, our reliance on testing to identify and label negative feature vectors may result in initial specifications that are overfitted to the examples enumerated by the test generator. There are two potential reasons such a positive example might be missed: (1) the input space of the program might be too large for a test generator to effectively explore, and (2) the provided implementation may simply not exhibit this behavior (e.g., it may be the case that the implementation of `push` that we are trying to verify against does indeed remove duplicates). While exhaustive or more effective enumeration can address the first cause, it cannot remedy the second. Elrond's weakening phase helps ameliorate both issues.

Our weakening algorithm iteratively weakens candidate specifications, focusing on one library function at a time. To weaken the specification of `push`, for example, we fix the specifications of the other library functions to their assignments in the current verification interface, and then try to find a maximal weakening of $\mathbf{R}_{\mathbf{push}}$ that admits a larger set of implementations of `push`. To do so, Elrond attempts to discover additional *weakening feature*

*vectors* for $\mathbf{R_{push}}$, or feature vectors corresponding to behaviors disallowed by the current specification but which would not lead to a violation of client safety. One possible weakening feature vector for our current example is shown in Table 3.5. Here, the head of both $\mathtt{s}_1$ and the result of the recursive call is $\mathbf{1}$; this scenario is mistakenly disallowed by the specification of `push` in Fig. 3.4.

**Table 3.5.** A weakening feature vector of $\mathbf{R_{push}}$ in Fig. 3.4

| $\mathbf{R_{push}}(\nu_{\mathbf{top}}, \nu_{\mathbf{concat}}, \nu)$ | $\forall \mathbf{u}$ | $hd(\nu_{\mathbf{concat}}, \mathbf{u})$ | $mem(\nu_{\mathbf{concat}}, \mathbf{u})$ | $hd(\nu, \mathbf{u})$ | $mem(\nu, \mathbf{u})$ | $\nu_{\mathbf{top}} = \mathbf{u}$ |
|---|---|---|---|---|---|---|
| | $\mathbf{u} = 1$ | true | true | true | true | true |

Elrond repeatedly queries the verifier to identify weakening feature vectors for `push`, indicating that the current specification is maximal when none can be found. It then moves on to the next function specification, iteratively weakening each until a fixpoint is reached. Fig. 3.6 shows the maximal verification interface Elrond builds by weakening the candidate specifications in Fig. 3.4. Compared with Fig. 3.4, the specification of `push` now permits duplicate elements in stacks. In addition, the specification of `is_empty` has been simplified by removing the redundant conjunction $\neg\nu \wedge hd(\mathtt{s}, u) \implies mem(\mathtt{s}, u)$, as $hd(\mathtt{s}, u) \implies mem(\mathtt{s}, u)$ can never be violated by a concrete stack value thanks to the observed semantics of *hd* and *mem*.

$$
\begin{aligned}
&\mathbf{R_{is_empty}}(\mathtt{s}, \nu) \mapsto \forall u, (\nu \implies \neg mem(\mathtt{s}, u)) \\
&\mathbf{R_{top}}(\mathtt{s}, \nu) \mapsto \forall u, \nu = u \iff (hd(\mathtt{s}, u) \wedge mem(\mathtt{s}, u)) \\
&\mathbf{R_{tail}}(\mathtt{s}, \nu) \mapsto \forall u, (mem(\mathtt{s}, u) \implies (mem(\nu, u) \vee hd(\mathtt{s}, u))) \wedge \\
&\qquad\qquad ((mem(\nu, u) \vee (hd(\nu, u) \wedge hd(\mathtt{s}, u))) \implies mem(\mathtt{s}, u)) \\
&\mathbf{R_{push}}(\mathtt{x}, \mathtt{s}, \nu) \mapsto \forall u, (mem(\nu, u) \wedge \neg \mathtt{x} = u \implies mem(\mathtt{s}, u) \wedge \neg hd(\nu, u)) \wedge \\
&\qquad\qquad (\mathtt{x} = u \vee mem(\mathtt{s}, u) \implies mem(\nu, u)) \wedge (\neg mem(\nu, u) \wedge hd(\nu, u) \implies hd(\mathtt{s}, u))
\end{aligned}
$$

**Figure 3.6.** Maximal specifications for `Stack` operations. The special variable $\nu$ represents the result of the method.

## 3.2 Problem Formulation

Having completed a high-level tour of Elrond in action, we now present a precise description of the specification synthesis problem and our data-driven inference procedure. We consider functional programs that use data structure libraries providing functions to access and construct instances of inductively-defined algebraic datatypes (e.g., list, stacks, trees, heaps, tries, etc.).

In the remainder of the chapter, we use $(\Sigma, \Phi)$ to refer to the *verification query* whose validity we are attempting to establish. These structures serve the same role as verification conditions in a typical verification framework. The first component of this query, $\Sigma$, is a conjunction of applications of specification placeholders ($R_f$) to arguments; these represent the library method calls made by the client program. The second component, $\Phi$, represents the client program's pre- and post-conditions, encoded as sentences built from logical connectives ($\wedge$, $\vee$, $\implies$) over prenex universally-quantified propositional formulae. Each verification query corresponds to a control flow path in the client program; the full algorithm considers the conjunction of all these verification queries at once. To keep the formalization and the description of our algorithms concise, our description considers a single verification query in isolation. The extension to sets of verification queries is provided Sec. 3.4.3.

**Definition 3.2.1 (Problem Definition).** A given verification query $(\Sigma, \Phi)$ with unknown library functions $F$ has the form $\Sigma \implies \Phi$, where:

$$\Sigma \equiv (\bigwedge_{f \in F, i=0}^{n} R_f(\vec{x_i}) \wedge \bigwedge_{x, x' \in \cup_{0 \leq j \leq i} \vec{x_j} \cup \mathcal{C}} x = x')$$

Here, the equality constraints are either between program variables $(x, x')$ or between variables and constants $\mathcal{C}$ of some base type (e.g., Booleans and integers). Each $R_f(\vec{x_i})$ in $\Sigma$ is an application of a placeholder predicate to some arguments; the conjunction of these placeholder applications and equality constraints represents a sequence of library method invocations in one control-flow path of the client.

To model the input and output behaviors of the blackbox implementations of library functions and method predicates, our formalization relies on a pair of partial functions with the same

signature as the implementations. We use partial functions to reflect the fact that we can only observe a subset of the full behaviors of these implementations when searching for specifications.

**Definition 3.2.2 (Specification Configuration).** Let $P$ be a set of method predicates and $F$ be the set of functions in a library used by the client. Let $\Gamma_f$ be a partial function from the domain of $f \in F$ to its codomain, and $\Gamma_p$ be a partial function with the same signature as $p \in P$. Let $\Gamma_P = \bigcup_{p \in P} \Gamma_p$ and $\Gamma_F = \bigcup_{f \in F} \Gamma_f$. A *specification configuration* is a 5-tuple $((\Sigma, \Phi), P, F, \Gamma_P, \Gamma_F)$, where $(\Sigma, \Phi)$ is the verification query extracted from the client.

**Example.** The specification configuration of our running example consists of a verification query $(\Sigma_{concat}, \Phi_{\texttt{concat}})$, a method predicate set $\{memhd\}$, and library functions $\{$**push**, **is_empty**, **top**, **tail**$\}$. The partial functions in $\Gamma_F$ and $\Gamma_P$ abstract over observations on their corresponding blackbox implementations; for an execution that produces the feature vectors shown in Table 3.2, they are:

$$
\begin{aligned}
\Gamma_{mem} &\equiv \lambda(l \in \{\mathbf{s_1}, \mathbf{s_2}, \nu_{\mathbf{tail}}, \nu_{\mathbf{concat}}, \nu\}), u. \\
&\quad (u = \texttt{a} \wedge (l = \mathbf{s_1} \vee l = \nu)) \vee (u = \texttt{b} \wedge (l = \mathbf{s_2} \vee l = \nu_{\mathbf{concat}} \vee l = \nu)) \\
\Gamma_{hd} &\equiv \lambda(l \in \{\mathbf{s_1}, \mathbf{s_2}, \nu_{\mathbf{tail}}, \nu_{\mathbf{concat}}, \nu\}), u. \\
&\quad (u = \texttt{a} \wedge (l = \mathbf{s_1} \vee l = \nu)) \vee (u = \texttt{b} \wedge (l = \mathbf{s_2} \vee l = \nu_{\mathbf{concat}})) \\
\Gamma_{\mathbf{push}} &\equiv \lambda x, (l \in \{\nu_{\mathbf{concat}})\}).\nu \\
\Gamma_{\mathbf{is_empty}} &\equiv \lambda(l \in \{\mathbf{s_1}\}), \mathit{false} \\
\Gamma_{\mathbf{top}} &\equiv \lambda(l \in \{\mathbf{s_1}\}), \texttt{a} \\
\Gamma_{\mathbf{tail}} &\equiv \lambda(l \in \{\mathbf{s_1}\}), \nu_{\mathbf{tail}}
\end{aligned}
$$

where stack arguments are limited to values in this particular execution, e.g. $\mathbf{s_1}$ or $\nu_{\mathbf{tail}}$.

Given a specification configuration as input, the output of our verification pipeline is a *verification interface* ($\Delta$), a logical interpretation of the method predicates that maps each placeholder predicate for a function $f \in F$ to a universally-quantified propositional formula

over the parameters and result of $f$. We impose two requirements on $\Delta$. The first is *safety*: an underlying theorem prover (e.g., a SMT solver) must be able to prove $\Sigma[\Delta] \implies \Phi$, where $\Sigma[\Delta]$ denotes the formula constructed by replacing all occurrences of specification placeholders with their interpretation in $\Delta$, and $\Sigma \implies \Phi$ is the verification query built from a client program:

**Definition 3.2.3 (Safe Verification Interface).** For a given verification query $(\Sigma, \Phi)$, a verification interface $\Delta$ is *safe* when:

1. it makes the VC valid: $\Sigma[\Delta] \models \Phi$, and
2. is not trivial: $\Sigma[\Delta] \not\models \bot$

In addition to safety, we also desire that any proposed mapping $\Delta$ be *consistent* with the provided implementations of method predicates and library functions, *i.e.* that $\Delta$ must accurately represent their observed behavior. Formally:

**Definition 3.2.4 (Interface Consistency).** A verification interface $\Delta$ is consistent with $\Gamma_P$ and $\Gamma_F$ when all specifications in $\Delta$ are consistent with the inputs on which $\Gamma_P$ and $\Gamma_F$ are defined. Formally,

$$\forall f \in F, \Gamma_f \in \Gamma_F, \Gamma_f(\vec{\alpha}) = \nu \Rightarrow \Delta(R_f)(\vec{\alpha}, \nu)[\Gamma_P]$$

The expression $\Delta(R_f)(\vec{\alpha}, \nu)$ denotes the instantiation of the formula bound to $R_f$ in $\Delta$ with the input arguments $\vec{\alpha}$ and observed output $\nu$. The expression $\Delta(R_f)(\vec{\alpha}, \nu)[\Gamma_P]$ replaces all free occurrences of $p$ in $\Delta(R_f)(\vec{\alpha}, \nu)$ with $\Gamma_p$ where $\Gamma_p \in \Gamma_P$.

This definition thus relates the observed behavior of a library method on test data, encoded by $\Gamma_P$ and $\Gamma_F$ with its logical characterization provided by $\Delta$. Note that there may be many possible verification interfaces for a given specification configuration. In order to identify the best such interface, we use an ordering based on a natural logical inclusion property:

**Definition 3.2.5 (Interface Order $\succ$).** The verification interface $\Delta'$ is weaker ($\succ$) than $\Delta$ when,

1. The two interfaces contain the same functions: $dom(\Delta) = dom(\Delta')$

2. They are not equal: $\exists R_f \in dom(\Delta), \Delta(R_f) \not\Longleftrightarrow \Delta'(R_f)$

3. The specifications in $\Delta'$ are logically weaker that those in $\Delta$: $\forall R_f \in dom(\Delta), \Delta(R_f) \implies \Delta'(R_f)$

Intuitively, weaker verification interfaces are preferable because they place fewer restrictions on the behavior of the underlying implementation. Given an ordering over verification interfaces, we seek to find the weakest safe and consistent interface, *i.e.* one that imposes the fewest constraints while still enabling verification of the client program.

**Definition 3.2.6 (Maximal Verification Interface).** For a specification configuration $((\Sigma, \Phi), P, F, \Gamma_P, \Gamma_F)$, $\Delta$ is a maximal verification interface when:

1. $\Delta$ is safe for the verification query $(\Sigma, \Phi)$.

2. $\Delta$ is consistent with $\Gamma_P$ and $\Gamma_F$.

3. For a given bound on the number of quantified variables $k$ used by the specifications in $\Delta$, there is no safe and consistent interface $\Delta'$ whose specifications use at most $k$ quantified variables such that $\Delta' \succ \Delta$.

We now refine our expectation for the output of our verification pipeline to be not just any safe and consistent verification interface, but also a *maximal* one. Notice that our notion of maximality is parameterized by the number of quantified variables used in the interpretation. As this bound increases, we can always find a weaker specification mapping. Thus we frame our definition of maximality to be relative to the number of quantified variables in the specification.

### 3.3 Learning Library Specifications

As Sec. 3.1 outlined, Elrond frames the search for a safe verification interface as a data-driven learning problem. At a high level, the goal of learning is to build a *classifier* (a

function from unlabeled data to a label) from a set of labeled data. More precisely, our goal is to learn classifiers for each of the library functions in a specification configuration that can correctly identify any input and output behavior that could induce an unsafe execution in the client.

Our first challenge is to find an encoding of program executions that is amenable to a data-driven learning framework. To begin, we need to identify the salient *features* used by a classifier to make its decisions.

**Definition 3.3.1 (Feature).** A *feature* of a function for a set of variables $\vec{x}$ is a method predicate applied to elements of $\vec{x}$ or equalities between variables in $\vec{x}$.

A feature is similar to a literal in first-order logic, but does not allow for method predicates as arguments (e.g. $hd(l, mem(l, u))$) or constant arguments (e.g. $hd(l, \mathtt{3})$).

**Definition 3.3.2 (Feature Set).** The *feature set* of a function $f$ with method predicates $P$ and quantified variables $\vec{u}$, denoted as $\mathcal{S} \equiv FSet(P, f(\vec{\alpha}_f) = \nu_f, \vec{u})$, is a list of all well-typed features in $P$ for the set of variables $\vec{\alpha}_f \cup \{\nu_f\} \cup \vec{u}$ which is *minimally linearly independent*:

$$\forall \eta' \notin \mathcal{S}, \exists \vec{\eta} \subseteq \mathcal{S}, \eta' \iff \bigwedge \vec{\eta}$$

**Example.** The feature set for the function **top** : $list_a \to a$ from the `Stack` library, where $a$ is some base type, for predicate set $P \equiv \{hd, mem\}$, equality operation $=_a$ and quantified variables $\vec{u} \equiv \{u : a\}$ is $FSet(P, \mathbf{top}(l) = \nu, \vec{u}) \equiv [hd(l, u),\ mem(l, u),\ \nu{=_a}u]$. Note that the features $mem(\nu, u)$ and $l =_a u$ are not included in this set because they are not well-typed. The feature $hd(l, \nu)$, on the other hand, is omitted because it can be represented by $hd(l, u) \wedge \nu{=_a}u$ and is thus not linearly independent with respect to the other features in the set. We use *feature vectors* to encode the features of observed tests:

**Definition 3.3.3 (Feature Vector).** A feature vector is a vector of Booleans that represents the value of each feature in the feature set for some test.

We also need to define the *hypothesis space* of possible solutions considered by our learning system. To easily integrate learned classifiers into the underlying theorem prover, we choose

to represent such solutions as Boolean combinations over terms consisting of applications of interpreted base relations and uninterpreted functions. In order to preserve decidability, we limit this space to a subset of effectively propositional sentences. This limitation was expressive enough for all of our benchmarks.[3]

**Definition 3.3.4 (Hypothesis Space).** The *hypothesis space* of specifications for a library function $f$, method predicate set $P$, and quantified variables $\vec{u}$ is the set of formulas in prenex normal form with the quantifier prefix $\forall\vec{u}$, and whose bodies are built from $FSet(P, f(\vec{\alpha}) = \nu, \vec{u})$, the logical connectives $\{\wedge, \vee, \neg, \implies\}$, and Boolean constants $\top$ (TRUE) and $\bot$ (FALSE). The hypothesis space of $f$ over $P$ and $\vec{u}$ is denoted $Hyp(P, f(\vec{\alpha}) = \nu, \vec{u})$.

In order to classify feature vectors, we ascribe them a semantics in logic:

**Definition 3.3.5 (Unitary classifier).** For a given feature vector $fv$ in a feature set $\mathcal{S} \equiv FSet(P, f(\alpha) = \nu, \vec{u})$, the *logical embedding* of $fv$ is a formula encoding the assignment to its features:

$$[\![fv]\!] \equiv \forall\vec{u}, \bigwedge_{i=0}^{|\mathcal{S}|} \mathcal{S}[i] \iff fv[i]$$

We say that a classifier $\phi$ labels a feature vector $fv$ as positive when $[\![fv]\!] \implies \phi$, and negative otherwise.

**Example.** Given the classifier $\phi \equiv \forall u, hd(\mathbf{s}_1, \mathbf{u}) \implies \nu_{\mathbf{top}} = \mathbf{u}$ for the **top** function from Sec. 3.1, the first row in Table 3.3 corresponds to the feature vector $fv^- \equiv \{hd(\mathbf{s}_1, \mathbf{u}) \mapsto \mathsf{false}; mem(\mathbf{s}_1, \mathbf{u}) \mapsto \mathsf{false}; \nu_{\mathbf{top}} = \mathbf{u} \mapsto \mathsf{true}\}$. The unitary classifier for $fv$ is $[\![fv^-]\!] \equiv \forall u, \neg hd(\mathbf{s}_1, \mathbf{u}) \wedge \neg mem(\mathbf{s}_1, \mathbf{u}) \wedge \nu_{\mathbf{top}} = \mathbf{u}$. $fv$ is labelled negative by $\phi$, as $[\![fv^-]\!] \not\Rightarrow \phi$. The other two feature vectors in Table 3.3 are labeled as positive by $\phi$.

[3]↑The main sorts of properties that we do not support as a consequence of this choice are those which use quantifier alternation, e.g., for every element in a stream, there exists another larger element that appears after it: $(\forall u, \exists v, mem(l, u) \implies (mem(l, v) \wedge ord(l, u, v) \wedge u \leq v))$.

**Definition 3.3.6 (Classification).** For a given classifier $\phi$ and feature set $\mathcal{S}$, it is straightforward to partition the feature vectors of $\mathcal{S}$ into positive ($\phi^+$) and negative ($\phi^-$) sets:

$$\begin{aligned}\phi^+ \equiv &\quad \{fv \in 2^{\mathcal{S}} \mid [\![fv]\!] \implies \phi\}\\ \phi^- \equiv &\quad \{fv \in 2^{\mathcal{S}} \mid [\![fv]\!] \not\Rightarrow \phi\}\end{aligned}$$

Notice that these two sets are trivially disjoint: $\phi^+ \cap \phi^- = \emptyset$.

For a particular configuration, we can straightforwardly lift this partitioning to verification interfaces:

$$\begin{aligned}\Delta(R_f)(\vec{\alpha},\nu)^+ \equiv &\quad \{fv \in 2^{FSet(P,f(\vec{\alpha})=\nu,\vec{u})} \mid [\![fv]\!] \implies \Delta(R_f)(\vec{\alpha},\nu)\}\\ \Delta(R_f)(\vec{\alpha},\nu)^- \equiv &\quad \{fv \in 2^{FSet(P,f(\vec{\alpha})=\nu,\vec{u})} \mid [\![fv]\!] \not\Rightarrow \Delta(R_f)(\vec{\alpha},\nu)\}\end{aligned}$$

### 3.3.1 Learning Safe and Consistent Verification Interfaces

We now confront the challenge of how to generate training data from a specification configuration in a way that guarantees the safety of the learned formulas (classifiers). To do so, we extract feature vectors from a set of *logical samples*:

**Definition 3.3.7 (Sample).** A *sample* $s$ of a formula $\phi$ is an instantiation of its quantified variables and a Boolean-valued interpretation for each application of a method predicate to those variables in $\phi$, which we denote as $s \models \phi$. The positive and negative samples of a verification query $(\Sigma, \Phi)$ are samples of $\Phi$ and $\neg\Phi$, respectively.

Intuitively, the positive samples of a verification query correspond to safe executions of a client program, while negative samples represent potential violations that safe verification interfaces need to prevent. For example, Cex from Sec. 3.1 corresponds to the following negative sample of $\Phi_{\texttt{concat}}$[4]:

$$\begin{aligned}&\{\texttt{s}_1 \mapsto \texttt{l}_0; \texttt{s}_2 \mapsto \texttt{l}_1; \nu \mapsto \texttt{l}_2; \nu_{\textbf{top}} \mapsto \texttt{a}; \nu_{\textbf{tail}} \mapsto \texttt{l}_3; \nu_{\textbf{concat}} \mapsto \texttt{l}_4; \nu_{\textbf{is_empty}} \mapsto \bot; \qquad (s^-)\\ &hd(l,u) \equiv \{\}; mem(l,u) \equiv \{(\texttt{l}_2,\texttt{a}); (\texttt{l}_3,\texttt{a}); (\texttt{l}_4,\texttt{a})\}\}\end{aligned}$$

[4]↑The interpretation of method predicates are represented as binary relations.

and the following sample, extracted from a concrete input and client execution result, is positive:

$$\{\mathtt{s}_1 \mapsto \mathtt{l}_0; \mathtt{s}_2 \mapsto \mathtt{l}_1; \nu \mapsto \mathtt{l}_2; \nu_{\mathbf{top}} \mapsto \mathtt{a}; \nu_{\mathbf{tail}} \mapsto \mathtt{l}_3; \nu_{\mathbf{concat}} \mapsto \mathtt{l}_4; \nu_{\mathbf{is_empty}} \mapsto \bot; \qquad (s^+)$$
$$hd(l,u) \equiv \{(\mathtt{l}_0,\mathtt{a}); (\mathtt{l}_1,\mathtt{b}); (\mathtt{l}_4,\mathtt{b}); (\mathtt{l}_2,\mathtt{a})\}; mem(l,u) \equiv \{(\mathtt{l}_0,\mathtt{a}); (\mathtt{l}_1,\mathtt{b}); (\mathtt{l}_4,\mathtt{b}); (\mathtt{l}_2,\mathtt{a}); (\mathtt{l}_2,\mathtt{b})\}\}$$

Although they come from different sources, both samples provide the values of variables (e.g., the value of $\nu_{\mathbf{is_empty}}$ in both $s^-$ and $s^+$ is $\bot$) and the values of predicate applications (e.g., $hd(\nu_{\mathbf{concat}}, \nu_{\mathbf{top}})$ is true in $s^-$ sample and false in $s^+$).

Using $[\![\cdot]\!]$, we can *extract* a collection of feature vectors under a feature set $\mathcal{S}$ from a sample $s$:

$$\chi_{\mathcal{S}}(s) \equiv \{fv \in 2^{\mathcal{S}} \mid s \models [\![fv]\!]\}$$

For example, the feature vectors extracted from $s^-$ and $s^+$ for the feature set of **top** are shown in the second and third rows of Table 3.1 and Table 3.2, respectively.

**Definition 3.3.8 (Classifier Consistency).** For a verification query $(\Sigma, \Phi)$, we say that a verification interface $\Delta$ is *consistent* with a negative sample $s^-$ if *at least one* of the library specifications in $\Delta$ classifies *one or more* features extracted from that sample as negative:

$$\exists R_f(\vec{\alpha}, \nu) \in \Sigma,\ \exists fv \in \chi_{FSet(P, f(\vec{\alpha})=\nu, \vec{u})}(s^-), fv \in \Delta(R_f)(\vec{a}, \nu)^-$$

Similarly, $\Delta$ is consistent with a positive sample $s^+$ if *all* specifications in $\Delta$ positively identify *every* feature vector extracted from $s^+$:

$$\forall R_f(\vec{\alpha}, \nu) \in \Sigma,\ \forall fv \in \chi_{FSet(P, f(\vec{\alpha})=\nu, \vec{u})}(s^+), fv \in \Delta(R_f)(\vec{a}, \nu)^+$$

**Example.** The verification interface $\Delta$ from Fig. 3.4 is consistent with $(s^-)$, as the specification of the **top** function labels as negative the following feature vector of $s^-$, $fv^- \equiv \{hd(\mathtt{s}_1, \mathbf{u}) \mapsto \mathsf{false}; mem(\mathtt{s}_1, \mathbf{u}) \mapsto \mathsf{false}; \nu_{\mathbf{top}} = \mathbf{u} \mapsto \mathsf{true}\}$ to negative. Furthermore, $\Delta$ is also consistent with all the feature vectors extracted from $s^+$.

**Lemma 3.3.9 (Specification Classification).** *For given classifier $\phi$,*

$$\bigvee_{fv \in \phi^+} [\![fv]\!] \iff \phi$$

*Proof.* According to Def 3.3.6, $fv \in \phi^+$ means $[\![fv]\!] \implies \phi$, thus $\bigvee_{fv\in\phi^+}[\![fv]\!] \implies \phi$. On the other hand, if $\phi \not\Longrightarrow \bigvee_{fv\in\phi^+}[\![fv]\!]$ then $\bigvee_{fv\in\phi^+}[\![fv]\!] \implies \neg\phi$. Thus there exists a feature vector $fv$ in $\phi^+$, $fv \not\Longrightarrow \phi$ which is conflict with Def 3.3.6. By contradiction, $\phi \implies \bigvee_{fv\in\phi^+}[\![fv]\!]$.

**Theorem 3.3.10.** *For a given specification configuration $((\Sigma, \Phi), P, F, \Gamma_P, \Gamma_F)$ and verification interface $\Delta$, $\Sigma[\Delta] \implies \Phi$ is valid iff $\Delta$ is consistent with all negative samples $s^-$; $\Delta$ is a consistent interface iff $\Delta$ is consistent with all positive samples $s^+$ entailed by $\Gamma_F$ and $\Gamma_P$.*[5]

*Proof.* We first prove if $\Sigma[\Delta] \implies \Phi$, then $\Delta$ is consistent with all negative samples. Consider $\Sigma[\Delta]$ which is the conjunction of application of library specifications, according to Lemma 3.3.9:

$$\Sigma[\Delta] \equiv \bigwedge_{R_f(\vec{\alpha},\nu)\in\Sigma}, \bigvee_{fv\in\Delta(R_f)(\vec{\alpha},\nu)^+} [\![fv]\!]$$

If there exists a negative sample $s^-$ which is not consistent with $\Delta$, by Def 3.3.8:

$$\forall R_f(\vec{\alpha},\nu) \in \Sigma,\ \forall fv \in \chi_{FSet(P,f(\vec{\alpha})=\nu,\vec{u})}(s^-), fv \notin \Delta(R_f)(\vec{a},\nu)^-$$

then

$$\forall R_f(\vec{\alpha},\nu) \in \Sigma,\ \forall fv \in \chi_{FSet(P,f(\vec{\alpha})=\nu,\vec{u})}(s^-), [\![fv]\!] \implies \Delta(R_f)(\vec{a},\nu)$$

which implies

$$\forall R_f(\vec{\alpha},\nu) \in \Sigma,\ s \models \Delta(R_f)(\vec{a},\nu)$$

[5]↑All positive samples $s^+$ entailed by $\Gamma_F$ and $\Gamma_P$ means every positive sample consistent with the observations encoded by $\Gamma_F$ and $\Gamma_P$.

that is $s \models \Sigma[\Delta]$. As $\Delta$ is safe, then $\Sigma[\Delta] \implies \Phi$, then $s \models \Phi$ which is conflict with $s$ is a negative sample. By contradiction, $\Delta$ is consistent with all negative samples.

Then we prove consistent verification interface $\Delta$ is consistent with all positive samples $s^+$ entailed by $\Gamma_F$ and $\Gamma_P$. If there exists a positive sample $s^+$ entailed by $\Gamma_F$ and $\Gamma_P$ which is not consistent with $\Delta$, by Def 3.3.8:

$$\exists R_f(\vec{\alpha}, \nu) \in \Sigma,\ \exists fv \in \chi_{FSet(P, f(\vec{\alpha})=\nu, \vec{u})}(s^+), fv \notin \Delta(R_f)(\vec{a}, \nu)^+$$

As $s^+$ is entailed by $\Gamma_F$ and $\Gamma_P$, then there exists a concrete value assignment which provides the input and output of each function call, specifically, the input and output of $f$ is $\vec{\alpha}_{s^+}$ and $\nu_{s^+}$:

$$s^+ \models FSet(P, f(\vec{\alpha}_{s^+}) = \nu_{s^+}, \vec{u})[\Gamma_P]$$

According to Def 3.2.4:

$$FSet(P, f(\vec{\alpha}_{s^+}) = \nu_{s^+}, \vec{u})[\Gamma_P] \implies \Delta(R_f)(\vec{\alpha}_{s^+}, \nu_{s^+})[\Gamma_P]$$

then

$$s^+ \models \Delta(R_f)(\vec{\alpha}_{s^+}, \nu_{s^+})[\Gamma_P]$$

On the other hand all feature vectors in $s^+$ is

$$\chi_{FSet(P, f(\vec{\alpha})=\nu, \vec{u})}(s^+) \equiv \{fv \mid s^+ \models [\![fv]\!]\}$$

which means

$$\forall fv, s^+ \models [\![fv]\!] \implies fv \in \chi_{FSet(P, f(\vec{\alpha})=\nu, \vec{u})}(s^+)$$

Then all feature vectors consistent with $\Delta(R_f)$ is in $\chi_{FSet(P, f(\vec{\alpha})=\nu, \vec{u})}(s^+)$ which is conflict with $\Delta$ is not consistent with $s^+$. By contradiction, $\Delta$ is consistent with all positive samples.

Now we prove, if a verification interface $\Delta$ is consistent with all negative samples then $\Sigma[\Delta] \implies \Phi$. As $\Delta$ is consistent with all negative $s^-$, then by Def 3.3.8:

$$\exists R_f(\vec{\alpha}, \nu) \in \Sigma,\ \exists fv \in \chi_{FSet(P, f(\vec{\alpha})=\nu, \vec{u})}(s^-), fv \in \Delta(R_f)(\vec{a}, \nu)^-$$

Then $s^- \not\models \Sigma[\Delta]$, then $\Sigma[\Delta] \implies \Phi$.

Finally we prove, if a verification interface $\Delta$ is consistent with all positive samples $s^+$ entailed by $\Gamma_F$ and $\Gamma_P$, it is consistent. If $\Delta$ is not a consistent interface, by Def 3.2.4, there exists a library function $f$ and one sample $s$ entailed by $\Gamma_F$ and $\Gamma_P$ which assigns the input and output of $f$ to $\vec{\alpha}_s$ and $\nu_s$,

$$s \not\models FSet(P, f(\vec{\alpha}_s) = \nu_s, \vec{u})[\Gamma_P]$$

As the verification queries is correct, then all samples entailed entailed by $\Gamma_F$ and $\Gamma_P$ are positive, then $s$ is the positive which is not consistent with $\Delta$. By contradiction, $\Delta$ is consistent.

### 3.3.2 Learning Maximal Verification Interfaces

While Theorem 3.3.10 identifies the conditions under which a verification interface is safe and consistent, it does not ensure that it is maximal. We frame the search for a maximal solution as a learning problem for a single function specification assuming all other specifications are fixed.

**Definition 3.3.11 (Weakest safe specification).** For a given verification query $(\Sigma, \Phi)$ and safe and consistent verification interface $\Delta$, $\phi$ is the weakest safe specification of $f$ iff

1. $\Delta[R_f \mapsto \phi]$ is safe
2. For a given bound on the number of quantified variables $k$ allowed in the specification of $f$, there is no other specification $\phi'$ with $k$ quantified variables that makes $\Delta[R_f \mapsto \phi']$ safe such that $\phi \implies \phi'$.

**Definition 3.3.12 (Sample with respect to library function).** For a verification query $(\Sigma, \Phi)$, and safe verification interface $\Delta$, a sample $s$ is positive (resp. negative) with respect to library function $f$ when $s$ is positive (resp. negative) and consistent with the specifications of all other library functions in the domain of $\Delta$:

$$s \models \Sigma[\Delta[R_f \mapsto \top]] \wedge \Phi$$
$$s \models \Sigma[\Delta[R_f \mapsto \top]] \wedge \neg\Phi \quad \text{(resp.)}$$

Ideally, the weakest safe specification of $f$ would be able to positively classify every such positive sample. Because this is not possible in general due to the intrinsic granularity of the hypothesis space, we must limit ourselves to covering some subset of this space instead. The weakening relation between classifiers can be viewed as the difference between the sets of positive and negative feature vectors induced by classifiers:

**Definition 3.3.13 (Weakening feature vector and samples).** For a verification query $(\Sigma, \Phi)$ and safe verification interface $\Delta$, a weakening feature vector $fv$ distinguishes between a weaker safe specification and $\Delta(R_f)$:

$$fv \in \Delta(R_f)^- \text{ and } \Sigma[\Delta[R_f \mapsto [\![fv]\!] \vee \Delta(R_f)]] \implies \Phi$$

A sample $s$ is a weakening sample when $s$ is positive with respect to $f$ and includes some weakening feature vector. Intuitively, such weakening samples can be used to safely generalize $f$. If we cannot find any weakening sample, then the specification must have converged to a maximal one.

**Example.** The sample

$$\{\mathtt{s}_1 \mapsto \mathtt{l}_0; \mathtt{s}_2 \mapsto \mathtt{l}_1; \nu \mapsto \mathtt{l}_2; \nu_{\textbf{top}} \mapsto \mathtt{a}; \nu_{\textbf{tail}} \mapsto \mathtt{l}_3; \nu_{\textbf{concat}} \mapsto \mathtt{l}_4; \nu_{\textbf{is_empty}} \mapsto \bot;$$
$$hd(l, u) \equiv \{(\mathtt{l}_0, \mathtt{a}); (\mathtt{l}_1, \mathtt{a}); (\mathtt{l}_2, \mathtt{a}); (\mathtt{l}_4, \mathtt{a})\}; mem(l, u) \equiv \{((\mathtt{l}_0, \mathtt{a}); (\mathtt{l}_1, \mathtt{a}); (\mathtt{l}_2, \mathtt{a}); (\mathtt{l}_4, \mathtt{a})\}\}$$

is a positive sample with respect to **push** since it makes both input stacks and the output stack contain $\mathtt{a}$ and also have the head element $\mathtt{a}$, which entails $\Phi_{\mathtt{concat}}$. It is also consistent

with all the specifications in the verification interface from Fig. 3.4 outside of the one for **push**. The feature vector $fv$ shown in Table 3.5 is extracted from this sample, and it is not included in $\Delta(R_{\mathbf{push}})^+$. Moreover, $\Sigma[\Delta[R_{\mathbf{push}} \mapsto [\![fv]\!] \vee \Delta(R_{\mathbf{push}})]] \implies \Phi$, thus $fv$ is a weakening feature vector for **push**, and the above sample is a weakening sample for **push**.

**Theorem 3.3.14.** *For a given specification configuration* $((\Sigma, \Phi), P, F, \Gamma_P, \Gamma_F)$ *and safe verification interface* $\Delta$, $\Delta(R_f)$ *is weakest safe specification of* $f$ *if and only if there are no weakening samples for* $f$.

*Proof.* We first prove if $\Delta(R_f)$ is weakest safe then there is no weakening sample. If there exists a weakening sample $s$, then $s$ includes a weakening feature vector $fv$. According to Def 3.3.13,

$$fv \in \Delta(R_f)^- \text{ and } \Sigma[\Delta[R_f \mapsto [\![fv]\!] \vee \Delta(R_f)]] \implies \Phi$$

Then $[\![fv]\!] \vee \Delta(R_f)$ is a weaker specification than $\Delta(R_f)$ and make $\Sigma[\Delta[R_f \mapsto [\![fv]\!] \vee \Delta(R_f)]] \implies \Phi$. By contradiction, there is no weakening sample.

Then we prove if there are no weakening sample for $f$, $\Delta(R_f)$ is weakest safe. Assume there is no weakening sample but exists a weaker specification $\phi_f^w \neq \Delta(R_f)$ and makes $\Sigma[\Delta[R_f \mapsto \phi_f^w]] \implies \Phi$, According to Lemma 3.3.9,

$$\bigvee_{fv \in \phi_f^{w+}} [\![fv]\!] \neq \bigvee_{fv \in \Delta(R_f)^+} [\![fv]\!]$$

$$\bigvee_{fv \in \Delta(R_f)^+} [\![fv]\!] \implies \bigvee_{fv \in \phi_f^{w+}} [\![fv]\!]$$

which means $\Delta(R_f)^+ \subset \phi_f^{w+}$, then there exists $fv$,

$$f \in \phi_f^{w+} \text{ and } fv \notin \Delta(R_f)^+ \text{ and } \{fv\} \cup \Delta(R_f)^+ \subseteq \phi_f^{w+}$$

As

$$\Sigma[\Delta[R_f \mapsto [\![fv]\!] \vee \Delta(R_f)]] \implies \Sigma[\Delta[R_f \mapsto \phi_f^w]]$$

**Algorithm 1:** Multi-Abductive Inference Algorithm.

```
Inputs  : Specification configuration ((Σ, Φ), F, P, Γ_F, Γ_P)
Output  : Maximal verification interface Δ or counterexample CEX
1  u⃗ ← ∅;
2  while true do
3      match SpecInfer((Σ, Φ), F, P, Γ_F, Γ_P, u⃗):
4          case Fail s⁻ do
5              match ExtractCex(s⁻):
6                  case CEX do return CEX ;
7                  case None do u⃗ ← u⃗ ∪ FreshVariable() ;
8          end
9          case Fail None do u⃗ ← u⃗ ∪ FreshVariable() ;
10         case Δ do
11             repeat
12                 Δ_0 ← Δ;
13                 for f ∈ F do Δ(R_f) ← Weaken((Σ, Φ), P, Δ, f, u⃗) ;
14             until Δ = Δ_0;
15             return Δ;
16         end
17 end
```

and

$$\Sigma[\Delta[R_f \mapsto \phi_f^w]] \implies \Phi$$

then

$$\Sigma[\Delta[R_f \mapsto [\![fv]\!] \vee \Delta(R_f)]] \implies \Phi$$

which means $fv$ is a weakening feature vector. By contradiction, no weaker specification exists.

### 3.4 Algorithm

Algorithm 1 presents the abductive inference procedure depicted in Fig. 3.1. The algorithm maintains a list of variables $\vec{u}$ that defines the hypothesis space it is currently exploring. The

main loop of the algorithm searches for a maximal verification interface in this hypothesis space, starting from an initially empty set of variables (line 1). We first try to use the *SpecInfer* subalgorithm (Algorithm 2) to infer a verification interface that is safe and consistent with the specification configuration. If inference fails with a sample, we search for corresponding inputs that trigger a safety violation using random testing via the *ExtractCex* subroutine (line 8). If we are unable to find such unsafe inputs, we extend $\vec{u}$ with a fresh variable (lines 7-8) and restart the loop under this enhanced hypothesis space. Otherwise, *SpecInfer* has found a safe and consistent verification interface, $\Delta$, which we then refine to a maximal solution via a weakening loop. This loop iteratively weakens the specification of each library function in $\Delta$ using the *Weaken* subalgorithm (Algorithm 3), and returns $\Delta$ once it has reached a fixed-point.

### 3.4.1 Specification Inference

Our data-driven multi-abduction specification inference algorithm (*SpecInfer*) given by Algorithm 2 assumes three key components: a property-based random sampler *Sample*, a satisfiability checker *Verify*, and a classifier learner *Learner*. *Sample* takes a first-order formula as input and attempts to find a counterexample via random sampling, returning data instances which violate this specification based on these counterexamples. *Verify* returns OK when given a valid formula, and produces an assignment of variables and member predicates (i.e. a sample) that invalidates its input formula otherwise. *Learner* takes two disjoint sets of feature vectors, $\pi$ and $\omega$, as inputs, and returns a formula that classifies elements of $\pi$ and $\omega$ as positive and negative, respectively. Elrond uses a decision tree algorithm [53] as the underlying learning framework.

In more detail, for each library function $f$, *SpecInfer* maintains a pair of positive and negative sets of feature vectors $\pi_f$ and $\omega_f$; these are initially empty (line 1). The algorithm holds the current candidate verification interface in the variable $\Delta$, which initially stores trivial specifications for each library function. *SpecInfer* implements a two-step refinement loop which first calls *Sample* to test the current candidate verification interface $\Delta$ against $\Gamma_F$ and $\Gamma_P$ in order to identify any library function behaviors that are inconsistent with their

**Algorithm 2:** Safe and Consistent Specification Inference (*SpecInfer*)

**Inputs** : Specification configuration $(\Sigma, \Phi), F, P, \Gamma_F, \Gamma_P$, variables in hypothesis space $\vec{u}$

**Output** : Consistent safe verification interface or reports failure

```
1  for f ∈ F do  π_f, ω_f ← ∅, ∅ ;
2  Δ ← {R_f ↦ ∀u⃗, ⊤};
3  while true do
4      while true do
5          match Sample(Δ(R_f), Γ_F, Γ_P):
6              case None do break;
7              case Some s⁺ do
8                  for f ∈ F do
9                      π_f ← FvecFromSample(FSet(P, f(α⃗_f) = ν_f, u⃗), s⁺)  ∪  π_f;
10                     ω_f ← ω_f \ π_f
11                 end
12                 for f ∈ F do  Δ(R_f) ← Learner(π_f, ω_f) ;
13             end
14     end
15     match Verify((Σ[Δ] ⟹ Φ)):
16         case OK do
17             match Verify(¬Σ[Δ]):
18                 case OK do  return Fail None ;
19                 case Sat _ do  return Δ;
20         end
21         case Sat s⁻ do
22             for f ∈ F do  ω'_f ← FVecFromSample(FSet(P, f(α⃗_f) = ν_f, u⃗), s⁻)  \  π_f ;
23             if ⋀_{f∈F} ω'_f = ∅ then  return Fail s⁻ ;
24             else  for f ∈ F do ω_f ← ω_f ∪ ω'_f ;
25             for f ∈ F do  Δ(R_f) ← Learner(π_f, ω_f) ;
26         end
27 end
```

current specifications (line 5). If so, *Sample* returns a positive sample $s^+$ that we then use to augment the set of positive feature vectors (line 7). We subsequently remove any feature vectors appearing in $\pi_f$ from $\omega_f$ in order to ensure the two sets are disjoint before invoking *Learner* to ascertain whether $\Delta$ is consistent with the newly observed behaviors (lines 9-11).

*SpecInfer* then queries *Verify* to determine if the current verification interface is sufficient to ensure the desired safety property (line 12). We then verify that the specifications are not

contradictory (line 14) before returning them, in order to ensure that they are not vacuously safe. Alternatively, if the verifier returns a sample $s^-$ witnessing an unsafe execution that $\Delta$ was not strong enough to disallow, we extract sets of feature vectors for each library function $f$ from $s^-$, filtering out any that also occur in $\pi_f$ (line 18) to maintain the invariant that $\pi_f$ and $\omega_f$ are disjoint. If no new negative feature vectors are found, *SpecInfer* cannot ensure that $s^-$ is spurious in the current hypothesis space, and it thus fails, returning $\mathsf{Fail}\ s^-$ to the main algorithm. Otherwise, the algorithm updates $\omega_f$ with the new negative feature vectors and strengthens $\Delta$ to rule out $s^-$ by invoking *Learner* and the refinement loop restarts (line 21).

**Theorem 3.4.1 (SpecInfer is Total and Sound).** *Algorithm 2 always halts and returns a verification interface that is safe and consistent.*

*Proof.* First we prove the decidability. Consider the size of positive feature vector set ($\pi_f$) and negative feature vector set ($\omega_f$) for each library function $f$. For the inner loop(line $4-10$), the algorithm always add new feature vector to $\pi_f$ on line 9. According to the correctness of the property based sampler, for given $s^+$ generated by *Sample* and each library function $f$, there always exists at least one feature vector extracted from $s^+$ not included by $\pi_f$. Thus the size of $\pi_f$ always increases, the total number of feature vectors of each library function is finite($2^{|\mathcal{S}|}$), the inner loop always halt. On the other hand, the algorithm always add new feature vectors to $\omega_f$(line 20) in the outer loop. If the outer loop continues, the condition on line 19 should be `false`, then there exists at least one library function whose negative feature vector set $\omega_f$ strictly increases. Again, since the total number of feature vectors of each library function is finite, we can conclude Algorithm 2 always halts.

The algorithm only terminates on line 16 which returns verification interface $\Delta$. Notice that SMT solver proves there are no negative samples which is not consistent with $\Delta$ on line 12 then according to Theorem 3.3.10, $\Sigma[\Delta] \implies \Phi$. SMT solver also proves all $\Sigma[\Delta]$ is satisfiable on line 14, then according to Definition 3.2.3, $\Delta$ is safe. On the other hand *Sampler* on line 5 shows all samples entailed by $\Gamma_F$ and $\Gamma_P$ are consistent with $\Delta$; according to Theorem 3.3.10, $\Delta$ is consistent. Thus the algorithm returns a safe and consistent verification interface.

**Algorithm 3:** Weakening (*Weaken*)

```
   Inputs   : (Σ, Φ), P, Δ, f, u⃗
   Output   : Weakest safe specification of f
 1 W_f ← ∀u⃗, ⊥;  π, ω ← ∅, ∅;
 2 while true do
 3     match Verify(Σ[Δ[R_f ↦ ∀u⃗, ⊤]] ∧ Φ  ⟹  Σ[Δ[R_f ↦ Δ(R_f) ∨ W_f ∨ ⟦ω⟧]]):
 4         case OK do
 5             if W_f = ∀u⃗, ⊥ then return Δ(R_f) else return W_f ∨ Δ(R_f);
 6         case Sat s do
 7             W_f, π, ω ← Update (s, W_f, π, ω);
 8             W_f, π, ω ← SafetyLoop (W_f, π, ω);
 9         end
10 end
11 Procedure Update(s, W_f, π, ω)
12     for fv ∈ FVecFromModel(Fset(P, f(α_f) = ν_f, u⃗),  s) do
13         match Verify((Σ[Δ[R_f ↦ W_f ∨ Δ(R_f) ∨ ⟦fv⟧]]  ⟹  Φ)):
14             case OK do  π ← π ∪ {fv} ;
15             case Sat _ do  ω ← ω ∪ {fv} ;
16         W_f ← Learner(π, ω)
17     end
18     return W_f, π, ω
19 Procedure SafetyLoop(W_f, π, ω)
20     match Verify((Σ[Δ[R_f ↦ W_f ∨ Δ(R_f)]]  ⟹  Φ)):
21         case OK do  return W_f, π, ω ;
22         case Sat s do
23             W_f, π, ω ← Update(s, W_f, π, ω);
24             return SafetyLoop(W_f, π, ω)
25         end
```

### 3.4.2 Weakening

Algorithm 3 shows the *Weaken* procedure that is used to generalize the specification of a library function $f$ from the initial solution returned by *SpecInfer*. This procedure takes a verification query, method predicate set, verification interface, library function, and a list of quantified variables as input, and infers a weakest safe specification for $f$. Notably, the arguments of *Weaken* do not include $\Gamma_P$ or $\Gamma_P$, reflecting that our approach to weakening is purely logical.

*Weaken* maintains a formula $W_f$ which is used to weaken the specification of $f$; the start of the function initializes this formula to false (line 1). Like *SpecInfer*, *Weaken* maintains two sets of feature vectors: $\pi$, which holds weakening feature vectors, and $\omega$, which holds feature vectors representing library behaviors that cause client safety violations. Both sets are initially empty (line 1). The body of *Weaken* uses a counterexample-guided refinement loop to iteratively construct $W_f$. The loop first queries the verifier to ensure that every safe execution consistent with the candidate specifications besides $\Delta(R_f)$ is either a) currently allowed by $W_f \vee \Delta(R_f)$ or b) involves a behavior of $f$ that can trigger a safety violation in other contexts (line 3). The clauses of the formula representing each of these pieces are highlighted with the corresponding color.

When this query is valid, there is no weakening sample for $f$; thus, by Theorem 3.3.14, the current specification for $f$ is the weakest safe one, and *Weaken* returns $W_f \vee \Delta(R_f)$ (or $\Delta(R_f)$ when $W_f \equiv \bot$) (line 5).

Alternatively, *Verify* will provide a sample $s$ witnessing a safe execution; this sample is passed to the *Update* subroutine to further weaken $W_f$. *Update* first uses *Verify* to identify feature vectors extracted from $s$ that trigger safety violations in other contexts. After inserting safe and unsafe feature vectors into $\pi$ and $\omega$, respectively (lines 12-13), *Update* uses *Learner* to refine $W_f$.

The weakened specification $W_f \vee \Delta(R_f)$ may violate client program safety, however. We use a learning based procedure for its intrinsic generalization ability: when we add a new feature vector to $\pi$, the learned classifier will not only classify *that* new feature vector to positive, but also potentially others. This ability results in faster convergence, since we do not need to explicitly enumerate each weakening feature vector. However, to avoid the learned classifier from being weakened to the point that it mistakenly classifies unsafe feature vectors as safe, *Weaken* then invokes *SafetyLoop* to filter any unsafe weakening (line 8). *SafetyLoop* implements a similar strategy as Algorithm 2 to progressively refine $W_f$ until it guarantees safety. After this step, the loop restarts in order to identify additional weakening possibilities.

**Theorem 3.4.2 (Weaken is Sound).** *For given* $(\Sigma, \Phi), P, \Delta, f, \vec{u}$*, if* $\Delta$ *is safe, Algorithm 3 will halt with the weakest safe specification for* $f$*.*

*Proof.* Now we first prove when algorithm halts with $W_f \vee \Delta(R_f)$ (or $\Delta(R_f)$, then this case $W_f \vee \Delta(R_f)$ is equal to $\Delta(R_f)$, we just unify all cases as $W_f \vee \Delta(R_f)$ in the proof) on line 5, then the result is always weakest safe. The Algorithm 3 terminates on line 5 which means $W_f \vee \Delta(R_f)$ is initially weakest safe or the algorithm has executed the *SafetyLoop*, then the SMT solver have proved $\Sigma[\Delta[R_f \mapsto W_f \vee \Delta(R_f)]] \implies \Phi$(line 17) is valid; as $\Delta$ is safe, notice that

$$\Sigma[\Delta] \equiv \Sigma[\Delta[R_f \mapsto \Delta(R_f)]] \implies \Sigma[\Delta[R_f \mapsto W_f \vee \Delta(R_f)]]$$

then

$$\Sigma[\Delta[R_f \mapsto \Delta(R_f)]] \not\models False$$

thus $\Delta[R_f \mapsto W_f \vee \Delta(R_f)]$ is also a safe verification interface. On the other hand, if $W_f \vee \Delta(R_f)$ is not weakest one, according to Theorem 3.3.14, there exists a weakening sample $s$. Then according to Def 3.3.13, $s$ is positive with respect to $f$ that is $s \models \Sigma[\Delta[R_f \mapsto \top]]$; $s$ contains weakening feature vectors which not included by $W_f \vee \Delta(R_f)$ and any non-weakening feature vectors in $\omega$, thus

$$s \models \Sigma[\Delta[R_f \mapsto \top]] \wedge \Phi \wedge \neg\Sigma[\Delta[R_f \mapsto [\![\omega]\!] \vee W_f \vee \Delta(R_f)]]$$

However, SMT solver have proved $s$ does not exists on line 3. By contradiction, the result $W_f \vee \Delta(R_f)$ is weakest safe.

Then we prove Algorithm 3 always halts. Consider the size of positive feature vector set($\pi$) and negative feature vector set($\omega$). $\pi$ and $\omega$ are always increased by *Update*, thus we can state $\mid \Pi \mid + \mid \Omega \mid$ *never decreases* during the iterations of the loop. Moreover, consider the sample $s$ on line 3,

$$s \models \neg\Sigma[\Delta[R_f \mapsto [\![\omega]\!] \vee W_f \vee \Delta(R_f)]]$$

then

$$\exists R_f(\vec{\alpha}, \nu) \in \Sigma,\ \exists fv \in \chi_{FSet(P, f(\vec{\alpha})=\nu, \vec{u})}(s), fv \notin (\llbracket \omega \rrbracket \vee W_f \vee \Delta(R_f))(\vec{a}, \nu)^+$$

there exists a feature vector extracted from $s$,

$$\exists fv \in \chi_{\mathcal{S}}(s), fv \notin \omega \text{ and } \llbracket fv \rrbracket \not\Longrightarrow W_f \vee \Delta(R_f)$$

As $W_f$ is the learning result from $\pi$ and $\omega$, thus according to Lemma 3.3.9,

$$\llbracket \pi \rrbracket \Longrightarrow W_f$$

thus $\llbracket fv \rrbracket \not\Longrightarrow \llbracket \pi \rrbracket$ which means $fv$ is not included in $\pi$. Then there is at least one feature vector not included in $\pi$ or $\omega$. Consider the sample $s$ on line 17,

$$s \models \Sigma[\Delta[R_f \mapsto W_f \vee \Delta(R_f)]]$$

which is a negative sample inconsistent with $\Delta[R_f \mapsto W_f \vee \Delta(R_f)]$. By Def 3.3.8 there exists a feature vector extracted from $s$ by not included in $\omega$. Moreover, if all feature vector extracted from $s$ are already included in $\pi$, thus $\mid \pi \mid + \mid \omega \mid$ always increases, we can conclude Algorithm 3 always halts.

**Lemma 3.4.3 (Locally maximality).** *For given verification query $(\Sigma, \Phi)$, a consistent verification interface $\Delta$ is maximal with respect to $\vec{u}$ iff for each library function $f$, $\Delta(f)$ is weakest safe.*

*Proof.* If $\Delta$ is maximal but for one library function $f$, there exists a specification $\phi_f$ which is weaker than $\Delta(R_f)$ and $\Sigma[\Delta[R_f \mapsto \phi_f]]$ is still safe. By Def 3.2.5, $\Delta[R_f \mapsto \phi_f] \succ \Delta$ which is conflict with $\Delta$ is maximal. By contradiction, each specification in $\Delta$ is weakest safe. On the other hand, if each specification in $\Delta$ is weakest safe but $\Delta$ is not maximal, then there exists another verification interface $\Delta' \succ \Delta$. By Def 3.2.5, all specification in $\Delta'$ is weaker than corresponding specification in $\Delta$, and exists a library function $f$, $\Delta'(R_f) \neq \Delta(R_f)$.

Then when we fixed the specification of functions besides $f$ to specification in $\Delta$, $\Delta'(R_f)$ is weaker than $\Delta(R_f)$ but still make the verification queries valid, which is conflict with $\Delta(R_f)$ is weakest safe. By contradiction, $\Delta$ is maximal verification interface.

**Theorem 3.4.4 (Relative Completeness).** *Algorithm 1 either returns either a maximal verification interface or a concrete counterexample.*

*Proof.* The algorithm terminates on line 6 and 14, returns a concrete valued counter-example; or a verification interface $\Delta$ literately weakened by *Weaken* from the result from *SpecInfer*, called $\Delta_{SpecInfer}$. According to Theorem 3.4.1, $\Delta_{SpecInfer}$ is safe and consistent. Notice that $\Delta \succ \Delta_{SpecInfer}$, according to Lemma 3.3.9, all positive samples consistent with $\Delta_{SpecInfer}$ are also consistent with $\Delta$. As $\Delta_{SpecInfer}$ is consistent, by Theorem 3.3.10, all positive samples entailed by $\Gamma_F$ and $\Gamma_P$ are consistent with $\Delta_{SpecInfer}$, thus also consistent with $\Delta$. According to Lemma 3.4.3 and Theorem 3.4.2, each $\Delta(R_f)$ is weakest safe, then $\Delta$ is maximal verification interface.

### 3.4.3 Multi-Query Algorithm

The multi-abductive inference algorithm for multiple verification queries are shown in Algorithm 4, which is the same as Algorithm 1 but takes a set of verification queries $\{(\Sigma, \Phi)\}$ as input.

The *SpecInfer* for set of verification queries are shown in Algorithm 5 which has loops on lines $14 - 20$ which verifies each verification queries and gather negative feature vectors. When algorithm cannot gather new feature vectors in any verification queries on line 19, then returns the negative sample. If all verification queries is valid and all $\Sigma_i[\Delta]$ is satisfiable, then returns the verification interface.

The weakening algorithm for set of verification queries are shown in Algorithm 6 where all SMT queries on line 3, 12 and 17 are conjunctions of corresponding queries in Algorithm 3 for each verification query.

**Algorithm 4:** Multi-Abductive Inference Algorithm.

```
Inputs  : Specification configuration ({(Σ, Φ)}, F, P, Γ_F, Γ_P)
Output  : Maximal verification interface Δ or counterexample CEX
1  u⃗ ← ∅;
2  while true do
3      match SpecInfer({(Σ, Φ)}, F, P, Γ_F, Γ_P, u⃗):
4          case Fail s⁻ do
5              match ExtractCex(s⁻):
6                  case CEX do return CEX ;
7                  case None do u⃗ ← u⃗ ∪ FreshVariable() ;
8          end
9          case Fail None do u⃗ ← u⃗ ∪ FreshVariable() ;
10         case Δ do
11             repeat
12                 Δ_0 ← Δ;
13                 for f ∈ F do Δ(R_f) ← Weaken({(Σ, Φ)}, P, Δ, f, u⃗) ;
14             until Δ = Δ_0;
15             return Δ;
16         end
17 end
```

**Algorithm 5:** Safe and Consistent Specification Inference(*SpecInfer*)

```
Inputs   : Specification configuration {(Σ, Φ)}, F, P, Γ_F, Γ_P, variables in
           hypothesis space u⃗
Output   : Consistent safe verification interface or reports failure
for f ∈ F do  π_f, ω_f ← ∅, ∅ ;
Δ ← {R_f ↦ ∀u⃗, ⊤};
while true do
    while true do
        match Sample(Δ(R_f), Γ_F, Γ_P):
            case None do break;
            case Some s⁺ do
                for f ∈ F do
                    π_f ← FvecFromSample(FSet(P, f(α⃗_f) = ν_f, u⃗), s⁺)  ∪  π_f;
                    ω_f ← ω_f \ π_f
                end
                for f ∈ F do  Δ(R_f) ← Learner(π_f, ω_f) ;
            end
    end
    if forall (Σ_i, Φ_i), Verify((Σ_i[Δ] ⟹ Φ_i)) = OK then
        if forall (Σ_i, Φ_i), Verify((Σ_i[Δ] ⟹ Φ_i)) = Sat _ then  return Δ  else  return
          Fail None ;
    end
    for (Σ_i, Φ_i) do
        match Verify((Σ_i[Δ] ⟹ Φ_i)):
            case OK do  continue;
            case Sat s⁻ do
                for f ∈ F do  ω'_f ← FVecFromSample(FSet(P, f(α⃗_f) = ν_f, u⃗), s⁻)  \  π_f ;
                if ⋀_{f∈F} ω'_f = ∅ then  return Fail s⁻ ;
                else  for f ∈ F do ω_f ← ω_f ∪ ω'_f ;
            end
    end
    for f ∈ F do  Δ(R_f) ← Learner(π_f, ω_f) ;
end
```

**Algorithm 6:** Weakening Algorithm

```
Inputs   : {(Σ, Φ)}, P, Δ, f, u⃗
Output   : Weakest safe specification of f
W_f ← ∀u⃗, ⊥;  π, ω ← emptyset, ∅;
while true do
    match Verify(⋀(Σ_i[Δ[R_f ↦ ∀u⃗, ⊤]] ∧ Φ_i  ⟹  Σ_i[Δ[R_f ↦ ⟦ω⟧ ∨ W_f ∨ Δ(R_f)]])):
        case OK do
            if W_f = ∀u⃗, ⊥ then return Δ(R_f) else return W_f ∨ Δ(R_f);
        case Sat s do
            W_f, π, ω ← Update (s, W_f, π, ω);
            W_f, π, ω ← SafetyLoop (W_f, π, ω);
        end
end
Procedure Update(s, W_f, π, ω)
    for fv ∈ FVecFromModel(Fset(P, f(α_f) = ν_f, u⃗),  s) do
        W_f ← Learner(π, ω);
        match Verify(⋀(Σ_i[Δ[R_f ↦ W_f ∨ Δ(R_f) ∨ ⟦fv⟧]]  ⟹  Φ_i)):
            case OK do  π ← π ∪ {fv} ;
            case Sat _ do  ω ← ω ∪ {fv} ;
    end
    return W_f, π, ω
Procedure SafetyLoop(W_f, π, ω)
    match Verify(⋀(Σ_i[Δ[R_f ↦ W_f ∨ Δ(R_f)]]  ⟹  Φ_i)):
        case OK do  return W_f, π, ω ;
        case Sat s do
            W_f, π, ω ← Update(s, W_f, π, ω);
            return SafetyLoop(W_f, π, ω)
        end
```

### 3.5 Implementation and Evaluation

We have implemented a verification pipeline based on the above approach, called Elrond[51], that targets OCaml programs which rely on libraries to manipulate algebraic data types. Elrond consists of 7267 lines of OCaml and uses Z3 [8] as its backend solver. Elrond's frontend generates verification queries from client programs via weakest liberal precondition predicate transformers. Elrond does not automatically infer inductive invariants for recursive client functions, and it expects client programs to provide such invariants, in addition to any pre- and post-conditions.

**Table 3.6.** Experimental results.

| | $(\mid F \mid, \mid R \mid)$ | $\mid P \mid$ | $\mid \vec{u} \mid$ | $\mid cex \mid$ | $\text{time}_c$(s) | #Gather / $\mid \phi^+ \mid$ | $\text{time}_w$(s) | $\text{time}_d$(ms) |
|---|---|---|---|---|---|---|---|---|
| queue | $(6, 10)$ | 2 | 2 | 26 | 1.85 | 366 / 1828 | 47.9 | 123.5 |
| | $(4, 5)$ | 2 | 1 | 10 | 0.28 | 235 / 1536 | 9.2 | 18.2 |
| | $(4, 5)$ | 3 | 2 | 23 | 0.87 | 2076 / 30054 | 318.9 | 42.7 |
| | $(6, 10)$ | 1 | 1 | 8 | 0.22 | 53 / 114 | 1.9 | Max |
| stack | $(4, 5)$ | 2 | 1 | 11 | 0.61 | 29 / 13 | 0.5 | 18.5 |
| | $(4, 5)$ | 3 | 2 | 39 | 2.94 | 3220 / 22716 | Limit | 169.3 |
| | $(4, 5)$ | 2 | 1 | 4 | 0.08 | | | |
| heap | $(3, 8)$ | 2 | 2 | 71 | 11.21 | 1790 / 26588 | Limit | 895.8 |
| | $(2, 21)$ | 1 | 1 | 10 | 0.37 | 172 / 432 | 21.5 | 21.3 |
| | $(2, 21)$ | 3 | 2 | 141 | 107.18 | 702 / 177388 | Limit | 328.3 |
| | $(2, 21)$ | 3 | 2 | 192 | 152.99 | 512 / 190215 | Limit | 149.7 |
| | $(3, 8)$ | 1 | 1 | 10 | 0.37 | 464 / 3308 | 46.0 | Max |
| stream | $(4, 10)$ | 1 | 1 | 4 | 0.15 | 50 / 110 | 1.3 | 21.1 |
| | $(4, 10)$ | 2 | 2 | 23 | 1.57 | 640 / 1376 | 1038.0 | 70.6 |
| | $(4, 10)$ | 3 | 2 | 18 | 1.69 | 1755 / 31276 | 741.8 | 24.3 |
| | $(4, 10)$ | 3 | 2 | 12 | 0.37 | | | |
| set | $(1, 21)$ | 4 | 2 | 91 | 77.75 | 1369 / 59797035 | Limit | 1268.7 |
| | $(2, 8)$ | 1 | 1 | 11 | 0.30 | 181 / 448 | 11.2 | 21.1 |
| | $(2, 8)$ | 2 | 1 | 21 | 0.81 | 3327 / 27609 | 1741.9 | 23.2 |
| | $(2, 8)$ | 2 | 2 | 63 | 8.28 | 2496 / 28495 | Limit | 314.1 |
| | $(2, 6)$ | 3 | 1 | 15 | 0.39 | 3644 / 22526 | 1203.3 | 19.5 |
| | $(2, 6)$ | 3 | 1 | 25 | 0.69 | 3091 / 20090 | 968.7 | 19.2 |
| | $(1, 21)$ | 1 | 1 | 10 | 0.27 | 228 / 772 | 34.0 | Max |
| | $(2, 8)$ | 2 | 1 | 8 | 0.14 | | | |
| | $(2, 6)$ | 3 | 3 | 29 | 4.69 | | | |
| trie | $(4, 18)$ | 2 | 1 | 12 | 0.47 | 167 / 404 | 13.6 | 28.0 |
| | $(4, 18)$ | 2 | 1 | 15 | 0.31 | | | |

Our experimental evaluation of Elrond addresses five key questions:

**Q1**: Is Elrond able to find specifications sufficient to verify a range of properties and client programs in a reasonable amount of time?

**Q2**: Can Elrond identify unsafe client programs?

**Q3**: Can Elrond efficiently find maximal solutions?

**Q4**: Is Elrond able to find useful intermediate generalizations of initial specifications?

**Q5**: Does weakening improve the quality of the inferred specifications?

All reported data was collected on a Linux server with an Intel(R) Core(TM) i7-8700 CPU @ 3.20GHz and 64GB of RAM.

To answer these questions, we have evaluated Elrond on a corpus[6] of client programs drawn from [50], the OCaml standard library [54], Verified Functional Algorithms [55] and Software Foundations [56]. Our benchmarks cover a range of abstract data types manipulating a diverse set of algebraic data types, including queues (bankers queues and batched queues), list-based stacks, heaps (leftist heaps and splay heaps), streams, sets (backed by trees, red black trees, and lists), and tries. The underlying representation of each algebraic data type provides a set of method predicates related to ordering, membership, and uniqueness (no duplicate elements) that can be queried as part of a test. These sorts of shape properties are relatively under-constrained and are thus amenable to property-based random sampling. We used Elrond to verify several different properties for each data type, including membership, ordering, distinct elements, and sorting.

**Example 3.5.1.** One of our benchmarks uses the following specification for an **insert** program that inserts an element `x` into an unbalanced set `s` using a binary tree for its underlying representation:

$$(\forall u, (root(\mathtt{s}, u) \implies (u < \mathtt{x} \implies root(\nu_{\mathbf{insert}}, u)) \wedge (u \geq \mathtt{x} \implies root(\nu_{\mathbf{insert}}, \mathtt{x})))$$
$$\wedge \ (mem(\nu_{\mathbf{insert}}, u) \iff mem(\mathtt{s}, u) \vee \mathtt{x} = u)$$

[6]↑An artifact containing this tool, our benchmark suite, results and corresponding Coq proofs is publicly available on Zenodo:10.5281/zenodo.5130646 [51].

Given this specification, Elrond infers the following specification for the $\mathbf{maket}(\mathbf{x}, \mathbf{l}, \mathbf{r}) = \nu$ function used by **add** that constructs a new tree from `x` and the left and right subtrees `l` and `r`:

$$\forall u, (mem(\nu, u) \iff mem(\mathtt{l}, u) \vee mem(\mathtt{r}, u) \vee (\mathtt{x} = u)) \wedge (root(\nu, u) \iff \mathtt{x} = u)$$
$$\wedge (root(\mathtt{l}, u) \implies mem(\mathtt{l}, u)) \wedge (root(\mathtt{r}, u) \implies mem(\mathtt{r}, u))$$

The first line of this specification captures the key semantic properties of the tree, while the second line encodes a key relationship between the *mem* and *hd* method predicates needed by the solver to verify the specification.

The detailed results of our experiments are shown in Table 3.6. The columns in the table can be divided into three groups. The first group presents the number of distinct library functions ($\mid F \mid$), the number of library function applications ($\mid R \mid$), and the size of the method predicate set ($\mid P \mid$) for each benchmark. The second column describes the number of quantified variables ($\mid \vec{u} \mid$), the number of counter-examples generated ($\mid cex \mid$), and the time in seconds ($\text{time}_c$) needed for Elrond to find a consistent and safe verification interface. Times indicate how long it took for counter-example generation, sampling, feature vector extraction, and labelling and DT learning. They do not include the time taken to generate the initial verification query. Red entries in the time column indicate how long it took to identify safety violations in the five unsafe benchmarks. The last group lists the number of gathered (#Gather) and total positive ($\mid \phi^+ \mid$) feature vectors in the space of weakenings, the time needed by the weakening phase ($\text{time}_w$), and the time needed for the SMT solver to find a sample allowed by a weakened solution but not the initial one ($\text{time}_d$). Blue entries in the $\mid \phi^+ \mid$ column indicate a lower bound. "Limit" entries in the $\text{time}_w$ column indicate that the one hour time bound was reached. "Max" entries in the $\text{time}_d$ column indicate that the initial solution was already maximally weak.

The first group of columns in Table 3.6 describes the salient features of our benchmarks. Each client specification uses between 1 and 4 member predicates, and the client programs make between 5 and 21 calls to library functions. In order to evaluate Elrond's ability to

identify faulty specifications, our experiments also included five unsafe client programs. Elrond can infer specifications for all the benchmarks with valid assertions, and returns concrete counter-examples for clients with unsafe post-conditions.

The second group of columns in Table 3.6 presents our evaluation of Elrond's ability to discover an initial safe and consistent verification interface (**Q1** and **Q2**). These columns show that Elrond is relatively efficient at finding safe specifications, with none of the benchmarks taking longer than three minutes to learn an initial solution (**Q1**). As expected, more complicated benchmarks (i.e. those with more function calls or member predicates) required more SMT-provided counterexamples and took longer to complete, as did benchmarks requiring a larger hypothesis space (i.e. specifications with more quantified variables). Notably, Elrond was able to quickly generate concrete counter-examples witnessing safety violations (**Q2**) in the five benchmarks with invalid postconditions (these benchmarks have red entries in the $\text{time}_c$ column).

The final group of columns in Table 3.6 addresses the questions dealing with Elrond's weakening phase (**Q3 - Q5**). Unsurprisingly, weakening requires more time than the initial inference phase; while the latter needs to identify a *single* solution, the former needs to account for *all* possible solutions in order to select the best one. When evaluating this phase, we choose to use a one hour time bound for each experiment. If this time bound was reached, we had Elrond return the current weakened solution. Three-fourths (16/22) of our safe benchmarks were able to find maximal solutions from the initial solution within this limit (**Q3**). In general, the time taken to weaken a solution was correlated with the complexity of the benchmark.

To further investigate how effective Elrond was at exploring the space of candidate weakenings (**Q4**), we calculated the ratio of the total number of feature vectors gathered during weakening (#Gather) against the total number of positive feature vectors admitted by the final maximal specification ($\mid \phi^+ \mid$). The latter number represents the set of vectors that a naïve exhaustive enumeration would need to consider in order to find our solution. In our experiments, Elrond only needed to consider at most 40% of the full search space.[7] To

[7]↑In the first stack benchmark, Elrond gathers more feature vectors than the total number because the algorithm may generate false feature vectors, as discussed in Sec. 3.4.

get $\mid \phi^+ \mid$ for the six benchmarks on which Elrond returned a partially weakened solution, we increased the time bound to 24 hours and ran Elrond until it converged on the maximal solution. Elrond was unable to converge under this longer bound for three of these benchmarks. For these three, we report the total number of feature vectors in the partial solution, which is indicated using a blue entry in the $\mid \phi^+ \mid$ column.

In an attempt to quantify how well Elrond was able to generalize an initial solution, we asked the SMT solver to identify samples permitted under the weakened solution but not the original (**Q5**). The more general a weakened solution, the larger this space should be, so the solver should be able to more readily identify one of its elements. The results of this experiment are presented under the $\text{time}_d$ column. The initial solution was already maximal for three of our benchmarks, which is indicated via a "Max" entry in the $\text{time}_d$ column. In general, the solver was able to quickly find such samples for the remaining nineteen benchmarks. This search took longer for the benchmarks that hit the time bound, suggesting those solutions are either closer to the initial specifications or are otherwise more complicated for the SMT solver to handle.

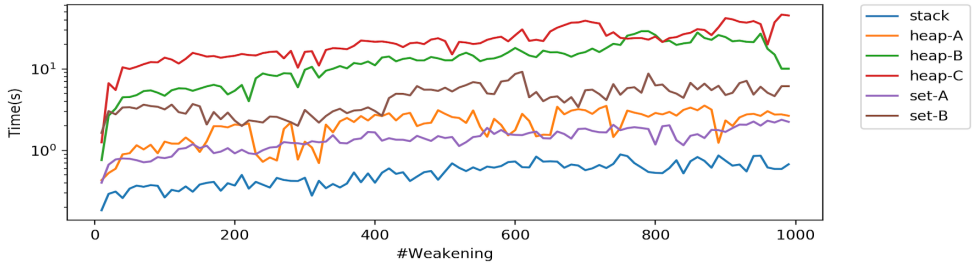


**Figure 3.7.** Run times of the first 1000 weakening iterations in benchmarks.

When running our experiments without a time bound, we observed that individual iterations of the weakening loop took longer to complete over time. As an example, in the first hour of the second stack benchmark, Elrond performed 46 weakening steps per minute on average, but only averaged 33 after that. Based on this observation, we conjectured that it became harder for Elrond to find further generalizations as the weakening phase progresses. To test this hypothesis, we measured the time required by each of the first 1000 iterations of the weakening loop for the six benchmarks that hit the time bound. The results of this

experiment are presented in Fig. 3.7; the y-axis uses a logarithmic scale in order to account for times that range from 0.1s to 50s. The trajectory agrees with our hypothesis, and suggests diminishing returns for running the weakening procedure for long periods of time.

Finally, in order to show that Elrond produces useful specifications (**Q1** and **Q5**), we used the Coq proof assistant to verify that implementations of the library functions from our benchmarks satisfied their inferred specifications. In total, Elrond inferred 68 specifications across all our experiments; we were able to verify 64 of these in Coq. Of these, three had initial specifications that were too strong; it was only after the weakening phase that the specifications could be used for verification. The four specifications that we were not able to verify came from the benchmarks whose weakening phase timed out, i.e., these specifications were not maximal. Taken together, these points suggest that the weakening phase results in more general (and thus more useful) specifications, and that Elrond is highly effective in finding meaningful specifications.

### 3.6 Discussion

As established in the introduction and background, refinement types serve as a potent specification language. The specifications inferred by Elrond can be directly encoded as refinement types. For instance, the maximal specification presented in Fig. 3.6 can be encoded as the refinement types illustrated in Fig. 3.8. The resulting refinement types are also maximal; they are sufficient to verify the client program, consistent with all observed behaviors from testing, and represent the weakest such specifications, thereby avoiding overfitting.

A naive transformation that simply qualifies the return type of each function with the inferred specification can produce undesirable results. Consider the following alternative type for the `top` function:

$$\tau'_{\texttt{top}} \doteq \texttt{s}{:}\{\nu{:}\texttt{int list} \mid \top\} \rightarrow \{\nu{:}\texttt{int} \mid \forall u, \nu = u \iff (hd(\texttt{s}, u) \wedge mem(\texttt{s}, u))\}$$

Unlike the type $\tau_{\texttt{top}}$ in Fig. 3.8, $\tau'_{\texttt{top}}$ permits any integer list as an argument, including the empty list, due to its $\top$ qualifier. This more permissive specification leads to an *unrealizable*

$$
\begin{aligned}
&\tau_{\texttt{is_empty}} \doteq \texttt{s}{:}\{\nu{:}\texttt{int list} \mid \top\} \to \{\nu{:}\texttt{bool} \mid \forall u, (\nu \implies \neg mem(\texttt{s}, u))\} \\
&\tau_{\texttt{top}} \doteq \texttt{s}{:}\{\nu{:}\texttt{int list} \mid \neg emp(\nu)\} \to \{\nu{:}\texttt{int} \mid \forall u, \nu = u \iff (hd(\texttt{s}, u) \wedge mem(\texttt{s}, u))\} \\
&\tau_{\texttt{tail}} \doteq \texttt{s}{:}\{\nu{:}\texttt{int list} \mid \neg emp(\nu)\} \to \\
&\qquad \{\nu{:}\texttt{int list} \mid \forall u, (mem(\texttt{s}, u) \implies (mem(\nu, u) \vee hd(\texttt{s}, u))) \wedge \\
&\qquad ((mem(\nu, u) \vee (hd(\nu, u) \wedge hd(\texttt{s}, u))) \implies mem(\texttt{s}, u))\} \\
&\tau_{\texttt{push}} \doteq \texttt{x}{:}\{\nu{:}\texttt{int} \mid \top\} \to \texttt{s}{:}\{\nu{:}\texttt{int list} \mid \top\} \to \\
&\qquad \{\nu{:}\texttt{int list} \mid \forall u, (mem(\nu, u) \wedge \neg \texttt{x} = u \implies mem(\texttt{s}, u) \wedge \neg hd(\nu, u)) \wedge \\
&\qquad (\texttt{x} = u \vee mem(\texttt{s}, u) \implies mem(\nu, u)) \wedge (\neg mem(\nu, u) \wedge hd(\nu, u) \implies hd(\texttt{s}, u))\}
\end{aligned}
$$

**Figure 3.8.** The refinement types that encoding maximal specifications shown in Fig. 3.6.

return type (i.e. a type that has no inhabitant), as demonstrated by specializing the return type for an empty list argument:

$$
\begin{aligned}
\tau'_{\texttt{return}} &\doteq \{\nu{:}\texttt{int} \mid \forall u, \nu = u \iff (hd([], u) \wedge mem([], u))\} \\
&\quad \text{when assign } u \text{ to } \nu \ \{\nu{:}\texttt{int} \mid hd([], \nu) \wedge mem([], \nu)\} \\
&= \{\nu{:}\texttt{int} \mid \bot\} \quad \text{by the definition of } hd \text{ and } mem
\end{aligned}
$$

The resulting type, $\tau'_{\texttt{return}}$, is uninhabited because its qualifier is logically equivalent to $\bot$. Ideally, the parameter type for the `top` function should be $\{\nu{:}\texttt{int list} \mid \neg emp(\nu)\}$, which rejects unsafe arguments like the empty list, as shown in type $\tau_{\texttt{top}}$ of Fig. 3.8.

To avoid unrealizable return types, we introduce a transformation procedure that also refines function parameter types, shown in Algorithm 7. The *Transform* algorithm takes as input the basic type $t$, parameter names $\vec{x}$, specification $\phi$, predicate set $P$, and a type context $\Gamma$ (initially empty) that caches previously inferred parameter types. The algorithm proceeds by structural induction on the parameter types (line 2): if $t$ is a base type (i.e., no more parameters remain), it places the specification $\phi$ directly in the return type (line 3). Otherwise, it selects the first parameter $x_{\texttt{param}}$ (line 4) and refines its qualifier. Following a learning-based approach similar to Algorithm 5 and Algorithm 6, it initializes a set of feature vectors using *FVecFromBasicType* (line 5), classifies each feature vector as positive or

**Algorithm 7:** Transforming the specification into refinement type.

**Inputs** : Type context $\Gamma$, basic type $t$, parameter names $\vec{x}$, specification $\phi$, and predicate set $P$.

**Output** : Refinement type $\tau'$ that is has the same meaning as $\phi$.

```
1  Procedure Transform(Γ, t, x⃗)
2      match t, x⃗:
3          case t is a base type, [] do  return {ν:t | φ} ;
4          case t_param → t_return, x_param :: x⃗' do
5              F ← FVecFromBasicType(Γ, x_param, t_param, P);
6              π, ω ← {}, {};
7              for fv ∈ F do
8                  match Satisfy(Γ ⊨ fv ∧ φ):
9                      case Sat s do  π ← π ∪ {fv} ;
10                     case Unsat do  ω ← ω ∪ {fv} ;
11             end
12             τ_param ← {ν:t_param | Learner(π, ω)};
13             Γ' ← Γ, x_param:τ_param;
14             τ_return ← Transform(Γ', t_return, x⃗');
15             return x_param:τ_param → τ_return
16         end
```

negative via SMT queries (lines 6–11), and learns the qualifier for the parameter type $\tau_{\texttt{param}}$ as a classifier of postitive and negative feature vectors (line 12). Crucially, a positive feature vector here means that it makes the specification satisfiable under the current type context (line 8). The algorithm then recursively refines the return type $\tau_{\texttt{return}}$ with the updated context (line 13), and finally constructs the full refinement type by combining the refined parameter and return types (line 15).

**Example 3.6.1.** Consider transforming the specification $\phi \doteq \forall u, \nu = u \iff (hd(\texttt{s}, u) \wedge mem(\texttt{s}, u))$ of the function `top` with the basic type `int list` $\rightarrow$ `int`. Let the predicate set be $P = \{emp, hd, mem\}$. When invoking *Transform* with $\Gamma = \emptyset$, $t =$ `int list` $\rightarrow$ `int`, and $\vec{x} = [\texttt{s}]$, the algorithm proceeds as follows. At line 4, the parameter $s$ is selected, and the feature vector set $F$ is initialized to $\{emp(s), \neg emp(s)\}$. For the feature vector $emp(s)$, the SMT query

$$\models emp(s) \wedge \forall u, \nu = u \iff (hd(\texttt{s}, u) \wedge mem(\texttt{s}, u))$$

is unsatisfiable, since an empty list ($emp(s)$) cannot have a head element. Thus, $emp(s)$ is classified as negative and added to $\omega$. Conversely, $\neg emp(s)$ is classified as positive and added to $\pi$, as non-empty lists can satisfy the specification. The learned parameter type is $\{\nu{:}\texttt{int list} \mid \neg emp(\nu)\}$, where the qualifier separates the positive and negative feature vectors. The return type is then directly assigned as

$$\{\nu{:}\texttt{int} \mid \forall u, \nu = u \iff (hd(\texttt{s}, u) \wedge mem(\texttt{s}, u))\}$$

following line 3 of Algorithm 7. Therefore, the final transformed refinement type is

$$\texttt{s}{:}\{\nu{:}\texttt{int list} \mid \neg emp(\nu)\} \rightarrow \{\nu{:}\texttt{int} \mid \forall u, \nu = u \iff (hd(\texttt{s}, u) \wedge mem(\texttt{s}, u))\}$$

which matches the type shown in Fig. 3.8.

**Theorem 3.6.2 (Correctness of Transformation).** *For given function with parameters and return variable $\vec{x}$, basic type $t$ and specification $\phi$, the transformed refinement type $\tau$ from $Transform(\top, t, \vec{x}, \phi)$ satisfies the following properties:*

1. ***Soundness**: a program assigned with refinement type $\tau$ also satisfies $\phi$.*
2. ***Realizablility**: for all possible argument assignments, the return type of the program is realizable.*

*Proof.* We first prove the soundness of the transformation. The last return type of the output refinement type is always has the same qualifier as the specification $\phi$ (line 3 in Algorithm 7). Thus a program assigned with refinement type $\tau$ also satisfies $\phi$. We prove the realizability of the transformation by induction on the structure of the function type.

- If the function type is a base type, there is no arguments thus the return type is realizable.
- If the function type is a function type, then the transformation refines the parameter type $\tau_{\texttt{param}}$ and the return type $\tau_{\texttt{return}}$. By the induction hypothesis, the return type

$\tau_{\texttt{return}}$ is realizable. Moreover, the parameter type $\tau_{\texttt{param}}$ is realizable is guaranteed by SMT queries (line 8 in Algorithm 7).

Therefore, the transformed refinement type is sound and realizable.

# 4. VERIFICATION OF REPRESENTATION INVARIANTS OF ABSTRACT DATATYPE LIBRARIES USING SYMBOLIC FINITE AUTOMATA

Functional programs often interact with stateful libraries that hide internal state behind their APIs. Such libraries are a vital component of a programming language's ecosystem, providing well-tested implementations of databases, key-value stores, logging and system services, and other stateful abstractions. A common use case for how these libraries are used is for programmers to build higher-level data structures on top of the abstractions they provide [57]. A set abstract data type (ADT) equipped with intersection and cardinality operations, for example, might use the aforementioned key-value library to maintain a persistent collection of its elements by associating those elements (represented as values) with unique ids (their associated keys). Since the internal state of the underlying stateful library is opaque to its clients, the implementations of these higher-level modules will necessarily be expressed in terms of the operations provided by the library: the methods of the set ADT might use methods like **put** and **get** to interact with the backing key-value store, for example. To maximize reusability, these low-level libraries are typically quite permissive in how client may use them, resulting in weak specifications that only describe simple algebraic or equational constraints. For example, the specification of the key-value store might state that two **put** operations over distinct keys commute or that a **get** operation on a key always returns the last value **put** in that key.

In contrast, the higher-level abstractions provided by clients built on top of these libraries are equipped with stronger guarantees: for example, the operations of our set ADT need to respect the semantics of mathematical sets, for example, so that $\forall A\ B : \texttt{Set.t}.\ \mid A \cup B \mid + \mid A \cap B \mid = \mid A \mid + \mid B \mid$. Verifying such semantically rich properties typically requires a strong guarantee that the internal state of the set library is consistent with the abstract states of the datatype, i.e., the ADT implementation should be equipped with a *representation invariant* [58, 59]. Ensuring that distinct keys are always associated with distinct values allows us to prove the above relationship between the intersection and cardinality operations of the set ADT, for example. Establishing such an invariant is complicated by the fact that

our set ADT is built on top of another library with an opaque internal state, and whose weak specifications do not directly address the notion of uniqueness central to the set ADT. This chapter presents an automated type-based verification procedure that addresses these complications.

Our proposed solution augments a refinement type system for a call-by-value functional core language that supports calls to stateful libraries, with *Hoare Automata Types* (HATs). HATs are a new type abstraction that qualifies basic types with pre- and post-conditions expressed as *symbolic finite-state automata* [60, 61], a generalization of finite-state automata equipped with an unbounded alphabet, and whose transitions are augmented with predicates over the elements of this alphabet. HATs serve as a compact representation of the trace of stateful API calls and returns made prior to a function invocation and those performed during its execution.

As a result, HATs serve as an expressive basis for a fine-grained effect system within a general refinement type framework, enabling compositional reasoning about invocations of stateful APIs in a form amenable to automated verification of a client module's representation invariant. For example, he representation invariant from our set ADT is captured by the following symbolic automaton, written using a variant of linear temporal logic on finite sequences [41]. The base predicates of this formula describe *events*, or invocations of the methods of a key-value store whose values are drawn from a `setElem` type:

$$I_{\texttt{Set}}(\texttt{el}) \doteq \Box(\langle \textbf{put}\, key\, val = \nu \mid val = \texttt{el}\rangle \Longrightarrow \bigcirc\neg\Diamond\langle \textbf{put}\, key\, val = \nu \mid val = \texttt{el}\rangle)$$

This representation invariant ($I_{\texttt{Set}}$) is expressed using standard temporal logic modalities (e.g., $\Box$ (always), $\bigcirc$ (next), and $\Diamond$ (eventually)) to express the history of valid interactions between the set implementation and the underlying key-value store that ensure the expected element uniqueness property holds. Informally, the invariant establishes that, at every step of the execution ($\Box(...)$), once a value `el` has been inserted into the set ($\langle \textbf{put}\, key\, val = \nu \mid val = \texttt{el}\rangle$), it will never be reinserted ($\bigcirc\neg\Diamond\langle \textbf{put}\, key\, val = \nu \mid val = \texttt{el}\rangle$). An implementation that preserves this invariant guarantees that every value is always associated with at most one key.

To verify it, we can check the `insert` method of the set ADT against the following type

$$\tau_{\texttt{insert}} \doteq \texttt{el}{:}\texttt{setElem} \dashrightarrow \texttt{x}{:}\{\nu{:}\texttt{setElem} \mid \top\} \rightarrow [I_{\texttt{Set}}(\texttt{el})]\ \{\nu{:}\texttt{unit} \mid \top\}\ [I_{\texttt{Set}}(\texttt{el})]$$

The output type of $\tau_{\texttt{insert}}$ ($[I_{\texttt{Set}}(\texttt{el})]$ $\{\nu{:}\texttt{unit} \mid \top\}$ $[I_{\texttt{Set}}(\texttt{el})]$) is a Hoare Automata Type (HAT) over the representation invariant $I_{\texttt{Set}}$. This type captures the behavior required of any correct implementation of `insert`: assuming the invariant holds for any value `el` (encoded here as a ghost variable scoped using $\dashrightarrow$) prior to the insertion of a new element `x`, it must also hold after the insertion as well. The precondition of the HAT captures the set of interactions that lead to a valid underlying representation state (namely, one in which every value in the store is unique), and the postcondition of the HAT specifies that this condition remains valid even after executing `insert`.

Our use of HATs differs in important ways from other type-based verification approaches for functional programs that allow computations to perform fine-grained effects, such as F* [62] or Ynot [63]. These systems use the power of rich dependent types to specify fine-grained changes to the heap induced by an stateful operation. The Dijkstra monad [64, 65] used by F*, for example, extends ideas first proposed by Nanevski *et. al* [66] in their development of Hoare Type Theory to embed a weakest precondition predicate transformer semantics into a type system that can be used to generate verification conditions consistent with a provided post-condition on the program heap. In contrast, our setting assumes that such operations are executed by a library with a *hidden* state, requiring the client's representation invariant to be expressed solely in terms of observations on calls (and returns) to (and from) the methods of the underlying library.

Our approach also bears some similarity to recent (refinement) type and effect systems for verifying temporal properties of event traces *produced* during a computation [67, 68]. While the idea of using symbolic traces of previous observations as a handle to reason about the hidden state of a library is similar to these other efforts, incorporating return values of operations as part of such traces is a significant point of difference. This capability, critical for the validation of useful representation invariants, allows the precondition encoded in a HAT to describe a fine-grained *context* of events in which a program can run, expressed in terms of both library method arguments *and* return values. These important differences with prior work position HATs as a novel middle-ground in the design space between explicit state-based verification techniques and trace-based effect systems. Our formulation also leads to a number of significant technical differences with other refinement type systems [22] and

their associated typing algorithms, since we must now contend with type-level reasoning over symbolic automata qualifiers.

In summary, this chapter makes the following contributions:

1. We formalize a new trace-based specification framework for expressing representation invariants of functional clients of stateful libraries that manage hidden state.

2. We show how this framework can be embedded within a compositional and fine-grained effect-tracking refinement type system, by encoding specifications as symbolic automata and instantiated as HATs in the type system.

3. We develop a bidirectional type-checking algorithm that translates the declarative type system into efficient subtype inclusion checks amenable to SMT-based automated verification.

4. Using a tool (Marple) that implements these ideas, we present a detailed evaluation study over a diverse set of non-trivial OCaml datatype implementations that interact with stateful libraries. To the best of our knowledge, Marple is the first system capable of automated verification of sophisticated representation invariants for realistic OCaml programs.

The remainder of this chapter is organized as follows. In the next section, we provide further motivation for the problem by developing a running example, which is used in the rest of the chapter. Sec. 4.2 introduces a core functional language equipped with operators that model stateful library methods. Sec. 4.3 presents a refinement type system that augments basic type qualifiers to also include HATs; a brief background on symbolic automata is also given. An efficient bidirectional typing algorithm is then presented in Sec. 4.4. Implementation details, as well as experimental results on its effectiveness in automatically verifying the representation invariants of a number of realistic data structure client programs that interact with stateful libraries is provided in Sec. 4.5.

## 4.1 Motivation

```
module type Bytes = sig
  type t
  ...
end

module type Path = sig
  type t = string
  val parent: t -> t
  val isRoot: t -> bool
  ...
end

module type File = sig
  val init: unit -> Bytes.t
  val isDir: Bytes.t -> bool
  val isFile: Bytes.t -> bool
  val isDel: Bytes.t -> bool
  val setDeleted: Bytes.t -> Bytes.t
  val delChild:
    Bytes.t -> Path.t -> Bytes.t
  val addChild:
    Bytes.t -> Path.t -> Bytes.t
  ...
end

module type KVStore = sig
  val put: Path.t -> Bytes.t -> unit
  val exists: Path.t -> bool
  val get: Path.t -> Bytes.t
end
```

```
1  module FileSystem = struct
2    open KVStore
3    let init () : unit = put "/" (File.init ())
4
5    let add (path: Path.t) (bytes: Bytes.t): bool =
6      if exists path then false
7      else
8        let parent_path = Path.parent path in
9        if not (exists parent_path) then false
10       else
11         let bytes' = get parent_path in
12         if File.isDir bytes' then (
13          put path bytes;
14          put parent_path (File.addChild bytes' path);
15          true)
16         else false
17
18   let delete (path : Path.t): bool =
19     if (isRoot path) or not (exists path) then false
20     else (
21       let bytes = get path in
22       if File.isDir bytes then deleteChildren(path);
23       let parent_path = Path.parent path in
24       let bytes' = get parent_path in
25       put path (File.setDeleted bytes);
26       put parent_path (File.delChild bytes' path);
27       true)
28 end
```

**Figure 4.1.** A file system datatype based on underline key-value store library.

To further motive our approach, consider the (highly-simplified) Unix-like file-system directory abstraction (`FileSystem`) shown in Fig. 4.1. This module is built on top of other libraries that maintain byte arrays (`Bytes.t`) which hold a file or a directory's contents and metadata, and which manipulate *paths* (`Path.t`), strings that represent fully-elaborated file and directory names. `FileSystem`'s implementation also uses an underlying key-value store library (`KVStore`) to provide a persistent and scalable storage abstraction, where keys represent file paths and values are byte arrays. The interface to this store is defined by three *stateful* library methods: **put** persistently stores a key-value pair $\langle k, v \rangle$, adding the pair to the store if $k$ does not already exist, and overwriting its old value with $v$ if it does; **exists** returns true if the given key has been previously **put** and false otherwise; and, **get** returns

the value associated with its key argument, and fails (i.e., raises an exception) if the key does not exist in the store.

Fig. 4.1 shows three of the methods provided by `FileSystem`: `init` initializes the file system by storing a root path ("`/`"); `add` allows users to add regular files or directories to the filesystem; and, `delete` logically removes files or directories from the filesystem. The latter two operations return a Boolean value to indicate whether they were successful. The `add` operation succeeds (line 15) only if (a) the given path does not already exist in the file system (line 6) and (b) a path representing the parent directory for the file does exist (line 9). Besides adding the file (line 13), `add` also updates the contents of the parent directory to make it aware that a new child has been added (line 14). Similar to `add`, `delete` requires that the supplied path exists and is not the root (line 19). If the path corresponds to a directory, its children are marked deleted (line 22) using procedure `deleteChildren` (not shown). Additionally, the directory's metadata, and that of its parent, are updated to record the deletion (lines 25 and 26). Implicit in `delete`'s implementation is the assumption that the parent path of the file to be deleted exists in the store: observe that there is no check to validate the existence of this path before the **get** of the parent's contents is performed on line 24. This assumption can be framed as a representation invariant that must hold prior to and after every call and return of `add` and `delete`. Intuitively, this invariant captures the following property:

**Invariant**$_{FS}$: *Any path that is stored as a key in the key-value store (other than the root path) must also have its parent stored as a non-deleted directory in the store.*

### Representation Invariants

In a language that guarantees proper data abstraction, library developers can safely assume that any guarantees about an API established using a representation invariant are independent of a particular client's usage. However, as we noted earlier, numerous challenges arise when validating a representation invariant in our setting because we can only characterize the state of the underlying library in terms of its admissible sequence of interactions with the client. In our example, this characterization must be able to capture the structure of

a valid filesystem, in terms of sequences of **put**, **get**, and **exists** calls and returns that are sufficient to ensure that the invariant is preserved. Verifying that $\mathbf{Invariant}_{FS}$ holds requires us to prove that, except for the root, the path associated with a file or directory can only be recorded in the store when its parent path exists, i.e., its parent has previously been **put** and has not yet been deleted. Establishing that this property holds involves reasoning over both data dependencies among filesystem calls and temporal dependencies among store operations.

### Events, Traces, and Symbolic Finite Automata

We address these challenges by encoding desired sets of stateful interactions between the client and library as *symbolic finite automata* (SFA). Besides constants and program variables, the alphabet of an automaton that captures these interactions includes a set of stateful operators `op` (e.g., **put** and **get**). The language recognized by an automaton specifies the sequences of permitted library method invocations. Each such invocation (called an *event*) $\texttt{op}\,\overline{v_i} = v$ records the operator invoked (`op`), a list of its arguments ($\overline{v_i}$), and a single result value $v$. It is critical that we record *both* inputs and outputs of an operation, in order to enable our type system to distinguish among different hidden library states. Consider, for example, a stateful library initialization operator **initialize** : `unit` → `bool`. The hidden state after observing "**initialize** () = `true`", representing a successful initialization action, is likely to be different from the hidden state that exists after observing the event "**initialize** () = `false`", which indicates an initialization error. The language accepted by an SFA is a potentially infinite set of finite sequences of events, or *traces*.

**Example 4.1.1 (Traces).** In contrast to the correct implementation of `add` shown in Fig. 4.1, consider the following incorrect version:

```
let add_bad (path: Path.t) (bytes: Bytes.t) = put path bytes
```

This implementation simply records a path without considering whether its parent exists. We assume the file system initially contains only the root path, "/", which is **put** when the

file system is created by the method `init`. The traces $\alpha_1$ and $\alpha_2$, shown below, represent executions of these two implementations that reflect their differences:

$$
\begin{aligned}
&\text{under context trace } \alpha_0 \doteq [\textbf{put}\ \text{“/”}\ \texttt{bytesDir} = ()] \\
&\qquad \texttt{add}_{\texttt{bad}}\ \text{“/a/b.txt”}\ \texttt{bytesFile} \\
&\qquad\qquad \text{yields } \alpha_1 \doteq [\textbf{put}\ \text{“/”}\ \texttt{bytesDir} = ();\ \textbf{put}\ \text{“/a/b.txt”}\ \texttt{bytesFile} = ()] \\
&\qquad \texttt{add}\ \text{“/a/b.txt”}\ \texttt{bytesFile} \\
&\qquad\qquad \text{yields } \alpha_2 \doteq [\textbf{put}\ \text{“/”}\ \texttt{bytesDir} = ();\ \textbf{exists}\ \text{“/a/b.txt”} = \texttt{false};\ \textbf{exists}\ \text{“/a”} = \texttt{false}]
\end{aligned}
$$

where `bytesDir` and `bytesFile` represent the contents of a directory and a regular file, resp. The trace $\alpha_1$ violates $\textbf{Invariant}_{FS}$ since it registers the path “`/a/b.txt`” in the file system even though the parent path (“`/a`”) does not exist. Thus, performing the operation `delete` “`/a/b.txt`” after executing the actions in this trace will cause a runtime error when it attempts to perform a **get** operation on the parent path “`/a`”. On the other hand, executing this operation after executing the actions in trace $\alpha_2$ terminates normally and preserves $\textbf{Invariant}_{FS}$.

**Example 4.1.2 (Symbolic Finite Automata).** We express symbolic automata as formulae in a symbolic version of linear temporal logic on finite sequences [41]. $\textbf{Invariant}_{FS}$ can be precisely expressed in this language as the following formula (denoted $I_{\texttt{FS}}(\texttt{p})$) parameterized over a path `p`:

$$I_{\texttt{FS}}(\texttt{p}) \doteq \Box\langle \texttt{isRoot}(\texttt{p})\rangle \vee (P_{\texttt{isFile}}(\texttt{p}) \vee P_{\texttt{isDir}}(\texttt{p}) \Longrightarrow P_{\texttt{isDir}}(\texttt{parent}(\texttt{p})))$$

where

$$
\begin{aligned}
P_{\texttt{isDir}}(\texttt{p}) \doteq\ & \Diamond(\langle \textbf{put}\ \mathit{key}\ \mathit{val} = \nu \mid \mathit{key} = \texttt{p} \wedge \texttt{isDir}(\mathit{val})\rangle \wedge \\
& \bigcirc\Box\neg\langle \textbf{put}\ \mathit{key}\ \mathit{val} = \nu \mid \mathit{key} = \texttt{p} \wedge (\texttt{isDel}(\mathit{val}) \vee \texttt{isFile}(\mathit{val}))\rangle) \\
P_{\texttt{isFile}}(\texttt{p}) \doteq\ & \Diamond(\langle \textbf{put}\ \mathit{key}\ \mathit{val} = \nu \mid \mathit{key} = \texttt{p} \wedge \texttt{isFile}(\mathit{val})\rangle \wedge \\
& \bigcirc\Box\neg\langle \textbf{put}\ \mathit{key}\ \mathit{val} = \nu \mid \mathit{key} = \texttt{p} \wedge (\texttt{isDel}(\mathit{val}) \vee \texttt{isDir}(\mathit{val}))\rangle)
\end{aligned}
$$

Here, `isRoot`, `isDir`, `isFile`, and `parent` are pure functions on paths and bytes. The invariant is comprised of two disjunctions involving `p`:

1. $\Box\langle \texttt{isRoot}(\texttt{p})\rangle$ asserts that $\texttt{p}$ is the filesystem root and is always a valid path, regardless of any other actions that have been performed on the underlying key-value store.

2. $P_{\texttt{isFile}}(\texttt{p}) \vee P_{\texttt{isDir}}(\texttt{p}) \Longrightarrow P_{\texttt{isDir}}(\texttt{parent}(\texttt{p}))$ asserts that if $\texttt{p}$ corresponds to either a file or directory in the file system, then its parent path must also correspond to a directory in the file system. The predicate $P_{\texttt{isDir}}(\texttt{p})$ captures contexts in which $\texttt{p}$ has been registered as a directory and then not subsequently deleted or replaced by a file (i.e., $\bigcirc\Box\neg\langle \textbf{put}\, \mathit{key}\, \mathit{val} = \nu \mid \mathit{key} = \texttt{p} \wedge (\texttt{isDel}(\mathit{val}) \vee \texttt{isFile}(\mathit{val}))\rangle$). The predicate ($P_{\texttt{isFile}}(\texttt{p})$) is defined similarly except that it requires that an existing file not be deleted or modified to become a directory. Observe that atomic predicates (e.g., $\langle \textbf{put}\, \mathit{key}\, \mathit{val} = \nu \mid \mathit{key} = \texttt{p} \wedge \texttt{isFile}(\mathit{val})\rangle$) not only capture when an event has been induced by a library operator (e.g., **put**), but also *qualify* the arguments to that operator and the result it produced (e.g., the argument *key* must be equal to the path $\texttt{p}$).[1]

This specification of the representation invariant $I_{\texttt{FS}}(\texttt{p})$ thus formalizes our informal characterization of a correct file system, $\textbf{Invariant}_{FS}$. The two subformulas capture the property that any path recorded as a key in the key-value store, other than the root, must have a parent directory. Notably, this specification is defined purely by constraining the sequence of **put** operations allowed on the store and does not require any knowledge of the store's underlying representation.

### Hoare Automata Types

While SFAs provide a convenient language in which to encode representation invariants, it is not immediately apparent how they can help with verification since, by themselves, they do not impose any safety constraints on expressions. To connect SFAs to computations, we embed them as refinements in a refinement type (and effect) system. In a standard refinement type system, the refinement type $\{\nu{:}b \mid \phi\}$ of an expression $e$ uses a qualifier $\phi$ to refine the value of $e$. Similarly, a Hoare Automata Type (HAT), $[A]\,\{\nu{:}b \mid \phi\}\,[B]$, additionally refines a pure refinement type $\{\nu{:}b \mid \phi\}$ with two symbolic automata: a precondition automaton $A$

[1] ↑Throughout this chapter, we italicize event arguments.

defining an effect *context* under which $e$ is allowed to evaluate, and a postcondition automata $B$ that augments this context with the effects induced by $e$ as it executes. A HAT thus allows us to track the sequence of stateful interactions between higher-level datatypes and the underlying stateful library, as well as to record the consequences of this interaction in terms of the return values of stateful calls.

Our type system leverages this information to verify the preservation of stated representation invariants in terms of these interactions. For example, `add` and `delete` can be ascribed the following HAT-enriched types:

$$\texttt{add} : \texttt{p:Path.t} \dashrightarrow \texttt{path:}\{\nu\texttt{:Path.t} \mid \top\} \to \texttt{bytes:}\{\nu\texttt{:Bytes.t} \mid \top\} \to [I_{\texttt{FS}}(\texttt{p})]\ \{\nu\texttt{:bool} \mid \top\}\ [I_{\texttt{FS}}(\texttt{p})] \qquad (\tau_{\texttt{add}})$$

$$\texttt{delete} : \texttt{p:Path.t} \dashrightarrow \texttt{path:}\{\nu\texttt{:Path.t} \mid \top\} \to [I_{\texttt{FS}}(\texttt{p})]\ \{\nu\texttt{:bool} \mid \top\}\ [I_{\texttt{FS}}(\texttt{p})] \qquad (\tau_{\texttt{delete}})$$

Here, the notation "$\texttt{p:Path.t} \dashrightarrow$" indicates that `p` is a ghost variable representing an arbitrary path; thus, the representation invariant must hold over *any* instantiation of `p`. While their input arguments ($\{\nu\texttt{:Path.t} \mid \top\}$ and ($\{\nu\texttt{:Bytes.t} \mid \top\}$)) are unconstrained, the return types of these functions are expressed as HATs in which both the precondition and postcondition automata refer to the representation invariant for the file system, $I_{\texttt{FS}}(\texttt{p})$. Informally, this type reads "if we invoke the function in a context that is consistent with $I_{\texttt{FS}}$, then the state after the function returns should also be consistent with $I_{\texttt{FS}}$". The traces admitted by an SFA are used to represent these contexts and states.

More concretely, observe that the trace $\alpha_2$ from Example 4.1.1 is derived by concatenating $\alpha_0$ with the library calls generated during the (symbolic) execution of `add`:

$$\alpha_{\texttt{new}} \doteq [\textbf{exists}\ \text{“}\texttt{/a/b.txt}\text{”} = \texttt{false};\ \textbf{exists}\ \text{“}\texttt{/a}\text{”} = \texttt{false}]$$

(i.e., $\alpha_2 = \alpha_0 \# \alpha_{\texttt{new}}$). Note that both $\alpha_0$ and $\alpha_2$ satisfy the representation invariant $I_{\texttt{FS}}$. To admit this trace, our type system automatically infers a new automata $A$ that accepts the sequence of events in $\alpha_{\texttt{new}}$ by type-checking `add`. Verifying that the representation invariant holds now reduces to an inclusion check between two symbolic automata, requiring that the

trace expected by the *concatenated* automata $I_{\mathtt{FS}}; A$ is also accepted by the representation invariant $I_{\mathtt{FS}}$.

## 4.2 Language

| | | |
|---|---|---|
| **Variables** | | $x, y, z, f, \nu, ...$ |
| **Pure Operations** | $op ::=$ | $+ \mid - \mid == \mid < \mid \leq \mid \mathtt{mod} \mid \mathtt{parent} \mid ...$ |
| **Effectful Operations** | $\mathtt{op} ::=$ | **put** \| **exists** \| **get** \| **insert** \| **mem** \| ... |
| **Data constructors** | $d ::=$ | $() \mid \mathtt{true} \mid \mathtt{false} \mid \mathtt{0} \mid \mathtt{S} \mid ...$ |
| **Constants** | $c ::=$ | $\mathbb{Z} \mid d(\overline{c})$ |
| **Values** | $v ::=$ | $c \mid x \mid d(\overline{v}) \mid \lambda x{:}t.e \mid \mathtt{fix} f{:}t.\lambda x{:}t.e$ |
| **Expressions (Computations)** | $e ::=$ | $v \mid \mathtt{let}\ x = \mathtt{op}\ \overline{v}\ \mathtt{in}\ e \mid \mathtt{let}\ x = op\ \overline{v}\ \mathtt{in}\ e \mid \mathtt{let}\ x = v\ v\ \mathtt{in}\ e$ |
| | | $\mid \mathtt{let}\ x = e\ \mathtt{in}\ e \mid \mathtt{match}\ v\ \mathtt{with}\ \overline{d\ \overline{y} \rightarrow e}$ |

**Figure 4.2.** $\boldsymbol{\lambda}^U$ term syntax.

To formalize our approach, we first introduce $\boldsymbol{\lambda}^U$, a core calculus for effectful programs.[2]

### Syntax

$\boldsymbol{\lambda}^U$ is a call-by-value lambda calculus with built-in inductive datatypes, pattern matching, a set of pure and effectful operators, and recursive functions. Its syntax is given in Fig. 4.2. For simplicity, programs are expressed in monadic normal-form (MNF) [69], a variant of A-Normal Form (ANF) [70] that permits nested let-bindings. The terms in $\boldsymbol{\lambda}^U$ are syntactically divided into two classes: values $v$ and expressions $e$, where the latter include both pure terms and those that have computational effects. Similar to the simply-typed lambda calculus, function parameters have explicit type annotations. $\boldsymbol{\lambda}^U$ is parameterized over two sets of primitive operators: pure operators ($op$) include basic arithmetic and logical operators, while examples of effectful operators ($\mathtt{op}$) include state manipulating operators (e.g., **put**/**exists**/**get** for interacting with a key-value store, or **insert**/**mem** for manipulating a stateful set library). We express sequencing of effectful computations $e_1; e_2$ in terms of

[2]↑Although the focus of this chapter is the verification of clients of stateful libraries, our proposed approach applies to effectful programs more generally. Our formalism reflects the more general setting of effectful programs.

`let`-bindings: $e_1; e_2 \doteq \texttt{let}\ x = e_1\ \texttt{in}\ e_2$, where $x$ does not occur free in $e_2$. The application $e_1\, e_2$ is syntactic sugar for $\texttt{let}\ x = e_1\ \texttt{in let}\ y = e_2\ \texttt{in let}\ z = x\, y\ \texttt{in}\ z$.

$$\textbf{Traces} \quad \alpha ::= \quad [\,] \mid (\texttt{op}\,\overline{v} = v) :: \alpha$$

**Figure 4.3.** Trace syntax.

### Operational Semantics

Our programming model does not have access to the underlying implementation of effectful operators, so the semantics of $\boldsymbol{\lambda}^U$ constrains the behavior of impure operations in terms of the outputs they may produce. To do so, we structure its semantics in terms of *traces* that record the history of previous effectful operations and their results. The syntax of traces adopts standard list operations (i.e., cons :: and concatenation $+\!\!+$), as shown in Fig. 4.3. *Traces* are lists of *effect events* $\texttt{op}\,\overline{v} = v$ that record the application of an effectful operator `op` to its arguments $\overline{v}$ as well as the resulting value $v$. The traces $\alpha_2$ and $\alpha_1$ from Example 4.1.1 record the events generated by correct and incorrect implementations of the `add` function of the `FileSystem` ADT, for example.

The operational semantics of $\boldsymbol{\lambda}^U$ is defined via a small-step reduction relation, $\alpha \vDash e \xhookrightarrow{\alpha'} e'$. This relation is read as "under an effect context (i.e., trace) $\alpha$, the term $e$ reduces to $e'$ in one step and performs the effect $\alpha'$", where $\alpha'$ is either empty $[\,]$ or holds a single event. The semantics is parameterized over two auxiliary relations, $e \Downarrow v$ and $\alpha \vDash e \Downarrow v$, used to evaluate pure and impure operations, respectively. Fig. 4.4 shows how the corresponding reduction rules use these relations. The reduction rule for pure operators (STPureOp) holds in an arbitrary effect context (including the empty one), and produces no additional events. The rule for effectful operators (STEffOp), in contrast, can depend on the current context $\alpha$, and records the result value of the effectful operator in the event it emits. The multi-step reduction relation $\alpha \vDash e \xhookrightarrow{\alpha'}{}^{*}\, e'$ defines the reflexive, transitive closure of the single-step relation; its trace $\alpha$ records all events generated during evaluation.

**Small-Step Reduction Rules** $\boxed{\alpha \vDash e \overset{\alpha}{\hookrightarrow} e}$

$$\frac{op\ \overline{v} \Downarrow v_x}{\alpha \vDash \texttt{let}\ x = op\ \overline{v}\ \texttt{in}\ e \overset{[]}{\hookrightarrow} e[x \mapsto v_x]}\ \textsc{StPureOp} \qquad \frac{\alpha \vDash \texttt{op}\,\overline{v} \Downarrow v_x}{\alpha \vDash \texttt{let}\ x = \texttt{op}\ \overline{v}\ \texttt{in}\ e \overset{[\texttt{op}\,\overline{v} = v_x]}{\hookrightarrow} e[x \mapsto v_x]}\ \textsc{StEffOp}$$

$$\frac{\alpha \vDash e_1 \overset{\alpha'}{\hookrightarrow} e_1'}{\alpha \vDash (\texttt{let}\ y = e_1\ \texttt{in}\ e_2) \overset{\alpha'}{\hookrightarrow} (\texttt{let}\ y = e_1'\ \texttt{in}\ e_2)}\ \textsc{StLetE1} \qquad \frac{}{\alpha \vDash \texttt{let}\ y = v\ \texttt{in}\ e \overset{[]}{\hookrightarrow} e[y \mapsto v]}\ \textsc{StLetE2}$$

$$\frac{}{\alpha \vDash (\texttt{let}\ y = \lambda x{:}t.e_1\ v_x\ \texttt{in}\ e_2) \overset{[]}{\hookrightarrow} (\texttt{let}\ y = e_1[x \mapsto v_x]\ \texttt{in}\ e_2)}\ \textsc{StLetAppLam}$$

$$\frac{}{\alpha \vDash \texttt{let}\ y = \texttt{fix}f{:}t.\lambda x{:}t_x.e_1\ v_x\ \texttt{in}\ e_2 \overset{[]}{\hookrightarrow} \texttt{let}\ y = (\lambda f{:}t.e_1[x \mapsto v_x])\ (\texttt{fix}f{:}t.\lambda x{:}t_x.e_1)\ \texttt{in}\ e_2}\ \textsc{StLetAppFix}$$

$$\frac{}{\alpha \vDash \texttt{match}\ d_i\ \overline{v_j}\ \texttt{with}\ \overline{d_i\ \overline{y_j} \to e_i} \overset{[]}{\hookrightarrow} e_i\overline{[y_j \mapsto v_j]}}\ \textsc{StMatch}$$

**Figure 4.4.** Operational Semantics

**Example 4.2.1.** We can define the semantics for the **put**, **exists**, and **get** operators found in a key-value store library and used in `FileSystem`, via the following rules:

$$\frac{}{\alpha \vDash \textbf{put}\,k\,v \Downarrow ()}\ \textsc{Put} \qquad \frac{\textbf{put}\,k\,v = () \notin \alpha}{\alpha \vDash \textbf{exists}\,k \Downarrow \texttt{false}}\ \textsc{ExistsF}$$

$$\frac{\textbf{put}\,k\,v = () \in \alpha}{\alpha \vDash \textbf{exists}\,k \Downarrow \texttt{true}}\ \textsc{ExistsT} \qquad \frac{\textbf{put}\,k\,v = () \notin \alpha'}{\alpha \mathbin{+\!\!+} [\textbf{put}\,k\,v = ()] \mathbin{+\!\!+} \alpha' \vDash \textbf{get}\,k \Downarrow v}\ \textsc{Get}$$

The rule Put asserts that **put** operations always succeed. For a particular key, the ExistsF and ExistsT rules stipulate that **exists** returns `false` or `true`, based on the presence of a corresponding **put** event in the effect context. Finally, the rule Get specifies that the result of a **get** event is the *last* value written by a **put** event to that key. Note that a program will get stuck if it attempts to **get** a key in a trace without an appropriate **put** event, since there is no rule covering this situation.

All the traces appearing in Sec. 4.1 can be derived from these rules. Under the effect context $\alpha_0$, for example, the expression, `add` "`/a/b.txt`" `bytesFile`, induces the trace $\alpha_2$ as follows: the `add` function uses the **exists** operator on the key "`/a/b.txt`" (line 6) to check if this path is in the file system. Since the context trace $\alpha_0$ lacks a **put** event with the key "`/a/b.txt`", we have

$$[\mathbf{put}\ \text{“/”}\ \texttt{bytesDir} = ()] \vDash \mathbf{exists}\ \text{“/a/b.txt”} \Downarrow \texttt{false} \quad (\textsc{existsF})$$

The trace used to evaluate subsequent expressions records this event: "`/a/b.txt`", we have

$$\alpha_0 +\!\!+ [\mathbf{exists}\ \text{“/a/b.txt”} = \texttt{false}]$$

The remaining evaluation of `add` "`/a/b.txt`" `bytesFile` is similar, yielding the following execution:

$$[\mathbf{put}\ \text{“/”}\ \texttt{bytesDir} = ()] \vDash \texttt{add}\ \text{“/a/b.txt”}\ \texttt{bytesFile} \xleftrightarrow{[\mathbf{exists}\ \text{“/a/b.txt”} = \texttt{false};\ \mathbf{exists}\ \text{“/a”} = \texttt{false}]}{}^{*}\ \texttt{false}$$

### 4.3 Type System

The syntax of $\boldsymbol{\lambda}^U$'s types is shown in Fig. 4.5. Our type system uses *pure* refinement types to describe pure computations and *Hoare Automata Types* (HATs) to describe effectful computations. Our pure refinement types are similar to those of other refinement type systems [22], and allow base types (e.g., `int`) to be further constrained by a logical formula or *qualifier*. Qualifiers in $\boldsymbol{\lambda}^U$ are universally quantified formulae over linear arithmetic operations ($op$) as well as uninterpreted functions, or *method predicates* ($mp$), e.g., `isRoot`. Verification conditions generated by our type-checking algorithm can nonetheless be encoded as effectively propositional (EPR) sentences [71], which can be efficiently handled by an off-the-shelf theorem prover such as Z3 [8]. We also allow function types to be prefixed with a set of *ghost variables* with base types ($\overline{x{:}b} \dashrightarrow \tau$). Ghost variables are purely logical – they can only appear in other qualifiers and are implicitly instantiated when the function is applied.

Unique to $\boldsymbol{\lambda}^U$ are the HATs ascribed to a stateful computation, which constrain the traces it may produce. HATs use *Symbolic Finite Automata* (SFAs) [42, 43, 61] to qualify traces,

**Type Syntax**

$$
\begin{array}{rrl}
\textbf{Base Types} & b ::= & \texttt{unit} \mid \texttt{bool} \mid \texttt{nat} \mid \texttt{int} \mid \ldots \\
\textbf{Basic Types} & s ::= & b \mid s \to s \\
\textbf{Value Literals} & l ::= & c \mid x \mid op\ \bar{l} \mid mp\ \bar{l} \\
\textbf{Qualifiers} & \phi ::= & l \mid \bot \mid \top \mid \neg\phi \mid \phi \wedge \phi \mid \phi \vee \phi \mid \phi \Longrightarrow \phi \mid \forall x{:}b.\, \phi \\
\textbf{Refinement Types} & t ::= & \{\nu{:}b \mid \phi\} \mid x{:}t \to \tau \mid x{:}b \dashrightarrow t \\
\textbf{Symbolic Finite Automata} & A, B ::= & \langle \texttt{op}\, \overline{x} = \nu \mid \phi \rangle \mid \langle \phi \rangle \mid \neg A \mid A \wedge A \mid A \vee A \mid A;A \mid \bigcirc A \mid A\, \mathcal{U}\, A \\
\textbf{Hoare Automata Types} & \tau ::= & [A]\, t\, [A] \mid \tau \sqcap \tau \\
\textbf{Type Contexts} & \Gamma ::= & \emptyset \mid x{:}t, \Gamma
\end{array}
$$

**Type Aliases**

$$
\begin{array}{ll}
\langle \texttt{op} \ldots \sim v_x \ldots = \nu \mid \phi \rangle \doteq \langle \texttt{op} \ldots x \ldots = \nu \mid x = v_x \wedge \phi \rangle & \Diamond A \doteq \langle \top \rangle\, \mathcal{U}\, A \quad \Box A \doteq \neg \Diamond \neg A \\
\textbf{LAST} \doteq \neg \bigcirc \langle \top \rangle & b \doteq \{\nu{:}b \mid \top\}
\end{array}
$$

**Figure 4.5.** $\boldsymbol{\lambda}^U$ types.

similar to how standard refinement types use formulae to qualify the types of pure terms. We adopt the symbolic version of *linear temporal logic on finite traces* [41] as the syntax of SFAs.[3]

As shown in Fig. 4.5, HATs support two kinds of atomic propositions, or *symbolic effect events*: $\langle \texttt{op}\, \overline{x} = \nu \mid \phi \rangle$ and $\langle \phi \rangle$. A symbolic effect event $\langle \texttt{op}\, \overline{x} = \nu \mid \phi \rangle$ describes an application of an effectful operator `op` to a set of argument variables $\overline{x}$ that produces a result variable $\nu$. The formula $\phi$ constrains the possible instantiation of $\overline{x}$ and $\nu$ in a concrete event. The other symbolic effect event, $\langle \phi \rangle$, is used to constrain the relationship between pure values (e.g., ghost variables and function arguments). In addition to the temporal next ($\bigcirc A$) and until operators ($A\, \mathcal{U}\, A$), the syntax of HATs includes negation ($\neg A$), intersection ($A \wedge A$), union ($A \vee A$), and concatenation ($A; A$). Fig. 4.5 defines notations for other useful logical and temporal operators: implication ($A \Longrightarrow B \doteq \neg A \vee B$), the eventually operator $\Diamond$, the globally operator $\Box$, and importantly, the last modality **LAST**, which describes a singleton trace, thus prohibiting a trace from including any other effects [41]. Fig. 4.5 also provides notations

[3]↑Our type system is agnostic to the syntax used to express SFAs, and our implementation uses the more developer-friendly syntax of Symbolic Regular Languages [41]; this syntax is provided in the appendix.

for specifying the value of an argument $\langle \mathtt{op}\,\overline{x}\ \sim v_x\ \overline{x} = \nu \mid \phi\rangle$. As is standard, we abbreviate the qualified type $\{\nu{:}b \mid \top\}$ as $b$.

**Example 4.3.1.** This syntax can capture rich properties over effect contexts. For example, the following formulae concisely specify traces under which a key holds a certain value ($P_{\mathtt{stored}}$) and when a particular key exists in the current store ($P_{\mathtt{exists}}$):

$$P_{\mathtt{stored}}(\mathtt{key}, \mathtt{value}) \doteq \Diamond(\langle \mathbf{put} \sim \mathtt{key} \sim \mathtt{value} = \nu \mid \top\rangle \wedge \bigcirc\Box\neg\langle \mathbf{put} \sim \mathtt{key}\, x = \nu \mid \top\rangle)$$

$$P_{\mathtt{exists}}(\mathtt{key}) \doteq \Diamond\langle \mathbf{put} \sim \mathtt{key}\, x = \nu \mid \top\rangle$$

### 4.3.1 HATs, By Example

Formally, a Hoare Automata Type $[A]\,t\,[B]$ qualifies a refinement type $t$ with two SFAs: a *precondition* SFA $A$ that captures the context traces in which a term can be executed and a *postcondition* SFA $B$ that describes the effect trace after the execution of the term.

Our type system also includes intersection types on HATs ($\tau \sqcap \tau$), in order to precisely specify the behaviors of a program under different effect contexts.

**Example 4.3.2 (Built-in Effectful Operators).** Our type system is parameterized over a (global) typing context $\Delta$ containing the types of built-in operators. Intuitively, the signatures of effectful operators in $\Delta$ captures the semantics the library developer ascribes to their API. Using the SFAs from Example 4.3.1, for example, the signatures of **put**, **exists**, and **get** are:

$$\Delta(\mathbf{put}) = \mathtt{k{:}Path.t} \to \mathtt{a{:}Bytes.t} \to [\Box\langle\top\rangle]\ \mathtt{unit}\ [\Box\langle\top\rangle; (\langle \mathbf{put} \sim \mathtt{k} \sim \mathtt{a} = \nu \mid \top\rangle) \wedge \mathbf{LAST}]$$

$$\Delta(\mathbf{get}) = \mathtt{a{:}Bytes.t} \dashrightarrow \mathtt{k{:}Path.t} \to$$
$$[P_{\mathtt{stored}}(\mathtt{k}, \mathtt{a})]\ \{\nu{:}\mathtt{Bytes.t} \mid \nu = \mathtt{a}\}\ [P_{\mathtt{stored}}(\mathtt{k}, \mathtt{a}); (\langle \mathbf{get} \sim \mathtt{k} = \nu \mid \nu = \mathtt{a}\rangle) \wedge \mathbf{LAST}]$$

$$\Delta(\mathbf{exists}) = \mathtt{k{:}Path.t} \to$$
$$[P_{\mathtt{exists}}(\mathtt{k})]\ \{\nu{:}\mathtt{bool} \mid \nu = \mathtt{true}\}\ [P_{\mathtt{exists}}(\mathtt{k}); (\langle \mathbf{exists} \sim \mathtt{k} = \nu \mid \nu = \mathtt{true}\rangle) \wedge \mathbf{LAST}] \sqcap$$
$$[\neg P_{\mathtt{exists}}(\mathtt{k})]\ \{\nu{:}\mathtt{bool} \mid \nu = \mathtt{false}\}\ [\neg P_{\mathtt{exists}}(\mathtt{k}); (\langle \mathbf{exists} \sim \mathtt{k} = \nu \mid \nu = \mathtt{false}\rangle) \wedge \mathbf{LAST}]$$

The postcondition SFAs of all three operators use **LAST**, to indicate that they append a single effect event to the end of the effect context ($\langle \mathtt{op}\,\overline{v} = \nu \mid \phi\rangle \wedge \mathbf{LAST}$). The type of **put**

permits it to be used under any context ($\Box\langle\top\rangle$). More interestingly, the built-in type of **get** uses a ghost variable (`a`) to represent the current value of the key `k`; its precondition stipulates that it should only be applied to keys that have been previously stored. Finally, the intersection type of **exists** describes two possible behaviors, depending on whether the key `k` has been previously added to the store. Note that the pre- and post-condition SFAs in these signatures express different constraints on the allowed contexts in which the operator may execute, and the effects the operator performs.

**Example 4.3.3 (MinSet).** Consider a developer who wants to implement an API for a set augmented with an additional operation that returns the minimum element in the set. This implementation is defined in terms of two other stateful libraries: a Set ADT that provides effectful operators **insert**:`int` → `unit` and **mem**:`int` → `bool`, and a MemCell library that provides a persistent cell with effectful **read**:`unit` → `int` and **write**:`int` → `unit` operators. One implementation strategy is to track the minimum element in a persistent cell, and maintain the representation invariant that the contents of this cell are always less than or equal to every element in the underlying set. Using SFAs, we can express this invariant as:

$$
\begin{aligned}
I_{\texttt{MinSet}}(\texttt{el}) \doteq \Diamond\ (\langle \textbf{write} \sim \texttt{el} = \nu \mid \top\rangle \wedge \bigcirc\Box\neg\langle \textbf{write}\, x = \nu \mid \top\rangle) \Longrightarrow \\
\Diamond\langle \textbf{insert} \sim \texttt{el} = \nu \mid \top\rangle \wedge \Box\neg\langle \textbf{insert}\, x = \nu \mid x < \texttt{el}\rangle \wedge \\
(\Box\neg\langle \textbf{write}\, x = \nu \mid \top\rangle \Longrightarrow \Box\neg\langle \textbf{insert}\, x = \nu \mid \top\rangle)
\end{aligned}
$$

which specifies that an element `el` is written into the persistent cell only when `el` has been inserted into the backing Set *and* no element less than `el` has also been inserted. The HAT ascribed to the function `minset_insert` enforces this invariant

$$
\texttt{el}{:}\texttt{int} \dashrightarrow \texttt{elem}{:}\texttt{int} \to [I_{\texttt{MinSet}}(\texttt{el})]\ \texttt{unit}\ [I_{\texttt{MinSet}}(\texttt{el})]
$$

**Example 4.3.4 (LazySet).** As our next example, consider a LazySet ADT that provides a lazy version of an `insert` operation, delaying when elements are added to an underlying Set ADT. The ADT does so by returning a thunk closed over a collection of elements that will be added to the set when it is forced. This ADT maintains the representation invariant that an element is never inserted twice:

$$
I_{\texttt{LSet}}(\texttt{el}) \doteq \Box(\langle \textbf{insert} \sim \texttt{el} = \nu \mid \top\rangle \Longrightarrow \bigcirc\neg\Diamond\langle \textbf{insert} \sim \texttt{el} = \nu \mid \top\rangle)
$$

We can specify a "*lazy*" insert operator using the following HAT:

$$\texttt{el:int} \dashrightarrow \texttt{elem:int} \to \texttt{thunk:}(\texttt{unit} \to [I_{\texttt{LSet}}(\texttt{el})]\ \texttt{unit}\ [I_{\texttt{LSet}}(\texttt{el})]) \to \texttt{unit} \to [I_{\texttt{LSet}}(\texttt{el})]\ \texttt{unit}\ [I_{\texttt{LSet}}(\texttt{el})]$$

This operation takes as input a (potentially new) element `elem` to be added to the set and a thunk holding any elements that have not yet been inserted into the backing set, and returns another thunk. This HAT stipulates that both input thunk and output thunk preserve the representation invariant of the ADT: $\texttt{unit} \to [I_{\texttt{LSet}}(\texttt{el})]\ \texttt{unit}\ [I_{\texttt{LSet}}(\texttt{el})]$.

**Example 4.3.5 (DFA).** Our final example is a library built on top of a stateful graph library that is used to maintain the states and transitions of a Deterministic Finite Automaton (DFA). The underlying graph library exports two methods to add (**connect**) and remove (**disconnect**) edges. We can specify the standard invariant that DFAs only have deterministic transitions:

$$\begin{aligned} I_{\texttt{DFA}}(\texttt{n}, \texttt{c}) \doteq\ & \Box\neg(\langle \textbf{connect} \sim\texttt{n} \sim\texttt{c}\ n_{end} = \nu \mid \top\rangle \wedge \\ & \bigcirc(\neg\langle \textbf{disconnect} \sim\texttt{n} \sim\texttt{c}\ n_{end} = \nu \mid \top\rangle\, \mathcal{U}\, \langle \textbf{connect} \sim\texttt{n} \sim\texttt{c}\ n_{end} = \nu \mid \top\rangle)) \end{aligned}$$

$I_{\texttt{DFA}}$ stipulates that a node `n` can have at most one transition on a character `c`. Moreover, adding a new transition from `n` on `c` requires that any previous transitions on `c` have first been removed ($\bigcirc(\neg\langle \textbf{disconnect} \sim\texttt{n} \sim\texttt{c}\ n_{end} = \nu \mid \top\rangle\, \mathcal{U}\, \langle \textbf{connect} \sim\texttt{n} \sim\texttt{c}\ n_{end} = \nu \mid \top\rangle)$. An `add_transition` method with the following signature is thus guaranteed to preserve determinism:

$$\begin{aligned} & \texttt{n:Node.t} \dashrightarrow \texttt{c:Char.t} \dashrightarrow \texttt{n_start:Node.t} \to \texttt{char:Char.t} \to \texttt{n_end:Node.t} \to \\ & \quad [I_{\texttt{DFA}}(\texttt{n}, \texttt{c})]\ \texttt{unit}\ [I_{\texttt{DFA}}(\texttt{n}, \texttt{c})] \end{aligned}$$

### 4.3.2 Typing Rules

#### Type contexts

A type context (shown in Fig. 4.5) is a sequence of bindings from variables to pure refinement types (i.e., $t$). Type contexts are not allowed to contain HATs, as doing so breaks several structural properties (e.g., weakening) that are used to prove type safety.[4]

**Type Erasure** $\boxed{\lfloor t \rfloor \quad \lfloor \tau \rfloor \quad \lfloor \Gamma \rfloor}$

$$\lfloor \{\nu{:}b \mid \phi\} \rfloor \doteq b \quad \lfloor x{:}t \to \tau \rfloor \doteq \lfloor t \rfloor \to \lfloor \tau \rfloor \quad \lfloor x{:}b \dashrightarrow t \rfloor \doteq \lfloor t \rfloor \quad \lfloor [A]\, t\, [B] \rfloor \doteq \lfloor t \rfloor \quad \lfloor \tau_1 \sqcap \tau_2 \rfloor \doteq \lfloor \tau_1 \rfloor$$
$$\lfloor \emptyset \rfloor \doteq \emptyset \quad \lfloor x{:}t, \Gamma \rfloor \doteq x{:}\lfloor t \rfloor, \lfloor \Gamma \rfloor$$

**Figure 4.6.** Type Erasure rules.

#### Auxiliary typing relations

There are three sets of auxiliary relations used by our type system, shown in Fig. 4.6, Fig. 4.7, and Fig. 4.8. The first set describes the type erasure functions $\lfloor t \rfloor$, $\lfloor \tau \rfloor$, and $\lfloor \Gamma \rfloor$. The former two functions return basic types by erasing all qualifiers and automata from types, while the latter $\lfloor \Gamma \rfloor$ is the type context derived by applying $\lfloor ... \rfloor$ to all of $\Gamma$'s bindings. The second set describes well-formedness conditions on SFAs ($\Gamma \vdash^{\mathbf{WF}} A$) and types ($\Gamma \vdash^{\mathbf{WF}} t$ and $\Gamma \vdash^{\mathbf{WF}} \tau$). These rules largely ensure that all the qualifiers appearing in a type are closed under the current typing context $\Gamma$. The exceptions are WFInter, which stipulates that only HATs with the same basic type can be intersected, and WFHoare, which requires the language accepted by the precondition SFA to be a prefix of the postcondition automata (i.e., $\mathcal{L}(B) \subseteq \mathcal{L}(A; \Box\langle\top\rangle)$, where $\Box\langle\top\rangle$ denotes the SFA that accepts an arbitrary trace) after performing a substitution consistent with the current typing context ($\sigma \in [\![\Gamma]\!]$); the functions $\mathcal{L}(...)$ and $[\![...]\!]$ are defined in the next subsection.

[4]↑Intuitively, since a type of the form $[A]\, t\, [B]$ describes a stateful computation, it cannot be duplicated or eliminated, and thus it would be unsafe to allow unrestricted use of "computational" variables, as would be possible if type contexts could contain such bindings.

**Well-Formed Types** $\boxed{\Gamma \vdash^{\mathbf{WF}} A \quad \Gamma \vdash^{\mathbf{WF}} t \quad \Gamma \vdash^{\mathbf{WF}} \tau}$

$$\frac{\lfloor\Gamma\rfloor, \nu{:}b \vdash_{\mathtt{s}} \phi : \mathtt{bool}}{\Gamma \vdash^{\mathbf{WF}} \{\nu{:}b \mid \phi\}}\ \textsc{WfBase} \qquad \frac{\Gamma \vdash^{\mathbf{WF}} t_x \qquad \Gamma, x{:}\lfloor t_x\rfloor \vdash^{\mathbf{WF}} t}{\Gamma \vdash^{\mathbf{WF}} x{:}t_x \to t}\ \textsc{WfArr} \qquad \frac{\Gamma, x{:}b \vdash^{\mathbf{WF}} t}{\Gamma \vdash^{\mathbf{WF}} x{:}b \dashrightarrow t}\ \textsc{WfGhost}$$

$$\frac{\mathbf{Ty}(\mathtt{op}) = \overline{x_i{:}b_i} \to b \quad \lfloor\Gamma\rfloor, \overline{x_i{:}b_i}, \nu{:}b \vdash_{\mathtt{s}} \phi : \mathtt{bool}}{\Gamma \vdash^{\mathbf{WF}} \langle \mathtt{op}\, \overline{x_i} = \nu \mid \phi\rangle}\ \textsc{WfOpEvent} \qquad \frac{\lfloor\Gamma\rfloor \vdash_{\mathtt{s}} \phi : \mathtt{bool}}{\Gamma \vdash^{\mathbf{WF}} \langle \phi\rangle}\ \textsc{WfTestEvent}$$

$$\frac{\Gamma \vdash^{\mathbf{WF}} A}{\Gamma \vdash^{\mathbf{WF}} \neg A}\ \textsc{WfNeg} \qquad \frac{\Gamma \vdash^{\mathbf{WF}} A_1 \quad \Gamma \vdash^{\mathbf{WF}} A_2}{\Gamma \vdash^{\mathbf{WF}} A_1 \wedge A_2}\ \textsc{WfAnd} \qquad \frac{\Gamma \vdash^{\mathbf{WF}} A_1 \quad \Gamma \vdash^{\mathbf{WF}} A_2}{\Gamma \vdash^{\mathbf{WF}} A_1 \vee A_2}\ \textsc{WfOr}$$

$$\frac{\Gamma \vdash^{\mathbf{WF}} A_1 \quad \Gamma \vdash^{\mathbf{WF}} A_2}{\Gamma \vdash^{\mathbf{WF}} A_1; A_2}\ \textsc{WfConcat} \qquad \frac{\Gamma \vdash^{\mathbf{WF}} A}{\Gamma \vdash^{\mathbf{WF}} \bigcirc A}\ \textsc{WfNext} \qquad \frac{\Gamma \vdash^{\mathbf{WF}} A_1 \quad \Gamma \vdash^{\mathbf{WF}} A_2}{\Gamma \vdash^{\mathbf{WF}} A_1 \,\mathcal{U}\, A_2}\ \textsc{WfUntil}$$

$$\frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} A \quad \Gamma \vdash^{\mathbf{WF}} B \quad \Gamma \vdash^{\mathbf{WF}} t \\ \forall \sigma \in [\![\Gamma]\!].\mathcal{L}(\sigma(B)) \subseteq \\ \mathcal{L}(\sigma(A; \Box\langle\top\rangle))\end{array}}{\Gamma \vdash^{\mathbf{WF}} [A]\, t\, [B]}\ \textsc{WFHoare} \qquad \frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} \tau_1 \quad \Gamma \vdash^{\mathbf{WF}} \tau_2 \\ \lfloor\tau_1\rfloor = \lfloor\tau_2\rfloor\end{array}}{\Gamma \vdash^{\mathbf{WF}} \tau_1 \sqcap \tau_2}\ \textsc{WFInter}$$

**Figure 4.7.** Type Well-formedness rules. $\lfloor\Gamma\rfloor \vdash_{\mathtt{s}} e : \lfloor t\rfloor$ is the standard typing judgment for basic types.

Our type system also uses a mostly-standard subtyping relation that aligns with the denotation of the types being related. Fig. 4.8 highlights key subtyping rules that do not have analogues in a standard refinement type system. The subtyping rule for symbolic automata (SubAutomata) checks inclusion between the languages of the automata, after performing substitutions consistent with the current typing context. The subtyping rule for (non-intersected) HATs (SubHoare) checks that the inclusion is contravariant over the precondition automata and covariant over the postcondition automata under the same context (i.e., the conjunction of $B_1$ and $B_2$ with $A_2; \Box\langle\top\rangle$). The subtyping rules for the intersection of HATs - SubIntLL, SubIntLR, and SubIntR - are standard.

The SubIntMerge rule additionally allows the precondition automata of intersected types to be merged. Finally, the subtyping rules for ghost variables either bind a ghost variable in the type context (SubGhostR), or instantiate it to some concrete value (SubGhostL).

**Automata Inclusion** $\boxed{\Gamma \vdash A \subseteq A}$ **Subtyping** $\boxed{\Gamma \vdash t <: t \quad \Gamma \vdash \tau <: \tau}$

$$\frac{\begin{array}{c}\forall \sigma \in [\![\Gamma]\!].\\ \mathcal{L}(\sigma(A_1)) \subseteq \\ \mathcal{L}(\sigma(A_2))\end{array}}{\Gamma \vdash A_1 \subseteq A_2}\ \textsc{SubAutomata} \qquad \frac{\begin{array}{c}\Gamma \vdash A_2 \subseteq A_1\\ \Gamma \vdash t_1 <: t_2\\ \Gamma \vdash (A_2; \Box\langle\top\rangle) \wedge B_1 \subseteq \\ (A_2; \Box\langle\top\rangle) \wedge B_2\end{array}}{\Gamma \vdash [A_1]\, t_1\, [B_1] <: [A_2]\, t_2\, [B_2]}\ \textsc{SubHoare}$$

$$\frac{}{\Gamma \vdash \tau_1 \sqcap \tau_2 <: \tau_1}\ \textsc{SubIntLL} \qquad \frac{}{\Gamma \vdash \tau_1 \sqcap \tau_2 <: \tau_2}\ \textsc{SubIntLR}$$

$$\frac{}{\Gamma \vdash [A_1]\, t\, [B] \sqcap [A_2]\, t\, [B] <: [A_1 \vee A_2]\, t\, [B]}\ \textsc{SubIntMerge}$$

$$\frac{\begin{array}{c}\Gamma \vdash \tau <: \\ \tau_1 \quad \Gamma \vdash \\ \tau <: \tau_2\end{array}}{\Gamma \vdash \tau <: \tau_1 \sqcap \tau_2}\ \textsc{SubIntR} \qquad \frac{\Gamma, x{:}\{\nu{:}b \mid \top\} \vdash t_1 <: t_2}{\Gamma \vdash t_1 <: x{:}b \dashrightarrow t_2}\ \textsc{SubGhostR} \qquad \frac{\begin{array}{c}\exists v. \lfloor \Gamma \rfloor \vdash_{\mathsf{s}} v : b\\ \Gamma \vdash t_1[x \mapsto v] <: t_2\end{array}}{\Gamma \vdash x{:}b \dashrightarrow t_1 <: t_2}\ \textsc{SubGhostL}$$

**Figure 4.8.** Subtyping rules.

Our typing rules is shown in Fig. 4.9. Note that an stateful computations can only be ascribed a HAT. As mentioned in Example 4.3.2, our type system is parameterized over $\Delta$, a typing context for built-in operators that provides HATs for both pure (TPOp) and effectful operators (TEOp). This system features the standard subsumption and intersection rules (TSub and TInter), which enables our type system to use fine-grained information about the effect context when typing different control flow paths. The rule TEPur allows a pure term to be treated as a computation that does not perform any effects.

**Typing** $\boxed{\Gamma \vdash op : t \quad \Gamma \vdash \mathtt{op} : t \quad \Gamma \vdash e : t \quad \Gamma \vdash e : \tau}$

$$\frac{\Gamma \vdash^{\mathbf{WF}} \{\nu{:}b \mid \nu = x\} \quad (x, \{\nu{:}b \mid \phi\}) \in \Gamma}{\Gamma \vdash x : \{\nu{:}b \mid \nu = x\}} \ \textsc{TBaseVar} \qquad \frac{\Gamma \vdash^{\mathbf{WF}} z{:}t \to \tau \quad (x, z{:}t \to \tau) \in \Gamma}{\Gamma \vdash x : z{:}t \to \tau} \ \textsc{TFuncVar} \qquad \frac{\Gamma \vdash^{\mathbf{WF}} \{\nu{:}b \mid \nu = c\} \quad \emptyset \vdash_{\mathtt{s}} c : b}{\Gamma \vdash c : \{\nu{:}b \mid \nu = c\}} \ \textsc{TConst}$$

$$\frac{\Gamma \vdash^{\mathbf{WF}} t \quad \Delta(op) = t}{\Gamma \vdash op : t} \ \textsc{TPOp} \qquad \frac{\Gamma \vdash^{\mathbf{WF}} t \quad \Delta(\mathtt{op}) = t}{\Gamma \vdash \mathtt{op} : t} \ \textsc{TEOp} \qquad \frac{\Gamma \vdash^{\mathbf{WF}} t_2 \quad \Gamma \vdash t_1 <: t_2 \quad \Gamma \vdash op : t_1}{\Gamma \vdash op : t_2} \ \textsc{TSubPOp} \qquad \frac{\Gamma \vdash^{\mathbf{WF}} t_2 \quad \Gamma \vdash t_1 <: t_2 \quad \Gamma \vdash \mathtt{op} : t_1}{\Gamma \vdash \mathtt{op} : t_2} \ \textsc{TSubEOp}$$

$$\frac{\Gamma \vdash^{\mathbf{WF}} \tau_2 \quad \Gamma \vdash \tau_1 <: \tau_2 \quad \Gamma \vdash e : \tau_1}{\Gamma \vdash e : \tau_2} \ \textsc{TSub} \qquad \frac{\Gamma \vdash^{\mathbf{WF}} t_2 \quad \Gamma \vdash t_1 <: t_2 \quad \Gamma \vdash e : t_1}{\Gamma \vdash e : t_2} \ \textsc{TSubPP} \qquad \frac{\Gamma \vdash^{\mathbf{WF}} A \quad \Gamma \vdash e : t}{\Gamma \vdash e : [A]\, t\, [A]} \ \textsc{TEPur}$$

$$\frac{\Gamma \vdash^{\mathbf{WF}} \tau_1 \sqcap \tau_2 \quad \Gamma \vdash e : \tau_1 \quad \Gamma \vdash e : \tau_2}{\Gamma \vdash e : \tau_1 \sqcap \tau_2} \ \textsc{TInter} \qquad \frac{\Gamma \vdash^{\mathbf{WF}} x{:}b \dashrightarrow t \quad \Gamma, x{:}\{\nu{:}b \mid \top\} \vdash v : t}{\Gamma \vdash v : x{:}b \dashrightarrow t} \ \textsc{TGhost} \qquad \frac{\Gamma \vdash^{\mathbf{WF}} x{:}t_x \to \tau \quad \Gamma, x{:}t_x \vdash e : \tau}{\Gamma \vdash \lambda x{:}t_x.e : x{:}t_x \to \tau} \ \textsc{TFun}$$

$$\frac{\Gamma \vdash^{\mathbf{WF}} \tau \quad \Gamma \vdash op : \overline{z_i{:}t_i} \to t \quad \forall i.\Gamma \vdash v_i : t_i \quad t_x = t[\overline{z_i \mapsto v_i}] \quad \Gamma, x{:}t_x \vdash e : \tau}{\Gamma \vdash \mathtt{let}\ x = op\ \overline{v_i}\ \mathtt{in}\ e : \tau} \ \textsc{TPOpApp} \qquad \frac{\Gamma \vdash^{\mathbf{WF}} [A]\, t\, [B] \quad \Gamma \vdash \mathtt{op} : \overline{z_i{:}t_i} \to \tau \quad \forall i.\Gamma \vdash v_i : t_i \quad [A]\, t_x\, [A'] = \tau[\overline{z_i \mapsto v_i}] \quad \Gamma, x{:}t_x \vdash e : [A']\, t\, [B]}{\Gamma \vdash \mathtt{let}\ x = \mathtt{op}\ \overline{v_i}\ \mathtt{in}\ e : [A]\, t\, [B]} \ \textsc{TEOpApp}$$

$$\frac{\Gamma \vdash^{\mathbf{WF}} [A]\, t\, [B] \quad \Gamma \vdash v_1 : y{:}t_2 \to \tau \quad \Gamma \vdash v_2 : t_2 \quad [A]\, t_x\, [A'] = \tau[y \mapsto v_2] \quad \Gamma, x{:}t_x \vdash e : [A']\, t\, [B]}{\Gamma \vdash \mathtt{let}\ x = v_1\ v_2\ \mathtt{in}\ e : [A]\, t\, [B]} \ \textsc{TApp} \qquad \frac{\Gamma \vdash^{\mathbf{WF}} [A]\, t\, [B] \quad \Gamma \vdash e_x : [A]\, t_x\, [A'] \quad \Gamma, x{:}t_x \vdash e : [A']\, t\, [B]}{\Gamma \vdash \mathtt{let}\ x = e_x\ \mathtt{in}\ e : [A]\, t\, [B]} \ \textsc{TLetE}$$

$$\frac{\Gamma \vdash^{\mathbf{WF}} \tau \quad \forall i.\Delta(d_i) = \overline{y_j{:}t_j} \to \{\nu{:}b \mid \phi_i\} \quad \Gamma, \overline{y_j{:}t_j}, z{:}\{\nu{:}b \mid \nu = v \wedge \phi_i\} \vdash e_i : \tau \quad \text{where } z \text{ is fresh}}{\Gamma \vdash (\mathtt{match}\ v\ \mathtt{with}\ \overline{d_i\, \overline{y_j} \to e_i}) : \tau} \ \textsc{TMatch} \qquad \frac{\Gamma \vdash^{\mathbf{WF}} t \quad \Gamma, f{:}t \vdash \lambda x{:}t_x.e : t}{\Gamma \vdash \mathtt{fix} f{:}t.\lambda x{:}t_x.e : t} \ \textsc{TFix}$$

**Figure 4.9.** Type rules. All typing judgements (i.e., $\Gamma \vdash e : t$ and $\Gamma \vdash e : \tau$) assume the corresponding basic type judgement ($\lfloor \Gamma \rfloor \vdash_{\mathtt{s}} e : \lfloor t \rfloor$ holds).

**Example 4.3.6.** The `add` function in Fig. 4.1 has four possible control flow paths, depending on the effect context under which it is called: (1) the input path exists in the file system (line 6); (2) neither the input path or its parent path exist (line 9); (3) its parent path exists and is a directory (line 13 – 15), or (4) it is not (line 16). The following four automata (recall that the SFA $P_{\texttt{isDir}}$ was defined in Sec. 4.1) indicate the effect context corresponding to each of these scenarios:

$$A_1 \doteq I_{\texttt{FS}}(\texttt{p}) \wedge P_{\texttt{exists}}(\texttt{path})$$
$$A_2 \doteq I_{\texttt{FS}}(\texttt{p}) \wedge \neg P_{\texttt{exists}}(\texttt{path}) \wedge \neg P_{\texttt{exists}}(\texttt{parent}(\texttt{path}))$$
$$A_2 \doteq I_{\texttt{FS}}(\texttt{p}) \wedge \neg P_{\texttt{exists}}(\texttt{path}) \wedge P_{\texttt{exists}}(\texttt{parent}(\texttt{path})) \wedge P_{\texttt{isDir}}(\texttt{parent}(\texttt{path}))$$
$$A_3 \doteq I_{\texttt{FS}}(\texttt{p}) \wedge \neg P_{\texttt{exists}}(\texttt{path}) \wedge P_{\texttt{exists}}(\texttt{parent}(\texttt{path})) \wedge \neg P_{\texttt{isDir}}(\texttt{parent}(\texttt{path}))$$

The union of these automata is exactly the representation invariant ($I_{\texttt{FS}}(\texttt{p})$), established from the following subtyping relation between their intersection and the return type of $\tau_{\texttt{add}}$:

$$\texttt{p}{:}\{\nu{:}\texttt{Path.t} \mid \top\}, \texttt{path}{:}\{\nu{:}\texttt{Path.t} \mid \top\}, \texttt{bytes}{:}\{\nu{:}\texttt{Bytes.t} \mid \top\} \vdash \bigsqcap_{i=1..4} [A_i]\,\texttt{bool}\,[I_{\texttt{FS}}(\texttt{p})] <: [I_{\texttt{FS}}(\texttt{p})]\,\texttt{bool}\,[I_{\texttt{FS}}(\texttt{p})] \quad \text{(SubIntMerge)}$$

Using the TInter and Tsub rules, our type system is able to reduce checking `add` against $\tau_{\texttt{add}}$ into checking `add` against each $A_i$ (i.e., $[A_i]\,\texttt{bool}\,[I_{\texttt{FS}}(\texttt{p})]$). Note that `add` only adds the given path to the file system in the third case (line 13), whose precondition automata ($A_2$) indicates that the input path does not exist in the file system ($\neg P_{\texttt{exists}}(\texttt{path})$), although its parent path does ($P_{\texttt{exists}}(\texttt{parent}(\texttt{path}))$), and is a directory ($P_{\texttt{isDir}}(\texttt{parent}(\texttt{path}))$). When coupled with the representation invariant that the parent of a path is always a directory in the file system, our type system is able to ensure that it is safe to perform the **put** operation.

### Effectful operator application

TPOpApp is the standard rule for operator application in a typical refinement type system [22]. On the other hand, rule TEOpApp, the rule for effect operator application, with the help of the subsumption rule (TSub), allows operators to have function types whose ghost variables are instantiated properly; moreover, the return type of effect operators is

a non-intersected HAT. After ensuring each argument $v_i$ types against the corresponding argument type $t_i$ and substituting all parameters ($\overline{z_i}$) with the supplied arguments ($\overline{v_i}$), the return type of an effectful operator must be in the form $[A]\, t_x\, [A']$ and have exactly the same precondition SFA ($A$) as the type of the surrounding term. Typing the `let`-body $e$ is similar to TPOpApp, where a new binding $x{:}t_x$ is added to the type context. Moreover, the rule replaces the precondition SFA used to type the body of the `let` with the postcondition SFA from the return type of the effect operator $A'$, so that the body is typed in a context reflecting the use of the effectful operator.

### 4.3.3 Type Soundness

**Trace Language** $\boxed{\alpha, i \models A \quad \mathcal{L}(A) \in \mathcal{P}(\alpha)}$

$$Tr^{\mathbf{WF}} \doteq \{\alpha \mid \forall (\mathtt{op}\, \overline{v_i} = v) \in \alpha.\emptyset \vdash_{\mathtt{s}} \mathtt{op} : \overline{b_i} \to b \wedge (\forall i.\emptyset \vdash_{\mathtt{s}} v_i : b_i) \wedge \emptyset \vdash_{\mathtt{s}} v : b\} \qquad \mathcal{L}(A) \doteq \{\alpha \in Tr^{\mathbf{WF}} \mid \alpha, 0 \models A\}$$

$$\alpha, i \models \langle \mathtt{op}\, \overline{x_j} = \nu \mid \phi \rangle \iff \alpha[i] = \mathtt{op}\, \overline{v_j} = v \wedge \phi[\overline{x_j \mapsto v_j}][\nu \mapsto v] \qquad \alpha, i \models A \wedge A' \iff \alpha, i \models A \wedge \alpha, i \models A'$$

$$\alpha, i \models \langle \phi \rangle \iff \alpha[i] = \mathtt{op}\, \overline{v_j} = v \wedge \phi \qquad \alpha, i \models A \vee A' \iff \alpha, i \models A \vee \alpha, i \models A'$$

$$\alpha, i \models \bigcirc A \iff \alpha, i{+}1 \models A \qquad \alpha, i \models A_1; A_2 \iff \alpha[i...len(\alpha)] = \alpha_1 \# \alpha_2 \wedge \alpha_1 \in \mathcal{L}(A_1) \wedge \alpha_2 \in \mathcal{L}(A_2)$$

$$\alpha, i \models \neg A \iff \alpha, i \not\models A \qquad \alpha, i \models A\, \mathcal{U}\, A' \iff \exists j.i \leq j < len(\alpha).\alpha, j \models A' \wedge \forall k.i \leq k < j \Longrightarrow \alpha, k \models A$$

**Type Denotation** $\boxed{[\![t]\!] \in \mathcal{P}(e) \quad [\![\tau]\!] \in \mathcal{P}(e)}$

$$\begin{aligned}
[\![\{\nu{:}b \mid \phi\}]\!] &\doteq \{e \mid \emptyset \vdash_{\mathtt{s}} e : b \wedge \forall \alpha\, \alpha'\, v.\, \alpha \vDash e \xhookrightarrow{\alpha'}{}^{*} v \Longrightarrow \alpha' = [\,] \wedge \phi[\nu \mapsto v]\} \\
[\![x{:}t \to \tau]\!] &\doteq \{e \mid \emptyset \vdash_{\mathtt{s}} e : \lfloor x{:}t \to \tau \rfloor \wedge \forall v \in [\![t]\!].\, e\ v \in [\![\tau[x \mapsto v]]\!]\} \\
[\![x{:}b \dashrightarrow \tau]\!] &\doteq \{e \mid \emptyset \vdash_{\mathtt{s}} e : \lfloor \tau \rfloor \wedge \forall v.\emptyset \vdash_{\mathtt{s}} v : b \Longrightarrow e \in [\![\tau[x \mapsto v]]\!]\} \\
[\![[A]\, t\, [B]]\!] &\doteq \{e \mid \emptyset \vdash_{\mathtt{s}} e : \lfloor t \rfloor \wedge \forall \alpha\, \alpha'\, v.\, \alpha \in \mathcal{L}(A) \wedge \alpha \vDash e \xhookrightarrow{\alpha'}{}^{*} v \Longrightarrow v \in [\![t]\!] \wedge \alpha \# \alpha' \in \mathcal{L}(B)\} \\
[\![\tau_1 \sqcap \tau_2]\!] &\doteq [\![\tau_1]\!] \cap [\![\tau_2]\!]
\end{aligned}$$

**Type Context Denotation** $\boxed{[\![\Gamma]\!] \in \mathcal{P}(\sigma)}$

$$[\![\emptyset]\!] \doteq \{\emptyset\} \qquad [\![x{:}t, \Gamma]\!] \doteq \{\sigma[x \mapsto v] \mid v \in [\![t]\!], \sigma \in [\![\Gamma[x \mapsto v]]\!]\}$$

**Figure 4.10.** Type denotations in $\boldsymbol{\lambda}^U$

The denotation of a SFA $\mathcal{L}(A)$, shown in Fig. 4.10, is the set of traces accepted or *recognized* by the automata; the denotation of refinement types $[\![t]\!]$ and HATs $[\![\tau]\!]$ are sets of terms; these functions are defined mutually recursively.

The language of traces ranges over the set of all *well-formed traces* $Tr^{\mathbf{WF}}$, i.e., those only containing events $(\mathtt{op}\,\overline{v_i} = v)$ that are well-typed according to the basic type system. As is standard [41], our denotation uses an auxiliary judgement $\alpha, i \models A$ that defines when the suffix of a trace $\alpha$ starting at the $i^{th}$ position is accepted by an automaton. The denotation of an SFA $A$ is the set of (well-formed) traces recognized by $A$, starting at index 0.

### Type denotations

Out type denotations use a basic typing judgement $\emptyset \vdash_{\mathtt{s}} e : s$ that types an expression $e$ according to the standard typing rules of the simply-typed lambda-calculus (STLC), erasing all qualifiers in function argument types. The denotation of pure refinement types is standard [22]. The denotation of ghost variables is similar to function types whose parameters are restricted to base types. The denotation of an intersection type $\tau_1 \sqcap \tau_2$ is the intersection of the denotations $\tau_1$ and $\tau_2$. An expression $e$ belongs to the denotation of a HAT $[A]\,t\,[B]$ iff every trace and value produced by $e$ is consistent with the SFA $B$ and refinement type $t$, under any effect context accepted by the SFA $A$. Intuitively, the denotation of a HAT is naturally derived from the language's operational semantics, as depicted by the following correspondence:

$$\begin{array}{lrll}
\text{Pure Language:} & e \hookrightarrow^* v \implies e \in [\![\{\nu{:}b \mid \phi\}]\!] & & \text{where } \phi[\nu \mapsto v] \text{ is valid} \\
\boldsymbol{\lambda}^U: & \alpha \vDash e \xhookrightarrow{\alpha'}{}^* v \implies e \in [\![[A]\,\{\nu{:}b \mid \phi\}\,[B]]\!] & & \text{where } \alpha \in \mathcal{L}(A), \\
& & & \alpha \uplus \alpha' \in \mathcal{L}(B), \text{and } \phi[\nu \mapsto v] \text{ is valid}
\end{array}$$

In a pure language with the simple multi-step reduction relation $e \hookrightarrow^* v$, refinement types qualify the value $v$ that $e$ produces. In contrast, the multi-step reduction relation of $\boldsymbol{\lambda}^U$, $\alpha \vDash e \xhookrightarrow{\alpha'}{}^* v$ depends on the effect context $\alpha$ and emits a trace $\alpha'$ that records the sequence of effects performed by $e$ during evaluation. Thus, HATs use precondition and postcondition

automata (i.e., $A$ and $B$) to qualify the trace before and after a term is evaluated (i.e., $\alpha$ and $\alpha \uplus \alpha'$).

### Type context denotation

The denotation of a type context is a set of *closing substitutions* $\sigma$, i.e., a sequence of bindings $\overline{[x_i \mapsto v_i]}$ consistent with the type denotations of the corresponding variables in the type context. The denotation of the empty context is a singleton set containing only the identity substitution $\emptyset$.

**Definition 4.3.7 (Well-formed built-in operator typing context).** The built-in operator typing context $\Delta$ is well-formed iff the semantics of every built-in operator $\theta$ is consistent with its type: $\Delta(\theta) \;=\; \overline{x{:}b_x} \dashrightarrow \overline{y{:}t_y} \to \tau \implies \overline{\forall x{:}b_x}.\, \overline{\forall v_y \in [\![t_y]\!]}.\, (\theta\, \overline{v_y}) \in [\![\tau \overline{[y \mapsto v_y]}]\!]$.[5]

**Theorem 4.3.8.** *[Fundamental Theorem] Given a well-formed typing context for built-in operators $\Delta$, the trace of effects produced by a well-typed term $e$ are captured by its corresponding HAT $\tau$: $\Gamma \vdash e : \tau \implies \forall \sigma, \sigma \in [\![\Gamma]\!] \implies \sigma(e) \in [\![\sigma(\tau)]\!]$.*[6]

**Corollary 4.3.9.** *[Type Soundness] Under a built-in operator type context that is well-formed, if a function $f$ preserves the representation invariant $A$, i.e. it has the type: $\emptyset \vdash f : \overline{x{:}b_x} \dashrightarrow \overline{y{:}b_y} \to [A]\, t\, [A]$, then for every set of well-typed terms $\emptyset \vdash \overline{v_x : b_x}$ and $\emptyset \vdash \overline{v_y : b_y}$, applying $f$ to $\overline{v_y}$ under an effect context $\alpha$ that is consistent with $A$ results in a context $\alpha \uplus \alpha'$ that is also consistent with $A$:*

$$\alpha \vDash (f\ \overline{v_y}) \xhookrightarrow{\alpha'}{}^{*} v \implies \alpha \uplus \alpha' \in \mathcal{L}(A[\overline{x \mapsto v_x}][\overline{y \mapsto v_y}]) \wedge v \in [\![t[\overline{x \mapsto v_x}][\overline{y \mapsto v_y}]]\!]$$

## 4.4 Typing Algorithm

Converting our declarative type system into an efficient algorithm requires resolving two key issues. First, we cannot directly use existing SFA inclusion algorithms [42, 43] to implement the check in SubAutomata, because our SFAs may involve non-local variables corresponding to function parameters and ghost variables; these variables can have refinement

[5]↑Recall that the semantics of an operator is defined by the auxiliary $e \Downarrow v$ and $\alpha \vDash e \Downarrow v$.

[6]↑The Coq formalization of our core language, typing rules, and the proof of Theorem 4.3.8 and Corollary 4.3.9 are available on Zenodo https://doi.org/10.5281/zenodo.10806686.

types. We deal with this wrinkle by integrating the subtyping algorithm for pure refinement types into the existing SFA algorithm. Second, typing the use of an effectful operator, e.g., **put** or **exists**, depends on the set of effects in the precondition of the current HAT. The declarative type system can ensure that this automaton has the right shape by applying the TInter and TSub rules at any point in a derivation, but an efficient implementation must perform this conversion more intelligently. Our solution to this problem is to employ a bidirectional type system [34] that closely tracks a context of effects that have preceded the term being typed, and selectively applies the subsumption rule to simplify the context when switching between checking and synthesis modes.

### 4.4.1 SFA Inclusion

Symbolic alphabets allow the character domains of SFAs to be infinite, making them strictly more expressive than standard FAs. This enhanced expressivity comes at a cost, however, as the standard FA inclusion algorithm cannot be directly applied to SFAs. Thankfully, prior work has shown how to reduce an inclusion check over SFAs to an inclusion check over FAs [42, 43]. The high-level approach is to first construct a finite set of equivalence classes over an SFA's infinite character domain, defined in terms of a set of maximal satisfiable Boolean combinations of logic literals (called *minterms*) in the automaton's symbolic alphabet. An alphabet transformation procedure then replaces characters in the original SFA with their corresponding equivalence classes, and replaces the SFA's symbolic alphabet with minterms, thus allowing SFAs to be translated into FAs. As long as the satisfiability of minterms is decidable, so is checking inclusion between two SFAs. Our use of EPR formulas for qualifiers guarantees that minterm satisfiability is decidable.

The pre- and post-conditions used in HATs have two features that distinguish them from standard SFAs, however: 1) characters are drawn from a set of distinct events and their qualifiers may range over multiple variables, instead of a single local variable; and, 2) event qualifiers may refer to variables in the typing context, i.e., function parameters and ghost variables. Our language inclusion algorithm, shown in Algorithm 8, accounts for all these

**Algorithm 8:** Inclusion Query

```
1  Procedure SFAInclusion(Γ, A, B) :=
2      L_Γ ← GetLits(Γ);
3      foreach φ_Γ ∈ BoolComb(L_Γ) do
4          M ← ∅;
5          foreach op where ∅ ⊢_s op : b̄_i → b do
6              L ← GetLits(Γ, A ∨ B);
7              foreach φ ∈ BoolComb(L) do
8                  if Γ, x̄_i:b̄_i ⊬ {ν:b | φ_Γ ∧ φ} <: {ν:b | ⊥} then
9                      M ← M ∪ {⟨op x̄_i = ν | φ_Γ ∧ φ⟩}
10                 end
11             end
12         end
13         if ℒ(AlphaTrans(M, A)) ⊈ ℒ(AlphaTrans(M, B)) then
14             return false;
15         end
16     end
17     return true;
```

differences by extending the existing SFA inclusion algorithm to incorporate a subtyping check between pure refinement types.

### Candidate Minterms

The first step in checking SFA inclusion is to construct the equivalence classes used to finitize its character set. With HATs, these characters range over events triggered by the use of library operations $\mathtt{op}\,\overline{v} = v$, and return events $\mathtt{ret}\,v$. As each class of events is already disjoint (e.g., **get** and **put** events will never overlap), we only need to build minterms that partition individual classes. The minterms of each class of events have the form $\langle \mathtt{op}\,\overline{x_i} = \nu \mid (\bigwedge \overline{l_j}) \wedge (\bigwedge \overline{\neg l_k}) \rangle$. To build the literals used in these minterms for an event $\mathtt{op}$, we collect all the literals $\bar{l}$ used to qualify $\mathtt{op}$, as well as any literals appearing in atomic predicates $\langle \phi \rangle$, and then construct a maximal set of satisfiable Boolean combinations of those literals.

Notably, these literals may contain variables bound in the current typing context. Our algorithm divides minterms into two categories: Boolean combinations of literals from the typing context (line 3), $\phi_\Gamma$, and Boolean combinations of literals appearing from the input

automata (line 7). The final set of candidate minterms are (satisfiable) conjunctions of the elements of $\phi_\Gamma$ and $\phi$ (line 9).

### Satisfiability Check

The soundness of the alphabet transfer procedure requires that *only* satisfiable minterms are used, so that every transition in the output FA corresponds to an actual transition in the original SFA.

To understand how this is done, observe that the typing context plays a similar role in checking both the satisfiability of minterms and the subtyping relation. Thus, the algorithm reduces checking the satisfiability of $\langle \mathtt{op}\,\overline{x_i}{=}\nu \mid (\bigwedge \overline{l_j}) \wedge (\bigwedge \overline{\neg l_k})\rangle$ to checking whether the refinement type $\{\nu{:}b \mid (\bigwedge \overline{l_j}) \wedge (\bigwedge \overline{\neg l_k})\}$ is *not* a subtype of $\{\nu{:}b \mid \bot\}$ (line 8), that is $\Gamma, \overline{x_i{:}b_i} \not\vdash \{\nu{:}b \mid (\bigwedge \overline{l_j}) \wedge (\bigwedge \overline{\neg l_k})\} <: \{\nu{:}b \mid \bot\}$. Since $\{\nu{:}b \mid \bot\}$ is uninhabited, this subtype check fails precisely when $(\bigwedge \overline{l_j}) \wedge (\bigwedge \overline{\neg l_k})$ is satisfiable under $\Gamma$.

### Inclusion Check

Equipped with the set of satisfiable minterms, the final inclusion check (line 10) is a straightforward application of the textbook inclusion check between the FA produced by the standard alphabet transformation algorithm (line 10). `SFAInclusion` returns `true` only when automata $A$ is included by $B$ under all instantiations of the variables in $\Gamma$.

**Algorithm 9:** Alphabet Transformation

```
1  Procedure AlphaTrans(M, A) :=
2      match A:
3          case ∅ or ϵ or • do return A;
4          case ⟨op x̄_i = ν | ϕ⟩ or ⟨ϕ⟩ do
5              M' ← {⟨op x̄_i = ν | ϕ'⟩ ∈ M | ϕ' ⟹ ϕ};
6              return ⋁_{m∈M'} Hash(m)
7          end
8          case A_1; A_2 do return AlphaTrans(M, A_1); AlphaTrans(M, A_2);
9          case A_1 ∨ A_2 do return AlphaTrans(M, A_1) ∨ AlphaTrans(M, A_2);
10         case A_1 ∧ A_2 do return AlphaTrans(M, A_1) ∧ AlphaTrans(M, A_2);
11         case A_1 \ A_2 do return AlphaTrans(M, A_1) \ AlphaTrans(M, A_2);
12         case A* do return AlphaTrans(M, A_1)*;
```

### Alphabet Transformation

As shown in Algorithm 9, the alphabet transformation function `AlphaTrans` translates a SFA into a FA that shares the same regular operators. Notably, the alphabet transformation for effect events gathers all minterms that can imply the qualifier of the effect event (line 17); moreover, the effect event $\langle \mathtt{op}\, \overline{x_i} = \nu \mid \phi \rangle$ requires the gathered minterms have the same operator (`op`).

### 4.4.2 Bidirectional Type System

As is standard, our bidirectional type system shown in Fig. 4.11 and Fig. 4.12 consists of both type synthesis ( $\Rightarrow$ ) and type checking ( $\Leftarrow$ ) judgments. The bidirectional system features a minor divergence from the declarative rules. While the declarative system was able to use the subsumption rule (TSub) to freely instantiate ghost variables, a typing algorithm requires a more deterministic strategy. Our solution is to treat ghost variables $x{:}b \dashrightarrow$ as instead being qualified by an unknown formula which is specialized as needed, based on the information in the current typing context. In order to efficiently infer the qualifiers of ghost variables, our algorithm only allows ghost variables to appear in SFAs. This restriction ensures that ghost variables are only used during SFA inclusion checks, and allows it to avoid the more sophisticated algorithms needed when ghost variables can appear in arbitrary refinement types [27]. This restriction is enforced by our algorithmic typing rules: the second line of ChkEOpApp ($\forall j.\Gamma \vdash v_j \Leftarrow t_j$) for example, only holds when no free ghost variables appear in the parameter types $t_j$ used by $\rightsquigarrow$ ($\Gamma \vdash [A]\ \overline{z_k{:}b_k} \dashrightarrow \overline{y_j{:}t_j} \to \tau_x \rightsquigarrow \overline{z_k{:}t_k}$).

With this minor tweak in hand, the top-level typing algorithm is mostly a matter of bidirectionalizing the typing rules of $\boldsymbol{\lambda}^U$ by choosing appropriate modes for the assumptions of each rule. This is largely straightforward: the type checking rule for ghost variables (ChkGhost) adapts the corresponding declarative rule (TGhost) in the standard way, for example. The rule for checking effectful operations (ChkEOpApp) is similar, although it adopts a fixed strategy for applying the subsumption rule: 1) instead of using TSub to instantiate ghost variables directly, it now relies on an auxiliary function, $\rightsquigarrow$, to strengthen their qualifiers based on the current context; 2) when the result of the library operation `op`

**Type Synthesis** $\boxed{\Gamma \vdash e \Rightarrow t \quad \Gamma \vdash e \Rightarrow \tau}$ **Type Check** $\boxed{\Gamma \vdash e \Leftarrow t \quad \Gamma \vdash e \Leftarrow \tau}$

$$\frac{\emptyset \vdash^{\mathbf{WF}} \{\nu{:}b \mid \nu = c\}}{\Gamma \vdash [A]\ c \Rightarrow \{\nu{:}b \mid \nu = c\}}\ \textsc{SynConst} \qquad \frac{\Gamma \vdash^{\mathbf{WF}} \{\nu{:}b \mid \nu = x\} \quad (x, \{\nu{:}b \mid \phi\}) \in \Gamma}{\Gamma \vdash [A]\ x \Rightarrow \{\nu{:}b \mid \nu = x\}}\ \textsc{SynBaseVar}$$

$$\frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} \\ \overline{y{:}b} \dashrightarrow z{:}t \to \tau \\ (x, \overline{y{:}b} \dashrightarrow z{:}t \to \tau) \in \\ \Gamma\end{array}}{\Gamma \vdash [A]\ x \Rightarrow \overline{y{:}b} \dashrightarrow z{:}t \to \tau}\ \textsc{SynFuncVar} \qquad \frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} x{:}b \dashrightarrow \tau \\ \Gamma, x{:}\{\nu{:}b \mid \top\} \vdash \\ e \Leftarrow \tau\end{array}}{\Gamma \vdash e \Leftarrow x{:}b \dashrightarrow \tau}\ \textsc{ChkGhost}$$

$$\frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} \tau_1 \sqcap \tau_2 \\ \Gamma \vdash e \Leftarrow \tau_1 \\ \Gamma \vdash e \Leftarrow \tau_2\end{array}}{\Gamma \vdash e \Leftarrow \tau_1 \sqcap \tau_2}\ \textsc{ChkInter}$$

$$\frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} [A]\, t\, [B] \\ \Gamma \vdash v_x \Rightarrow t_x \\ \Gamma, x{:}t_x \vdash \\ e \Leftarrow [A]\, t\, [B]\end{array}}{\Gamma \vdash \texttt{let}\ x = v_x\ \texttt{in}\ e \Leftarrow [A]\, t\, [B]}\ \textsc{ChkLetV} \qquad \frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} [A]\, t\, [B] \\ \Gamma \vdash e_x \Rightarrow \bigsqcap [A_i]\, t_i\, [A'_i] \\ \Gamma \vdash A \subseteq \bigvee A_i \\ \forall i.\Gamma, x{:}t_i \vdash \\ e \Leftarrow [(A; \Box\langle\top\rangle) \wedge A'_i]\, t\, [B]\end{array}}{\Gamma \vdash \texttt{let}\ x = e_x\ \texttt{in}\ e \Leftarrow [A]\, t\, [B]}\ \textsc{ChkLetE}$$

$$\frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} [A]\, t\, [B] \\ \Delta(op) = \overline{y_j{:}t_j} \to t_x \\ \forall j.\Gamma \vdash v_j \Leftarrow t_j \\ \Gamma, x{:}t_x \vdash \\ e \Leftarrow [A]\, t\, [B]\end{array}}{\Gamma \vdash \texttt{let}\ x = \texttt{op}\ \overline{v_i}\ \texttt{in}\ e \Leftarrow [A]\, t\, [B]}\ \textsc{ChkPOpApp}$$

$$\frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} [A]\, t\, [B] \\ \Delta(\texttt{op}) = \overline{z_k{:}b_k} \dashrightarrow \overline{y_j{:}t_j} \to \tau_x \\ \forall j.\Gamma \vdash v_j \Leftarrow t_j \\ \Gamma \vdash [A]\ \overline{z_k{:}b_k} \dashrightarrow \overline{y_j{:}t_j} \to \tau_x \rightsquigarrow \overline{z_k{:}t_k} \\ \bigsqcap [A_i]\, t_i\, [A'_i] = \tau_x[y_j \mapsto v_j] \\ \forall i.\Gamma, \overline{z_k{:}t_k}, x{:}t_i \vdash \\ e \Leftarrow [(A; \Box\langle\top\rangle) \wedge A'_i]\, t\, [B]\end{array}}{\Gamma \vdash \texttt{let}\ x = \texttt{op}\ \overline{v_i}\ \texttt{in}\ e \Leftarrow [A]\, t\, [B]}\ \textsc{ChkEOpApp}$$

**Figure 4.11.** Bidirectional Typing Rules (Part 1).

$$\frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} [A]\, t\, [B] \\ \Gamma \vdash v_1 \;\Rightarrow\; \overline{z_k{:}b_k} \dashrightarrow y{:}t_y \to \tau_x \\ \Gamma \vdash v_2 \;\Leftarrow\; t_y \\ \Gamma \vdash [A]\; \overline{z_k{:}b_k} \dashrightarrow y{:}t_y \to \tau_x \;\rightsquigarrow\; \overline{z_k{:}t_k} \\ \bigsqcap \overline{[A_i]\, t_i\, [A'_i]} = \tau_x[y \mapsto v_y] \\ \forall i.\Gamma, \overline{z_k{:}t_k}, x{:}t_i \vdash \\ e \;\Leftarrow\; [(A; \Box\langle\top\rangle) \wedge A'_i]\, t\, [B]\end{array}}{\Gamma \vdash \texttt{let}\; x = v_1\; v_2\; \texttt{in}\; e \;\Leftarrow\; [A]\, t\, [B]}\;\textsc{ChkApp}$$

$$\frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} x{:}t'_x \to \tau \\ \Gamma \vdash t'_x <: t_x \\ \Gamma, x{:}t'_x \vdash e \;\Leftarrow\; \tau\end{array}}{\Gamma \vdash \lambda x{:}t_x.e \;\Leftarrow\; x{:}t'_x \to \tau}\;\textsc{ChkFunc}$$

$$\frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} t'_f \\ \Gamma \vdash t'_f <: t_f \\ \Gamma, f{:}t'_f \vdash \\ \lambda x{:}t_x.e \;\Leftarrow\; t'_f\end{array}}{\Gamma \vdash \texttt{fix} f{:}t_f.\lambda x{:}t_x.e \;\Leftarrow\; t'_f}\;\textsc{ChkFix}$$

$$\frac{\begin{array}{c}\forall i.(d_i, \overline{y_j{:}t_j} \to \{\nu{:}b \mid \phi_i\}) \in \Delta \\ \Gamma, \overline{y_j{:}t_j}, z{:}\{\nu{:}b \mid \nu = v \wedge \phi_i\} \vdash e_i \;\Leftarrow\; \tau \\ \text{where } z \text{ is fresh}\end{array}}{\Gamma \vdash \texttt{match}\; v\; \texttt{with}\; \overline{d_i\, \overline{y_j} \to e_i} \;\Leftarrow\; \tau}\;\textsc{ChkMatch}$$

$$\frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} [A_2]\, t_2\, [B_2] \\ \Gamma \vdash e \;\Rightarrow\; [A_1]\, t_1\, [B_1] \\ \Gamma \vdash A_2 <: A_1 \quad \Gamma \vdash t_1 <: t_2 \\ \Gamma \vdash (A_2; \Box\langle\top\rangle) \wedge \\ (A_2; \Box\langle\top\rangle) \wedge B_1 \subseteq B_2\end{array}}{\Gamma \vdash e \;\Leftarrow\; [A_2]\, t_2\, [B_2]}\;\textsc{ChkSub}$$

$$\frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} t \\ \Gamma \vdash v \;\Rightarrow\; t' \\ \Gamma \vdash t' <: t\end{array}}{\Gamma \vdash v \;\Leftarrow\; t}\;\textsc{ChkSubV}$$

**Figure 4.12.** Bidirectional Typing Rules (Part 2).

is an intersection type ($\bigsqcap \overline{[A_i]\, t_i\, [A'_i]}$), ChkEOpApp considers each case of the intersection, instead of using TSub to focus on a single case; and 3) instead of relying on TSub to align the postcondition automata ($A'_i$) of `op` with the precondition of the HAT being checked against ($A$), the rule uses the conjunction of the automata ($(A; \Box\langle\top\rangle) \wedge A'_i$) as the precondition automata used to check the term that uses the result of `op`.

$$\frac{\begin{array}{c}\Gamma = \overline{x_i{:}\{\nu{:}b_i \mid \phi_i\}}, \overline{y{:}(\overline{u{:}b_u} \dashrightarrow z{:}t \to \tau)} \\ \forall \overline{x_i{:}b_i}.\forall \nu{:}b.\, \bigwedge \phi_i \Longrightarrow (\phi_1 \Longrightarrow \phi_2)\end{array}}{\Gamma \vdash \{\nu{:}b \mid \phi_1\} <: \{\nu{:}b \mid \phi_2\}}\;\textsc{SubBaseAlg}$$

$$\frac{\begin{array}{c}\bigsqcap \overline{[A_i]\, t_i\, [A'_i]} = \tau \quad \Gamma' = \Gamma, \overline{y_j{:}t_j} \\ \texttt{Abduce}(\Gamma, A, \bigvee A_i, \overline{z_k{:}b_k}) = \overline{z_k{:}t_k}\end{array}}{\Gamma \vdash [A]\; \overline{z_k{:}b_k} \dashrightarrow \overline{y_j{:}t_j} \to \tau \;\rightsquigarrow\; \overline{z_k{:}t_k}}\;\textsc{InstAlg}$$

**Figure 4.13.** Auxiliary Typing Functions

As shown in Fig. 4.13, The implementation of $\rightsquigarrow$ relies on the `Abduce` subroutine. Given a typing context $\Gamma$, set of ghost variables $\overline{x{:}b}$, and a pair of automata $A$ and $A'$, `Abduce` infers a set of qualifiers $\overline{\phi}$ for $\overline{x{:}b}$ sufficient enough to ensure that $A$ is included in $A'$, i.e. $\Gamma, \overline{x{:}\{\nu{:}b \mid \phi\}} \vdash A \subseteq A'$, or reports that none exists. `Abduce` is an adaption of an existing algorithm [72] which infers the *weakest* qualifiers $\phi_i$ needed for the inclusion check.

**Algorithm 10:** Ghost Variable Inference

```
1  Procedure Abduce(Γ, A, A', x_i:b_i) :=
2      if Γ, x_i:{ν:b_i | ⊤} ⊭ A ⊆ A' then return ⊥ ;
3      L ← GetLits(Γ, A ∨ B);
4      M^? ← ⋃_i BoolComb(L ∪ x_i);
5      M^+ ← ∅; M^- ← ∅; φ ← ⊤;
6      while ∃m ∈ M^?.m ⟹ φ do
7          M^? ← M^? \ {m};
8          φ' ← Synthesize(M^+ ∪ M^?, M^- ∪ {m});
9          if Γ, x_i:{ν:b_i | ⊤}, z:{ν:unit | φ'} ⊭ A ⊆ A' then
10             M^- ← M^- ∪ {m};
11             φ ← φ';
12         else M^+ ← M^+ ∪ {m} ;
13     end
14     foreach x_i:b_i do
15         M ← {m ∈ BoolComb(L_Γ ∪ x_i) | m ⟹ φ};
16         t_i ← {ν:b_i | ⋁_{m∈M} m[x_i ↦ ν]};
17     end
18     return x_i:t_i;
```

### Ghost Variable Instantiatation

As we show in Algorithm 10, the abductive synthesizer `Abduce` adapts an existing algorithm [72] that synthesizes a *maximal* assignment of unknown qualifier $\phi_i$ for the given ghost variables $x_i{:}b_i$. These inferred qualifiers are strong enough to prove the given inclusion relation between two input automata under the given type context (i.e., $\Gamma \vdash A \subseteq A'$). Note that the ghost variables only make sense when the inclusion relation holds with all unknown formulae $\phi_i$ are maximally weak (i.e., true) (line 2). Similar to the minterm algorithm in Algorithm 9, the abductive algorithm gathers all literals that appear in the pre- and post-condition automata and type context (line 3) that will be used to generate candidate

minterms (feature vectors, in the literature of abductive synthesis [72]), to build the hypothesis space for synthesis. The function `Abduce` uses a counterexample-guided refinement loop (line 6 - 12) adapted from the existing algorithm [72] manipulating three sets of minterms: unknown minterms $M^?$, positive minterms $M^+$, and negative minterms $M^-$. Thanks to the multi-abduction approach from the existing algorithm [72], inferring a formula $\phi$ from the union of all minterms induces all the assignment of unknown qualifiers $\overline{\phi_i}$ at one shot. The auxiliary synthesizer `Synthesize` (line 8) returns a formula that can be implied by given positive minterm and unknown minterms but cannot be implied by any given negative minterms. Following the existing algorithm [72], function `Synthesize` is implemented as a Decision Tree (DT) that classifies these two sets of minterms. On the other hand, the inclusion query on line 9 decides if unknown minterms should be positive or negative; it also guarantees the soundness of the algorithm, that is, a synthesized type is always consistent with the given automaton $A$ and $A'$. Finally, the function `Abduce` returns refinement type $\{\nu{:}b_i \mid \phi_i\}$ for each ghost variable $x_i$, where $\phi_i$ is the union of all local minterms that can imply $\phi$ (line 14 - 15).

### 4.4.3 Soundness and Decidability of Typing Algorithm

We introduce the following lemmas about the `SFAInclusion` subroutine and the instantiation function `Abduce` for the soundness theorem of our typing algorithm.

**Lemma 4.4.1.** *[Soundness of Inclusion Algorithm] Given type context $\Gamma$, automaton $A$ and $A'$,*

$$\texttt{SFAInclusion}(\Gamma, A, A') \Longrightarrow \Gamma \vdash A \subseteq A'$$

*Proof.* By routine induction on $A$.

**Lemma 4.4.2.** *[Soundness of Instantiation Function] Given a type context $\Gamma$, ghost variables $\overline{x_i{:}b_i}$, automaton $A$ and $A'$,*

$$\texttt{Abduce}(\Gamma, A, A', \overline{x_i{:}b_i}) \implies \Gamma, \overline{x_i{:}\{\nu{:}b_i \mid \phi_i\}} \vdash A \subseteq A'$$

*Proof.* By routine induction on $A$.

The soundness of SFAInclusionestablishes that it is safe to treat the subtyping relation in our typing algorithm as in declarative type system. On the other hand, the soundness of instantiation function induces the following lemma.

**Lemma 4.4.3.** *[Instantiation Implies Precondition Automata Inclusion] Given a type context* $\Gamma$*, term* $e$*,*

$$\Gamma \vdash [A]\ \overline{z_k{:}b_k} \dashrightarrow \overline{y_j{:}t_j} \to \overline{[A_i]\, t_i\, [A'_i]} \approx \overline{z_k{:}t_k} \implies \Gamma, \overline{z_k{:}t_k} \vdash A \subseteq \bigvee A_i \wedge$$
$$\Gamma, \overline{z_k{:}t_k} \vdash \overline{z_k{:}b_k} \dashrightarrow \overline{y_j{:}t_j} \to \overline{[A_i]\, t_i\, [A'_i]} <: \overline{y_j{:}t_j} \to \overline{[A_i]\, t_i\, [A'_i]}$$

*Proof.* By routine induction on $A$.

We need three auxiliary lemmas about declarative typing, which establish that we can drop unused binding from the type context, well-typed substitution preserves typing judgement, and well-formedness implies subtyping.

**Lemma 4.4.4.** *[Strengthening] Given a type context* $\Gamma$*, term* $e$ *and HAT* $\tau$*, for any free variable* $x$ *and pure refinement type* $t$*:* $\Gamma, x{:}t \vdash e : \tau \wedge \Gamma \vdash^{\boldsymbol{WF}} \tau \implies \Gamma \vdash e : \tau$

*Proof.* By routine induction on type judgement $\Gamma \vdash e : \tau$.

**Lemma 4.4.5.** *[Substitution Preserves Typing] Given a type context* $\Gamma$*, term* $e$ *and HAT* $\tau$*, for any free variable* $x$ *and pure refinement type* $t$*:* $\Gamma, x{:}t \vdash e : \tau \wedge \Gamma \vdash v : t \implies \Gamma \vdash e : \tau[x \mapsto v]$

*Proof.* By routine induction on type judgement $\Gamma \vdash e : \tau$.

**Lemma 4.4.6.** *[Well-formedness implies Subtyping] Given a type context* $\Gamma$*, term* $e$ *and HAT* $[A]\, t\, [B]$*, we have* $\Gamma \vdash^{\boldsymbol{WF}} [A]\, t\, [B] \implies \Gamma \vdash [A]\, t\, [B] <: [A \wedge A']\, t\, [B \wedge (A'; \Box\langle\top\rangle)]$*.*

*Proof.* Consider the denotation of HAT $[A]\, t\, [B]$ and $[A \wedge A']\, t\, [B \wedge (A'; \Box\langle\top\rangle)]$:

$$[\![[A]\, t\, [B]]\!] = \{e \mid \emptyset \vdash_{\mathsf{s}} e : \lfloor t \rfloor \wedge \forall \alpha\, \alpha'\, v.\ \alpha \in \mathcal{L}(A) \wedge \alpha \vDash e \xrightarrow{\alpha'}{}^{*} v \implies v \in [\![t]\!] \wedge \alpha +\!\!+ \alpha' \in \mathcal{L}(B)\}$$
$$[\![[A \wedge A']\, t\, [B \wedge (A'; \Box\langle\top\rangle)]]\!] = \{e \mid \emptyset \vdash_{\mathsf{s}} e : \lfloor t \rfloor \wedge \forall \alpha\, \alpha'\, v.\ \alpha \in \mathcal{L}(A \wedge A') \wedge \alpha \vDash e \xrightarrow{\alpha'}{}^{*} v \implies$$
$$v \in [\![t]\!] \wedge \alpha +\!\!+ \alpha' \in \mathcal{L}(B \wedge (A'; \Box\langle\top\rangle))\}$$

Consider a trace $\alpha \in \mathcal{L}(A \wedge A')$, it also can be recognized by automata $A$, thus for any $\alpha \vDash e \xhookrightarrow{\alpha'}{}^{*} v$, we have $\alpha \# \alpha' \in \mathcal{L}(B)$. Moreover, the trace $\alpha \# \alpha'$ has a prefix $\alpha$, thus is naturally recognized by automata $(A \wedge A'); \Box\langle\top\rangle$. Thus, we know $\alpha \# \alpha' \in \mathcal{L}((A \wedge A'); \Box\langle\top\rangle \wedge B)$. Note that the rule WFHoare requires that precondition is always the "prefix" of the postcondition automata, thus $\Gamma \vdash^{\mathbf{WF}} [A]\, t\, [B]$ implies $(A; \Box\langle\top\rangle) \subseteq B$, which means that $\mathcal{L}((A \wedge A'); \Box\langle\top\rangle \wedge B) = \mathcal{L}(A'; \Box\langle\top\rangle \wedge B)$. Thus the term $e$ is in the denotation of type $[A \wedge A']\, t\, [B \wedge (A'; \Box\langle\top\rangle)]$ when it is in the denotation of type $[A]\, t\, [B]$.

Now we can prove the soundness theorem of our typing algorithm with respect to our declarative type system.

**Theorem 4.4.7.** *[Soundness of Algorithmic Typing] Given a well-formed built-in typing context $\Delta$, type context $\Gamma$, term $e$, and HAT $\tau$, $\Gamma \vdash e \Leftarrow \tau \implies \Gamma \vdash e : \tau$.*

*Proof.* As the type synthesis rules are defined mutually recursively, we simultaneously prove both are correct. The following theorem states a stronger inductive invariant that is sufficient to prove the soundness of the typing algorithm.

**Theorem 4.4.8.** *[Soundness of the type synthesis and type check algorithm] For all type context $\Gamma$, term $e$, pure refinement type $t$ and HAT $\tau$,*

$$\Gamma \vdash e \Leftarrow \tau \implies \Gamma \vdash e : \tau$$

$$\Gamma \vdash e \Leftarrow t \implies \Gamma \vdash e : t$$

$$\Gamma \vdash e \Rightarrow \tau \implies \Gamma \vdash e : \tau$$

$$\Gamma \vdash e \Rightarrow t \implies \Gamma \vdash e : t$$

We proceed by induction over the mutual recursive structure of type synthesis ( $\Rightarrow$ ) and type check ( $\Leftarrow$ ) rules. In the cases for; synthesis and checking rules of rule SynConst, SynBaseVar, SynFuncVar, ChkGhost, ChkInter, ChkFunc, ChkFix, ChkMatch, the typing rules in Fig. 4.9 align exactly with these rules. There are seven remaining cases that are about let-bindings (ChkLetV and ChkLetE), application (ChkApp, ChkPOpApp, and ChkEOpApp), and subtyping (ChkSubV and ChkSub). The rule

ChkLetV, ChkPOpApp, and ChkSubV are less interesting, which are similar with standard refinement typing algorithm[22] and simply drop the pre- and post-condition automata from their effectful version. Consequently, we show the proof of three interesting cases (ChkSub, ChkLetE, ChkEOpApp) below, where the case ChkApp is almost the same as ChkEOpApp.

Case ChkSub:

$$\frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} [A_2]\, t_2\, [B_2] \\ \Gamma \vdash e \;\Rightarrow\; [A_1]\, t_1\, [B_1] \\ \Gamma \vdash A_2 <: A_1 \quad \Gamma \vdash t_1 <: t_2 \\ \Gamma \vdash (A_2; \Box\langle\top\rangle) \wedge B_1 \subseteq \\ (A_2; \Box\langle\top\rangle) \wedge B_2\end{array}}{\Gamma \vdash e \;\Leftarrow\; [A_2]\, t_2\, [B_2]}\ \textsc{ChkSub}$$

This rule connects the relation between the type synthesis and type check judgement. From the induction hypothesis and the precondition of ChkSub, we know

| | | |
|---|---|---|
| $\Gamma \vdash A_2 <: A_1$ | from precondition | (4.1) |
| $\Gamma \vdash t_1 <: t_2$ | from precondition | (4.2) |
| $\Gamma \vdash (A_2; \Box\langle\top\rangle) \wedge B_1 \subseteq (A_2; \Box\langle\top\rangle) \wedge B_2$ | from precondition | (4.3) |
| $\Gamma \vdash [A_1]\, t_1\, [B_1] <: [A_2]\, t_2\, [B_2]$ | via rule SubHoare and (1) (2) (3) | (4.4) |
| $\Gamma \vdash e : [A_1]\, t_1\, [B_1]$ | inductive hypothesis | (4.5) |
| $\Gamma \vdash e : [A_2]\, t_2\, [B_2]$ | via rule TSub and (4) (5) | (4.6) |

Then the proof immediate holds in this case.

Case ChkLetE:

$$\frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} [A]\, t\, [B] \\ \Gamma \vdash e_x \;\Rightarrow\; \bigsqcap \overline{[A_i]\, t_i\, [A'_i]} \\ \Gamma \vdash A \subseteq \bigvee A_i \\ \forall i. \Gamma, x{:}t_i \vdash \\ e \;\Leftarrow\; [(A; \Box\langle\top\rangle) \wedge A'_i]\, t\, [B]\end{array}}{\Gamma \vdash \texttt{let}\ x = e_x\ \texttt{in}\ e \;\Leftarrow\; [A]\, t\, [B]}\ \textsc{ChkLetE}$$

This rule can be treated as a combination of subtyping of intersection types (e.g., SubIntLL) and TLet. From the induction hypothesis and the precondition of ChkApp, we know

| | | |
|---|---|---|
| $\Gamma \vdash e_x : \bigsqcap [A_i]\, t\, [A'_i]$ | inductive hypothesis | (4.1) |
| $\forall i.\Gamma \vdash e_x : \overline{[A_i]\, t_i\, [A'_i]}$ | via rule TSub, rule SubIntLL and (1) | (4.2) |
| $\forall i.\Gamma \vdash e_x : \overline{[A \wedge A_i]\, t_i\, [(A; \Box\langle\top\rangle) \wedge A'_i]}$ | via Lemma 4.4.6, precondition, and (2) | (4.3) |
| $\forall i.\Gamma, x{:}t_i \vdash e : [(A; \Box\langle\top\rangle) \wedge A'_i]\, t\, [B]$ | inductive hypothesis | (4.4) |
| $\forall i.\Gamma \vdash \texttt{let}\ x = e_x\ \texttt{in}\ e : [A \wedge A'_i]\, t\, [B]$ | rule TLet and (3) (4) | (4.5) |
| $\Gamma \vdash \texttt{let}\ x = e_x\ \texttt{in}\ e : \bigsqcap [A \wedge A_i]\, t\, [B]$ | rule TInter and (5) | (4.6) |
| $\Gamma \vdash \texttt{let}\ x = e_x\ \texttt{in}\ e : [A \wedge \bigvee A_i]\, t\, [B]$ | via rule TSub, rule SubIntMerge and (6) | (4.7) |
| $\Gamma \vdash A \subseteq \bigvee A_i$ | precondition | (4.8) |
| $\Gamma \vdash \texttt{let}\ x = e_x\ \texttt{in}\ e : [A]\, t\, [B]$ | via rule TSub, rule SubHoare and (7) (8) | (4.9) |

which is exactly what we needed to prove for this case.

Case ChkEOpApp:

$$\frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} [A]\, t\, [B] \\ \Delta(\texttt{op}) = \overline{z_k{:}b_k} \dashrightarrow \overline{y_j{:}t_j} \to \tau_x \\ \forall j.\Gamma \vdash v_j \Leftarrow t_j \\ \Gamma \vdash [A]\ \overline{z_k{:}b_k} \dashrightarrow \overline{y_j{:}t_j} \to \tau_x \rightsquigarrow \overline{z_k{:}t_k} \\ \bigsqcap \overline{[A_i]\, t_i\, [A'_i]} = \tau_x[\overline{y_j \mapsto v_j}] \\ \forall i.\Gamma, \overline{z_k{:}t_k}, x{:}t_i \vdash \\ e \Leftarrow [(A; \Box\langle\top\rangle) \wedge A'_i]\, t\, [B]\end{array}}{\Gamma \vdash \texttt{let}\ x = \texttt{op}\ \overline{v_i}\ \texttt{in}\ e \Leftarrow [A]\, t\, [B]}\ \text{ChkEOpApp}$$

This rule can be treated as a combination of TSub and TEffOp. From the induction hypothesis and the precondition of ChkEffOp, we know

| | | |
|---|---|---|
| $\Delta(\mathtt{op}) = \overline{z_k{:}b_k} \dashrightarrow \overline{y_j{:}t_j} \to \tau_x$ | precondition | (4.1) |
| $\Gamma \vdash \mathtt{op} : \overline{z_k{:}b_k} \dashrightarrow \overline{y_j{:}t_j} \to \tau_x$ | via rule TEOp and (1) | (4.2) |
| $\forall j.\Gamma \vdash v_j \Leftarrow t_j$ | precondition | (4.3) |
| $\Gamma \vdash \mathtt{op} : \overline{z_k{:}b_k} \dashrightarrow \overline{y_j{:}t_j} \to \tau_x\overline{[y_j \mapsto v_j]}$ | Lemma 4.4.5 and (2) (3) | (4.4) |
| $\Gamma \vdash \mathtt{op} : \overline{z_k{:}b_k} \dashrightarrow \overline{y_j{:}t_j} \to \bigsqcap\overline{[A_i]\, t_i\, [A'_i]}$ | precondition and (4) | (4.5) |
| $\Gamma \vdash [A]\ \overline{z_k{:}b_k} \dashrightarrow \overline{y_j{:}t_j} \to \tau_x \rightsquigarrow \overline{z_k{:}t_k}$ | precondition | (4.6) |
| $\Gamma, \overline{z_k{:}t_k} \vdash A \subseteq \bigvee A_i$ | via Lemma 4.4.3 | (4.7) |
| $\Gamma, \overline{z_k{:}t_k} \vdash \mathtt{op} : \overline{y_j{:}t_j} \to \bigsqcap\overline{[A_i]\, t_i\, [A'_i]}$ | via Lemma 4.4.3 and rule TSub | (4.8) |

Now the operator op has the return type $\bigsqcap\overline{[A_i]\, t_i\, [A'_i]}$. The rest of proof it almost the same as the proof in the case ChkLetE:

| | | |
|---|---|---|
| $\forall i.\Gamma, \overline{z_k{:}t_k} \vdash \mathtt{op} : \overline{[A_i]\, t_i\, [A'_i]}$ | via rule TSub, rule SubIntLL and (8) | (4.9) |
| $\forall i.\Gamma, \overline{z_k{:}t_k} \vdash \mathtt{op} : \overline{[A \wedge A_i]\, t_i\, [(A; \Box\langle\top\rangle) \wedge A'_i]}$ | via Lemma 4.4.6, precondition, and (9) | (4.10) |
| $\forall i.\Gamma, \overline{z_k{:}t_k}, x{:}t_i \vdash e : [(A; \Box\langle\top\rangle) \wedge A'_i]\, t\, [B]$ | inductive hypothesis | (4.11) |
| $\forall i.\Gamma, \overline{z_k{:}t_k} \vdash \mathtt{let}\ x = \mathtt{op}\ \overline{v_i}\ \mathtt{in}\ e : [A \wedge A'_i]\, t\, [B]$ | rule TEOpApp and (10) (11) | (4.12) |
| $\Gamma, \overline{z_k{:}t_k} \vdash \mathtt{let}\ x = \mathtt{op}\ \overline{v_i}\ \mathtt{in}\ e : \bigsqcap[A \wedge A_i]\, t\, [B]$ | rule TInter and (12) | (4.13) |
| $\Gamma, \overline{z_k{:}t_k} \vdash \mathtt{let}\ x = \mathtt{op}\ \overline{v_i}\ \mathtt{in}\ e : [A \wedge \bigvee A_i]\, t\, [B]$ | via rule TSub, rule SubIntMerge and (13) | (4.14) |
| $\Gamma, \overline{z_k{:}t_k} \vdash \mathtt{let}\ x = \mathtt{op}\ \overline{v_i}\ \mathtt{in}\ e : [A]\, t\, [B]$ | via rule TSub, rule SubHoare and (7) (14) | (4.15) |
| $\Gamma \vdash \mathtt{let}\ x = \mathtt{op}\ \overline{v_i}\ \mathtt{in}\ e : [A]\, t\, [B]$ | via Lemma 4.4.4, precondition and (15) | (4.16) |

which are valid.

The following lemmas about the `SFAInclusion` subroutine and the instantiation function `Abduce` are required by the proof of decidability of algorithmic typing.

**Lemma 4.4.9.** *[Decidability of Base Subtyping] the* $\Gamma \vdash \{\nu{:}b \mid \phi_1\} <: \{\nu{:}b \mid \phi_2\}$ *is decidable.*

*Proof.* As we mentioned in Section 5.1, similar to standard refinement type system, the subtyping check will be encoded as a query in EPR, which is in the decidable fragment of FOL. Thus, the base subtyping check is decidable.

**Lemma 4.4.10.** *[Decidability of Inclusion Algorithm] the* `SFAInclusion` *is decidable.*

*Proof.* The gathered literals are finite, thus the induced boolean combination is also finite. Then the loop in `SFAInclusion` (line 3 in Algorithm 8) has a finite bound. According to Lemma 4.4.9, `SFAInclusion` is decidable.

**Lemma 4.4.11.** *[Decidability of Instantiation Algorithm] the* `Abduce` *is decidable.*

*Proof.* The gathered literals are finite, thus the induced boolean combination is also finite. Then the loop in `Abduce` (line 6 in Algorithm 8) has a finite bound. According to Lemma 4.4.9, `Abduce` is decidable.

**Lemma 4.4.12.** *[Decidability of Well-Formedness] For all type context* $\Gamma$*, type* $t$*, and* $\tau$*, the* $\Gamma \vdash^{\boldsymbol{WF}} t$ *and* $\Gamma \vdash^{\boldsymbol{WF}} \tau$ *is decidable.*

*Proof.* According to the well-formedness rules shown in Fig. 4.7, these rules are inductively defined and structural decreasing over the types ($t_1$ or $\tau_1$). Thus, they always terminate.

**Lemma 4.4.13.** *[Decidability of Subtyping] For all type context* $\Gamma$*, type* $t_1$*,* $t_2$*,* $\tau_1$*, and* $\tau_2$ *, the* $\Gamma \vdash t_1 <: t_2$ *and* $\Gamma \vdash \tau_1 <: \tau_2$ *is decidable.*

*Proof.* According to the subtyping rules shown in Fig. 4.8, Lemma Theorem 4.4.9 and Lemma Theorem 4.4.10, these rules are inductively defined and structural decreasing over the types ($t_1$ or $\tau_1$). Thus, they always terminate.

**Lemma 4.4.14.** *[Decidability of Instantiation] For all type context* $\Gamma$ *and automata* $A$*,* $\Gamma \vdash [A]\ \overline{z_k{:}b_k} \dashrightarrow \overline{y_j{:}t_j} \to \tau_x \ \rightsquigarrow \overline{z_k{:}t_k}$ *is decidable.*

*Proof.* According to the rule shown in Fig. 4.8 and Lemma 4.4.11, it always terminates.

Now we can prove the decidability theorem of our typing algorithm.

**Theorem 4.4.15.** *[Decidability of Algorithmic Typing] Type checking a term $e$ against a type $\tau$ in a typing context $\Gamma$, i.e., $\Gamma \vdash e \Leftarrow \tau$, is decidable.*

*Proof.* As the type synthesis rules are defined mutually recursively, we simultaneously prove both are decidable. The following theorem states a stronger inductive invariant that is sufficient to prove the decidability of the typing algorithm.

**Theorem 4.4.16 (Decidability of Algorithmic Typing).** *Type checking a term $e$ against a type $\tau$ in a typing context $\Gamma$, all $\Gamma \vdash e \Leftarrow \tau$, $\Gamma \vdash v \Leftarrow t$, $\Gamma \vdash e \Rightarrow \tau$, and $\Gamma \vdash v \Rightarrow t$ is decidable.*

According to Lemma 4.4.12, Lemma 4.4.13, and Lemma 4.4.14, the auxiliary functions (well-formedness, subtyping, instantiation) always terminate. Thus, the key of the decidability

proof is finding a ranking function over the inputs of typing algorithm, which is based on the size of terms ($\texttt{TermSize}$) and types ($\texttt{TypeSize}$):

$$
\begin{aligned}
\texttt{TermSize}(c) &\doteq 1 \\
\texttt{TermSize}(x) &\doteq 1 \\
\texttt{TermSize}(d(\overline{v_i})) &\doteq 1 + \Sigma_i \texttt{TermSize}(v_i) \\
\texttt{TermSize}(\lambda x{:}t.e) &\doteq 1 + \texttt{TermSize}(e) \\
\texttt{TermSize}(\texttt{fix} f{:}t.\lambda x{:}t.e) &\doteq 2 + \texttt{TermSize}(e) \\
\texttt{TermSize}(\texttt{let}\ x = \texttt{op}\ \overline{v_i}\ \texttt{in}\ e) &\doteq 1 + \texttt{TermSize}(e) + \Sigma_i \texttt{TermSize}(v_i) \\
\texttt{TermSize}(\texttt{let}\ x = op\ \overline{v_1}\ \texttt{in}\ e) &\doteq 1 + \texttt{TermSize}(e) + \Sigma_i \texttt{TermSize}(v_i) \\
\texttt{TermSize}(\texttt{let}\ x = v_1\ v_2\ \texttt{in}\ e) &\doteq 1 + \texttt{TermSize}(e) + \texttt{TermSize}(v_1) + \texttt{TermSize}(v_2) \\
\texttt{TermSize}(\texttt{let}\ x = e_1\ \texttt{in}\ e_2) &\doteq 1 + \texttt{TermSize}(e_1) + \texttt{TermSize}(e_2) \\
\texttt{TermSize}(\texttt{match}\ v\ \texttt{with}\ \overline{d\ \overline{y} \to e_i}) &\doteq 1 + \texttt{TermSize} v + \Sigma_i \texttt{TermSize}(e_i) \\
\texttt{TypeSize}(\{\nu{:}b \mid \phi\}) &\doteq 1 \\
\texttt{TypeSize}(x{:}t \to \tau) &\doteq 1 + \texttt{TypeSize}(t) + \texttt{TypeSize}(\tau) \\
\texttt{TypeSize}(x{:}b \dashrightarrow \tau) &\doteq 1 + \texttt{TypeSize}(\tau) \\
\texttt{TypeSize}([A]\, t\, [A]) &\doteq 1 + \texttt{TypeSize}(t) \\
\texttt{TypeSize}(\tau_1 \sqcap \tau_2) &\doteq 1 + \texttt{TypeSize}(\tau_1) + \texttt{TypeSize}(\tau_2)
\end{aligned}
$$

With the help of $\texttt{TermSize}$, we define a lexicographic ranking function as a string contains two natural numbers (a pair of natural numbers).

$$
\begin{aligned}
\texttt{Rank}(\Gamma \vdash e \ \Leftarrow\ \tau) &\doteq (\texttt{TermSize}(e), \texttt{TypeSize}(\tau)) \\
\texttt{Rank}(\Gamma \vdash e \ \Rightarrow\ \tau) &\doteq (\texttt{TermSize}(e), 0)
\end{aligned}
$$

where one rank is less than another iff

$$
(a_1, b_1) < (a_2, b_2) \iff a_1 < b_1 \vee (a_1 = b_1 \wedge b_1 < b_2)
$$

We proceed by induction over the mutual recursive structure of type synthesis ( $\Rightarrow$ ) and type check ( $\Leftarrow$ ) rules. In the cases for; synthesis rule SynConst, SynBaseVar, and SynFuncVar don't have recursive call, thus always terminates. Consequently, we try to prove that the rank current call is greater than recursive calls in the rest of cases below.

Case ChkGhost:

$$\frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} x{:}b \dashrightarrow \tau \\ \Gamma, x{:}\{\nu{:}b \mid \top\} \vdash \\ e \Leftarrow \tau\end{array}}{\Gamma \vdash e \Leftarrow x{:}b \dashrightarrow \tau}\ \textsc{ChkGhost}$$

We the rank current call is

$$\begin{aligned}&\texttt{Rank}(\Gamma \vdash e \Leftarrow x{:}b \dashrightarrow \tau)\\ &\quad = (\texttt{TermSize}(e), 1 + \texttt{TypeSize}(\tau))\end{aligned}$$

which is greater than recursive call:

$$\texttt{Rank}(\Gamma, x{:}\{\nu{:}b \mid \top\} \vdash e \Leftarrow \tau) = (\texttt{TermSize}(e), \texttt{TypeSize}(\tau))$$

Case ChkInter:

$$\frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} \tau_1 \sqcap \tau_2 \\ \Gamma \vdash e \Leftarrow \tau_1 \\ \Gamma \vdash e \Leftarrow \tau_2\end{array}}{\Gamma \vdash e \Leftarrow \tau_1 \sqcap \tau_2}\ \textsc{ChkInter}$$

We the rank current call is

$$\begin{aligned}&\texttt{Rank}(\Gamma \vdash e \Leftarrow \tau_1 \sqcap \tau_2)\\ &\quad = (\texttt{TermSize}(e), 1 + \texttt{TypeSize}(\tau_1) + \texttt{TypeSize}(\tau_2))\end{aligned}$$

which is greater than recursive call:

$$\texttt{Rank}(\Gamma \vdash e \ \Leftarrow \ \tau_1) = (\texttt{TermSize}(e), \texttt{TypeSize}(\tau_1))$$
$$\texttt{Rank}(\Gamma \vdash e \ \Leftarrow \ \tau_2) = (\texttt{TermSize}(e), \texttt{TypeSize}(\tau_2))$$

Case ChkLetV:

$$\frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} [A]\, t\, [B] \\ \Gamma \vdash v_x \ \Rightarrow \ t_x \\ \Gamma, x{:}t_x \vdash \\ e \ \Leftarrow \ [A]\, t\, [B]\end{array}}{\Gamma \vdash \texttt{let}\ x = v_x\ \texttt{in}\ e \ \Leftarrow \ [A]\, t\, [B]}\ \textsc{ChkLetV}$$

We the rank current call is

$$\begin{aligned}&\texttt{Rank}(\Gamma \vdash \texttt{let}\ x = v_x\ \texttt{in}\ e \ \Leftarrow \ [A]\, t\, [B]) \\ &\quad = (1 + \texttt{TermSize}(v_x) + \texttt{TermSize}(e), 1 + \texttt{TypeSize}(t))\end{aligned}$$

which is greater than recursive calls:

$$\texttt{Rank}(\Gamma \vdash v_x \ \Rightarrow \ t_x) = (\texttt{TermSize}(v_x), 0)$$
$$\texttt{Rank}(\Gamma, x{:}t_x \vdash e \ \Leftarrow \ [A]\, t\, [B]) = (\texttt{TermSize}(e), 1 + \texttt{TypeSize}(t))$$

Case ChkLetE:

$$\frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} [A]\, t\, [B] \\ \Gamma \vdash e_x \ \Rightarrow \ \bigsqcap \overline{[A_i]\, t_i\, [A'_i]} \\ \Gamma \vdash A \subseteq \bigvee A_i \\ \forall i.\Gamma, x{:}t_i \vdash \\ e \ \Leftarrow \ [(A; \Box\langle\top\rangle) \wedge A'_i]\, t\, [B]\end{array}}{\Gamma \vdash \texttt{let}\ x = e_x\ \texttt{in}\ e \ \Leftarrow \ [A]\, t\, [B]}\ \textsc{ChkLetE}$$

We the rank current call is

$$\begin{aligned}
&\texttt{Rank}(\Gamma \vdash \texttt{let}\ x = e_x\ \texttt{in}\ e \ \Leftarrow\ [A]\,t\,[B]) \\
&\quad = (1 + \texttt{TermSize}(e_x) + \texttt{TermSize}(e), 1 + \texttt{TypeSize}(t))
\end{aligned}$$

which is greater than recursive calls:

$$\begin{aligned}
\texttt{Rank}(\Gamma \vdash e_x \ \Rightarrow\ \prod \overline{[A_i]\,t_i\,[A'_i]}) &= (\texttt{TermSize}(e_x), 0) \\
\texttt{Rank}(\Gamma, x{:}t_i \vdash e \ \Leftarrow\ [(A; \Box\langle\top\rangle) \wedge A'_i]\,t\,[B]) &= (\texttt{TermSize}(e), 1 + \texttt{TypeSize}(t))
\end{aligned}$$

Case ChkPOpApp:

$$\frac{\begin{array}{c}
\Gamma \vdash^{\mathbf{WF}} [A]\,t\,[B] \\
\Delta(op) = \overline{y_j{:}t_j} \to t_x \\
\forall j.\Gamma \vdash v_j \ \Leftarrow\ t_j \\
\Gamma, x{:}t_x \vdash \\
e \ \Leftarrow\ [A]\,t\,[B]
\end{array}}{\Gamma \vdash \texttt{let}\ x = \texttt{op}\ \overline{v_i}\ \texttt{in}\ e \ \Leftarrow\ [A]\,t\,[B]}\ \textsc{ChkPOpApp}$$

We the rank current call is

$$\begin{aligned}
&\texttt{Rank}(\Gamma \vdash \texttt{let}\ x = \texttt{op}\ \overline{v_i}\ \texttt{in}\ e \ \Leftarrow\ [A]\,t\,[B]) \\
&\quad = (1 + \texttt{TermSize}(e) + \Sigma\texttt{TermSize}(v_i), 1 + \texttt{TypeSize}(t))
\end{aligned}$$

which is greater than recursive calls:

$$\begin{aligned}
\texttt{Rank}(\Gamma \vdash v_j \ \Leftarrow\ t_j) &= (\texttt{TermSize}(v_j), \texttt{TypeSize}(t_j)) \\
\texttt{Rank}(\Gamma, x{:}t_i \vdash e \ \Leftarrow\ [(A; \Box\langle\top\rangle) \wedge A'_i]\,t\,[B]) &= (\texttt{TermSize}(e), 1 + \texttt{TypeSize}(t))
\end{aligned}$$

Case ChkEOpApp:

$$\frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} [A]\, t\, [B] \\ \Delta(\mathtt{op}) = \overline{z_k{:}b_k} \dashrightarrow \overline{y_j{:}t_j} \to \tau_x \\ \forall j.\Gamma \vdash v_j \;\Leftarrow\; t_j \\ \Gamma \vdash [A]\; \overline{z_k{:}b_k} \dashrightarrow \overline{y_j{:}t_j} \to \tau_x \;\rightsquigarrow \overline{z_k{:}t_k} \\ \bigsqcap \overline{[A_i]\, t_i\, [A'_i]} = \tau_x[\overline{y_j \mapsto v_j}] \\ \forall i.\Gamma, \overline{z_k{:}t_k}, x{:}t_i \vdash \\ e \;\Leftarrow\; [(A; \Box\langle\top\rangle) \wedge A'_i]\, t\, [B]\end{array}}{\Gamma \vdash \mathtt{let}\ x = \mathtt{op}\ \overline{v_i}\ \mathtt{in}\ e \;\Leftarrow\; [A]\, t\, [B]}\ \textsc{ChkEOpApp}$$

We the rank current call is

$$\begin{aligned}&\mathtt{Rank}(\Gamma \vdash \mathtt{let}\ x = \mathtt{op}\ \overline{v_i}\ \mathtt{in}\ e \;\Leftarrow\; [A]\, t\, [B]) \\ &\quad = (1 + \mathtt{TermSize}(e) + \Sigma\mathtt{TermSize}(v_i), 1 + \mathtt{TypeSize}(t))\end{aligned}$$

which is greater than recursive calls:

$$\mathtt{Rank}(\Gamma \vdash v_j \;\Leftarrow\; t_j) = (\mathtt{TermSize}(v_j), \mathtt{TypeSize}(t_j))$$

$$\mathtt{Rank}(\Gamma, \overline{z_k{:}t_k}, x{:}t_i \vdash e \;\Leftarrow\; [(A; \Box\langle\top\rangle) \wedge A'_i]\, t\, [B]) = (\mathtt{TermSize}(e), 1 + \mathtt{TypeSize}(t))$$

Case $\textsc{ChkApp}$:

$$\frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} [A]\, t\, [B] \\ \Gamma \vdash v_1 \;\Rightarrow\; \overline{z_k{:}b_k} \dashrightarrow y{:}t_y \to \tau_x \\ \Gamma \vdash v_2 \;\Leftarrow\; t_y \\ \Gamma \vdash [A]\; \overline{z_k{:}b_k} \dashrightarrow y{:}t_y \to \tau_x \;\rightsquigarrow \overline{z_k{:}t_k} \\ \bigsqcap \overline{[A_i]\, t_i\, [A'_i]} = \tau_x[y \mapsto v_y] \\ \forall i.\Gamma, \overline{z_k{:}t_k}, x{:}t_i \vdash \\ e \;\Leftarrow\; [(A; \Box\langle\top\rangle) \wedge A'_i]\, t\, [B]\end{array}}{\Gamma \vdash \mathtt{let}\ x = v_1\ v_2\ \mathtt{in}\ e \;\Leftarrow\; [A]\, t\, [B]}\ \textsc{ChkApp}$$

We the rank current call is

$$\begin{aligned}&\mathtt{Rank}(\Gamma \vdash \mathtt{let}\ x = v_1\ v_2\ \mathtt{in}\ e \;\Leftarrow\; [A]\, t\, [B]) \\ &\quad = (1 + \mathtt{TermSize}(e) + \mathtt{TermSize}(v_1) + \mathtt{TermSize}(v_2), 1 + \mathtt{TypeSize}(t))\end{aligned}$$

which is greater than recursive calls:

$$\begin{aligned}
\texttt{Rank}(\Gamma \vdash v_1 \ \Rightarrow\ \overline{z_k{:}b_k} \dashrightarrow y{:}t_y \to \tau_x) &= (\texttt{TermSize}(v_1), 0) \\
\texttt{Rank}(\Gamma \vdash v_2 \ \Leftarrow\ t_y) &= (\texttt{TermSize}(v_2), \texttt{TypeSize}(t_y)) \\
\texttt{Rank}(\Gamma, \overline{z_k{:}t_k}, x{:}t_i \vdash e \ \Leftarrow\ [(A; \Box\langle\top\rangle) \wedge A'_i]\, t\, [B]) &= (\texttt{TermSize}(e), 1 + \texttt{TypeSize}(t))
\end{aligned}$$

Case CHKFUNC:

$$\frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} x{:}t'_x \to \tau \\ \Gamma \vdash t'_x <: t_x \\ \Gamma, x{:}t'_x \vdash e \ \Leftarrow\ \tau\end{array}}{\Gamma \vdash \lambda x{:}t_x.e \ \Leftarrow\ x{:}t'_x \to \tau}\ \textsc{ChkFunc}$$

We the rank current call is

$$\begin{aligned}
&\texttt{Rank}(\Gamma \vdash \lambda x{:}t_x.e \ \Leftarrow\ x{:}t'_x \to \tau) \\
&\quad = (1 + \texttt{TermSize}(e), 1 + \texttt{TypeSize}(t'_x) + \texttt{TypeSize}(\tau))
\end{aligned}$$

which is greater than recursive calls:

$$\texttt{Rank}(\Gamma, x{:}t'_x \vdash e \ \Leftarrow\ \tau) = (\texttt{TermSize}(e), \texttt{TypeSize}(\tau))$$

Case CHKFIX:

$$\frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} t'_f \\ \Gamma \vdash t'_f <: t_f \\ \Gamma, f{:}t'_f \vdash \\ \lambda x{:}t_x.e \ \Leftarrow\ t'_f\end{array}}{\Gamma \vdash \texttt{fix} f{:}t_f.\lambda x{:}t_x.e \ \Leftarrow\ t'_f}\ \textsc{ChkFix}$$

We the rank current call is

$$\texttt{Rank}(\Gamma \vdash \texttt{fix} f{:}t_f.\lambda x{:}t_x.e \ \Leftarrow\ t'_f) = (2 + \texttt{TermSize}(e), \texttt{TypeSize}(t'_f))$$

which is greater than recursive calls:

$$\texttt{Rank}(\Gamma, f{:}t'_f \vdash \lambda x{:}t_x.e \;\Leftarrow\; t'_f) = (1 + \texttt{TermSize}(e), \texttt{TypeSize}(t'_f))$$

Case ChkMatch:

$$\frac{\begin{array}{c}\forall i.(d_i, \overline{y_j{:}t_j} \to \{\nu{:}b \mid \phi_i\}) \in \Delta \\ \Gamma, \overline{y_j{:}t_j}, z{:}\{\nu{:}b \mid \nu = v \wedge \phi_i\} \vdash e_i \;\Leftarrow\; \tau \\ \text{where } z \text{ is fresh}\end{array}}{\Gamma \vdash \texttt{match}\ v\ \texttt{with}\ \overline{d_i\,\overline{y_j} \to e_i} \;\Leftarrow\; \tau}\ \textsc{ChkMatch}$$

We the rank current call is

$$\begin{aligned}&\texttt{Rank}(\Gamma \vdash \texttt{match}\ v\ \texttt{with}\ \overline{d_i\,\overline{y_j} \to e_i} \;\Leftarrow\; \tau) \\ &\quad = (1 + \texttt{TermSize}(v) + \Sigma\texttt{TermSize}(e_i), \texttt{TypeSize}(\tau))\end{aligned}$$

which is greater than recursive calls:

$$\texttt{Rank}(\Gamma, \overline{y_j{:}t_j}, z{:}\{\nu{:}b \mid \nu = v \wedge \phi_i\} \vdash e_i \;\Leftarrow\; \tau) = (\texttt{TermSize}(e_i), \texttt{TypeSize}(\tau))$$

Case ChkSub:

$$\frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} [A_2]\,t_2\,[B_2] \\ \Gamma \vdash e \;\Rightarrow\; [A_1]\,t_1\,[B_1] \\ \Gamma \vdash A_2 <: A_1 \quad \Gamma \vdash t_1 <: t_2 \\ \Gamma \vdash (A_2; \square\langle\top\rangle) \wedge \\ (A_2; \square\langle\top\rangle) \wedge B_1 \subseteq B_2\end{array}}{\Gamma \vdash e \;\Leftarrow\; [A_2]\,t_2\,[B_2]}\ \textsc{ChkSub}$$

We the rank current call is

$$\texttt{Rank}(\Gamma \vdash e \;\Leftarrow\; [A_2]\,t_2\,[B_2]) = (\texttt{TermSize}(e), 1 + \texttt{TypeSize}(t_2))$$

which is greater than recursive calls:

$$\texttt{Rank}(\Gamma \vdash e \;\Rightarrow\; [A_1]\, t_1\, [B_1]) = (\texttt{TermSize}(e), 0)$$

Case ChkSubV:

$$\frac{\begin{array}{c}\Gamma \vdash^{\mathbf{WF}} t \\ \Gamma \vdash v \;\Rightarrow\; t' \\ \Gamma \vdash t' <: t\end{array}}{\Gamma \vdash v \;\Leftarrow\; t}\ \textsc{ChkSubV}$$

We the rank current call is

$$\texttt{Rank}(\Gamma \vdash v \;\Leftarrow\; t) = (\texttt{TermSize}(v), \texttt{TypeSize}(t))$$

which is greater than recursive calls:

$$\texttt{Rank}(\Gamma \vdash v \;\Rightarrow\; t') = (\texttt{TermSize}(v), 0)$$

Thus, the typing algorithm is decidable.

### 4.5 Implementation and Evaluation

We have implemented a tool based on the above approach, called Marple, that targets ADTs implemented in terms of other stateful libraries. Marple consists of approximately 12K lines of OCaml and uses Z3 [8] as its backend solver for both SMT and FA inclusion queries.[7]

Marple takes as an input the implementation of an ADT in OCaml, enhanced signatures of ADT operations that include representation invariants expressed as HATs, and specifications of the supporting libraries as HATs (e.g., the signatures in Example 4.3.2). The typing context used by Marple includes signatures for a number of (pure) OCaml primitives, including the pure operators listed in Fig. 4.2. Marple also includes a set of predefined method predicates (i.e., *mp* in Fig. 4.5) that allow qualifiers to capture non-trivial datatype properties. For

[7]↑Our artifact [73] provides a docker image that contains the source code of Marple and all our benchmarks.

example, the method predicates `isDir(x)` and `isDel(y)` from the motivating example in Sec. 4.1 encode that `x` holds the contents of a directory, and that `y` is marked as deleted, respectively. The semantics of method predicates are defined via a set of lemmas in FOL, in order to enable automated verification; e.g., the axiom $\forall \texttt{x}.\texttt{isDir}(\texttt{x}) \implies \neg\texttt{isDel}(\texttt{x})$ encodes that a set of bytes cannot simultaneously be an active directory and marked as deleted.

**Table 4.1.** Experimental results of using Marple to verify representation invariants of ADTs.

| ADT | Library | #Method | #Ghost | $s_I$ | $t_{total}$ (s) | #Branch | #App | #SAT | $\#FA_{\subseteq}$ | avg. $s_{FA}$ | $t_{SAT}$ (s) | $t_{FA_{\subseteq}}$ (s) |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Stack† | LinkedList | 7 | 0 | 4 | 5.94 | 4 | 7 | 297 | 7 | 98 | 1.63 | 0.06 |
| | KVStore | 7 | 1 | 9 | 17.88 | 4 | 10 | 874 | 17 | 226 | 5.78 | 0.23 |
| Queue† | LinkedList | 6 | 0 | 4 | 4.88 | 4 | 9 | 190 | 4 | 96 | 1.08 | 0.06 |
| | Graph | 6 | 1 | 24 | 28.24 | 5 | 12 | 1212 | 14 | 525 | 8.19 | 0.86 |
| Set⋆ | Tree | 5 | 0 | 12 | 27.12 | 5 | 12 | 1589 | 11 | 531 | 9.34 | 0.53 |
| | KVStore | 3 | 1 | 9 | 2.39 | 3 | 5 | 245 | 6 | 160 | 1.51 | 0.06 |
| Heap⋆ | Tree | 7 | 0 | 12 | 25.71 | 5 | 12 | 1589 | 11 | 531 | 9.33 | 0.52 |
| | LinkedList | 6 | 0 | 4 | 8.84 | 4 | 8 | 497 | 8 | 118 | 3.03 | 0.08 |
| MinSet | Set | 4 | 1 | 28 | 10.55 | 3 | 6 | 612 | 17 | 294 | 4.19 | 1.58 |
| | KVStore | 4 | 1 | 25 | 32.84 | 5 | 8 | 2227 | 23 | 519 | 10.93 | 9.34 |
| LazySet | Tree | 6 | 0 | 12 | 27.10 | 5 | 12 | 1589 | 11 | 531 | 9.33 | 0.57 |
| | Set | 4 | 1 | 9 | 1.42 | 2 | 3 | 101 | 4 | 106 | 0.57 | 0.04 |
| | KVStore | 5 | 1 | 9 | 3.10 | 3 | 5 | 245 | 6 | 160 | 1.49 | 0.06 |
| FileSystem | Tree | 6 | 1 | 20 | 58.80 | 3 | 8 | 2085 | 17 | 652 | 14.15 | 2.21 |
| | KVStore | 4 | 1 | 17 | 157.27 | 4 | 10 | 8144 | 43 | 481 | 56.64 | 16.54 |
| DFA | KVStore | 5 | 2 | 18 | 42.62 | 3 | 3 | 3604 | 25 | 228 | 18.84 | 1.60 |
| | Graph | 5 | 2 | 11 | 78.44 | 4 | 4 | 3625 | 27 | 225 | 25.53 | 3.42 |
| ConnectedGraph | Set | 5 | 2 | 9 | 80.46 | 4 | 4 | 3889 | 50 | 357 | 27.39 | 12.80 |
| | Graph | 5 | 1 | 20 | 176.89 | 4 | 3 | 1349 | 17 | 360 | 19.19 | 53.90 |

Our experimental evaluation addresses three research questions:

**Q1**: Is Marple *effective*? Can it verify a diverse range of ADTs that are implemented using a variety of backing stateful libraries?

**Q2**: Is Marple *expressive*? Are HATs able to capture a rich set of representation invariants?

**Q3**: Is Marple *efficient*? Is it able verify clients of stateful APIs in a reasonable amount of time?

We have evaluated Marple on a corpus of complex and realistic ADTs drawn from a variety of sources (shown in the title of Table 4.1), including general datatypes such as stacks, queues, sets, heaps, graphs, as well as custom data structures including the Unix-style file system from Sec. 4.1 and the deterministic finite automata described in Sec. 4.3 (**Q1**). Benchmarks from

prior works are annotated with their source: Okasaki [50](†), Hanoi [59](⋆). We have rewritten functional implementations of these benchmarks into stateful versions by assuming there is a hidden mutable datatype instance that ADT methods use to perform their operations. The backing stateful libraries are drawn from a verified OCaml library [57]. Table 4.1 presents the results of this evaluation. Each datatype includes multiple implementations using different underlying libraries (Library column), each of which provides a distinct set of operations. Our backing APIs include linked lists, a persistent key-value store, sets, trees, and graphs, each of which induces a different encoding of the target ADT's representation invariant.

Table 4.1 divides the results of our experiments into two categories, separated by the double bars; the first gives comprehensive results about analyzing the entire datatype, while the second reports information about the most complex method of each ADT implementation.[8] The first group of columns describes broad characteristics of each ADT, including its number of methods (#Method), the number of ghost variables in its representation invariant (#Ghost), and the size of the formula encoding the representation invariant ($s_I$), after it is desugared into a symbolic regular expression. The next column reports the total time needed to verify all the method of each ADT ($t_{total}$). Marple performs quite reasonably on all of our benchmarks, taking between 1.42 to 176.89 seconds for each ADT. As expected, more complicated ADT implementations (i.e., those with larger values in the #Method, #Ghost, and $s_I$ columns), take longer to verify (**Q3**).

Details about the representation invariants used in our benchmarks are shown in Table 4.2 (**Q2**). The stack and queue ADTs require a low-level guarantee that the underlying linked list implementation is not circular. The set ADTs maintain the invariant that there are no duplicate elements. The connected graph benchmark requires that every pair of nodes is connected, necessitating the need for ghost variables. As shown in Table 4.1, all of these properties, as well as those discussed in previous sections, can be expressed using $LTL_f$ using no more than 2 ghost variables (#Ghost) and 28 literals ($s_I$).

The first group of columns in the second half of Table 4.1 list features relevant to the complexity of the most demanding method in the ADT. These methods feature between 3 and 5 branches (#Branch) and 3 to 12 uses of built-in operators (#App). The last two groups

[8]†Our appendix includes a full table listing the corresponding information for each ADT method.

**Table 4.2.** Details of the properties used in the benchmarks shown in Table 4.1. Representation invariants of client ADTs are expressed in terms of their interactions with the backing stateful library.

| Client ADT | Representation Invariant | Underlying Library | Policy governing interactions with underlying API |
|---|---|---|---|
| Stack | LIFO-property | LinkedList<br>KVStore | The addresses that store elements are unique<br>Not a circular linked list |
| Queue | FIFO-property | LinkedList<br>Graph | Not a circular linked list<br>No self-loops in graph, degree of nodes in graph are at most 1 |
| Set | Unique elements | Tree<br>KVStore | The underlying tree is a binary search tree<br>Every key is associated with a distinct value |
| Heap | Min-heap property | Tree<br>LinkedList | The value of the parent node in the tree is less than the value of its child nodes<br>Not a circular linked list; the elements are sorted |
| MinSet | Uniqueness and minimality | Set<br>KVStore | The cached element is less than all inserted elements in the set and has been inserted into the set<br>The cached element is less than all put elements in the store and has been put before |
| LazySet | Uniqueness of elements | Tree<br>Set<br>KVStore | The underlying tree is a binary search tree<br>An element has never been inserted twice<br>Every key is associated with a distinct value |
| FileSystem | Policy of Unix-like filesystem paths | Tree<br>KVStore | The Parent node stores a path that is prefix of its child nodes; a non-root parent node stores directories<br>Any non-root path stored as a key in the key-value store must have its parent stored as a non-deleted directory in the store |
| DFA | Determinism of Transitions | KVStore<br>Graph | All stored transitions are represented as tuples (start, char, end); starting state has at most have one transition for a character<br>Two nodes (which represent states in FA) can have at most one edge, which is labeled with a character |
| ConnectedGraph | Connectivity | Set<br>Graph | The set stores unique pairs (fst, snd); only elements that have both in and out edges are treated as a value node in the graph<br>All nodes in the graph are connected |

of columns present type checking results for these methods. The #SAT and $\#FA_{\subseteq}$ columns list the number of SMT queries and finite automata (FA) inclusion checks needed to type check the method. The next column (avg. $s_{FA}$) gives the average number of transitions in the finite state automata (FA) after the alphabet transformation described in Sec. 4.4. These numbers are roughly proportional to code complexity (column #Branch and #App) and invariant complexity (column #Ghost and $s_I$) — intuitively, programs that have more uses of operators and larger specifications, require more queries and produce more complicated FAs. The last two columns report verification time for the method (**Q3**). These times are typically dominated by the SFA inclusion checks (ranging from .63 to 73.09 seconds) that result from minterm satisfiability checks during the alphabet transformation phase ($t_{SAT}$) and FA inclusion checks ($t_{FA_{\subseteq}}$). Unsurprisingly, generating more queries (both #SMT and $\#FA_{\subseteq}$) result in more time spent checking minterm satisfiability and FA inclusion. Our results

also indicate that FA inclusion checks are not particularly sensitive to the size of the FA involved; rather, the cost of these checks corresponds more closely to the deep semantics of the representation invariant, in which the choice of the underlying library is an important factor. Taking the FileSystem benchmark as an example, both Tree and KVStore implementations lead to similar sizes for the invariant, in term of #Ghost. $s_I$, and FA (avg.$s_{FA}$). However, the use of the former library results in a significantly shorter verification time since the relevant property captured by the invariant "*every child has its parent in the file system*" is naturally supported by the Tree library; the only remaining verification task in this case for the client to handle involves ensuring these parents are indeed directories in the file system. In contrast, the KVStore provides no such guarantees on how its elements are related, requiring substantially more verification effort to ensure the invariant is preserved.

# 5. POLYMORPHIC COVERAGE TYPES

Property-based testing (PBT) is a popular technique for automatically testing deep semantic and structural properties of programs. Originally pioneered by the QuickCheck [74] library for Haskell, PBT frameworks now exist for many programming languages, including QCheck [75], JavaScript [76], Rust [77], Python [78], Scala [79], and Coq [23]. The PBT methodology rests on two key components: *executable properties* that capture the expected input-output behaviors of the program under test, and *test input generators* that generate random values of the input types needed to validate these behaviors. In contrast to unit tests, which rely on single examples of inputs and outputs, generators are meant to provide a *family* of inputs against which programs can be tested, with the goal of ensuring the set of generated tests provide good coverage of all possible inputs. In order to prune out irrelevant inputs, PBT frameworks allow users to define custom generators that reflect the specific shape of data that the developer believes is most likely to trigger interesting (aka faulty) behavior. As one simple example, to test a tree compression or balancing function, the developer may want to use a generator that produces $n$-ary trees with randomly chosen height and arity but whose leaves are ordered according to a user-provided ordering relation.

Given the critical role they play in the assurance case provided by PBT frameworks, it is reasonable to ask what constitutes a "good" specification for a test generator. For our example, one answer could be that it should only produce ordered trees. Of course, this is not a very satisfactory characterization of the behavior we desire: the "constant" generator that always produces trees of height one trivially meets this specification, but it is unlikely to produce useful tests for a compression function! Ideally, we would like a generator to intelligently enumerate the space of *all* possible ordered trees, thereby helping to maximize the likelihood of finding bugs in the function under test. Because defining such an enumeration procedure for arbitrary datatypes can be hard, even when complete enumeration is computationally feasible, PBT frameworks instead give developers the ability to assemble generators for complex data structures *compositionally*, building on generators for simpler types where randomly sampling elements of the type is straightforward and sufficient. For example, we could implement an ordered tree generator in terms of a primitive random number generator that is used to

non-deterministically select the height, arity, and elements of a candidate tree, checking (or enforcing) the orderness of the tree before returning it as a feasible test input. Although the random number generator might provide a guarantee that its underlying probability density function (PDF) is always non-zero on all elements in its sample space, determining that a tree generator that is built using it can actually enumerate all the ordered trees desired is a substantially harder problem. Even if we know the generator is capable of eventually yielding all trees, constraints imposed by the function's precondition might require the generator to perform further filtering or transformations over generated trees. However, proving that any filtering operations the generator uses do not mistakenly prune out valid ordered trees or that any transformations the generator performs over candidate trees preserve the elements of the random tree being transformed, pose additional challenges. In other words, verifying that the generator is *complete* with respect to our desired orderness property entails reasoning that is independent of the behavior of the primitive generators used to build the tree. Consequently, we require some alternative mechanism to help qualify the part of the target function's input space the generator is actually guaranteed to cover. Devising such a mechanism is challenging precisely because the properties that need to be tested may impose complex structural and semantic constraints on the generated output (e.g., requiring that an output tree be a binary search tree, or that it satisfies a red-black property, etc.); the complexity of these constraints is directly correlated to the sparseness of the function's input space preconditions.

```
1 type 'a tree =
2   | Leaf
3   | Node of ('a * 'a tree * 'a tree)
4 let rec bst_gen (lo: int) (hi: int): int tree =
5   if lo + 1 >= hi then Leaf else
6   (* Leaf ⊕ *)
7   (let (x: int) = int_range (lo + 1, hi - 1) in
8       Node (x, bst_gen lo x, bst_gen x hi))
```

**Figure 5.1.** A BST generator.

To illustrate this distinction more concretely, consider the input test generator shown in Figure 5.1 that is intended to generate all binary search trees (BSTs) whose elements

are between the interval `lo` and `hi`. If we ignore the comment on line 6, we can conclude this generator always produces a non-empty BST whenever `lo` $<$ `hi`. While the generator is correct - it always generates a well-formed BST - it is also incomplete; the call `bst_gen 0 10`, for example, will never produce a tree containing just `Leaf` or a tree with a shape like `Node(1,Leaf,Leaf)`, even though these instances are valid trees consistent with the constraints imposed by the generator's argument bounds. In fact, this implementation *never* generates a BST that only contains a proper subset of the elements that reside within the interval defined by `lo` and `hi`. By uncommenting line 6, however, we allow the generator to non-deterministically choose (via operator $\oplus$) to either return a `Leaf` or left and right BST subtrees based on value returned by the `int_range` generator, enabling it to potentially produce BSTs containing all valid subsets of the provided interval, thus satisfying our desired desired completeness behavior. The subtleties involved in reasoning about such coverage properties is clearly non-trivial. We reiterate that recognizing the distinction between these two implementations is not merely a matter of providing a precise output type capturing the desired sortedness property of a BST: the incomplete implementation clearly satisfies such a type! Furthermore, simply knowing that the underlying `int_range` generator used in the implementation samples all elements within the range of the arguments it is provided is also insufficient to conclude that the BST generator can yield *all* possible BSTs within the supplied interval. Similar observations have led prior work to consider ways to improve a generator's coverage through mechanisms such as fuzzing [80, 81], or to automatically generate complete-by-construction generators for certain classes of datatypes [82].

In contrast to these approaches, this chapter embeds the notion of coverage as an integral part of a test input generator's *type* specification. By doing so, a generator's type now specifies the set of behaviors the generator is *guaranteed* to exhibit; a well-typed generator is thus guaranteed to produce *every possible* value satisfying a desired structural property, e.g., that the repaired (complete) version of `bst_gen` is capable of producing every valid BST. By framing the notion of coverage in type-theoretic terms, our approach neither requires instrumentation of the target program to assess the coverage effectiveness of a candidate generator (as in [80]) nor does it depend on a specific compilation strategy for producing generators (as in [82]). Instead, our approach can automatically verify the coverage properties

of an *arbitrary* test input generator, regardless of whether it was hand-written or automatically synthesized.

Key to our approach is a novel formulation of a *must*-style analysis [38, 83, 84] of a test input generator's behavior in type-theoretic terms. In our proposed type system, we say an expression $e$ has *coverage type* $\tau$ if every value contained in $\tau$ *must* be producible by $e$. Note how this definition differs from our usual notion of what a type represents: ordinarily, if $e$ has type $\tau$ then we are allowed to conclude only that any value contained in $\tau$ *may* be produced by $e$. Informally, types interpreted in this usual way define an *overapproximation* of the values an expression $e$ can yield, without obligating $e$ to produce any specific such value. In contrast, coverage types define an *underapproximation* - they characterize the values an expression $e$ has to produce, potentially eliding other values that $e$ may also evaluate to. When the set of elements denoted by a generator's (underapproximate) coverage type matches that of its (overapproximate) normal type, however, we can soundly assert that the generator is complete. As we illustrate in the remainder of the chapter, this characterization allows us to reason about a program's coverage behavior on the same formal footing as its safety properties.

In this sense, our solution can be seen a type-theoretic interpretation of recently proposed Incorrectness Logics (IL) [39, 85, 86], in much the same way that refinement-type systems like Liquid Types [13, 22] relate to traditional program logics [87]. Despite the philosophical similarities with IL, however, we use underapproximate reasoning for a very different goal. While IL has been primarily used to precisely capture the conditions that will lead a program to fault, this work explores how type-based underapproximate reasoning can be used to verify the completeness properties of a test generator in the context of PBT.

This interpretation leads to a fundamental recasting of how types relate to one another: ordinarily, we are always allowed to assert that $\tau <: \top$. This means that any typing context that admits an expression with type $\tau$ can also admit that expression at a type with a logically weaker structure. In contrast, the subtyping relation for coverage types inverts this relation, so that $\top <: \tau$. Intuitively, $\top$ represents the coverage type that obligates an expression ascribed this type to be capable of producing *all* elements in $\tau$. But, any context that requires an expression to produce all such elements can always guarantee that the expression will

also produce a subset of these elements. In other words, we are always allowed to weaken an overapproximation (i.e., grow the set of values an expression may evaluate to), and strengthen an underapproximation (i.e., shrink the set of values an expression must evaluate to). Thus, in our setting, a random number generator over the integers has coverage type $\top$ under the mild assumption that its underlying PDF provides a non-zero likelihood of returning every integer. In contrast, a faulty computation like `1 div 0` has coverage type $\bot$ since there are no guarantees provided by the computation on the value(s) it must return. Here, $\bot$ represents a type that defines a degenerate underapproximation, imposing no constraints on the values an expression ascribed this type must produce.

```
1     val return: 'a -> 'a Gen.t
2     val bind: 'a -> ('a -> 'b Gen.t) -> 'b Gen.t
3     val frequency: (int -> (int * 'a Gen.t)) -> 'a Gen.t
4     val fix: (int -> (int -> 'a -> 'a Gen.t) -> 'a Gen.t) -> int -> 'a Gen.t
5
6     type 'a tree = Leaf of 'a | Node of 'a tree * 'a tree
7     let tree_gen : int -> int tree Gen.t = fix
8       (fun n self -> match n with
9         | 0 -> return err
10        | 1 -> bind nat_gen (fun x -> return (Leaf x))
11        | n ->
12          frequency
13            (function
14            | 0 -> (1, bind nat_gen (fun x -> return (Leaf x)))
15            | _ -> (2, bind (self n/2) (fun tr1 ->
16                        bind (self n/2) (fun tr2 ->
17                          return (Node (tr1, tr2))))))
18      )
```

**Figure 5.2.** A tree generator expressed using `QCheck`.

In practice, developers often implement input generators using *test generator combinators* using combinators provided by the PBT framework. In this setting, generators are defined in terms of a monad that facilitates the compositional construction of complex generators from simpler ones in a modular way. The OCaml PBT framework `QCheck` [75], for example, ascribes the type `'aM` to such generators, as can be seen in the signature for the binary tree generator `tree_gen` shown in Fig. 5.2. This generator is implemented by 4 combinators: in

addition to the basic monad operators (`return` and `bind`), `tree_gen` also uses the `frequency` combinator to sample from multiple generators according to the given weights, as well as the `fix` combinator to recursively construct trees of greater height from smaller ones. Extending coverage types to support monadic combinators and type polymorphism (on both types and qualifiers) in requires significant extensions over the core monomorphic, non-monadic formulation.

In summary, this chapter makes the following contributions:

1. Introduces the notion of *coverage types*, types that characterize the values that an input test generator is guaranteed to produce.

2. Formalizes the semantics of coverage types in an ML-like functional language with support for higher-order functions and inductive datatypes.[1]

3. Develops a bi-directional type-checking algorithm for coverage types in this language.

4. Extends this core calculus and typing algorithm to support coverage type polymorphism and monadic composition.

5. Incorporates these ideas in a tool (Poirot) that operates over OCaml programs equipped with input generators and typed using coverage types.

6. Presents an extensive empirical evaluation justifying the utility of coverage types, by using Poirot to verify the coverage properties of both hand-written generators drawn from real-world open source PBT applications and automatically synthesized generators, covering a rich class of datatypes and their structural properties.

The remainder of this chapter is structured as follows. In the next section, we present an informal overview of the key features of our type system. Section 5.2 presents the syntax and semantics for a core call-by-value higher-order functional language with inductive datatypes that we use to formalize our approach. Section 5.3 describes our coverage type system, and its metatheory is given in Section 5.4. A bidirectional typing algorithm is then given in

[1] ↑A Coq formalization of this calculus, its type system, and its metatheory is provided on Zenodo [88].

Section 5.5. We describe details about the implementation of Poirot and provide benchmark results in Section 5.8.

## 5.1 Overview

Before presenting the full details of our type system, we begin with an informal overview of its key features.

### Base types

In the following, we write $[\nu{:}\mathtt{b} \mid \phi]$ to denote the coverage type that qualifies the base type $b$ using the predicate $\phi$. As described in the previous section, an application of the primitive built-in generator for random numbers: $\mathtt{int_gen} : \mathtt{unit} \rightarrow \mathtt{int}$ has the coverage type $\mathtt{int_gen}\ () : [\nu{:}\mathtt{int} \mid \top]$. We use brackets [...] to emphasize that a coverage type has a different meaning from the types typically found in other refinement type systems [13, 22] where a qualified type $b$, written as $\{\nu{:}\mathtt{b} \mid \phi\}$, uses a predicate $\phi$ to constrain the set of values a program *might* evaluate to. To illustrate this distinction, consider the combinations of expressions and types shown in Table 5.1. We use $\vdash$ and $\nvdash$ to identify whether a term can or cannot be assigned the corresponding type, resp. The constant `err` represents a special error value, which causes the program to halt when encountered.

**Table 5.1.** Examples of overapproximate and underapproximate (coverage) typings.

| | | | |
|---|---|---|---|
| `int_gen ()` | $\vdash [\nu{:}\mathtt{int} \mid \top]$<br>$\vdash [\nu{:}\mathtt{int} \mid \bot]$<br>$\nvdash \{\nu{:}\mathtt{int} \mid \nu = 1\}$ | $\vdash [\nu{:}\mathtt{int} \mid \nu = 1 \vee 2]$<br>$\vdash \{\nu{:}\mathtt{int} \mid \top\}$<br>$\nvdash \{\nu{:}\mathtt{int} \mid \bot\}$ | $\vdash [\nu{:}\mathtt{int} \mid \nu = 1]$<br>$\nvdash \{\nu{:}\mathtt{int} \mid \nu = 1 \vee 2\}$ |
| `1` | $\vdash [\nu{:}\mathtt{int} \mid \nu = 1]$<br>$\vdash \{\nu{:}\mathtt{int} \mid \nu = 1 \vee 2\}$<br>$\nvdash [\nu{:}\mathtt{int} \mid \nu = 1 \vee 2]$ | $\vdash [\nu{:}\mathtt{int} \mid \bot]$<br>$\vdash \{\nu{:}\mathtt{int} \mid \nu = 1\}$<br>$\nvdash \{\nu{:}\mathtt{int} \mid \bot\}$ | $\vdash \{\nu{:}\mathtt{int} \mid \top\}$<br>$\nvdash [\nu{:}\mathtt{int} \mid \top]$ |
| `err` | $\vdash [\nu{:}\mathtt{int} \mid \bot]$<br>$\nvdash [\nu{:}\mathtt{int} \mid \nu = 1]$<br>$\nvdash \{\nu{:}\mathtt{int} \mid \nu = 1\}$ | $\nvdash [\nu{:}\mathtt{int} \mid \top]$<br>$\nvdash \{\nu{:}\mathtt{int} \mid \top\}$<br>$\nvdash \{\nu{:}\mathtt{int} \mid \bot\}$ | $\nvdash [\nu{:}\mathtt{int} \mid \nu = 1 \vee 2]$<br>$\nvdash \{\nu{:}\mathtt{int} \mid \nu = 1 \vee 2\}$ |

These examples demonstrate the previous observation that it is always possible to strengthen the refinement predicate used in an underapproximate type and weaken such a predicate in an overapproximate type. A similar phenomena appears in IL's rule of

consequence, which inverts the direction of the implications on pre- and postconditions in the overapproximate version of the rule. As a result, the bottom type $[\nu{:}\texttt{int} \mid \bot]$ is the universal supertype in our type hierarchy, as it places no restrictions on the values a term *must* produce. Thus, we sometimes abbreviate $[\nu{:}\texttt{int} \mid \bot]$ as `int`, since the information provided by both types is the same. Importantly, the coverage type for the error term (`err`) can *only* be qualified with $\bot$, since an erroneous computation is unconstrained with the respect to the values it is obligated to produce.

Coverage types can also qualify inductive datatypes, like lists and trees. In particular, the complete generator for BSTs presented in the introduction can be successfully type-checked using the following result type:

$$[\nu{:}\texttt{int tree} \mid bst(\nu) \wedge \forall u, mem(\nu, u) \Longrightarrow \texttt{lo} < u < \texttt{hi}]$$

where $bst(\nu)$ and $mem(\nu, u)$ are *method predicates*, i.e., uninterpreted functions used to encode semantic properties of the datatype. In the type given above, the qualifier requires that `bst_gen`'s result is a BST (encoded by the predicate $bst(\nu)$) and that every element $u$ stored in the tree (encoded by the predicate $mem(\nu, u)$) is between `lo` and `hi`; the coverage type thus constrains the implementation to produce *all* trees that satisfy this qualifier predicate. In contrast, the incomplete version of the generator (i.e., the implementation that does not allow prematurely terminating tree generation with a `Leaf` node) could only be type-checked using the following (stronger) type:

$$[\nu{:}\texttt{int tree} \mid bst(\nu) \wedge \forall u, mem(\nu, u) \iff \texttt{lo} < u < \texttt{hi}]$$

This signature asserts that all trees produced by the generator are BSTs, that any element contained in the tree is within the interval bounded by `lo` and `hi`, *and* moreover, any element in that interval *must* be included in the tree. The subtle difference between the two implementations, reflected in the different implication constraints expressed in their respective refinements, precisely captures how their coverage properties differ.

```
let even_gen () =
  let (n: int) = int_gen () in
  let (b: bool) = n mod 2 == 0 in
  if b then n else err
```

**Figure 5.3.** An even number generator defined in terms of an integer number generator.

### Control Flow

Just as underapproximate coverage types invert the standard overapproximate subtyping relationship, they also invert the standard relationship between a control flow construct and its subexpressions. To see how, consider the simple generator for even numbers shown in Fig. 5.3. When the integer generator, `int_gen`(), yields an odd number, `even_gen` faults; otherwise it simply returns the generated number. Consider the following type judgment that arises when type checking this program:

$$\begin{aligned} &\texttt{n}{:}[\nu{:}\texttt{int} \mid \top], \texttt{b}{:}[\nu{:}\texttt{bool} \mid \nu \iff n \texttt{ mod } 2 = 0] \\ &\quad \vdash \texttt{if b then n else err} : [\nu{:}\texttt{int} \mid \nu \texttt{ mod } 2 = 0] \end{aligned} \tag{5.1}$$

Intuitively, this judgment asserts that the `if` expression covers all even numbers (i.e., has the type $[\nu{:}\texttt{int} \mid \nu \texttt{ mod } 2 = 0]$) assuming that the *local variable* `n` can be instantiated with an arbitrary number, and that the variable `b` is true precisely when $n$ is even. Notice how the typing context encodes the potential control-flow path that *must* reach the non-faulting branch of the conditional expression. Enforcing the requirement that the conditional be able to return all even numbers does *not* require each of its branches to be a subtype of the expected type, in contrast to standard type systems. Our type system must instead establish that, *in total*, the values produced by each of the branches cover the even numbers. Because the `false` branch of the conditional faults, it is only typeable at the universal supertype, i.e., $[\nu{:}\texttt{int} \mid \bot]$. Thus, if the standard subtyping relationship between this conditional and its branches held, it could only be typed at $[\nu{:}\texttt{int} \mid \bot]$! This is not the case in our setting, as

the `true` branch contributes all the desired outputs. Formally, this property is checked by the following assumption of the coverage typing rule for conditionals:

$$\begin{aligned}&\mathtt{n}{:}[\nu{:}\mathtt{int} \mid \top], \mathtt{b}{:}[\nu{:}\mathtt{bool} \mid \nu \iff \mathtt{n}\ \mathtt{mod}\ 2 = 0]\\ &\qquad \vdash [\nu{:}\mathtt{int} \mid (\mathtt{b} \wedge \nu = \mathtt{n}) \vee (\neg\mathtt{b} \wedge \bot)] <: [\nu{:}\mathtt{int} \mid \nu\ \mathtt{mod}\ 2 == 0]\end{aligned}$$

The $\mathtt{b} \wedge \nu = n$ and $\neg\mathtt{b} \wedge \bot$ subformulas correspond to the types of the `true` and `false` branches[2], respectively. Taking the disjunction of these two formulas describes the set of values that can be produced by either branch[3]; this subtyping relationship guarantees this type is at least as large as the type expected by the entire conditional. To check that this subtyping relationship holds, our type checker generates the following formula:

$$\forall \nu, (\nu\ \mathtt{mod}\ 2 = 0) \implies (\exists \mathtt{n}, \top \wedge \exists \mathtt{b}, \mathtt{b} \iff \mathtt{n}\ \mathtt{mod}\ 2 = 0 \wedge (\mathtt{b} \wedge \nu = \mathtt{n}) \vee (\neg\mathtt{b} \wedge \bot)) \tag{5.2}$$

This formula aligns with the intuitive meaning of (5.1): in our type system, coverage types of variables in the typing context tell us what values they must (at least) produce. When checking whether a particular subtyping or typing relationship holds, we are free to choose *any* instantiation of the variables that entails the desired property. Accordingly, in (5.2), the variables $n$ and $b$ are *existentially* quantified to indicate there exists an execution path that instantiates these local variables in a way that produces the output $\nu$, instead of being universally quantified as they would be in a standard refinement type system.

### Function types

To type functions, most refinement type systems add a restricted form of the *dependent function types* found in full-spectrum dependent type systems. Such types allow the qualifiers in the result type of a function to refer to its parameters, enabling the expression of rich safety conditions governing the arguments that may be supplied to the function. To see how

[2]↑As is standard in dependent type systems, the types of both branches have been refined to reflect the path conditions under which they will be executed.

[3]↑This is similar to how the derived rule of choice in IL uses disjunction to reason about both branches of a nondeterministic choice statement.

this capability might be useful in our setting, consider the test generator `bst_gen` from the introduction. The complete version of this function produces all BSTs whose elements fall between the range specified by its two parameters, `lo` and `hi`. For the bounds `0` and `3`, the application `bst_gen 0 3` can be typed as: $[\nu{:}\texttt{int} \mid bst(\nu) \wedge \forall u, mem(\nu, u) \Longrightarrow 0 < u < 3]$. Using the standard typing rule for functions, the only way to encode this relationship in the type of `bst_gen` is:

$$[\nu{:}\texttt{int} \mid \nu = 0] \rightarrow [\nu{:}\texttt{int} \mid \nu = 3] \rightarrow [\nu{:}\texttt{int tree} \mid bst(\nu) \wedge \forall u, mem(\nu, u) \Longrightarrow 0 < u < 3]$$

Of course, this specification fails to account for the behaviors of `bst_gen` when supplied with different bounds: for example, the application `bst_gen 2 7` will fail to typecheck against this type.

Since the desired coverage property of `bst_gen` fundamentally depends on the kinds of inputs given to it, our type system includes dependent products of the form:

$$\begin{aligned}&\texttt{lo}{:}\{\nu{:}\texttt{int} \mid \top\} \rightarrow \texttt{hi}{:}\{\nu{:}\texttt{int} \mid \texttt{lo} \leq \nu\} \rightarrow \\ &\qquad [\nu{:}\texttt{int tree} \mid bst(\nu) \wedge \forall u, mem(\nu, u) \Longrightarrow \texttt{lo} < u < \texttt{hi}]\end{aligned}$$

We use the notation $\{...\}$ to emphasize that the argument types of a dependent arrow have a similar purpose and interpretation as in standard refinement type systems. Thus, the above type can be read as "if the inputs `lo` and `hi` are *any* number such that $\texttt{lo} \leq \texttt{hi}$, then the output *must* cover all possible BSTs whose elements are between `lo` and `hi`". Using this type for `bst_gen` allows our system to seamlessly type-check both (`bst_gen 0 3`) and (`bst_gen 2 7`). Our typing algorithm will furthermore flag the call (`bst_gen 3 1`) as being ill-typed, since the function's type dictates that the generator's second argument (`1`) *may* only be greater than or equal to its first (`3`).

```
let bst_gen_low_bound (low: int) =
    let (high: int) = int_gen () in
    bst_gen low high
```

**Figure 5.4.** This function generates a BST with a supplied lower bound, `low`.

### Function Application

Since the type of a function parameter is interpreted as a normal (overapproximate, "may") refinement type, while arguments in an application may be typed using (underapproximate, "must") coverage types, we need to be able to bridge the gap between may and must types when typing function applications. Intuitively, our type system does so by ensuring that the set of values in the coverage type of the argument has a nonempty overlap with the set of possible values expected by the function. We establish this connection by using the fact that the typing context captures the control flow paths that may and must exist when the function is called. To illustrate this intuition concretely, consider the function `bst_gen_low_bound` shown in Fig. 5.4. This function generates all non-empty BSTs whose elements are integers with the lower bound given by its parameter. The judgment we need to check is of the form:

$$\texttt{bst_gen} : \texttt{lo}{:}\{\nu{:}\texttt{int} \mid \top\} \rightarrow \texttt{hi}{:}\{\nu{:}\texttt{int} \mid \texttt{lo} \leq \nu\} \rightarrow [\nu{:}\texttt{int tree} \mid ...],$$
$$\texttt{low}{:}\{\nu{:}\texttt{int} \mid \top\},\ \texttt{high}{:}[\nu{:}\texttt{int} \mid \top]$$
$$\vdash \texttt{bst_gen low high} : \ldots$$

Note that the type for `low` is a normal refinement type that specifies a safety condition for function `bst_gen_low_bound`, namely that `low` *may* be any number. In contrast, the type for `high` is a coverage type, representing the result of `int_gen()` that indicates that it *must* (i.e., guaranteed to) be any possible integer. However, the signature for `bst_gen` demands that parameter `hi` only be supplied values greater than its first argument (`lo`); we incorporate this requirement by *strengthening* `high`'s type (via a subsumption rule) to reflect this additional constraint when typing the body of the let expression in which `high` is bound. This strengthening, which is tantamount to a more refined underapproximation, allows us to typecheck the application (`bst_gen low high`) in the following context:

$$\texttt{bst_gen} : \texttt{lo}{:}\{\nu{:}\texttt{int} \mid \top\} \rightarrow \texttt{hi}{:}\{\nu{:}\texttt{int} \mid \texttt{lo} \leq \nu\} \rightarrow [\nu{:}\texttt{int tree} \mid ...],$$
$$\texttt{low}{:}\{\nu{:}\texttt{int} \mid \top\},\ \texttt{high}{:}[\nu{:}\texttt{int} \mid \texttt{low} \leq \nu]$$
$$\vdash \texttt{bst_gen low high} : \ldots$$

The coverage type associated with `high` guarantees that `int_gen()` must produce values greater than `low` (along with possibly other values). To ensure that the result type of the call reflects the underapproximate (coverage) dependences that exist between `low` and `high`, we introduce existential quantifiers in the type's qualifier:

$$\ldots, \texttt{low}{:}\{\nu{:}\texttt{int} \mid \top\}$$
$$\vdash [\nu{:}\texttt{int tree} \mid bst(\nu) \wedge \exists \texttt{high}, \texttt{low} \leq \texttt{high} \wedge \forall u, mem(\nu, u) \Longrightarrow \texttt{low} < u < \texttt{high}]$$

This type properly captures the behavior of the generator: it is guaranteed to generate all BSTs characterized by a lower bound given `low` such that there exists an upper bound `high` where `low` $\leq$ `high` and in which every element in the tree is contained within these bounds.

#### Summary

Coverage types invert many of the expected relationships that are found in a normal refinement type system. Here, qualifiers provide an *underapproximation* of the values that an expression may evaluate to, in contrast to the typically provided *overapproximation*. This, in turn, causes the subtyping relation to invert the standard relationship entailed by logical implication between type qualifiers. Our coverage analysis also considers the *disjunction* of the coverage guarantees provided by the branches of control-flow constructs, instead of their *conjunction*. Finally, when applying a function with a dependent arrow type to a coverage type, we check semantic inclusion between the overapproximate and underapproximate constraints provided by the two types, and manifest the paths that witness the elements guaranteed to be produced by the coverage type through existentially-quantified variables in the application's result type.

### 5.2 Language

#### Terms

In order to formalize our typed-based verification approach of input test generators, we introduce a core calculus for test generators, $\boldsymbol{\lambda}^U$. The language, whose syntax is summarized

| | | |
|---|---|---|
| **Variables** | | $x, y, z, f, u, ...$ |
| **Data constructors** | $d ::=$ | $() \mid \texttt{true} \mid \texttt{false} \mid \texttt{O} \mid \texttt{S} \mid \texttt{Cons} \mid \texttt{Nil} \mid \texttt{Leaf} \mid \texttt{Node}$ |
| **Constants** | $c ::=$ | $\mathbb{B} \mid \mathbb{N} \mid \mathbb{Z} \mid \ldots \mid d\ \overline{c}$ |
| **Operators** | $op ::=$ | $d \mid + \mid == \mid < \mid \texttt{mod} \mid \texttt{NatGen} \mid \texttt{int_gen} \mid ...$ |
| **Values** | $v ::=$ | $c \mid op \mid x \mid \lambda x{:}\tau.e \mid \texttt{fix} f{:}\tau.\lambda x{:}\tau.e$ |
| **Terms** | $e ::=$ | $v \mid \texttt{err} \mid \texttt{let}\ x = e\ \texttt{in}\ e \mid \texttt{let}\ x = op\ \overline{v}\ \texttt{in}\ e$ |
| | | $\mid \texttt{let}\ x = v\ v\ \texttt{in}\ e \mid \texttt{match}\ v\ \texttt{with}\ \overline{d\ \overline{y} \rightarrow e}$ |
| **Base Types** | $b ::=$ | $\texttt{unit} \mid \texttt{bool} \mid \texttt{nat} \mid \texttt{int} \mid \texttt{float} \mid \texttt{char} \mid \texttt{string}$ |
| | | $\mid b \times b \mid \{\overline{x{:}b}\} \mid b\ \texttt{option} \mid b\ \texttt{list} \mid b\ \texttt{tree} \mid \ldots$ |
| **Basic Types** | $t ::=$ | $b \mid t \rightarrow t$ |
| **Method Predicates** | $mp ::=$ | $emp \mid hd \mid mem \mid ...$ |
| **Literals** | $l ::=$ | $c \mid x$ |
| **Propositions** | $\phi ::=$ | $l \mid \bot \mid \top \mid op(\bar{l}) \mid mp(\overline{x})$ |
| | | $\mid \neg\phi \mid \phi \wedge \phi \mid \phi \vee \phi \mid \phi \Longrightarrow \phi \mid \forall u{:}b.\ \phi \mid \exists u{:}b.\ \phi$ |
| **Refinement Types** | $\tau ::=$ | $[\nu{:}b \mid \phi] \mid \{\nu{:}\texttt{b} \mid \phi\} \mid x{:}\tau \rightarrow \tau$ |
| **Type Contexts** | $\Gamma ::=$ | $\emptyset \mid \Gamma, x{:}\tau$ |

**Figure 5.5.** $\boldsymbol{\lambda}^U$ syntax.

in Fig. 5.5, is a call-by-value lambda-calculus with pattern-matching, inductive datatypes, and well-founded (i.e., terminating) recursive functions whose argument must be structurally decreasing in all recursive calls made in the function's body. The syntax of $\boldsymbol{\lambda}^U$ is expressed in monadic normal-form (MNF) [69], a variant of A-Normal Form (ANF) [70] that allows nested let-bindings. Terms in a standard lambda-calculus can be expressed as syntax sugar in ANF, e.g., an application $(e_1\ e_2)$ can be defined as:

$$e_1\ e_2 \doteq \texttt{let}\ f = e_1\ \texttt{in}\ \texttt{let}\ x = e_2\ \texttt{in}\ \texttt{let}\ y = f\ x\ \texttt{in}\ y$$

To enable type-checking, the parameters of lambda function (i.e. $\lambda x{:}\tau.e$) and fixpoint functions (i.e. $\texttt{fix} f{:}\tau.\lambda x{:}\tau.e$) are also notated with coverage types. The language additionally allows faulty programs to be expressed using the error term `err`. As discussed in Sec. 5.1,

this term is important in our investigation because coverage types capture an expression's reachability properties, and we need to ensure the guarantees offered by such types are robust even in the presence of stuck computations induced by statements like `err`. The language is also equipped with primitive operators to generate natural numbers, integers, etc. ($\texttt{NatGen}\ (), \texttt{int_gen}(), \ldots$) that can be used to express various kinds of non-deterministic behavior relevant to test input generation. As an example, the $\oplus$ choice operator used in Fig. 5.1 can be defined as:

$$e_1 \oplus e_2 \doteq \texttt{let}\ n = \texttt{int_gen}\ ()\ \texttt{mod}\ 2\ \texttt{in}\ \texttt{match}\ n\ \texttt{with}\ 0 \rightarrow e_1 \mid _ \rightarrow e_2$$

Note that the primitive generators of $\boldsymbol{\lambda}^U$ are completely agnostic to the specific sampling strategy they employ, as long as they ensure every value in their range has a nonzero likelihood of being generated.

$\boldsymbol{\lambda}^U$ has a completely standard small-step operational semantics.

### Types

Like other refinement type systems [22], $\boldsymbol{\lambda}^U$ supports three classes of types: *base types*, *basic types*, and *refinement types*. Base types ($b$) include primitive types such as `unit`, `bool`, `nat`, etc., product types ($b \times b$), record types $\{\overline{x{:}b}\}$, and inductive datatypes (e.g., `int list`, `bool tree`, `int list list`, etc.). Basic types ($t$) extend base types with function types. Refinement types ($\tau$) qualify base types with both underapproximate and overapproximate propositions, expressed as predicates defined in first-order logic (FOL). Function parameters can also be qualified with overapproximate refinements that specify when it is safe to apply this function. In contrast, the return type of a function can only be qualified using an underapproximate refinement, reflecting the coverage property of the function's result and thus characterizing the values the function is guaranteed to produce. The type context is defined normally, i.e., as a sequence of variable-type bindings consisting of overapproximate refinement types, underapproximate coverage types, and arrow (function) types.

### Qualifiers

To express rich shape properties over inductive datatypes, we allow propositions to reference method predicates, as it is straightforward to generate verification conditions using these uninterpreted functions that can be handled by an off-the-shelf theorem prover like Z3 [8]. As we describe in Section 5.5, our typechecking algorithm imposes additional constraints on the form propositions can take, in order to ensure that its validity is decidable. In particular, we ensure that Z3 queries generated by our typechecker to check refinement validity are always over effectively propositional (EPR) sentences (i.e., prenex-quantified formulae of the form $\exists^*\forall^*\varphi$ where $\varphi$ is quantifier-free.)

## 5.3 Type System

Similar to other contemporary refinement type systems [22], our type system assumes all terms are well-typed in a basic (aka non-refined) type system[4], i.e., $\emptyset \vdash_{\mathtt{s}} e : t$. For example, a term $e$ that has refinement type $\tau$ has basic type $\lfloor \tau \rfloor$ where $\lfloor ... \rfloor$ is a type erasure function, as shown in Fig. 5.6, which erases all qualifiers from refinement types and type contexts.

Despite superficial similarities to other contemporary type systems, the typing rules of $\boldsymbol{\lambda}^U$ differ in significant ways from those of its peers, due to the fundamental semantic distinction that arises when viewing types as an underapproximation and not overapproximation of program behavior.

Our type system depends on three auxiliary relations shown in Fig. 5.6. The first group defines well-formedness conditions on type contexts ($\vdash^{\mathbf{WF}} \Gamma$) as well as refinement types under a particular type context ($\Gamma \vdash^{\mathbf{WF}} \tau$). A type $\tau$ that is well-formed under a well-formed type context $\Gamma$ needs to meet three criteria: (1) the qualifier in $\tau$ needs to be well-typed under the erasure of the current typing context in the basic type system (e.g., requirement $\lfloor \Gamma \rfloor, \nu{:}b \vdash_{\mathtt{s}} \phi : \mathtt{bool}$ in WfBase), (2) overapproximate types may only appear in the domain of a function type (WfArg); and, (3) underapproximate coverage types may only appear in the range of a function type (only underapproximate and function type are allowed as parameter type in WfArg andWfRes respectively). A type context is well-formed when all types in a

[4]↑The basic typing rules are provided in appendix.

**Type Erasure** $\boxed{\lfloor \tau \rfloor \quad \lfloor \Gamma \rfloor}$

$$\lfloor \{\nu{:}\mathtt{b} \mid \phi\} \rfloor \doteq b \quad \lfloor [\nu{:}\mathtt{b} \mid \phi] \rfloor \doteq b \quad \lfloor x{:}t \to \tau \rfloor \doteq \lfloor t \rfloor \to \lfloor \tau \rfloor \quad \lfloor \emptyset \rfloor \doteq \emptyset \quad \lfloor x{:}\tau, \Gamma \rfloor \doteq x{:}\lfloor \tau \rfloor, \lfloor \Gamma \rfloor$$

**Well-Formedness** $\boxed{\vdash^{\mathbf{WF}} \Gamma \quad \Gamma \vdash^{\mathbf{WF}} \tau}$

$$\frac{\begin{array}{c}\vdash^{\mathbf{WF}} \Gamma \\ \lfloor \Gamma \rfloor, \nu{:}b \vdash_{\mathtt{s}} \phi : \mathtt{bool}\end{array}}{\Gamma \vdash^{\mathbf{WF}} [\nu{:}\mathtt{b} \mid \phi]} \ \textsc{WfBase} \qquad \frac{\Gamma, x{:}\{\nu{:}\mathtt{b} \mid \phi\} \vdash^{\mathbf{WF}} \tau}{\Gamma \vdash^{\mathbf{WF}} x{:}\{\nu{:}\mathtt{b} \mid \phi\} \to \tau} \ \textsc{WfRes}$$

$$\frac{\Gamma, x{:}(y{:}\tau_y \to \tau) \vdash^{\mathbf{WF}} \tau}{\Gamma \vdash^{\mathbf{WF}} x{:}(y{:}\tau_y \to \tau) \to \tau} \ \textsc{WfRes}$$

$$\frac{}{\vdash^{\mathbf{WF}} \emptyset} \ \textsc{WfCtxEmp} \qquad \frac{\vdash^{\mathbf{WF}} \Gamma \quad \Gamma \vdash^{\mathbf{WF}} [\nu{:}\mathtt{b} \mid \phi]}{\vdash^{\mathbf{WF}} \Gamma, x{:}\{\nu{:}\mathtt{b} \mid \phi\}} \ \textsc{WfCtxOBase}$$

$$\frac{\begin{array}{c}\vdash^{\mathbf{WF}} \Gamma \quad \Gamma \vdash^{\mathbf{WF}} [\nu{:}\mathtt{b} \mid \phi] \\ \mathtt{err} \notin [\![ [\nu{:}\mathtt{b} \mid \phi]_\Gamma ]\!]\end{array}}{\vdash^{\mathbf{WF}} \Gamma, x{:}[\nu{:}\mathtt{b} \mid \phi]} \ \textsc{WfCtxUBase} \qquad \frac{\vdash^{\mathbf{WF}} \Gamma \quad \Gamma \vdash^{\mathbf{WF}} y{:}\tau_y \to \tau}{\vdash^{\mathbf{WF}} \Gamma, x{:}(y{:}\tau_y \to \tau)} \ \textsc{WfCtxArr}$$

**Subtyping** $\boxed{\Gamma \vdash \tau_1 <: \tau_2}$

$$\frac{\begin{array}{c}[\![ [\nu{:}\mathtt{b} \mid \phi_1] ]\!]_\Gamma \subseteq \\ [\![ [\nu{:}\mathtt{b} \mid \phi_2] ]\!]_\Gamma\end{array}}{\Gamma \vdash [\nu{:}\mathtt{b} \mid \phi_1] <: [\nu{:}\mathtt{b} \mid \phi_2]} \ \textsc{SubUBase} \qquad \frac{\begin{array}{c}[\![ \{\nu{:}b \mid \phi_1\} ]\!]_\Gamma \subseteq \\ [\![ \{\nu{:}b \mid \phi_2\} ]\!]_\Gamma\end{array}}{\Gamma \vdash \{\nu{:}b \mid \phi_1\} <: \{\nu{:}b \mid \phi_2\}} \ \textsc{SubOBase}$$

$$\frac{\begin{array}{c}\Gamma \vdash \tau_{21} <: \tau_{11} \\ \Gamma, x{:}\tau_{21} \vdash \tau_{12} <: \tau_{22}\end{array}}{\Gamma \vdash x{:}\tau_{11} \to \tau_{12} <: x{:}\tau_{21} \to \tau_{22}} \ \textsc{SubArr}$$

**Disjunction** $\boxed{\Gamma \vdash \tau_1 \vee \tau_2 = \tau_3}$

$$\frac{[\![ \tau_1 ]\!]_\Gamma \cap [\![ \tau_2 ]\!]_\Gamma = [\![ \tau_3 ]\!]_\Gamma}{\Gamma \vdash \tau_1 \vee \tau_2 = \tau_3} \ \textsc{Disjunction}$$

**Figure 5.6.** Auxiliary typing relations

type context are well-formed under the context derived from the former's bindings, moreover,

the denotations[5] of coverage base types ($[\nu{:}\mathtt{b} \mid \phi]$) should not include $\mathtt{err}$ (WfCtxUBase). To understand the motivation for this constraint, observe that a type context in our setting provides a witness to feasible execution paths in the form of bindings to local variables. Accordingly, the type context $x{:}[\nu{:}\mathtt{nat} \mid \bot]$ or $x{:}\{\nu{:}\mathtt{nat} \mid \nu > 0\}, y{:}[\nu{:}\mathtt{nat} \mid x = 0 \wedge \nu = 2]$ are not well-formed, as neither context corresponds to a valid manifest execution path. On the other hand, a well-formed type is allowed to include an error term in its denotation, e.g., type $[\nu{:}\mathtt{nat} \mid \bot]$ is well-formed under type context $x{:}\{\nu{:}\mathtt{nat} \mid \nu > 0\}$ as it always corresponds to a valid underapproximation.

Our second set of judgments defines a largely standard subtyping relation based on the underlying denotation of the types being related. Note also that over- and under-approximate types are incomparable— our typing rules tightly control when one can be treated as another.

The disjunction rule (Disjunction), which was informally introduced in Sec. 5.1, merges the types found along distinct control paths. Intuitively, the type $[\nu{:}\mathtt{nat} \mid \nu = 1 \vee \nu = 2]$ is the disjunction of the types $[\nu{:}\mathtt{nat} \mid \nu = 1]$ and $[\nu{:}\mathtt{nat} \mid \nu = 2]$. Notice that only an inhabitant of both $[\nu{:}\mathtt{nat} \mid \nu = 1]$ and $[\nu{:}\mathtt{nat} \mid \nu = 2]$ should be included in their disjunction: e.g., the term $1 \oplus 2$ is one such inhabitant. Thus, we formally define this relation as the *intersection* of the denotations of two types.

The rules of our type system are defined in Fig. 5.7 and Fig. 5.8, and include typing judgments for terms ($\Gamma \vdash e : \tau$), primitive operators ($\Gamma \vdash op : \tau$), and data constructors ($\Gamma \vdash d : \tau$). All our typing rules assume that terms are well-typed according to the basic type system; the basic type system is completely standard and we omit it here. The rules collectively maintain the invariant that terms can only be assigned a well-formed type. The rules for constants (TConst), operators (TOp), and data constructors (TDt) are straightforward. It relies on an auxiliary function, **Ty**, to assign types to the primitives of $\boldsymbol{\lambda}^U$. Table 5.2 presents some examples of the typings provided by **Ty**. We use method predicates in the types of constructors: the types for list constructors, for example, use *emp*, *hd* and *tl*, to precisely capture that $\mathtt{[]}$ constructs an empty list, and that $(\mathtt{Cons}\, x\, y)$ builds a list containing $x$ as its head and $y$ as its tail.

[5]↑The definition of a type's denotation is given in Sec. 5.4.

**Typing** $\boxed{\Gamma \vdash e : \tau \quad \Gamma \vdash op : \tau \quad \Gamma \vdash d : \tau}$

$$\frac{\Gamma \vdash^{\mathbf{WF}} [\nu{:}\mathtt{b} \mid \bot]}{\Gamma \vdash \mathtt{err} : [\nu{:}\mathtt{b} \mid \bot]}\ \textsc{TErr} \qquad \frac{\Gamma \vdash^{\mathbf{WF}} \mathbf{Ty}(c)}{\Gamma \vdash c : \mathbf{Ty}(c)}\ \textsc{TConst} \qquad \frac{\Gamma \vdash^{\mathbf{WF}} \mathbf{Ty}(d)}{\Gamma \vdash d : \mathbf{Ty}(d)}\ \textsc{TDt}$$

$$\frac{\Gamma \vdash^{\mathbf{WF}} \mathbf{Ty}(op)}{\Gamma \vdash op : \mathbf{Ty}(op)}\ \textsc{TOp}$$

$$\frac{\Gamma \vdash^{\mathbf{WF}} [\nu{:}\mathtt{b} \mid \nu = x]}{\Gamma \vdash x : [\nu{:}\mathtt{b} \mid \nu = x]}\ \textsc{TVarBase} \qquad \frac{\Gamma(x) = (a{:}\tau_a \to \tau_b) \quad \Gamma \vdash^{\mathbf{WF}} a{:}\tau_a \to \tau_b}{\Gamma \vdash x : (a{:}\tau_a \to \tau_b)}\ \textsc{TVarFun}$$

$$\frac{\Gamma, x{:}\tau_x \vdash e : \tau \quad \Gamma \vdash^{\mathbf{WF}} x{:}\tau_x \to \tau}{\Gamma \vdash \lambda x{:}\tau_x.e : (x{:}\tau_x \to \tau)}\ \textsc{TFun}$$

$$\frac{\begin{array}{c}\Gamma, x{:}\{\nu{:}b \mid \phi\}, f{:}(x{:}\{\nu{:}b \mid \nu \prec x \wedge \phi\} \to \tau) \vdash e : \tau \\ \Gamma \vdash x{:}\{\nu{:}b \mid \phi\} \to \tau\end{array}}{\Gamma \vdash \mathtt{fix} f{:}(x{:}\{\nu{:}b \mid \phi\} \to \tau).\lambda x{:}\{\nu{:}b \mid \phi\}.e \ : \ x{:}\{\nu{:}b \mid \phi\} \to \tau}\ \textsc{TFix}$$

**Figure 5.7.** Typing rules (Part 1)

**Table 5.2.** Example typings for $\boldsymbol{\lambda}^U$ primitives

| | |
|---|---|
| Constants | $\mathbf{Ty}(\mathtt{true}) = [\nu{:}\mathtt{bool} \mid \nu] \quad \mathbf{Ty}(8) = [\nu{:}\mathtt{nat} \mid \nu = 8]$... |
| Data Constructors | $\mathbf{Ty}([]) = [\nu{:}\mathtt{b}\ \mathtt{list} \mid emp(\nu)]$<br>$\mathbf{Ty}(\mathtt{Cons}) = x{:}\{\nu{:}\mathtt{b} \mid \top\} \to y{:}\{\nu{:}\mathtt{b}\ \mathtt{list} \mid \top\} \to [\nu{:}\mathtt{b}\ \mathtt{list} \mid hd(\nu, x) \wedge tl(\nu, y)]$... |
| Operators | $\mathbf{Ty}(\mathtt{NatGen}) = [\nu{:}\mathtt{unit} \mid \top] \to [\nu{:}\mathtt{int} \mid 0 \leq \nu]$<br>$\mathbf{Ty}(+) = x{:}\{\nu{:}\mathtt{int} \mid \top\} \to y{:}\{\nu{:}\mathtt{int} \mid \top\} \to [\nu{:}\mathtt{int} \mid \nu = x + y]$... |

The typing rules for function abstraction (TFun) and error (TErr) are similarly straightforward. In a $\lambda$-abstraction, the type of the bound variable ($\tau_x$) is explicitly given. The error term can be assigned an arbitrary bottom coverage base type. The variable rule (TVarBase) establishes that the variable $x$ in the type context with a base type can also be typed with the tautological qualifier $\nu = x$ (the rule's well-formedness assumption ensures that $x$ is not free). This judgment allows us to, for example, type the function $\lambda x : nat.x$ with the type $x{:}\{\nu{:}\mathtt{nat} \mid \top\} \to [\nu{:}\mathtt{nat} \mid \nu = x]$, indicating that the return value is guaranteed to be exactly equal to the input $x$. Observe that the type of $x$ under the type context $x{:}\{\nu{:}\mathtt{nat} \mid \top\}$ (generated by the function rule TFun) is *not* $[\nu{:}\mathtt{nat} \mid \top]$. We cannot simply duplicate the

$$\frac{\emptyset \vdash \tau <: \tau' \quad \emptyset \vdash e : \tau \quad \Gamma \vdash^{\mathbf{WF}} \tau'}{\Gamma \vdash e : \tau'} \ \textsc{TSub} \qquad \frac{\Gamma \vdash \tau' <: \tau \quad \Gamma \vdash \tau <: \tau' \quad \Gamma \vdash e : \tau \quad \Gamma \vdash^{\mathbf{WF}} \tau'}{\Gamma \vdash e : \tau'} \ \textsc{TEq}$$

$$\frac{\Gamma \vdash e : \tau_1 \quad \Gamma \vdash e : \tau_2 \quad \Gamma \vdash \tau_1 \vee \tau_2 = \tau \quad \Gamma \vdash^{\mathbf{WF}} \tau}{\Gamma \vdash e : \tau} \ \textsc{TMerge} \qquad \frac{\Gamma \vdash e_x : \tau_x \quad \Gamma, x{:}\tau_x \vdash e : \tau \quad \Gamma \vdash^{\mathbf{WF}} \tau}{\Gamma \vdash \texttt{let}\ x = e_x\ \texttt{in}\ e : \tau} \ \textsc{TLetE}$$

$$\frac{\Gamma \vdash op : \overline{a_i{:}\{\nu{:}\texttt{b}_\texttt{i} \mid \phi_i\}} \to \tau_x \quad \forall i, \Gamma \vdash v_i : [\nu{:}\texttt{b}_\texttt{i} \mid [\phi_i] \quad \Gamma, x{:}\tau_x[\overline{a_i \mapsto v_i}] \vdash e : \tau \quad \Gamma \vdash^{\mathbf{WF}} \tau}{\Gamma \vdash \texttt{let}\ x = op\ \overline{v_i}\ \texttt{in}\ e : \tau} \ \textsc{TAppOp}$$

$$\frac{\Gamma \vdash v_1 : (\tau_1 \to \tau_2) \to \tau_x \quad \Gamma \vdash v_2 : \tau_1 \to \tau_2 \quad \Gamma, x{:}\tau_x \vdash e : \tau \quad \Gamma \vdash^{\mathbf{WF}} \tau}{\Gamma \vdash \texttt{let}\ x = v_1\ v_2\ \texttt{in}\ e : \tau} \ \textsc{TAppFun}$$

$$\frac{\Gamma \vdash v_1 : a{:}\{\nu{:}\texttt{b} \mid \phi\} \to \tau_x \quad \Gamma \vdash v_2 : [\nu{:}\texttt{b} \mid \phi] \quad \Gamma, x{:}\tau_x[a \mapsto v_2] \vdash e : \tau \quad \Gamma \vdash^{\mathbf{WF}} \tau}{\Gamma \vdash \texttt{let}\ x = v_1\ v_2\ \texttt{in}\ e : \tau} \ \textsc{TApp} \qquad \frac{\Gamma \vdash v : \tau_v \quad \Gamma \vdash^{\mathbf{WF}} \tau \quad \Gamma, \overline{y{:}\tau_y} \vdash d_i(\overline{y}) : \tau_v \quad \Gamma, \overline{y{:}\tau_y} \vdash e_i : \tau}{\Gamma \vdash (\texttt{match}\ v\ \texttt{with}\ \overline{d_i\ \overline{y} \to e_i}) : \tau} \ \textsc{TMatch}$$

**Figure 5.8.** Typing rules (Part 2)

qualifier for $x$ from the type context here, as this is only sound when types characterize an overapproximation of program behavior. As an example, $\{\nu{:}\texttt{nat} \mid \top\}$ is a subtype of $\{\nu{:}\texttt{nat} \mid \nu = x\}$ under the type context $x{:}\{\nu{:}\texttt{nat} \mid \top\}$; in our underapproximate coverage type system, in contrast, $[\nu{:}\texttt{nat} \mid \top]$ is not a subtype of $[\nu{:}\texttt{nat} \mid \nu = x]$ under the type context $x{:}\{\nu{:}\texttt{nat} \mid \top\}$ .

The typing rule for application (TApp) requires both its underapproximate argument type and the overapproximate parameter type to have the same qualifier, and furthermore requires that the type of the body ($\tau$) is well-formed under the original type context $\Gamma$, enforcing that $x$ (the result of the application) does not appear free in $\tau$. When argument and parameter qualifiers are not identical, a subsumption rule is typically used to bring the

two types into alignment. Recall the following example from Sec. 5.1, suitably modified to conform to $\boldsymbol{\lambda}^U$'s syntax:

$$\texttt{bst_gen} : \texttt{lo}{:}\{\nu{:}\texttt{int} \mid \top\} \to \texttt{hi}{:}\{\nu{:}\texttt{int} \mid \texttt{lo} \leq \nu\} \to [\nu{:}\texttt{int tree} \mid \ldots], \texttt{low} : \{\nu{:}\texttt{int} \mid \top\} \vdash$$

```
let (g: unit -> int) = int_gen in let (x: unit) = () in
let (high: int) = g x in let (y: int tree) = bst_gen low high in y
```

Here, the type of `high`, $[\nu{:}\texttt{int} \mid \top]$ is stronger than the type expected for the second parameter of `bst_gen`, $[\nu{:}\texttt{int} \mid \texttt{lo} \leq \nu]$. The subsumption rule (TSub), that would normally allow us to strengthen the type of `high` to align with the required parameter type, is applicable to only closed terms, which `high` is not. For the same reason, we cannot use TSub to strengthen the type of `high` when it is bound to `g x`. Thankfully, we can strengthen `g` when it is bound to `int_gen`: according to Table 5.2, the operator `int_gen` has type $\{\nu{:}\texttt{unit} \mid \top\} \to [\nu{:}\texttt{int} \mid \top]$ and is also closed, and can thus be strengthened via TSub, allowing us to type the call to `bst_gen` under the following, stronger type context:

$$\texttt{bst_gen} : \texttt{lo}{:}\{\nu{:}\texttt{int} \mid \top\} \to \texttt{hi}{:}\{\nu{:}\texttt{int} \mid \texttt{lo} \leq \nu\} \to [\nu{:}\texttt{int tree} \mid \ldots], \texttt{low} : \{\nu{:}\texttt{int} \mid \top\},$$
$$\texttt{g} : [\nu{:}\texttt{unit} \mid \top][\nu{:}\texttt{int} \mid \texttt{low} \leq \nu], \texttt{x} : [\nu{:}\texttt{unit} \mid \top], \texttt{high} : [\nu{:}\texttt{int} \mid \texttt{low} \leq \nu] \vdash$$

```
let (y: int tree) = bst_gen low high in y
```

The subsumption rule allows us to use `int_gen` in a context that requires *fewer* guarantees than `int_gen` actually provides, namely those values of `high` required by the signature of `bst_gen`. Intuitively, since our notion of coverage types records feasible executions in the type context in the form of existentials that serve as witnesses to an underapproximation, the strengthening provided by the subsumption rule establishes an invariant that all bindings introduced into a type context only characterize valid behaviors in a program execution. When coupled with TMerge, this allows us to *split* a typing derivation into multiple plausible strengthenings when a variable is introduced into the typing context and then *combine* the resulting types to reason about multiple feasible paths.

Now, using TApp to type `bst_gen low high`, and TVarBase to type the body of the `let` gives us:

$$\texttt{bst_gen} : \texttt{lo}{:}\{\nu{:}\texttt{int} \mid \top\} \to \texttt{hi}{:}\{\nu{:}\texttt{int} \mid \texttt{lo} \leq \nu\} \to [\nu{:}\texttt{int tree} \mid \ldots],$$
$$\texttt{low} : \{\nu{:}\texttt{int} \mid \top\}, \texttt{high} : [\nu{:}\texttt{int} \mid \texttt{low} \leq \nu],$$

$\texttt{y} : [\nu{:}\texttt{int tree} \mid bst(\nu) \wedge \forall u, mem(\nu, u) \Longrightarrow \texttt{lo} < u < \texttt{hi}][\texttt{lo} \mapsto \texttt{low}][\texttt{hi} \mapsto \texttt{high}]$
$\vdash \texttt{y} : [\nu{:}\texttt{int tree} \mid \nu = \texttt{y}]$

Observe that TVarBase types the body as: $[\nu{:}\texttt{int tree} \mid \nu = \texttt{y}]$, which is not closed. To construct a well-formed term, we need a formula equivalent to this type that accounts for the type of `y` in the current type context. The TEq rule allows us to interchange formulae that are equivalent under a given type context to ensure the well-formedness of the types constructed. Unlike TSub, it simply changes the form of a type's qualifiers, *without altering* the scope of feasible behaviors under the current context. In this example, such an equivalent closed type, given the binding for `y` in the type context under which the expression is being type-checked, would be:

```
let (y: int tree) = bst_gen low high in y :
```
$[\nu{:}\texttt{int tree} \mid \exists \texttt{y}, (bst(\texttt{y}) \wedge \forall u, mem(\texttt{y}, u) \Longrightarrow \texttt{low} < u < \texttt{high}) \wedge \nu = \texttt{y}]$

With these pieces in hand, we can see that the typing rule for `match` is a straightforward adaptation of the components we have already seen, where the type of matched variable $v$ is assumed to have been strengthened by the rule TSub to fit the type required to take the $i$th branch $\Gamma, \overline{y{:}\tau_y} \vdash d_i(\overline{y}) : \tau_v$. We can also safely assume the type of the branch $\tau_i$ is closed under original type context $\Gamma$, relying on TEq to meet this requirement. While TMatch only allows for a single branch to be typechecked, applying TMerge allows us to reason about the coverage provided by multiple branches, which have all been typed according to this rule.

As in TFun, the typing rule for recursive functions (TFix) requires the types of the self-reference to $f$ and the function parameter $x$ to be explicitly given.

Since types in our language serve as witnesses to feasible executions, the result type of any recursive procedure must characterize the set of values the procedure can plausibly return. Thus, the TFix rule forces its first argument to always decrease according to some well-founded relation $\prec$. To see why we impose this restriction, consider the function **loop**:

```
let rec loop (n: nat) = loop n
```

Without termination check, this function can be assigned the following coverage type

$$\{\nu{:}\texttt{nat} \mid \top\} \to [\nu{:}\texttt{nat} \mid \nu = 3]$$

despite the fact that this function never returns `3`— or any value at all! The body of this expression can be type-checked under the following type context (via TFix and TFun):

$$\texttt{n}{:}\{\nu{:}\texttt{nat} \mid \top\},\ \texttt{loop}{:}(\texttt{n}{:}\{\nu{:}\texttt{nat} \mid \top\} \to [\nu{:}\texttt{nat} \mid \nu = 3]) \vdash \texttt{loop n} : [\nu{:}\texttt{nat} \mid \nu = 3]$$

This judgment reflects an infinitely looping execution, however. Indeed, the same reasoning allows us to type this function with any result type. Constraining `loop`'s argument type to be decreasing according to $\prec$ yields the following typing obligation:

$$\texttt{n}{:}\{\nu{:}\texttt{nat} \mid \top\},\ \texttt{loop}{:}(\texttt{n}{:}\{\nu{:}\texttt{nat} \mid \nu \prec \texttt{n}\} \to [\nu{:}\texttt{nat} \mid \nu = 3]) \vdash \texttt{n} : [\nu{:}\texttt{nat} \mid \nu = \texttt{n}]$$

where the qualifiers $\nu \prec \texttt{n}$ and $\nu = \texttt{n}$ conflict, raising a type error, and preventing `loop` from being recursively applied to `n`.

## 5.4 Type Soundness

### Type Denotations

Assuming a standard typing judgement for basic types, $\emptyset \vdash_{\mathsf{s}} e : t$, a type denotation for a type $\tau$, $[\![\tau]\!]$, is a set of closed expressions:

$$\begin{aligned}
[\![\{\nu{:}b \mid \phi\}]\!] &\doteq \{v \mid \emptyset \vdash_{\mathsf{s}} v : b \wedge \phi[\nu \mapsto v]\} \\
[\![[\nu{:}\mathtt{b} \mid \phi]]\!] &\doteq \{e \mid \emptyset \vdash_{\mathsf{s}} e : b \wedge \forall v{:}b, \phi[\nu \mapsto v] \Longrightarrow e \hookrightarrow^* v\} \\
[\![x{:}\tau_x \to \tau]\!] &\doteq \{f \mid \emptyset \vdash_{\mathsf{s}} f : \lfloor \tau_x \to \tau \rfloor \wedge \forall v_x \in [\![\tau_x]\!] \Longrightarrow f\ v_x \in [\![\tau[x \mapsto v_x]]\!]\}
\end{aligned}$$

In the case of an overapproximate refinement type, $\{\nu{:}b \mid \phi\}$, the denotation is simply the set of all values of type $b$ whose elements satisfy the type's refinement predicate ($\phi$), when substituted for all free occurrences of $\nu$ in $\phi$.[6] Dually, the denotation of an underapproximate

[6]↑The denotation of an overapproximate refinement type is more generally $\{e{:}b \mid \emptyset \vdash e : b \wedge \forall v{:}b, e \hookrightarrow^* v \Longrightarrow \phi[x \mapsto v]\}$. However, because such types are only used for function parameters, and our language syntax only admits values as arguments, our denotation uses the simpler form.

coverage type is the set of expressions that evaluate to $v$ whenever $\phi[\nu \mapsto v]$ holds, where $\phi$ is the type's refinement predicate. Thus, every expression in such a denotation serves as a witness to a feasible, type-correct, execution. The denotation for a function type is defined in terms of the denotations of the function's argument and result in the usual way, ensuring that our type denotation is a logical predicate.

### Type Denotations under a Type Context

The denotation of a refinement type $\tau$ under a type context $\Gamma$ (written $[\![\tau]\!]_\Gamma$) is:[7],

$$
\begin{aligned}
[\![\tau]\!]_\emptyset &\doteq [\![\tau]\!] \\
[\![\tau]\!]_{x:\tau_x,\Gamma} &\doteq \{e \mid \forall v_x \in [\![\tau_x]\!].\texttt{let}\ x = v_x\ \texttt{in}\ e \in [\![\tau[x \mapsto v_x]]\!]_{\Gamma[x \mapsto v_x]}\} && \text{if } \tau \equiv \{\nu{:}b \mid \phi\} \\
[\![\tau]\!]_{x:\tau_x,\Gamma} &\doteq \{e \mid \exists \hat{e}_x \in [\![\tau_x]\!].\forall e_x \in [\![\tau_x]\!]. \\
&\qquad \texttt{let}\ x = e_x\ \texttt{in}\ e \in \bigcap_{\hat{e}_x \hookrightarrow^* v_x} [\![\tau[x \mapsto v_x]]\!]_{\Gamma[x \mapsto v_x]}\} && \text{otherwise}
\end{aligned}
$$

The denotation of an overapproximate refinement type under a type context is mostly unsurprising, other than our presentation choice to use a let-binding, rather than substitution, to construct the expressions included in the denotations.

For a coverage type, however, the definition precisely captures our notion of a reachability witness by explicitly constructing an execution path as a sequence of `let`-bindings that justifies the inhabitant of the target type $\tau$. Using `let`-bindings forces expressions in the denotation to make consistent choices when evaluated. The existential introduced in the definition captures the notion of an underapproximation, while the use of set intersection allows us to reason about non-determinism introduced by primitive generators like `nat_gen ()`.

**Example 5.4.1.** The term $x + 1$ is included in the denotation of the following type

$$[\nu{:}\texttt{nat} \mid \nu = x + 1 \lor \nu = x + x]$$

[7]↑In the last case, since $\hat{e}_x$ may nondetermistically reduce to multiple values, we employ intersection (not union), similar to the Disjunction rule.

under the type context $x : [\nu{:}\texttt{nat} \mid \nu = 1]$. This is justified by picking 1 for $\hat{e}_x$, which yields a set intersection that is equivalent to $[\![[\nu{:}\texttt{nat} \mid \nu = 2]]\!]$. Observe that *any* expression in $[\![[\nu{:}\texttt{nat} \mid \nu = 1]]\!]$, e.g. $0 \oplus 1$ and $1 \oplus 2$, yields an expression, $\texttt{let}\ x = 0 \oplus 1\ \texttt{in}\ x + 1$ or $\texttt{let}\ x = 1 \oplus 2\ \texttt{in}\ x + 1$, included in this intersection.

**Example 5.4.2.** On the other hand, the term $x$ is not a member of this denotation. To see why, let us pick `nat_gen()` for $\hat{e}_x$. This yields a set intersection that is equivalent to $[\![[\nu{:}\texttt{nat} \mid \top]]\!]$. While specific choices for $e_x$, e.g., `nat_gen()`, are included in this denotation, it does not work for all terms $e_x \in [\![[\nu{:}\texttt{nat} \mid \nu = 1]]\!]$. As one example, $0 \oplus 1 \oplus 2$ is an element of this set, but $\texttt{let}\ x = 0 \oplus 1 \oplus 2\ \texttt{in}\ x$ is clearly not a member of $[\![[\nu{:}\texttt{nat} \mid \top]]\!]$. Suppose instead that we picked a more restrictive expression for $\hat{e}_x$, like the literal 1 from the previous example. Here, it is easy to choose $e_x \in [\![[\nu{:}\texttt{nat} \mid \nu = 1]]\!]$ (e.g., the literal 1) such that $\texttt{let}\ x = e_x\ \texttt{in}\ x \notin [\![[\nu{:}\texttt{nat} \mid \nu = 2]]\!]$.

When reasoning about a subset relation between the denotations of two types under a type context $[\![[\nu{:}\texttt{b} \mid \phi_1]]\!]_\Gamma \subseteq [\![[\nu{:}\texttt{b} \mid \phi_2]]\!]_\Gamma$ we require that the denotations be computed using the same $\Gamma$. Then, the subset relation between the denotations of two refinement types $\tau_1$ and $\tau_2$ under a type context $\Gamma$ (written $[\![\tau_1]\!]_\Gamma \subseteq [\![\tau_2]\!]_\Gamma$) is defined as follows:

$$
\begin{aligned}
[\![\tau_1]\!]_\emptyset \subseteq [\![\tau_2]\!]_\emptyset &\doteq [\![\tau_1]\!] \subseteq [\![\tau_2]\!] \\
[\![\tau_1]\!]_{x:\tau_x,\Gamma} \subseteq [\![\tau_2]\!]_{x:\tau_x,\Gamma} &\doteq \forall v_x \in [\![\tau_x]\!], \\
&\qquad [\![\tau_1[x \mapsto v_x]]\!]_{\Gamma[x\mapsto v_x]} \subseteq [\![\tau_2[x \mapsto v_x]]\!]_{\Gamma[x\mapsto v_x]} \quad \text{if } \tau \equiv \{\nu{:}\texttt{b} \mid \phi\} \\
[\![\tau_1]\!]_{x:\tau_x,\Gamma} \subseteq [\![\tau_2]\!]_{x:\tau_x,\Gamma} &\doteq \exists \hat{e}_x \in [\![\tau_x]\!], \forall v_x, \hat{e}_x \hookrightarrow^* v_x \implies \\
&\qquad [\![\tau_1[x \mapsto v_x]]\!]_{\Gamma[x\mapsto v_x]} \subseteq [\![\tau_2[x \mapsto v_x]]\!]_{\Gamma[x\mapsto v_x]} \qquad \text{otherwise}
\end{aligned}
$$

The way we interpret the type context $\Gamma$ here is the same as in the definition of type denotation under a type context, but we maintain the denotations of $\tau_1$ and $\tau_2$ as a subset relation under the same interpretation of $\Gamma$, that is, under the *same* substitution $[x \mapsto v_x]$. This constraint is also required by other refinement type systems, which define the denotation of the type context $\Gamma$ as a set of substitutions, with the subset relation between the denotations of two types holding under the *same* substitution. However, our type context is more complicated, since

it contains both under- and overapproximate types that are interpreted via existential and universal quantifiers, and thus cannot simply be denoted as a set of substitutions. Therefore, we define a subset relation over denotations under a type context to ensure that the same substitution is applied to both types.

Our main soundness result establishes the correctness of type-checking in the presence of coverage types with respect to a type's denotation:

**Theorem 5.4.3.** *[Type Soundness] For all type contexts* $\Gamma$*, terms* $e$ *and coverage types* $\tau$*,* $\Gamma \vdash e : \tau \implies e \in [\![\tau]\!]_\Gamma$.[8]

It immediately follows that a closed input generator `e` with coverage type $[\nu{:}\mathtt{b} \mid \phi]$ must produce every value satisfying $\phi$, as desired.

## 5.5 Typing Algorithm

The declarative typing rules are highly nondeterministic, relying on a combination of the TMerge and TSub rules to both explore and combine the executions needed to establish the desired coverage properties. In addition, each of the auxillary typing relations depend on logical properties of the semantic interpretation of types. Any effective type checking algorithm based on these rules must address both of these issues. Our solution to the first problem is to implement a bidirectional type checker [34] whose type synthesis phase characterizes a set of feasible paths and whose type checking phase ensures those paths produce the desired results. Our solution to the second is to encode the logical properties into a *decidable* fragment of first order logic that can be effectively discharged by an SMT solver.

### 5.5.1 Bidirectional Typing Algorithm

As is standard in bidirectional type systems, our typing algorithm consists of a type synthesis judgement ($\Gamma \vdash e \Rightarrow \tau$) and a type checking judgment ($\Gamma \vdash e \Leftarrow \tau$) as presented in Fig. 5.9 and Fig. 5.10. Most of rules are similar to the declarative typing rules, e.g., the type synthesis rule SynVarBase has the same premise as the declarative typing rule

[8]↑The Coq formalization and proof of type soundness are publicly available on Zenodo[88].

**Type Synthesis** $\boxed{\Gamma \vdash e \Rightarrow \tau}$ **Type Check** $\boxed{\Gamma \vdash e \Leftarrow \tau}$

$$\frac{\Gamma \vdash^{\mathbf{WF}} \mathbf{Ty}(c)}{\Gamma \vdash c \Rightarrow \mathbf{Ty}(c)}\ \textsc{SynConst} \qquad \frac{\Gamma \vdash^{\mathbf{WF}} [\nu{:}\mathtt{b} \mid \bot]}{\Gamma \vdash \mathtt{err} \Rightarrow [\nu{:}\mathtt{b} \mid \bot]}\ \textsc{SynErr}$$

$$\frac{\Gamma \vdash^{\mathbf{WF}} [\nu{:}\mathtt{b} \mid \nu = x]}{\Gamma \vdash x \Rightarrow [\nu{:}\mathtt{b} \mid \nu = x]}\ \textsc{SynVarBase}$$

$$\frac{\Gamma(x) = a{:}\tau_a \to \tau_b \quad \Gamma \vdash^{\mathbf{WF}} a{:}\tau_a \to \tau_b}{\Gamma \vdash x \Rightarrow a{:}\tau_a \to \tau_b}\ \textsc{SynVarFun} \qquad \frac{\begin{array}{c}\Gamma, x{:}\{\nu{:}\mathtt{b} \mid \top\} \vdash e \Rightarrow \tau \\ \Gamma \vdash^{\mathbf{WF}} x{:}\{\nu{:}\mathtt{b} \mid \top\} \to \tau\end{array}}{\begin{array}{c}\Gamma \vdash \\ \lambda x{:}b.e \Rightarrow x{:}\{\nu{:}\mathtt{b} \mid \top\} \to \tau\end{array}}\ \textsc{ChkFun}$$

$$\frac{\begin{array}{c}\Gamma \vdash v_1 \Rightarrow \tau_2 \to \tau_x \\ \lfloor \tau_2 \rfloor \notin b \quad \Gamma \vdash v_2 \Leftarrow \tau_2 \\ \Gamma' = x{:}\tau_x \\ \Gamma, \Gamma' \vdash e \Rightarrow \tau \\ \tau' = \mathtt{Ex}(\Gamma', \tau) \\ \Gamma \vdash^{\mathbf{WF}} \tau'\end{array}}{\Gamma \vdash \mathtt{let}\ x = v_1\ v_2\ \mathtt{in}\ e \Rightarrow \tau'}\ \textsc{SynAppFun} \qquad \frac{\begin{array}{c}\Gamma \vdash \\ v_1 \Rightarrow a{:}\{\nu{:}\mathtt{b} \mid \phi\} \to \tau_x \\ \Gamma' = \\ a{:}[\nu{:}\mathtt{b} \mid \nu = v_2 \wedge \phi], x{:}\tau_x \\ \Gamma, \Gamma' \vdash e \Rightarrow \tau \\ \tau' = \mathtt{Ex}(\Gamma', \tau) \quad \Gamma \vdash^{\mathbf{WF}} \tau'\end{array}}{\Gamma \vdash \mathtt{let}\ x = v_1\ v_2\ \mathtt{in}\ e \Rightarrow \tau'}\ \textsc{SynAppBase}$$

$$\frac{\begin{array}{c}\mathbf{Ty}(op) \Rightarrow \overline{a_i{:}\{\nu{:}\mathtt{b_i} \mid \phi_i\}} \to \tau_x \\ \Gamma' = \\ \overline{a_i{:}[\nu{:}\mathtt{b_i} \mid \nu = v_i \wedge \phi_i]}, x{:}\tau_x \\ \Gamma, \Gamma' \vdash e \Rightarrow \tau \\ \tau' = \mathtt{Ex}(\Gamma', \tau) \quad \Gamma \vdash^{\mathbf{WF}} \tau'\end{array}}{\Gamma \vdash \mathtt{let}\ x = op\ \overline{v_i}\ \mathtt{in}\ e \Rightarrow \tau'}\ \textsc{SynAppOp} \qquad \frac{\begin{array}{c}\Gamma \vdash e_x \Rightarrow \tau_x \quad \Gamma' = x{:}\tau_x \\ \Gamma, \Gamma' \vdash e \Rightarrow \tau \\ \tau' = \mathtt{Ex}(\Gamma', \tau) \quad \Gamma \vdash^{\mathbf{WF}} \tau'\end{array}}{\Gamma \vdash \mathtt{let}\ x = e_x\ \mathtt{in}\ e \Rightarrow \tau}\ \textsc{SynLetE}$$

**Figure 5.9.** Bidirectional Typing Rules (Part 1)

TVarBase shown in Fig. 5.7. We use one type checking rule ChkMatch and one synthesis rule SynAppBase as examples to illustrate how we convert declarative typing rules into a typing algorithm.

### Typing `match`

As we saw in Sec. 5.3, applying the declarative typing rule for `match` expressions typically requires first using several other rules to get things into the right form: TMerge is required

$$\frac{\begin{array}{c}\forall i, \mathbf{Ty}(d_i) = \overline{y{:}\{\nu{:}\mathtt{b_y} \mid \theta_y\}} \to [\nu{:}\mathtt{b} \mid \psi_i] \\ \Gamma'_i = \overline{y{:}[\nu{:}\mathtt{b_y} \mid \theta_y]}, a{:}[\nu{:}\mathtt{b} \mid \nu = v_a \wedge \psi_i] \quad \Gamma, \Gamma'_i \vdash e_i \;\Rightarrow\; \tau_i \\ \tau'_i = \mathtt{Ex}(\Gamma'_i, \tau_i) \quad \Gamma \vdash^{\mathbf{WF}} \mathtt{Disj}(\overline{\tau'_i})\end{array}}{\Gamma \vdash \mathtt{match}\ v_a\ \mathtt{with}\ \overline{d_i\ \overline{y} \to e_i} \;\Rightarrow\; \mathtt{Disj}(\overline{\tau'_i})}\ \textsc{SynMatch}$$

$$\frac{\emptyset \vdash e \;\Rightarrow\; \tau \quad \Gamma \vdash \tau <: \tau' \quad \Gamma \vdash^{\mathbf{WF}} \tau'}{\Gamma \vdash e \;\Leftarrow\; \tau'}\ \textsc{ChkSub} \qquad \frac{\Gamma, x{:}\tau_x \vdash e \;\Leftarrow\; \tau \quad \Gamma \vdash^{\mathbf{WF}} x{:}\tau_x \to \tau}{\Gamma \vdash \lambda x{:}\lfloor \tau_x \rfloor.e \;\Leftarrow\; (x{:}\tau_x \to \tau)}\ \textsc{ChkFun}$$

$$\frac{\begin{array}{c}\forall i, \mathbf{Ty}(d_i) = \overline{y{:}\{\nu{:}\mathtt{b_y} \mid \theta_y\}} \to [\nu{:}\mathtt{b} \mid \psi_i] \\ \Gamma'_i = \overline{y{:}[\nu{:}\mathtt{b_y} \mid \theta_y]}, a{:}[\nu{:}\mathtt{b} \mid \nu = v_a \wedge \psi_i] \quad \Gamma, \Gamma'_i \vdash e_i \;\Rightarrow\; \tau_i \\ \tau'_i = \mathtt{Ex}(\Gamma'_i, \tau_i) \quad \Gamma \vdash \mathtt{Disj}(\overline{\tau'_i}) <: \tau' \quad \Gamma \vdash^{\mathbf{WF}} \tau'\end{array}}{\Gamma \vdash \mathtt{match}\ v_a\ \mathtt{with}\ \overline{d_i\ \overline{y} \to e_i} \;\Leftarrow\; \tau'}\ \textsc{ChkMatch}$$

$$\frac{\begin{array}{c}\Gamma \vdash \\ \lambda x{:}b.\lambda f{:}(b \to \lfloor \tau \rfloor).e \;\Leftarrow\; (x{:}\{\nu{:}\mathtt{b} \mid \phi\} \to f{:}(x{:}\{\nu{:}\mathtt{b} \mid \nu \prec x \wedge \phi\} \to \tau) \to \tau) \\ \Gamma \vdash^{\mathbf{WF}} x{:}\{\nu{:}\mathtt{b} \mid \phi\} \to \tau\end{array}}{\Gamma \vdash \mathbf{fix} f{:}(b \to \lfloor \tau \rfloor).\lambda x{:}b.e \;\Leftarrow\; (x{:}\{\nu{:}\mathtt{b} \mid \phi\} \to \tau)}\ \textsc{ChkFix}$$

**Figure 5.10.** Bidirectional Typing Rules (Part 2)

to analyze and combine the types of each branch, TSub is used to equip each branch with the right typing context, and TEq is used to remove any local or pattern variables from the type of a branch. Our bidirectional type system combines all of these into the single ChkMatch rule shown in Fig. 5.10. At a high level, this rule synthesizes a type for all the branches and then ensures that, in combination, they cover the desired type.

Similarly to other refinement type systems, when synthesizing the type for the branch for constructor $d_i$ , we use a ghost variable $a{:}[\nu{:}\mathtt{b} \mid \nu = v_a \wedge \psi_i]$ to ensure that the types of any pattern variables $\overline{y}$ are consistent with the parameters of $d_i$. This strategy allows us to avoid having to apply TSub to focus on a particular branch: instead, we simply infer a type for *each* branch, and then combine them using our disjunction operation. In order for the inferred type of a branch to make sense, we need to remove any occurrences of pattern variables or the ghost variable $a$. To do, we use the $\mathtt{Ex}$ function, which intuitively allows us to embed information from the typing context into a type. This function takes as input a typing context $\Gamma$ and type $\tau$ and produces an equivalent type $\tau <: \tau' <: \tau$ in which pattern

and ghost variables do not appear free. Finally, ChkMatch uses Disj to ensure that the combination of the types of all the branches cover the required type $\Gamma \vdash \texttt{Disj}(\overline{\tau_i'}) <: \tau$.

**Example 5.5.1.** Consider how we might check that the body of the generator for even numbers introduced in Sec. 5.1 has the expected type $[\nu{:}\texttt{int} \mid \nu \texttt{ mod } 2 = 0]$:[9]

$$\texttt{int_gen}{:}\{\nu{:}\texttt{unit} \mid \top\} \to [\nu{:}\texttt{int} \mid \top] \vdash$$

```
let (n: int) = int_gen() in let (b: bool) = n mod 2 == 0 in
match b with true -> err | false -> n
```

$\Leftarrow [\nu{:}\texttt{int} \mid \nu \texttt{ mod } 2 = 0]$

Our typing algorithm first adds the local variable `n` and `b` to the type context, and then checks the pattern-matching expression against the given type:

$$\texttt{int_gen}{:}\{\nu{:}\texttt{unit} \mid \top\} \to [\nu{:}\texttt{int} \mid \top], \texttt{n}{:}[\nu{:}\texttt{int} \mid \top], \texttt{b}{:}[\nu{:}\texttt{bool} \mid \nu \iff \texttt{n mod } 2 = 0] \vdash$$

```
match b with true -> err | false -> n
```

$\Leftarrow [\nu{:}\texttt{int} \mid \nu \texttt{ mod } 2 = 0]$

The ChkMatch rule first synthesizes types for the two branches separately. Inferring a type of the first branch using the existing type context:

$$\ldots, \texttt{b}{:}[\nu{:}\texttt{bool} \mid \nu \iff \texttt{n mod } 2 = 0], \texttt{b}'{:}[\nu{:}\texttt{bool} \mid \nu = \texttt{b} \wedge \nu] \vdash \texttt{err} \quad \Rightarrow \quad [\nu{:}\texttt{int} \mid \bot]$$

adds a ghost variable $\texttt{b}'$ to reflect the fact that `n` must be less than 0 in this branch. By next applying the TErr rule, our algorithm infers the type $[\nu{:}\texttt{int} \mid \bot]$ for this branch. The rule next uses Ex to manifest $\texttt{b}'$ in the inferred type, encoding the path constraints under which this type holds (i.e. $b$ is true).

$$\ldots, \texttt{b}{:}[\nu{:}\texttt{bool} \mid \nu \iff \texttt{n mod } 2 = 0], \texttt{b}'{:}[\nu{:}\texttt{bool} \mid \nu = \texttt{b} \wedge \nu] \vdash$$

$$\texttt{err} \quad \Rightarrow \quad [\nu{:}\texttt{int} \mid \exists \texttt{b}', \texttt{b}' = \texttt{b} \wedge \texttt{b}' \wedge \bot]$$

Thus, the synthesized type for the first branch is $[\nu{:}\texttt{int} \mid \texttt{b} \wedge \bot]$ modulo some trivial simplification. The type of the second branch provides a better demonstration of why Ex is needed:

$$\ldots, \texttt{b}{:}[\nu{:}\texttt{bool} \mid \nu \iff \texttt{n mod } 2 = 0], \texttt{b}'{:}[\nu{:}\texttt{bool} \mid \nu = \texttt{b} \wedge \neg\nu] \vdash$$

$$\texttt{n} \quad \Rightarrow \quad [\nu{:}\texttt{int} \mid \nu = \texttt{n}]$$

[9]↑We have replaced the `if` from the original example with a `match` expression, to be consistent with the syntax of $\boldsymbol{\lambda}^U$.

After applying Ex, the inferred type is $[\nu{:}\texttt{int} \mid \exists \texttt{b}', \texttt{b}' = \texttt{b} \wedge \neg\texttt{b}' \wedge \nu = \texttt{n}]$; after simplification, this becomes $[\nu{:}\texttt{int} \mid \neg\texttt{b} \wedge \nu = \texttt{n}]$. The disjunction of these two types:

$$\texttt{Disj}([\nu{:}\texttt{int} \mid \texttt{b} \wedge \bot], [\nu{:}\texttt{int} \mid \neg\texttt{b} \wedge \nu = \texttt{n}]) = [\nu{:}\texttt{int} \mid (\texttt{b} \wedge \bot) \vee (\neg\texttt{b} \wedge \nu = \texttt{n})]$$

results in exactly the type shown in Sec. 5.1. This can be then successfully checked against the target type $[\nu{:}\texttt{nat} \mid \nu \texttt{ mod } 2 = 0]$.

#### Application

Our type synthesis rules for function application adopt a strategy similar to ChkMatch's, trying to infer the strongest type possible for an expression that uses the result of a function application. The rule for a function whose parameter is an overapproximate refinement type (SynAppBase) is most interesting, since it has to bridge the gap with an argument that has an underapproximate coverage type. When typing $e$, the expression that uses the result of the function call, the rule augments the typing context with a ghost variable $a$. This variable records that the coverage type of the argument must overlap with the type expected by the function (both must satisfy the refinement predicate $\phi$): if this intersection is empty, i.e., the type of $a$ is equivalent to $\bot$, we will fail to infer a type for $e$, as no type will be well-formed in this context. As with ChkMatch, SynAppBase uses Ex to ensure that it does not infer a type that depends on $a$.

### 5.5.2 Auxiliary Typing Functions

#### Disjunction Function

We implement our disjunction function Disj as a function with type $\texttt{Disj} : \tau \to \tau \to \tau$. The disjunction of multiple types is defined compositionally:

$$\texttt{Disj}(\tau_1, \tau_2, ..., \tau_{n-1}, \tau_n) \doteq \texttt{Disj}(\tau_1, \texttt{Disj}(\tau_2, ..., \texttt{Disj}(\tau_{n-1}, \tau_n)))$$

As shown in Fig. 5.11, the Disj and Conj functions call each other recursively. As discussed in Sec. 5.3, the disjunction of two base coverage types (underapproximate types) $[\nu{:}\texttt{b} \mid \nu = 1]$ and $[\nu{:}\texttt{b} \mid \nu = 2]$ takes the disjunction of their qualifiers: $[\nu{:}\texttt{b} \mid \nu = 1 \vee \nu = 2]$. Similarly, the

**Algorithm 11:** Disjunction

```
1  Procedure Disj(τ1, τ2) :=
2      match τ1, τ2:
3          case [ν:b | φ1], [ν:b | φ2] do
4              return [ν:b | φ1 ∨ φ2];
5          end
6          case {ν:b | φ1}, {ν:b | φ2} do
7              return {ν:b | φ1 ∨ φ2};
8          end
9          case a:τa1 → τ1, a:τa2 → τ2 do
10             τa ← Conj(τa1, τa2);
11             return a:τa → Disj(τ1, τ2);
12         end
```

**Algorithm 12:** Conjunction

```
1  Procedure Conj(τ1, τ2) :=
2      match τ1, τ2:
3          case [ν:b | φ1], [ν:b | φ2] do
4              return [ν:b | φ1 ∧ φ2];
5          end
6          case {ν:b | φ1}, {ν:b | φ2} do
7              return {ν:b | φ1 ∧ φ2};
8          end
9          case a:τa1 → τ1, a:τa2 → τ2 do
10             τa ← Disj(τa1, τa2);
11             return a:τa → Conj(τ1, τ2);
12         end
```

**Figure 5.11.** Disjunction and Conjunction

disjunction of normal refinement types (overapproximate types) is also the disjunction of their corresponding qualifiers. The disjunction of function types conjuncts their argument types and disjuncts their return types.

**Algorithm 13:** Exists

```
1  Procedure Ex(x, [ν:b | φx], τ) :=
2      match τ:
3          case [ν:b | φ] do
4              φ' ← ∃x.φx[ν ↦ x] ∧ φ;
5              return [ν:b | φ'];
6          end
7          case {ν:b | φ} do
8              φ' ← ∀x.φx[ν ↦ x] ⟹ φ;
9              return {ν:b | φ'};
10         end
11         case a:τa → τ do
12             τa' ← Fa(x, [ν:b | φx], τa);
13             τ' ← Ex(x, [ν:b | φx], τ);
14             return a:τa' → τ';
15         end
```

**Algorithm 14:** Forall

```
1  Procedure Fa(x, [ν:b | φx], τ) :=
2      match τ:
3          case [ν:b | φ] do
4              φ' ← ∀x.φx[ν ↦ x] ⟹ φ;
5              return [ν:b | φ'];
6          end
7          case {ν:b | φ} do
8              φ' ← ∃x.φx[ν ↦ x] ∧ φ;
9              return {ν:b | φ'};
10         end
11         case a:τa → τ do
12             τa' ← Ex(x, [ν:b | φx], τa);
13             τ' ← Fa(x, [ν:b | φx], τ);
14             return a:τa' → τ';
15         end
```

**Figure 5.12.** Exists and Forall

### "Exists" Function

We implement our "Exists" function $\texttt{Ex}$ as a function with type $\texttt{Ex}(x, \tau_x, \tau) : \mathit{Var} \to \tau \to \tau \to \tau$, where $x$ and $\tau_x$ is a variable and its corresponding binding type that we want to existentialize into the type $\tau$. Thus, it can also be denoted as $\texttt{Ex}(x{:}\tau_x,\ \tau)$. Existentializing a type context $x_1{:}\tau_1, x_2{:}\tau_2, \ldots, x_n{:}\tau_n$ into a type $\tau$ is equivalent to existentializing each binding consecutively:

$$\texttt{Ex}(x_1{:}\tau_1, x_2{:}\tau_2, \ldots, x_n{:}\tau_n,\ \tau) \doteq \texttt{Ex}(x_1{:}\tau_1,\ \texttt{Ex}(x_2{:}\tau_2,\ \ \ldots, \texttt{Ex}(x_n{:}\tau_n,\ \tau)))$$

As shown in Fig. 5.11, the $\texttt{Ex}$ function relies on the **Fa** function. More specifically, as we mentioned in Sec. 5.5, existentializing a binding $x{:}[\nu{:}\texttt{nat} \mid \nu > 0]$ into the type $[\nu{:}\texttt{nat} \mid \nu = x + 1]$ will derive type $[\nu{:}\texttt{nat} \mid \exists x.x > 0 \wedge \nu = x + 1]$, which has an existentially-quantified qualifier. The function type is contravariant in its argument types and covariant in its return types.

### Subtyping Function

The auxillary $\texttt{Disj}$ and $\texttt{Ex}$ operations are a straightforward syntactic transformations. More interesting is how we check well-formedness and subtyping. Our type checking algorithm translates both obligations into logical formulae that can be discharged by a SMT solver. Both obligations are encoded by the $\texttt{Query}$ subroutine shown in Algorithm 15. $\texttt{Query}(\Gamma, [\nu{:}\texttt{b} \mid \phi_1], [\nu{:}\texttt{b} \mid \phi_2])$ encodes the bindings in $\Gamma$ in the typing context from right to left, before checking whether $\phi_1$ implies $\phi_2$. Variables with function types, on the other hand, are omitted entirely, as qualifiers cannot have function variables in FOL. Variables with an overapproximate (underapproximate) type are translated as a universally (existential) quantified variable, and are encoded into the refinement of both coverage types.

**Example 5.5.2.** Consider the subtyping obligation generated by Example 5.5.1 above:

$$\texttt{int_gen}{:}\{\nu{:}\texttt{unit} \mid \top\} \to [\nu{:}\texttt{int} \mid \top], \texttt{n}{:}[\nu{:}\texttt{int} \mid \top], \texttt{b}{:}[\nu{:}\texttt{bool} \mid \nu \iff \texttt{n mod } 2 = 0] \vdash$$
$$[\nu{:}\texttt{int} \mid (b \wedge \bot) \vee (\neg b \wedge \nu = n)] <: [\nu{:}\texttt{int} \mid \nu \geq 0]$$

This obligation is encoded by the following call to Query

$$
\begin{aligned}
&\texttt{Query}(\texttt{int_gen}{:}\{\nu{:}\texttt{unit} \mid \top\} \to [\nu{:}\texttt{int} \mid \top], \texttt{n}{:}[\nu{:}\texttt{int} \mid \top], \texttt{b}{:}[\nu{:}\texttt{bool} \mid \nu \iff \texttt{n mod } 2 = 0]) \\
&\qquad [\nu{:}\texttt{int} \mid (\texttt{b} \wedge \bot) \vee (\neg\texttt{b} \wedge \nu = \texttt{n})], [\nu{:}\texttt{int} \mid \nu \geq 0]) \\
\equiv\ &\texttt{Query}(\texttt{int_gen}{:}\{\nu{:}\texttt{unit} \mid \top\} \to [\nu{:}\texttt{int} \mid \top], \texttt{n}{:}[\nu{:}\texttt{int} \mid \top], \\
&\qquad [\nu{:}\texttt{int} \mid \exists \texttt{b}, \texttt{b} \iff \texttt{n mod } 2 = 0 \wedge (\texttt{b} \wedge \bot) \vee (\neg\texttt{b} \wedge \nu = \texttt{n})], \\
&\qquad [\nu{:}\texttt{int} \mid \exists \texttt{b}, \texttt{b} \iff \texttt{n mod } 2 = 0 \wedge \nu \geq 0]) \\
\equiv\ &\texttt{Query}(\texttt{int_gen}{:}\{\nu{:}\texttt{unit} \mid \top\} \to [\nu{:}\texttt{int} \mid \top], \\
&\qquad [\nu{:}\texttt{int} \mid \exists \texttt{n}, \top \wedge \exists \texttt{b}, \texttt{b} \iff \texttt{n mod } 2 = 0 \wedge (\texttt{b} \wedge \bot) \vee (\neg\texttt{b} \wedge \nu = \texttt{n})], \\
&\qquad [\nu{:}\texttt{int} \mid \exists \texttt{n}, \top \wedge \exists \texttt{b}, \texttt{b} \iff \texttt{n mod } 2 = 0 \wedge \nu \geq 0]) \\
\equiv\ &\texttt{Query}(\emptyset, [\nu{:}\texttt{int} \mid \exists \texttt{n}, \top \wedge \exists \texttt{b}, \texttt{b} \iff \texttt{n mod } 2 = 0 \wedge (\texttt{b} \wedge \bot) \vee (\neg\texttt{b} \wedge \nu = \texttt{n})], \\
&\qquad [\nu{:}\texttt{int} \mid \exists \texttt{n}, \top \wedge \exists \texttt{b}, \texttt{b} \iff \texttt{n mod } 2 = 0 \wedge \nu \geq 0]) \\
\equiv\ &\forall \nu, \exists \texttt{n}, \top \wedge \exists \texttt{b}, \texttt{b} \iff \texttt{n mod } 2 = 0 \wedge \nu \geq 0) \implies \\
&\quad \exists \texttt{n}, \top \wedge \exists \texttt{b}, \texttt{b} \iff \texttt{n mod } 2 = 0 \wedge (\texttt{b} \wedge \bot) \vee (\neg\texttt{b} \wedge \nu = \texttt{n})
\end{aligned}
$$

This is equivalent to formula (5.2) from Sec. 5.1:

$$
\forall \nu, (\nu \geq 0) \implies (\exists \texttt{n}, \exists \texttt{b}, \texttt{b} \iff \texttt{n mod } 2 = 0 \wedge (\texttt{b} \wedge \bot) \vee (\neg\texttt{b} \wedge \nu = \texttt{n}))
$$

Using Query, it is straightforward to discharge well-formedness and subtyping obligations using the rules shown in Fig. 5.13. In the case of WfBase, for example, observe that the error term err is always an inhabitant of the type $[\nu{:}\texttt{b} \mid \bot]$ for arbitrary base type $b$. Thus, to check the last assumption of WfBase, it suffices to iteratively check if any coverage types in the type context are a supertype of their associated bottom type.

$$
\frac{\not\models \texttt{Query}(\Gamma, [\nu{:}\texttt{b} \mid \bot], [\nu{:}\texttt{b} \mid \phi])}{\texttt{err} \notin [\![ [\nu{:}\texttt{b} \mid \phi] ]\!]_\Gamma} \qquad \frac{\models \texttt{Query}(\Gamma, [\nu{:}\texttt{b} \mid \phi_1], [\nu{:}\texttt{b} \mid \phi_2])}{\Gamma \vdash [\nu{:}\texttt{b} \mid \phi_1] <: [\nu{:}\texttt{b} \mid \phi_2]}
$$

$$
\frac{\models \texttt{Query}(\Gamma, [\nu{:}\texttt{b} \mid \phi_2], [\nu{:}\texttt{b} \mid \phi_1])}{\Gamma \vdash \{\nu{:}\texttt{b} \mid \phi_1\} <: \{\nu{:}\texttt{b} \mid \phi_2\}}
$$

**Figure 5.13.** Auxiliary Algorithmic Typing Functions

**Algorithm 15:** Subtyping Query Encoding

**Inputs** : A type context $\Gamma$, and two base coverage types $[\nu{:}\mathtt{b} \mid \phi_1]$ and $[\nu{:}\mathtt{b} \mid \phi_2]$.
**Output** : A FOL formula that can be automated checked by SMT solver.

```
1  Procedure Query(Γ, [ν:b | φ1], [ν:b | φ2]) :=
2      match Γ:
3          case ∅ do
4              return ∀ν:b, φ2 ⟹ φ1;
5          end
6          case Γ, x:(a:τa → τ) do
7              φ ← Query((Γ, [ν:b | φ1], [ν:b | φ2]);
8              return φ;
9          end
10         case Γ, x:{ν:bx | φx} do
11             τ1 ← [ν:b | ∀x:bx, φx[ν ↦ x] ⟹ φ1];
12             τ2 ← [ν:b | ∀x:bx, φx[ν ↦ x] ⟹ φ2];
13             return Query(Γ, [ν:b | τ1], [ν:b | τ2]);
14         end
15         case Γ, x:[ν:bx | φx] do
16             τ1 ← [ν:b | ∃x:bx, φx[ν ↦ x] ∧ φ1];
17             τ2 ← [ν:b | ∃x:bx, φx[ν ↦ x] ∧ φ2];
18             return Query(Γ, [ν:b | τ1], [ν:b | τ2]);
19         end
```

Discharging subtyping obligations is slightly more involved, as we need to ensure that the formulas sent to the SMT solver are decidable. Observe that in order to produce effectively decidable formulas, the encoding strategy realized by `Query` always generates a formula of the form $\forall\overline{x}.\exists\overline{y}.\phi$, i.e. it does not allow for arbitrary quantifier alternations. To ensure that this is sound strategy, we restrict all overapproximate refinement types in a type context to not have any free variables that have a coverage type. This constraint allows us to safely lift all universal quantifiers to the top level, thus avoiding arbitrary quantifier alternations.

As an example of a scenario disallowed by this restriction, consider the following type checking judgment:

$$x{:}[\nu{:}\mathtt{nat} \mid \nu > 0] \vdash \lambda \mathtt{y} : nat.\, \mathtt{x} + \mathtt{y} \;\Leftarrow\; y{:}\{\nu{:}\mathtt{nat} \mid \nu > x + 1\} \to [\nu{:}\mathtt{nat} \mid \phi]$$

This judgment produces the following subtyping check:

$$x{:}[\nu{:}\texttt{nat} \mid \nu > 0], y{:}\{\nu{:}\texttt{nat} \mid \nu > x+1\} \vdash [\nu{:}\texttt{nat} \mid \nu = x+y] <: [\nu{:}\texttt{nat} \mid \phi]$$

where the normal refinement type $\{\nu{:}\texttt{nat} \mid \nu > x+1\}$ in the type context has free variable $x$ that has coverage type. Evaluating this judgment entails solving the formula:

$$\forall \nu, (\exists x, x > 0 \wedge (\forall y, y > x+1 \Longrightarrow \phi)) \implies (\exists x, x > 0 \wedge (\forall y, y > x+1 \Longrightarrow \nu = x+y))$$

which is not decidable due to the quantifier alternation $\forall \nu \exists x \forall y$.

#### SMT Query Encoding for Data Types

In order to reason over data types, we allow the user to specify refinement types with method predicates (e.g., *mem*) and quantifiers (e.g., $\forall u.\neg mem(\nu, u)$). These method predicates are encoded as uninterpreted functions. To ensure the query is an EPR sentence, we require that a normal refinement type (overapproximate type) can only use universal quantifiers. In addition, as shown in Fig. 5.5, we disallow nested method predicate applications (e.g., $mem(\nu, mem(\nu, u))$) and only allow method predicates to be applied to constants, such as $mem(\nu, 3)$ (which can be encoded as $\forall u.u = 3 \Longrightarrow mem(\nu, u)$).

### 5.5.3 Soundness and Completeness of the Algorithmic Typing

The proof of soundness of algorithmic typing requires the following lemmas about the `Query` subroutine, `Ex` and `Disj` functions.

**Lemma 5.5.3.** *[Soundness of* `Query` *subroutine] For all type context* $\Gamma$ *and coverage type* $[\nu{:}\texttt{b} \mid \phi_1]$ *and* $[\nu{:}\texttt{b} \mid \phi_2]$*,* $\texttt{Query}(\Gamma, [\nu{:}\texttt{b} \mid \phi_1], [\nu{:}\texttt{b} \mid \phi_2])$ *implies* $\Gamma \vdash [\nu{:}\texttt{b} \mid \phi_1] <: [\nu{:}\texttt{b} \mid \phi_2]$*.*

*Proof.* By routine induction on type context $\Gamma$.

**Lemma 5.5.4 (The `Disj` Function implies disjunction judgement).** *For all type context* $\Gamma$*, type* $\tau_1$ *and* $\tau_2$*,* $\Gamma \vdash \tau_1 \vee \tau_2 = \texttt{Disj}(\tau_1, \tau_2)$*.*

*Proof.* By routine induction on type context $\Gamma$.

**Lemma 5.5.5 (The `Ex` Function implies type judgement transformation).** *For all type context $\Gamma$, $\Gamma'$, term $e$, and type $\tau$,*

$$\Gamma, \Gamma' \vdash e : \tau \implies \Gamma, \Gamma' \vdash e : \texttt{Ex}(\Gamma',\ \tau) \wedge \Gamma \vdash \texttt{Ex}(\Gamma',\ \tau)$$

*Proof.* By routine induction on type context $\Gamma$.

We also lift the subtyping relation to type contexts.

**Definition 5.5.6 (Subtyping relation over Type Contexts).** As in the subtyping relation between types, the subtyping relation between two type context $\Gamma_1 \sqsubseteq \Gamma_2$ means that if a term have type $\tau$ under one context, it should also have the same type in the second context.

$$\Gamma_1 \sqsubseteq \Gamma_2 \doteq \forall \tau, \forall e, e \in [\![\tau]\!]_{\Gamma_1} \implies e \in [\![\tau]\!]_{\Gamma_2}$$

**Lemma 5.5.7.** *[Sub Type Context Implies Type Judgement Transformation] For two type context $\Gamma_1 \sqsubseteq \Gamma_2$, term $e$ and coverage type $\tau$,*

$$\Gamma_1 \vdash e : \tau \implies \Gamma_2 \vdash e : \tau$$

*Proof.* Intuitively, modifying a type binding in a type context is equivalent to applying the subsumption rule before we introduce this binding into the type context. This subtype context relation allows us to prove the correctness of the typing algorithm, which lazily strengthens the types in the type context by need.

Now we can prove the soundness theorem of our typing algorithm with respect to our declarative type system.

**Theorem 5.5.8.** *[Soundness of Algorithmic Typing] For all type context $\Gamma$, term $e$ and coverage type $\tau$, $\Gamma \vdash e \Leftarrow \tau \Longrightarrow \Gamma \vdash e : \tau$*

*Proof.* As the type synthesis rules are defined mutually recursively, we simultaneously prove both are correct:

**Theorem 5.5.9.** *[Soundness of the type synthesis and type check algorithm] For all type context $\Gamma$, term $e$ and coverage type $\tau$,*

$$\Gamma \vdash e \Rightarrow \tau \implies \Gamma \vdash e : \tau$$

$$\Gamma \vdash e \Leftarrow \tau \implies \Gamma \vdash e : \tau$$

We proceed by induction of the mutual recursive structure of $\Gamma \vdash e \Rightarrow \tau$ and $\Gamma \vdash e \Leftarrow \tau$. In the cases for; synthesis and checking rules of rule SynConst, SynOp, SynErr, SynVarBase, SynVarFun, ChkSub,ChkFun, and ChkFix, the coverage typing rules in Fig. 5.7 aligns exactly with these rules, thus the soundness is immediate in these cases.

In addition, the rule SynAppOp is similar to SynAppBase, but has multiple arguments; the rule SynLetE is the same as SynAppFun but has no application; the rule SynMatch is similar with ChkMatch, thus we discuss one rule in each of these pairs while the second follows in a similar fashion. Consequently, there are three interesting cases, corresponding to the rules shown in Fig. 5.9.

Case SynAppFun: This rule can be treated as a combination of TAppFun and TEq. From the induction hypothesis and the precondition of SynAppFun, we know

$$\Gamma \vdash v_1 : (a{:}\tau_a \to \tau_b) \to \tau_x \qquad \text{since} \quad \Gamma \vdash v_1 \Rightarrow (a{:}\tau_a \to \tau_b) \to \tau_x$$

$$\Gamma \vdash v_2 : a{:}\tau_a \to \tau_b \qquad \text{since} \quad \Gamma \vdash v_2 \Leftarrow a{:}\tau_a \to \tau_b$$

$$\Gamma, x{:}\tau_x \vdash e : \tau \qquad \text{since} \quad \Gamma, x{:}\tau_x \vdash e \Rightarrow \tau$$

For the $\tau' = \texttt{Ex}(x{:}\tau_x,\ \tau)$, according to Lemma 5.5.5, we know

$$\Gamma, x{:}\tau_x \vdash e : \tau' \wedge \Gamma \vdash \tau'$$

Using the above conclusions, Since all the preconditions of TAppFun hold, applying the rule TAppFun, we have $\Gamma \vdash \texttt{let}\ x = v_1\ v_2\ \texttt{in}\ e : \tau'$.

Case SynAppBase: Notice that the value $v_2$ has the base type $t$, and can only be a constant or a variable, doing a case split on this:

(a) If $v_2$ is a constant $c_2$, notice that $\phi[\nu \mapsto c_2]$ has to be true, otherwise the binding $a{:}[\nu{:}\mathtt{b} \mid \nu = c_2 \wedge \phi]$ has the bottom type, and the type context that contains it is not well formed. Thus, using the well-formedness of the context, it follows that

$$\nu = c_2 \wedge \phi \equiv \nu = c_2$$

Thus, again using the Induction Hypothesis on the antecedents of the rule we have:

$$\Gamma \vdash v_1 : a{:}\{\nu{:}\mathtt{b} \mid \nu = c_2 \wedge \phi\} \to \tau_x[a \mapsto c_2]$$

$$\text{since } \Gamma \vdash v_1 \;\Rightarrow\; a{:}\{\nu{:}\mathtt{b} \mid \nu = c_2 \wedge \phi\} \to \tau_x \text{ and TSub}$$

$$\Gamma \vdash c_2 : [\nu{:}\mathtt{b} \mid \nu = c_2 \wedge \phi]$$

$$\text{since TConst and } \nu = c_2 \equiv \nu = c_2 \wedge \phi$$

$$\Gamma, a{:}[\nu{:}\mathtt{b} \mid \nu = c_2 \wedge \phi], x{:}\tau_x[a \mapsto c_2] \vdash e : \tau[a \mapsto c_2]$$

$$\text{since } \Gamma, a{:}[\nu{:}\mathtt{b} \mid \nu = c_2 \wedge \phi], x{:}\tau_x \vdash e \;\Rightarrow\; \tau$$

Since the variable $a$ is not free in the type judgment, we can remove it from the type context

$$\Gamma, x{:}\tau_x[a \mapsto c_2] \vdash e : \tau[a \mapsto c_2]$$

According to the Lemma 5.5.5, we know that

$$\Gamma, x{:}\tau_x[a \mapsto c_2] \vdash e : \mathtt{Ex}(x{:}\tau_x[a \mapsto c_2], \tau[a \mapsto c_2])$$

The type $\mathtt{Ex}(x{:}\tau_x[a \mapsto c_2], \tau)$ is well formed under the type context $\Gamma$, and all preconditions of the rule TApp are satisfied, so we can conclude

$$\Gamma \vdash e : \mathtt{let}\ x = v_1\ c_2\ \mathtt{in}\ e : \mathtt{Ex}(x{:}\tau_x[a \mapsto c_2], \tau[a \mapsto c_2])$$

Notice that, $\phi[\nu \mapsto c_2]$ is true, thus we have

$$
\begin{aligned}
& \texttt{Ex}(x{:}\tau_x[a \mapsto c_2], \tau[a \mapsto c_2]) \\
\equiv\ & \texttt{Ex}(a{:}[\nu{:}\texttt{b} \mid \nu = c_2], x{:}\tau_x[a \mapsto a], \tau[a \mapsto a]) \\
\equiv\ & \texttt{Ex}(a{:}[\nu{:}\texttt{b} \mid \nu = c_2 \wedge \phi], x{:}\tau_x, \tau) \\
\equiv\ & \tau'
\end{aligned}
$$

Thus, we can conclude $\Gamma \vdash e : \texttt{let}\ x = v_1\ c_2\ \texttt{in}\ e : \tau'$.

(b) If $v_2$ is a variable $x_2$, we first construct a subcontext of $\Gamma$ where we modify the type of $x_2$ in the type context $\Gamma$. Since the variable $x_2$ has a type in the context $\Gamma$, then[10]

$$\Gamma \equiv \Gamma_1, x_2{:}[\nu{:}\texttt{t}_2 \mid \phi_2], \Gamma_2$$

we build a type context $\Gamma^*$

$$\Gamma \equiv \Gamma_1, x_2{:}[\nu{:}\texttt{t}_2 \mid \phi_2 \wedge \phi], \Gamma_2$$

Intuitively, this new context gives us an assumption similar to the constant case above:

$$\nu = x_2 \wedge \phi \iff \nu = x_2$$

In fact, the new context $\Gamma^*$ implies two subtyping relations over the context:

$$\Gamma^* \sqsubseteq \Gamma$$

$$\Gamma, a{:}[\nu{:}\texttt{b} \mid \nu = x_2 \wedge \phi] \sqsubseteq \Gamma^*, a{:}[\nu{:}\texttt{b} \mid \nu = x_2 \wedge \phi]$$

[10]↑We use the same way when the variable $x_2$ having a normal refinement type $\{\nu{:}\texttt{t}_2 \mid \phi_2\}$, thus we omitted this situation.

The first is obvious, since we only add a conjunction into the type of $x_2$. On the other hand, $\Gamma, a{:}[\nu{:}\mathtt{b} \mid \nu = x_2 \wedge \phi]$ is a subtype of $\Gamma^*, a{:}[\nu{:}\mathtt{b} \mid \nu = x_2 \wedge \phi]$ in reverse, since we strengthen the coverage type of $x_2$ in the last binding $a{:}[\nu{:}\mathtt{b} \mid \nu = x_2 \wedge \phi]$. Then, according to the second subtype context relation, we have

$$\Gamma^* \vdash v_1 : a{:}\{\nu{:}\mathtt{b} \mid \nu = x_2 \wedge \phi\} \to \tau_x \quad \text{since} \quad \Gamma \vdash v_1 \;\Rightarrow\; a{:}\{\nu{:}\mathtt{b} \mid \phi\} \to \tau_x \text{ and TSub}$$

Based on the fact $\nu = x_2 \wedge \phi \iff \nu = x_2$, we have

$$\Gamma^* \vdash v_2 : [\nu{:}\mathtt{b} \mid \nu = x_2 \wedge \phi] \qquad \text{According to the rule TVar}$$

According to the second subtype context relation, we have

$$\Gamma^*, a{:}[\nu{:}\mathtt{b} \mid \nu = x_2 \wedge \phi], x{:}\tau_x[a \mapsto x_2] \vdash e : \tau[a \mapsto x_2]$$
$$\text{since } \Gamma, a{:}[\nu{:}\mathtt{b} \mid \nu = x_2 \wedge \phi], x{:}\tau_x \vdash e \;\Rightarrow\; \tau$$

Again, since the variable $a$ is not free, we can also remove it. Moreover, according to the typing rule TApp and the Lemma 5.5.5, we know

$$\Gamma^* \vdash \texttt{let}\ x = v_1\ x_2\ \texttt{in}\ e : \texttt{Ex}(x{:}\tau_x[a \mapsto x_2], \tau[a \mapsto x_2])$$

Again, we have

$$\begin{aligned}
& \texttt{Ex}(x{:}\tau_x[a \mapsto x_2], \tau[a \mapsto x_2]) \\
\equiv\ & \texttt{Ex}(a{:}[\nu{:}\mathtt{b} \mid \nu = x_2], x{:}\tau_x[a \mapsto a], \tau[a \mapsto a]) \\
\equiv\ & \texttt{Ex}(a{:}[\nu{:}\mathtt{b} \mid \nu = x_2 \wedge \phi], x{:}\tau_x, \tau) \\
\equiv\ & \tau'
\end{aligned}$$

Then we have

$$\Gamma^* \vdash \texttt{let}\ x = v_1\ x_2\ \texttt{in}\ e : \tau'$$

Finally, by combining Lemma 5.5.7 and $\Gamma^* \sqsubseteq \Gamma$, we have

$$\Gamma \vdash \texttt{let}\ x = v_1\ x_2\ \texttt{in}\ e : \tau'$$

Case ChkMatch: The rule is a combination of TMatch and TMerge. For the $i^{th}$ branch of the pattern matching branch, we have the following judgment after unfolding $\Gamma'_i$

$$\Gamma, \overline{y{:}[\nu{:}\mathtt{b_y} \mid \theta_y]}, a{:}[\nu{:}\mathtt{b} \mid \nu = v_a \wedge \psi_i] \vdash e_i : \tau_i \qquad \text{since } \Gamma, \Gamma'_i \vdash e_i \Rightarrow \tau_i$$

Similarly to the approach we used for the SynAppBase case, since $v_a$ is a value of base type, it can only be a constant or a variable. Then we can derive the following judgement without the variable $a$:

$$\Gamma, \overline{y{:}[\nu{:}\mathtt{b_y} \mid \theta_y]} \vdash e_i : \texttt{Ex}(\overline{y{:}[\nu{:}\mathtt{b_y} \mid \theta_y]}, \tau_i[a \mapsto v_a]) \equiv \tau'_i$$

According to the rule TMatch, we have the following judgement for all branches

$$\Gamma \vdash \texttt{match}\ v_a\ \texttt{with}\ \overline{d_i\ \overline{y} \rightarrow e_i} : \tau'_i$$

Then according to the Lemma 5.5.4, we have

$$\Gamma \vdash \texttt{match}\ v_a\ \texttt{with}\ \overline{d_i\ \overline{y} \rightarrow e_i} : \texttt{Disj}(\overline{\tau'_i})$$

Finally, according to TSub, for a type $\tau'$ that $\Gamma \vdash \texttt{Disj}(\overline{\tau'_i}) <: \tau'$, we have

$$\Gamma \vdash \texttt{match}\ v_a\ \texttt{with}\ \overline{d_i\ \overline{y} \rightarrow e_i} : \tau'$$

which is exactly what we needed to prove for this case.

Now we prove the completeness of algorithm typing. The theorem assumes"an oracle for all formulas produced by the Query subroutine", which can be stated as the following lemma.

**Lemma 5.5.10.** *[An oracle of* `Query` *subroutine exists] For all type context* $\Gamma$ *and coverage type* $[\nu{:}\mathtt{b} \mid \phi_1]$ *and* $[\nu{:}\mathtt{b} \mid \phi_2]$, $\Gamma \vdash [\nu{:}\mathtt{b} \mid \phi_1] <: [\nu{:}\mathtt{b} \mid \phi_2]$ *iff* $\mathtt{Query}(\Gamma, [\nu{:}\mathtt{b} \mid \phi_1], [\nu{:}\mathtt{b} \mid \phi_2])$.

*Proof.* By routine induction on type context $\Gamma$.

With the assumption above, we introduce the following lemmas about the `Query` subroutine, `Disj` and `Ex` functions as we did in the soundness proof.

**Lemma 5.5.11 (`Query` subroutine implies propositional equality).** *For all type context* $\Gamma$, *type* $[\nu{:}\mathtt{b} \mid \phi_1]$, $[\nu{:}\mathtt{b} \mid \phi_2]$

$$\mathtt{Query}(\Gamma, [\nu{:}\mathtt{b} \mid \phi_1], [\nu{:}\mathtt{b} \mid \phi_2]) \wedge \mathtt{Query}(\Gamma, [\nu{:}\mathtt{b} \mid \phi_2], [\nu{:}\mathtt{b} \mid \phi_1]) \implies \phi_1 = \phi_2$$

*Proof.* By routine induction on type context $\Gamma$.

**Lemma 5.5.12 (Disjunction judgement can be simulated by the `Disj` Function).** *For all type context* $\Gamma$, *type* $\tau_1$, $\tau_2$, $\tau_3$, $\Gamma \vdash \tau_1 \vee \tau_2 = \tau_3 \implies \Gamma \vdash \tau_3 <: \mathtt{Disj}(\tau_1, \tau_2) \wedge \Gamma \vdash \mathtt{Disj}(\tau_1, \tau_2) <: \tau_3$.

*Proof.* By routine induction on type context $\Gamma$.

**Lemma 5.5.13 (`Ex` Function is identical when well-fromed).** *For all type context* $\Gamma$, $\Gamma'$ *and type* $\tau$, $\Gamma \vdash \tau \implies \forall e, \Gamma, \Gamma' \vdash e \Rightarrow \tau \iff \Gamma, \Gamma' \vdash e \Rightarrow \mathtt{Ex}(\Gamma', \tau)$.

*Proof.* By routine induction on type context $\Gamma$.

We also have the corresponding lemma about the subtyping judgement.

**Lemma 5.5.14 (Subtyping judgement iff the `Ex` Function).** *For all type context* $\Gamma$, *and type* $\tau_1$, $\tau_2$, $\Gamma \vdash \tau_1 <: \tau_2 \iff \emptyset \vdash \mathtt{Ex}(\Gamma, \tau_1) <: \mathtt{Ex}(\Gamma, \tau_2)$.

*Proof.* By routine induction on type context $\Gamma$.

Now we can prove the completeness theorem of our typing algorithm with respect to our declarative type system.

**Theorem 5.5.15.** *[Completeness of Algorithmic Typing] Assume an oracle for all formulas produced by the* `Query` *subroutine. Then for any type context* $\Gamma$*, term* $e$ *and coverage type* $\tau$*,* $\Gamma \vdash e : \tau \Longrightarrow \Gamma \vdash e \Leftarrow \tau$.

*Proof.* We proceed by induction of $\Gamma \vdash e : \tau$. In the cases for typing rules of rule TErr, TConst, TOp, TVarBase, TVarFun, TFun, and TFix, the coverage typing synthesis rules in Fig. 5.9 aligns exactly with these rules. By applying the rule ChkSub to shift from the typing synthesis judgement to the typing check judgement, the completeness is immediate in these cases. For the same reason, in the case for the rule TSub, the completeness also holds.

Case TLetE, TAppOp, TAppFun, TApp, TMatch: The coverage typing synthesis rules in Fig. 5.9 aligns similar rules (SynLetE, SynAppOp, SynAppFun, SynAppBase, SynMatch) in these cases, which synthesis the type $\texttt{Ex}(\Gamma', \tau)$ instead of $\tau$. This difference can be fixed by the Lemma 5.5.13 and the precondition that $\tau$ is well formed under type context $\Gamma$.

Case TEq: The key idea of this case is to use the auxiliary term $\texttt{let}\ x = e\ \texttt{in}\ x$ and the rule SynLetE to simulate the type judgment transformation. Notice that the auxiliary term $\texttt{let}\ x = e\ \texttt{in}\ x$ is equivalent to $e$ with respect to the operational semantics, that is,

$$\forall v, e \hookrightarrow^* v \iff \texttt{let}\ x = e\ \texttt{in}\ x \hookrightarrow^* v$$

It also implies that

$$\forall \Gamma\ e\ \tau, e \in [\![e]\!]_\Gamma \iff [\![\texttt{let}\ x = e\ \texttt{in}\ x]\!]_\Gamma$$

Thus, the goal of this case can be simplified as

$$\forall \Gamma\ e\ \tau_1\ \tau_2, \Gamma \vdash e \Rightarrow \tau_1 \wedge \Gamma \vdash \tau_1 <: \tau_2 \wedge \Gamma \vdash \tau_2 <: \tau_1 \implies \Gamma \vdash \texttt{let}\ x = e\ \texttt{in}\ x \Rightarrow \tau_2$$

According to the Lemma 5.5.14, we know

$$\emptyset \vdash \texttt{Ex}(\Gamma, \tau_1) <: \texttt{Ex}(\Gamma, \tau_2) \wedge \emptyset \vdash \texttt{Ex}(\Gamma, \tau_1) <: \texttt{Ex}(\Gamma, \tau_2)$$

According to the Lemma 5.5.11, we know $\texttt{Ex}(\Gamma, \tau_1) = \texttt{Ex}(\Gamma, \tau_2)$. On the other hand, according to the rule SynVarBase (or, SynVarFun) and the rule SynLetE, we can infer the type of the auxiliary term $\texttt{let}\ x = e\ \texttt{in}\ x$ as $\texttt{Ex}(\Gamma, \tau_1)$, thus we know

$$\Gamma \vdash \texttt{let}\ x = e\ \texttt{in}\ x \ \Rightarrow\ \texttt{Ex}(\Gamma, \tau_2)$$

The according to Lemma 5.5.14 and the type $\texttt{Ex}(\Gamma, \tau_2)$ has no free variable, we know

$$\Gamma \vdash \texttt{let}\ x = e\ \texttt{in}\ x \ \Rightarrow\ \tau_2$$

which is exactly what we needed to prove for this case.

Case TMerge: Similarly to the approach we used in the case TEq, the key idea is to use the auxiliary term $e \oplus e$ (non-deterministic choice between two $e$) and the rule SynMatch to simulate the typing rule TMerge. Notice that the auxiliary term $e \oplus e$ is equivalent to $e$ with respect to the operational semantics, which implies that

$$\forall \Gamma\ e\ \tau, e \in [\![e]\!]_\Gamma \iff [\![e \oplus e]\!]_\Gamma$$

Thus, the goal of this case can be simplified as

$$\forall \Gamma\ e\ \tau_1\ \tau_2\ \tau_3, \Gamma \vdash e \ \Rightarrow\ \tau_1 \wedge \vdash e \ \Rightarrow\ \tau_2 \wedge \Gamma \vdash \tau_1 \vee \tau_2 = \tau_3 \implies \Gamma \vdash e \oplus e \ \Rightarrow\ \tau_3$$

With the rule SynMatch, we can infer the type of the term $e \oplus e$ as $\texttt{Disj}(\tau_1, \tau_2)$. On the other hand, according to the Lemma 5.5.12, we know

$$\Gamma \vdash \tau_3 <: \texttt{Disj}(\tau_1, \tau_2) \wedge \Gamma \vdash \texttt{Disj}(\tau_1, \tau_2) <: \tau_3$$

Finally, it falls back to the same situation of the case TEQ, obviously can be proved in the same way.

## 5.6 Polymorphic Refinements

```
1   val union: (unit -> 'a) -> (unit -> 'a) -> (unit -> 'a)
2   let union g1 g2 = fun () -> (g1 () ⊕ g2 ())
3
4   val pos_gen: unit -> int (* positive number generator *)
5   val neg_gen: unit -> int (* negative number generator *)
6
7   let bool_gen = union (fun () -> true) (fun () -> false)
8   let nat_gen = union pos_gen (fun () -> 0)
9   let int_gen = union neg_gen nat_gen
```

**Figure 5.14.** A `union` combinator example.

Most mainstream PBT frameworks provide a set of combinators that enable users to compositionally build test generators. For example, the `QCheck` framework provides a `union` combinator shown in Fig. 5.14 that produces a new generator which covers the combined test cases of two test input generators. Unlike $\oplus$ which takes two base values as arguments, the `union` combinator combines two input generators, defined as function values. As shown in Fig. 5.14, the `union` combinator is polymorphic and cannot be assigned a (principal) type in the monomorphic coverage type system we have discussed so far. As a consequence, we need to extend this type system in order to, e.g., type the simple generator on lines 7-9.

To support this flexibility, we extend this type system to align with how OCaml implements polymorphism. Thus, a polymorphic term is assigned a *principal* type (i.e., a most general type) that includes universally quantified type variables like `'a`.

**Example 5.6.1.** In this extended type system, the non-deterministic choice operator $\oplus$ is now assigned the following polymorphic function coverage type:

$$\tau_{\oplus} \doteq x{:}\{\nu{:}\texttt{'a} \mid \top\} \to y{:}\{\nu{:}\texttt{'a} \mid \top\} \to [\nu{:}\texttt{'a} \mid \nu = x \vee \nu = y]$$

The type $\tau_{\oplus}$ imposes no constraint on its two parameters $x$ and $y$, and guarantees the return value ($\nu$) must cover both (i.e., $\nu = x \vee \nu = y$).

While type polymorphism is sufficient to capture the basic behavior of generator combinators, it falls short in a type system, where we aim to specify and provide coverage guarantees. Consider lines 8-9 in Fig. 5.14: the `union` combinator combines a generator that produces only positive integers (`pos_gen`) with the constant generator that always returns 0 (`fun () → 0`), to build a natural number generator (`nat_gen`). It is then used again to create an integer generator (`int_gen`) from a negative number generator (`neg_gen`) and the aforementioned natural number generator. Thus, `union` takes test generators with *arbitrary* coverage guarantees, and build generator whose coverage guarantees combines those guarantees.

Our goal is to assign the `union` combinator a principal coverage type $\tau_{\texttt{union}}$ that reflects that it builds a generator that returns a superset of the values produced by its arguments. This guarantee is similar to that of the non-deterministic choice operator $\oplus$ shown in Example 5.6.1; but because the `union` combinator takes functions as parameters, the analog of the qualifier "$\nu = x \vee \nu = y$" for the `union` combinator cannot be expressed in our current qualifier language. One potential solution is to add union and intersection types (e.g., $\tau \sqcap \tau$ and $\tau \sqcup \tau$) into our type system, but this strategy will not work for more complicated combinators like `bind`, `frequency`, and `fix` shown in Fig. 5.2.

To address this, we draw inspiration from other variants of refinement type systems ([89] and [90]) and introduce *qualifier polymorphism* to allow type to be parameterized over *predicate variables*. With this extension, the principal type of $\tau_{\texttt{union}}$ becomes:

$$\tau_{\texttt{union}} \doteq \forall P_1\, P_2 : \texttt{'a} \to \texttt{bool}.\, (\texttt{unit} \to [\nu{:}\texttt{'a} \mid P_1(\nu)]) \to (\texttt{unit} \to [\nu{:}\texttt{'a} \mid P_2(\nu)]) \to (\texttt{unit} \to [\nu{:}\texttt{'a} \mid P_1(\nu) \vee P_2(\nu)])$$

Here, $P_1$ and $P_2$ are universally quantified predicate variable of type $\texttt{'a} \to \texttt{bool}$. This type states that "for all possible qualifiers $P_1(\nu)$ and $P_2(\nu)$, if the two input generators cover $P_1(\nu)$

and $P_2(\nu)$ respectively, then the generator that results from combining them with a union operation covers $P_1(\nu) \vee P_2(\nu)$".

Much like type variables, predicate variable must appear at the front of the type and be instantiated with concrete qualifiers during function application. For instance, when applying `union` combinator on line 8 in Fig. 5.14, we instantiate the predicate variables according to the following assignment:

$$P_1 \mapsto \lambda\nu.0 < \nu \quad \text{and} \quad P_2 \mapsto \lambda\nu.\nu = 0$$

This allows us to assign `union` combinator the following coverage type:

$$\tau_{\texttt{union}_1} \doteq (\texttt{unit} \to [\nu{:}\texttt{int} \mid 0 < \nu]) \to (\texttt{unit} \to [\nu{:}\texttt{int} \mid \nu = 0]) \to (\texttt{unit} \to [\nu{:}\texttt{int} \mid 0 \leq \nu])$$

The rest of program can be checked using this type, which align the types of `neg_gen` and `fun () → 0`. Our extended bidirectional typing rules can automatically infer predicate variable assignments from the qualifier in the corresponding arguments using a simple form of syntactic unification, which we discuss below.

### 5.6.1 Extended Type Syntax

| | | |
|---|---|---|
| **Type Variable** | | 'a, 'b, 'c, ... |
| **Base Types** | $b ::=$ | 'a $\mid$ `unit` $\mid$ `bool` $\mid$ `nat` $\mid$ `int` $\mid$ $b$ `list` $\mid$ $b$ `tree` $\mid \ldots$ |
| **Basic Types** | $t ::=$ | $b \mid t \to t \mid$ 'a.$t$ |
| **Predicate Variable** | | $P, \ldots$ |
| **Predicates** | $pred ::=$ | $P \mid \overline{\lambda x}.\phi$ |
| **Qualifier** | $\phi ::=$ | $l \mid \bot \mid \top \mid op(\bar{l}) \mid mp(\overline{x}) \mid \neg\phi \mid \phi \wedge \phi \mid \phi \vee \phi \mid \phi \Longrightarrow \phi$ |
| | | $\mid \forall u{:}b.\ \phi \mid \exists u{:}b.\ \phi \mid pred(\bar{l})$ |
| **Refinement Types** | $\tau ::=$ | $[\nu{:}\texttt{b} \mid \phi] \mid \{\nu{:}\texttt{b} \mid \phi\} \mid x{:}\tau \to \tau \mid$ 'a.$\tau \mid \forall P{:}(\bar{b} \to \texttt{bool}).\tau$ |
| **Type context** | $\Gamma ::=$ | $\emptyset \mid \Gamma, x{:}\tau \mid \Gamma, P{:}t \mid \Gamma,$ 'a |

**Figure 5.15.** Extended $\boldsymbol{\lambda}^U$ syntax.

The syntax of our extended type system is presented in Fig. 5.15, with new additions highlighted in green.

### Basic Types

Following the conventions of standard polymorphic type systems [91], the basic type system of $\boldsymbol{\lambda}^U$ supports both type variables ('a) and type quantification ('a.$t$). The type variable 'a in our type system can be used in refinement types $[\nu{:}\texttt{'a} \mid \phi]$, and thus can only be instantiated with base type ($b$). As in OCaml, type and predicate quantifiers are omitted when the context is clear, e.g., $\texttt{'a}.\forall P{:}\texttt{'a} \rightarrow \texttt{bool}.\texttt{unit} \rightarrow [\nu{:}\texttt{'a} \mid P(\nu)]$ is written as $\texttt{unit} \rightarrow [\nu{:}\texttt{'a} \mid P(\nu)]$.

### Qualifiers and Refinement Types

In addition to supporting type quantification (e.g., $\texttt{'a}.\tau$), coverage refinement type also allow quantified *predicate variables* $P$. As is standard, *predicates* are functions (e.g., $\overline{\lambda x}.\phi$) whose parameters must be base types arguments and which return boolean values ($\bar{b} \rightarrow \texttt{bool}$). Fully applied predicate variables may appear in qualifiers, like $P(\overline{x})$, analogous to primitive operators ($op$). Predicate application $(\overline{\lambda x}.\phi)(\bar{l})$ is simplified as $\phi[\overline{x \mapsto l}]$. Notably, the extension of the qualifier language yields both universal (for type-checking polymorphic functions) and existential (for type-checking polymorphic function application) quantified type variables and predicate variables. Our subtyping rules instantiate these variables such that derived verification conditions only contain universal quantified variables. Type and predicate variables are encoded as *uninterpreted sorts* and *uninterpreted functions* in the verification conditions generated by our type system, ensuring that type-checking remains within the decidable fragment of first order logic.

### Type context and type erasure function

To support these extensions, the type context $\Gamma$ is extended to include type variables ($\Gamma, \texttt{'a}$) as well as predicate variables ($\Gamma, P{:}t$). The updated type erasure function $\lfloor \tau \rfloor$ preserves all quantified type variables and discards all quantified predicate variables.

### 5.6.2 Extended Typing Rules

**Well-Formedness** $\boxed{\Gamma \vdash^{\mathbf{WF}} t \quad \Gamma \vdash^{\mathbf{WF}} \tau \quad \vdash^{\mathbf{WF}} \Gamma}$

$$\frac{\textbf{FreeTypeVar}(t) \subseteq \Gamma}{\Gamma \vdash^{\mathbf{WF}} t}\ \textsc{WfBasicType} \qquad \frac{\begin{array}{c}\vdash^{\mathbf{WF}} \Gamma \\ \lfloor \Gamma \rfloor, \nu{:}b \vdash_{\mathtt{s}} \phi : \mathtt{bool} \\ \Gamma \vdash^{\mathbf{WF}} b\end{array}}{\Gamma \vdash^{\mathbf{WF}} [\nu{:}\mathtt{b} \mid \phi]}\ \textsc{WfBase}$$

$$\frac{\begin{array}{c}\Gamma, \overline{\mathtt{'a}} \vdash^{\mathbf{WF}} \tau \\ \text{no type quantification in } \tau\end{array}}{\Gamma \vdash^{\mathbf{WF}} \overline{\mathtt{'a}}.\tau}\ \textsc{WfPolyType} \qquad \frac{\begin{array}{c}\Gamma, \overline{P{:}\bar{b} \to \mathtt{bool}} \vdash^{\mathbf{WF}} \tau \\ \text{no predicate quantification in } \tau\end{array}}{\Gamma \vdash^{\mathbf{WF}} \overline{\forall P{:}\bar{b} \to \mathtt{bool}}.\tau}\ \textsc{WfPolyPred}$$

**Subtyping** $\boxed{\Gamma \vdash \tau_1 <: \tau_2}$

$$\frac{\Gamma \vdash^{\mathbf{WF}} \tau[\mathtt{'a} \mapsto b]}{\Gamma \vdash \mathtt{'a}.\tau <: \tau[\mathtt{'a} \mapsto b]}\ \textsc{SubPolyType} \qquad \frac{\Gamma, \overline{x{:}b} \vdash^{\mathbf{WF}} \phi}{\Gamma \vdash \forall P : \bar{b} \to \mathtt{bool}.\tau <: \tau[P \mapsto \overline{\lambda x}.\phi]}\ \textsc{SubPolyPred}$$

**Typing** $\boxed{\Gamma \vdash e : \tau}$

$$\frac{\begin{array}{c}\Gamma, \mathtt{'a} \vdash e : \tau \\ \Gamma \vdash^{\mathbf{WF}} \tau\end{array}}{\Gamma \vdash e : \mathtt{'a}.\tau}\ \textsc{TPolyType} \qquad \frac{\begin{array}{c}\Gamma, P{:}\bar{b} \to \mathtt{bool} \vdash e : \tau \\ \Gamma \vdash^{\mathbf{WF}} \tau\end{array}}{\Gamma \vdash e : \forall P{:}\bar{b} \to \mathtt{bool}.\tau}\ \textsc{TPolyPred}$$

**Figure 5.16.** Extended typing relations

The introduction of polymorphism induces the three new sets of typing rules shown in Fig. 5.16. The additional well-formedness WfPolyType and WfPolyPred rules ensure that refinement types are in prenex normal form, i.e., all quantifiers appear at the front of types. The subtyping extensions are also straightforward: the SubPolyType rule permits instantiating type variables with any valid base type, while the SubPolyPred rule allows a predicate variable $P$ to be instantiated with any well-formed predicate $\overline{\lambda x}.\phi$. Finally, we have two new declarative typing rules (TPolyType and TPolyPred) that move type and predicates quantifiers from the type into the type context.

### Type Denotation and Soundness

The extended denotational semantics for types is:

$$[\![\texttt{'a}.\tau]\!] \doteq \bigcup_{b} [\![\tau[\texttt{'a} \mapsto b]]\!] \quad \text{where } b \text{ contains no type variables}$$

$$[\![\forall P{:}(\overline{b} \to \texttt{bool}).\tau]\!] \doteq \bigcup_{\phi} [\![\tau[P \mapsto \overline{\lambda x}.\phi]]\!] \quad \text{where } \overline{x{:}b} \vdash_{\texttt{s}} \phi : \texttt{bool}$$

The fundamental theorem and overall type soundness remain unchanged.

### 5.6.3 Extended Typing Algorithm

**Type Synthesis** $\boxed{\Gamma \vdash e \Rightarrow \tau}$ **Type Check** $\boxed{\Gamma \vdash e \Leftarrow \tau}$

$$\frac{\Gamma, \texttt{'a} \Leftarrow e : \tau \qquad \Gamma \vdash^{\mathbf{WF}} \tau}{\Gamma \vdash e \Leftarrow \texttt{'a}.\tau}\ \textsc{ChkPolyType} \qquad \frac{\Gamma, p{:}\overline{b} \to \texttt{bool} \Leftarrow e : \tau \qquad \Gamma \vdash^{\mathbf{WF}} \tau}{\Gamma \vdash e \Leftarrow \forall p{:}\overline{b} \to \texttt{bool}.\tau}\ \textsc{ChkPolyPred}$$

$$\frac{\begin{array}{c}\Gamma \vdash v_1 \Rightarrow \tau_1 \quad \Gamma \vdash v_2 \Rightarrow \tau_2 \quad \lfloor \tau_2 \rfloor \notin b \quad \mathbf{Inst}(\Gamma, \tau_1, \tau_2) = a{:}\tau_2' \to \tau_x \quad \Gamma \vdash \tau_2 <: \tau_2' \\ \Gamma' = x{:}\tau_x \quad \Gamma, \Gamma' \vdash e \Rightarrow \tau \quad \tau' = \texttt{Ex}(\Gamma', \tau) \quad \Gamma \vdash^{\mathbf{WF}} \tau'\end{array}}{\Gamma \vdash \texttt{let}\ x = v_1\ v_2\ \texttt{in}\ e \Rightarrow \tau'}\ \textsc{SynAppFun}$$

$$\frac{\begin{array}{c}\Gamma \vdash v_1 \Rightarrow \tau_1 \quad \Gamma \vdash v_2 \Rightarrow [\nu{:}\texttt{b} \mid \phi_2] \quad \mathbf{Inst}(\Gamma, \tau_1, [\nu{:}\texttt{b} \mid \phi_2]) = a{:}\{\nu{:}\texttt{b} \mid \phi\} \to \tau_x \\ \Gamma' = a{:}[\nu{:}\texttt{b} \mid \nu = v_2 \wedge \phi], x{:}\tau_x \quad \Gamma, \Gamma' \vdash e \Rightarrow \tau \quad \tau' = \texttt{Ex}(\Gamma', \tau) \\ \Gamma \vdash^{\mathbf{WF}} \tau'\end{array}}{\Gamma \vdash \texttt{let}\ x = v_1\ v_2\ \texttt{in}\ e \Rightarrow \tau'}\ \textsc{SynAppBase}$$

**Figure 5.17.** Extended bidirectional Typing Rules

The extended bidirectional typing algorithm is shown in Fig. 5.17. We introduce two new rules ChkPolyType and ChkPolyPred to type-check polymorphic terms, which are similar to TPolyType and TPolyPred from Fig. 5.16. More challenging is, function application, which in the declarative type system relies on the auxiliary SubPolyType and SubPolyPred subtyping rules to instantiate both type variables and predicate variables in polymorphic refinement types. We modify the previous typing algorithm for function application (i.e., SynAppFun and SynAppBase) as shown in Fig. 5.17, where modified

**Algorithm 16:** Predicate Variable Instantiation

**Inputs** : A type context $\Gamma$, a function type $\forall \overline{P{:}t}.x{:}\tau_{\texttt{param}} \to \tau_{\texttt{ret}}$, and an argument type $\tau_2$

**Output** : the instantiated function type

1 **Procedure** **Inst**$(\Gamma, \forall \overline{P{:}t}.x{:}\tau_{\texttt{param}} \to \tau_{\texttt{ret}}, \tau_2) :=$

2 $\quad \overline{P{:}t} \leftarrow \overline{P{:}t} \setminus$ **FreePredVar**$(\tau_{\texttt{param}})$;

3 $\quad \sigma \leftarrow \texttt{UnifyType}(\Gamma, \emptyset, [(\tau_{\texttt{param}}, \tau_2)])$;

4 $\quad x{:}\tau_{\texttt{param}} \to \tau_{\texttt{ret}} \leftarrow \sigma(x{:}\tau_{\texttt{param}} \to \tau_{\texttt{ret}})$;

5 $\quad$ **return** $x{:}\tau_{\texttt{param}} \to (\forall \overline{P{:}t}.\tau_{\texttt{ret}})$;

parts are colored in gray. These two rules first infer a function type ($v_1$) and type for the corresponding argument ($v_2$), then invoke the instantiation subroutine **Inst**, which instantiates the polymorphic function type with respect to the argument type (e.g., instantiates $\tau_1$ on $\tau_2$ in the rule SynAppFun). Since we perform type synthesis instead of type checking for the function $v_1$ in SynAppFun, we additionally inline the rule ChkSub to ensure the argument type $\tau_2$ is a subtype of the instantiated parameter type $\tau_2'$. The output of the **Inst** subroutine also ensures that the resulting parameter type (e.g., $\{\nu{:}\texttt{b} \mid \phi\}$ in SynAppBase) contains no free predicate variables. This guarantees that the remaining typing rules can be directly copied from the original typing algorithm. We show the instantiation subroutine **Inst** in Algorithm 17.

As shown in Algorithm 16, the **Inst** subroutine assumes that type qualifiers (e.g., $\texttt{'a}.\tau$) have been eliminated, and divides the input function type $\tau_1$ into three parts: quantified predicates $\overline{P{:}t}$, parameter type $\tau_{\texttt{param}}$, and the return type $\tau_{\texttt{ret}}$. Newly quantified predicates are updated as shown on line 2 to guarantee that all predicate variables appearing in $\tau_{\texttt{param}}$ (i.e., **FreePredVar**$(\tau_{\texttt{param}})$) are instantiated. The algorithm then uses the unification subroutine to obtain a predicate assignment $\sigma$ that ensures predicate variables within the parameter type $\tau_{\texttt{param}}$ are consistent with the argument type $\tau_2$ (line 3). After substituting predicate variables with the corresponding assignments (line 4), **Inst** subroutine then returns an instantiated function type (line 5) whose return type is properly quantified.

The unification algorithm is shown in Algorithm 17 and Algorithm 18, which consists of two subroutines `UnifyType` and `UnifyQualifier` that collectively unify refinement type and

**Algorithm 17:** Predicate Unification over Types

**Inputs** : A type context $\Gamma$, an initial predicate variable assignment $\sigma$, and a list of type alignment constraints $l$

**Output** : A predicate variable assignment that satifies all input constraints.

```
1  Procedure UnifyType(Γ, σ, l) :=
2      match l:
3          case [] do return σ ;
4          case ({ν:b | φ1}, {ν:b | φ2}) :: l or ([ν:b | φ1], [ν:b | φ2]) :: l do
5              if FreePredVar(φ1) \ Dom(Γ) ≠ ∅ then
6                  σq = UnifyQualifier(Γ, ∅, [φ1, φ2]);
7                  σ = σq(σ);
8                  l = σq(l);
9                  return UnifyType(Γ, σ ∪ σq, l);
10             end
11             else
12                 return UnifyType(Γ, σ, l);
13             end
14         end
15         case (x:τ11 → τ12, x:τ21 → τ22) :: l do
16             return UnifyType(σ, (τ11, τ21) :: (τ21, τ22) :: l);
17         end
18         case (τ1, τ2) :: l do
19             Raise Failure;
20         end
```

qualifiers in similar manner to the standard type unification algorithm [91]. The `UnifyType` subroutine takes an initial predicate assignment solution ($\sigma$) and a list of type constraints ($l$). When encountering a base refinement type containing a free predicate variable that is not included in the type context (line 5), the algorithm invokes the qualifier unification subroutine (line 6) to get a predicate assignment $\sigma_q$. It then unions the predicate assignment and continues to process the remaining constraints after substitution (line 6 and 7). A function type is unified piecewise (line $12 - 13$); otherwise, a mismatched case leads to a unification error (line 15). The `UnifyQualifier` subroutine works similarly but recursively processes over the syntactic structure of the qualifier instead.

**Example 5.6.2.** As shown on line 5 in Fig. 5.14, type-checking the application of function `union neg_gen nat_gen` requires instantiating the quantified types predicates of $\tau_{\texttt{union}}$ based

**Algorithm 18:** Predicate Unification over Qualifiers

**Inputs** : A type context $\Gamma$, an initial predicate variable assignment $\sigma$, and a list of qualifier alignment constraints $l$

**Output** : A predicate variable assignment that satifies all input constraints.

```
1  Procedure UnifyQualifier(Γ, σ, l) :=
2      match l:
3          case [] do return σ ;
4          case (P(x̄), φ) :: l do
5              σ_q = {P ↦ λx̄.φ};
6              σ = σ_q(σ);
7              l = σ_q(l);
8              return UnifyQualifier(Γ, σ ∪ σ_q, l);
9          end
10         case (φ_1 ∨ φ_2, φ'_1 ∨ φ'_2) :: l do
11             return UnifyQualifier(Γ, σ, (φ_1, φ'_1) :: (φ_2, φ'_2) :: l);
12         end
13         case (φ_1 ∧ φ_2, φ'_1 ∧ φ'_2) :: l do
14             return UnifyQualifier(Γ, σ, (φ_1, φ'_1) :: (φ_2, φ'_2) :: l);
15         end
16         case (φ_1 ⟹ φ_2, φ'_1 ⟹ φ'_2) :: l do
17             return UnifyQualifier(Γ, σ, (φ_1, φ'_1) :: (φ_2, φ'_2) :: l);
18         end
19         case (¬φ, ¬φ') :: l do return UnifyQualifier(Γ, σ, (φ, φ') :: l) ;
20         case (∀x.φ, ∀x.φ') :: l do return UnifyQualifier(Γ, σ, (φ, φ') :: l) ;
21         case (∃x.φ, ∃x.φ') :: l do return UnifyQualifier(Γ, σ, (φ, φ') :: l) ;
22         case (φ, φ') :: l do Raise Failure ;
```

on its negative and natural number generators arguments. We type check the inner application (`union neg_gen`) first.

$$\texttt{union} : (\texttt{unit} \to [\nu{:}\texttt{int} \mid P_1(\nu)]) \to$$

$$(\texttt{unit} \to [\nu{:}\texttt{int} \mid P_2(\nu)]) \to (\texttt{unit} \to [\nu{:}\texttt{int} \mid P_1(\nu) \vee P_2(\nu)]),$$

$$\texttt{neg_gen} : \texttt{unit} \to [\nu{:}\texttt{int} \mid \nu < 0], \texttt{nat_gen} : \texttt{unit} \to [\nu{:}\texttt{int} \mid 0 \leq \nu] \vdash$$

$$(\texttt{union neg_gen})\ \texttt{nat_gen} \ \Rightarrow\ ?$$

The rule SynAppFun uses the **Inst** subroutine to unify the type $\texttt{unit} \to [\nu{:}\texttt{int} \mid P_1(\nu)]$ against $\texttt{unit} \to [\nu{:}\texttt{int} \mid \nu < 0]$. The predicate assignment after unification is $P_1 \mapsto \lambda\nu.\nu < 0$, and the synthesized type of the application ($\texttt{union}\ \texttt{neg_gen}$) is

$$(\texttt{unit} \to [\nu{:}\texttt{int} \mid P_2(\nu)]) \to (\texttt{unit} \to [\nu{:}\texttt{int} \mid \nu < 0 \lor P_2(\nu)])$$

Now we can type check the second application ($\texttt{union}\ \texttt{neg_gen}$) $\texttt{nat_gen}$ in a similar way, where the **Inst** subroutine unifies the parameter type $\texttt{unit} \to [\nu{:}\texttt{int} \mid P_2(\nu)]$ against $\texttt{unit} \to [\nu{:}\texttt{int} \mid 0 \leq \nu]$. The predicate assignment after unification is $P_2 \mapsto \lambda\nu.0 \leq \nu$, and the coverage type of the final application is $\texttt{unit} \to [\nu{:}\texttt{int} \mid \nu < 0 \lor 0 \leq \nu]$, which covers all integers.

Note that our instantiation algorithm is not complete, as it relies on the assumption that the syntactic structures of two types under unification are aligned. For example, the **Inst** subroutine cannot unify $[\nu{:}\texttt{int} \mid P_1(\nu) \lor P_2(\nu)]$ with $[\nu{:}\texttt{int} \mid \nu \geq 0]$, although one possible solution is assigning $P_1$ to $\lambda\nu.\nu > 0$ and $P_2$ to $\lambda\nu.\nu = 0$.

We present the proof for soundness of extended algorithmic typing. This proof additionally requires the following lemma about the **Inst** subroutine.

**Lemma 5.6.3.** *[Soundness of* **Inst** *subroutine] For all type context* $\Gamma$ *and coverage type* $\tau_1$*,* $\tau_2$ *and* $\tau_3$*,* $\textbf{Inst}(\Gamma, \tau_1, \tau_2) = \tau_3$ *implies* $\Gamma \vdash \tau_1 <: \tau_3$*.*

*Proof.* By routine induction on type context $\Gamma$.

**Theorem 5.6.4.** *[Soundness of Extended Algorithmic Typing] For all type contexts* $\Gamma$*, term* $e$ *and coverage type* $\tau$*,* $\Gamma \vdash e \Leftarrow \tau \Longrightarrow \Gamma \vdash e : \tau$

*Proof.* As the type synthesis rules are defined mutually recursively, we simultaneously prove both are correct:

**Theorem 5.6.5.** *[Soundness of the type synthesis and type check algorithm] For all type context $\Gamma$, term $e$ and coverage type $\tau$,*

$$\Gamma \vdash e \Rightarrow \tau \implies \Gamma \vdash e : \tau$$

$$\Gamma \vdash e \Leftarrow \tau \implies \Gamma \vdash e : \tau$$

We proceed by induction of the mutual recursive structure of $\Gamma \vdash e \Rightarrow \tau$ and $\Gamma \vdash e \Leftarrow \tau$ and focus on the extended rules shown in Fig. 5.17. In addition, the rule ChkPolyType and ChkPolyPred aligns exactly to declarative tying rule TPolyType and TPolyPred shown in Fig. 5.16, thus the soundness is immediate in thse cases. Consequently, we now provide the proof of two modified rules, SynAppFun and SynAppBase.

Case SynAppFun: This rule can be treated as a combination of TAppFun and TEq. From the induction hypothesis and the precondition of SynAppFun, we know

$$\begin{array}{lr}
\Gamma \vdash v_1 : \tau_1 & \text{since} \quad \Gamma \vdash v_1 \Rightarrow \tau_1 \\
\Gamma \vdash v_1 : a{:}\tau_2' \to \tau_x & \text{since} \quad \mathbf{Inst}(\Gamma, \tau_1, \tau_2) = a{:}\tau_2' \to \tau_x \text{ and Lemma 5.6.3} \\
\Gamma \vdash v_1 : a{:}\tau_2 \to \tau_x & \text{since} \quad \Gamma \vdash \tau_2 <: \tau_2' \text{ and TSub} \\
\Gamma \vdash v_2 : a{:}\tau_2 & \text{since} \quad \Gamma \vdash v_2 \Rightarrow a{:}\tau_2 \\
\Gamma, x{:}\tau_x \vdash e : \tau & \text{since} \quad \Gamma, x{:}\tau_x \vdash e \Rightarrow \tau
\end{array}$$

For the $\tau' = \texttt{Ex}(x{:}\tau_x,\ \tau)$, according to Lemma 5.5.5, we know

$$\Gamma, x{:}\tau_x \vdash e : \tau' \wedge \Gamma \vdash \tau'$$

Using the above conclusions, Since all the preconditions of TAppFun hold, applying the rule TAppFun, we have $\Gamma \vdash \texttt{let}\ x = v_1\ v_2\ \texttt{in}\ e : \tau'$.

Case SynAppBase: From the induction hypothesis and the precondition of SynAppBase, we know

$$\Gamma \vdash v_1 : \tau_1$$

$$\text{since} \quad \Gamma \vdash v_1 \;\Rightarrow\; \tau_1$$

$$\Gamma \vdash v_1 : a{:}\{\nu{:}\mathtt{b} \mid \phi\} \to \tau_x$$

$$\text{since} \quad \mathbf{Inst}(\Gamma, \tau_1, [\nu{:}\mathtt{b} \mid \phi_2]) = a{:}\{\nu{:}\mathtt{b} \mid \phi\} \to \tau_x \text{ and Lemma 5.6.3}$$

which leads the same precondition of original rule of SynAppBase as shown in Fig. 5.9, thus the proof is the same as in the proof of Theorem 5.5.8.

We present the proof for completeness of extended algorithmic typing. The theorem besides assumes"an oracle for all formulas produced by the Query subroutine", also assumes "**Inst** subroutine always succeeds" which can be stated as the following lemma.

**Lemma 5.6.6.** *[**Inst** subroutine always succeeds] For all type context $\Gamma$ and coverage type $\tau_1$ and $\tau_2$, if there exists a type $x{:}\tau_2' \to \tau_{\mathtt{ret}}$ such that $\Gamma \vdash a{:}\tau_2 \to \tau_{\mathtt{ret}} <: \tau_1$ and $\tau_2' <: \tau_2'$, then $\mathbf{Inst}(\Gamma, \tau_1, \tau_2) = x{:}\tau_2' \to \tau_{\mathtt{ret}}$.*

*Proof.* By routine induction on type context $\Gamma$.

**Theorem 5.6.7.** *[Completeness of Extended Algorithmic Typing] Assuming an oracle for all formulas produced by the* `Query` *subroutine, and assuming* **Inst** *subroutine always succeeds, then for any type context $\Gamma$, term $e$ and coverage type $\tau$, $\Gamma \vdash e : \tau \Longrightarrow \Gamma \vdash e \;\Leftarrow\; \tau$.*

*Proof.* We proceed by induction of $\Gamma \vdash e : \tau$ and focus on the extended rules shown in Fig. 5.17. In addition, the rule TPolyType and TPolyPred aligns exactly to rule ChkPolyType and ChkPolyPred, thus the completeness also holds. The remaining proof for the modified rules ChkAppFun and ChkAppBase can be simplified to showing that all terms type-checkable under the original rules remain type-checkable under the modified versions.

Case ChkAppFun: According to Theorem 5.5.15, we just needs to show the precondition of original rule ChkAppFun can implies the precondition of modified rule ChkAppFun. We know

| | |
|---|---|
| $\Gamma \vdash v_1 \Rightarrow \tau_2 \to \tau_x$ | precondition of original ChkAppFun |
| $\mathbf{Inst}(\Gamma, \tau_2 \to \tau_x, \tau_2) = \tau_2 \to \tau_x$ | since Lemma 5.6.6 |
| $\Gamma \vdash v_2 \Leftarrow \tau_2$ | precondition of original ChkAppFun |
| $\Gamma \vdash v_2 \Rightarrow \tau_2$ | since CheckSub |
| $\Gamma \vdash \tau_2 <: \tau_2$ | since soundness of subtyping |

Then all preconditions of modified rule ChkAppFun hold, then the completeness also holds.

Case ChkAppBase: According to Theorem 5.5.15, we just needs to show the precondition of original rule ChkAppBase can implies the precondition of modified rule ChkAppBase. We know

$$\Gamma \vdash v_1 \Rightarrow a{:}\{\nu{:}\mathtt{b} \mid \phi\} \to \tau_x$$

precondition of original ChkAppBase

$$\Gamma \vdash v_2 \Rightarrow [\nu{:}\mathtt{b} \mid \nu = v_2]$$

since rule SynVarBase

$$\mathbf{Inst}(\Gamma, a{:}\{\nu{:}\mathtt{b} \mid \phi\} \to \tau_x, [\nu{:}\mathtt{b} \mid \nu = v_2]) = a{:}\{\nu{:}\mathtt{b} \mid \phi \wedge \nu = v_2\} \to \tau_x$$

Notice that $a{:}\{\nu{:}\mathtt{b} \mid \phi \wedge \nu = v_2\} \to \tau_x$ is equal to $a{:}\{\nu{:}\mathtt{b} \mid \phi \wedge \nu = v_2 \wedge \nu = v_2\} \to \tau_x$, then all preconditions of modified rule ChkAppFun hold, then the completeness also holds.

## 5.7 The Coverage Monad and Monadic Generator Combinators

Real-world PBT frameworks like `QCheck` define the type of input generators as a monad and provide combinators for compositionally defining input generators. In order to align with this practice, this section defines a *coverage monad* that provides coverage guarantees about input generators defined in this style. This monad and the monadic generator combinators described in this section can all be desugared into the core language from the previous section.

### 5.7.1 Coverage Monad

```
module Gen = struct
    (* RS.t is the internal state of PRNG *)
    type 'a t = RS.t -> 'a
    let return (x: 'a) = fun (st: RS.t) -> x
    let bind (g: 'a t) (f: 'a -> 'b t) =
       fun (st: RS.t) -> let (x: 'a) = gen st in f x s
end
```

**Figure 5.18.** The definition of the test generator monad in `QCheck`.

In real-world programming languages, computational nondeterminism is typically introduced via a primitive pseudorandom number generator (PRNG) that produces unpredictable sequences of numbers based on an random seed. This facility is used by PBT frameworks like the generator library `Gen` of `QCheck` to parameterize the behavior of generators by a PRNG state, allowing test executions to be replayed by reusing the same initial seed. Fig. 5.18 shows the definition of `Gen` in `QCheck`. The type of a generator `'a`M is a function type `RS.t` → `'a` (line 3), which takes a random seed and returns generated test value of type `'a`. The `return` operator returns a constant value, regardless of the input state (line 4); while the `bind` operator passes the PRNG state through both the generator and the bind function (lines 5–6), respectively.

**Example 5.7.1.** In `QCheck`, a generator for positive integers can be compositionally defined by using the monadic `bind` combinator to get a random value from the natural number generator `nat_gen` and replacing it with 1 when it is a non-positive integer:

```
val nat_gen: int Gen.t
let pos_gen: int Gen.t =
    bind nat_gen (fun (n: int) ->
        if 0 < n then return n else return 1)
```

$$\mathcal{M}\ [\nu{:}\mathtt{b} \mid \phi] \doteq \{\nu{:}\mathtt{RS.t} \mid \top\} \to [\nu{:}\mathtt{b} \mid \phi]$$

$$\mathtt{bind}^{\mathtt{C}}\ \mathcal{M}\ [\nu{:}\mathtt{b}_1 \mid \phi_1]\ \ (x{:}\{\nu{:}\mathtt{b}_1 \mid \phi_2\} \to \mathcal{M}\ [\nu{:}\mathtt{b}_2 \mid \phi_3]) \doteq$$
$$\mathcal{M}\ [\nu{:}\mathtt{b}_2 \mid \exists x{:}\ b_1.(\phi_1 \wedge \phi_2)[\nu \mapsto x] \wedge \phi_3]$$

$$\mathtt{return} : x{:}\{\nu{:}\mathtt{'a} \mid \top\} \to \mathcal{M}\ [\nu{:}\mathtt{'a} \mid \nu = x]$$

$$\mathtt{bind} : \mathcal{M}\ [\nu{:}\mathtt{'a} \mid P_1(\nu)] \to (x{:}\{\nu{:}\mathtt{'a} \mid P_2(\nu)\} \to \mathcal{M}\ [\nu{:}\mathtt{'b} \mid P_3(x,\nu)]) \to$$
$$\mathtt{bind}^{\mathtt{C}}\ \mathcal{M}\ [\nu{:}\mathtt{'a} \mid P_1(\nu)]\ (x{:}\{\nu{:}\mathtt{'a} \mid P_2(\nu)\} \to \mathcal{M}\ [\nu{:}\mathtt{'b} \mid P_3(x,\nu)])$$
$$\equiv \mathcal{M}\ [\nu{:}\mathtt{'b} \mid \exists x.P_1(x) \wedge P_2(x) \wedge P_3(x,\nu)] \quad \text{(after unfolding } \mathtt{bind}^{\mathtt{C}}\text{)}$$

**Figure 5.19.** The coverage monad and the signatures of its operators.

While `nat_gen` and `pos_gen` produce different sets of values, they both have the same basic type: `int Gen.t`. To better capture the coverage guarantees of these sorts of monadic generators, we define the coverage monad $\mathcal{M}\ [\nu{:}\mathtt{b} \mid \phi]$, shown in Fig. 5.19. As expected, the shape of this monad augments the type parameter of `Gen.t` with a qualifier that captures its coverage properties. The definition of $\mathcal{M}\ [\nu{:}\mathtt{b} \mid \phi]$ has two consequences: first, since only non-function values can be qualified, the coverage monad can only range over base types. Second, the coverage guarantees of a monadic generator cannot reference the random seed. This latter restriction makes intuitive sense, as a monadic test generator of type $\mathcal{M}\ [\nu{:}\mathtt{b} \mid \phi]$ should be guaranteed to produce values satisfying predicate $\phi$ regardless of the random seed. We can use the coverage monad to capture the different coverages of `nat_gen` and `pos_gen`, as $\mathcal{M}\ [\nu{:}\mathtt{int} \mid 0 < \nu]$ and $\mathcal{M}\ [\nu{:}\mathtt{int} \mid 0 \leq \nu]$, respectively.

Fig. 5.19 also gives the signatures of the operators of the coverage monad; their definitions are the same as in Fig. 5.18. The signature of `return` reflects that lifts an arbitrary value into a generator that always produces that value (i.e., $\nu = x$). The signature of the `bind` operator uses a special notation, $\mathtt{bind}^{\mathtt{C}}$, for its codomain; this alias will be helpful in the

upcoming discussion of generator combinators, which are built using `bind`. Intuitively, the qualifier in this type introduces an existentially quantified variable $x$ that represents the intermediate value by the first argument of `bind`. This value is required to satisfy both the coverage guarantee of the test generator ($\phi_1$) as well as the safety constraint associated with the bind function ($\phi_2$).

To understand the definition of $\texttt{bind}^{\texttt{C}}$, consider how the definition of `bind` in Fig. 5.18 can be typed using the polymorphic type system of the previous section. After unfolding $\texttt{bind}^{\texttt{C}}$, the initial typing goal is:

$$\begin{aligned}&\texttt{g}{:}\mathcal{M}\ [\nu{:}\texttt{'a} \mid P_1(\nu)], \texttt{f}{:}(x{:}\{\nu{:}\texttt{'a} \mid P_2(\nu)\} \to \mathcal{M}\ [\nu{:}\texttt{'b} \mid P_3(x,\nu)]),\\ &\quad \vdash \lambda\texttt{st}{:}\texttt{RS.t}.\texttt{let x} = \texttt{g st in f x st} : [\nu{:}\texttt{'b} \mid \exists x.P_1(x) \wedge P_2(x) \wedge P_3(x,\nu)]\end{aligned}$$

After moving the PRNG state and the value produce by the first generator into the context, the typing goal is:

$$\begin{aligned}&\texttt{g}{:}\mathcal{M}\ [\nu{:}\texttt{'a} \mid P_1(\nu)], \texttt{f}{:}(x{:}\{\nu{:}\texttt{'a} \mid P_2(\nu)\} \to \mathcal{M}\ [\nu{:}\texttt{'b} \mid P_3(x,\nu)]), \texttt{s}{:}\texttt{RS.t}, \texttt{x}{:}[\nu{:}\texttt{'a} \mid P_1(\nu)]\\ &\quad \vdash \texttt{f x s} : [\nu{:}\exists\texttt{x}.\texttt{'b} \mid P_1(x) \wedge P_2(x) \wedge P_3(\texttt{x},\nu)]\end{aligned}$$

Similarly to how `bst_gen` was typed in Sec. 5.3, we can use TEq to remove the existential quantifier in the target type, and then use TSub to bring `x` into alignment with the argument type of `f` to yield the expected type.

```
1   val power: m:{ν:int | 0 ≤ v} → x:{ν:int | 0 ≤ v} → [ν:int | ν = m^x]
2   let power_gen = bind (int_bound 10: [ν:int | ν ≤ 10])
3        (fun (x: {ν:int | 0 ≤ v}) -> return (power 2 x))
```

**Figure 5.20.** A generator for powers of 2 less than $2^{10}$.

**Example 5.7.2.** The coverage guarantees of generators built from monadic combinators are also derived compositionally.As an example, consider the definition of a generator for powers of 2 from $2^0$ to $2^{10}$ shown in Fig. 5.20. After randomly generating an integer using `int_bound 10`, `power_gen` raises 2 to this power and returns the result. Note that the coverage

guarantees of `power` are only provided for arguments greater than 0, however, a fact that will be reflected in the final type of `power_gen`. Importantly, the instantiations of three predicate variables in the type of $\texttt{bind}^{\mathsf{C}}$ come directly from the qualifiers of its arguments:

$$P_1 \mapsto \underbrace{\lambda\nu.\nu \leq 10}_{\texttt{int_bound 10}} \qquad P_2 \mapsto \underbrace{\lambda\nu.0 \leq \nu}_{\texttt{fun x:}\{\nu:\texttt{int}|0\leq\texttt{v}\}\ \texttt{->}} \qquad P_3 \mapsto \underbrace{\lambda x.\lambda\nu.\nu = 2^x}_{\texttt{power 2 x}}$$

Allowing us to derive the desired coverage type for `power_gen`:

$$\texttt{power} : \ldots \vdash \texttt{power_gen} : \mathcal{M}\ [\nu{:}\texttt{int} \mid \exists x.0 \leq x \wedge x \leq 10 \wedge \nu = 2^x]$$

```
let biased_nat_gen (st: RS.t) =
    union (int_range 0 10) (return Int.max_int)

⊢ biased_nat_gen: M {ν:int | 0 ≤ ν ≤ Int.max_int}
```

**Figure 5.21.** Example of safety monad.

### Over- and under-approximate monads

Monadic combinators also lend themselves well to an over-approximate analysis via traditional refinement types. In the context of test input generators, such specifications are useful in contexts where developers want to prioritize certain *safe* values that always meet the preconditions of the system under test, e.g., when probing boundary cases. As an example of this situation, consider the following generator:

```
let biased_nat_gen (st: RS.t) =
union (int_range 0 10) (return Int.max_int)

⊢ biased_nat_gen: M {ν:int | 0 ≤ ν ≤ Int.max_int}
```

`biased_nat_gen` does not enumerate all natural numbers; instead, it samples from a biased distribution that emphasizes small numbers (e.g., $[0, 10]$) and the maximum integer value

(`Int.max_int`), the latter of which covers the critical edge case for overflow testing. To cover this scenario, we can similarly define a dual *safety* monad for generators $\mathcal{M}\ \{\nu{:}\mathtt{b} \mid \phi\}$. Here, `biased_nat_gen` can be assigned the safety monad type $\mathcal{M}\ \{\nu{:}\mathtt{int} \mid 0 \leq \nu \leq \mathtt{Int.max_int}\}$, ensuring that all sampled values fall within this range. We further discuss real-world instances of such usage in Sec. 5.8.

Using monad combinators to define generators enables the seamless use of both safety and coverage monads to provide richer specifications that capture their ability to produce every value that meets a precondition and only such values. Consider a generator $g_1$ for lists of integers which has the following type:

$$\vdash \mathtt{g_1} : \mathtt{bind}^{\mathtt{C}}\ \mathcal{M}\ \{\nu{:}\mathtt{int} \mid 0 \leq \nu\}\ (n{:}\mathtt{int} \rightarrow \mathcal{M}\ [\nu{:}\mathtt{int\ list}] \mid \mathit{len}(\nu, n) \wedge \mathit{sorted}(\nu)])$$

This type specifies that $\mathtt{g_1}$ produces all possible sorted lists of integers with length $n$, where $n$ is drawn from a generator assigned a safety monad type (e.g., `biased_nat_gen`) which is biased toward values believed to be important by developers. Conversely, consider a generator $\mathtt{g_2}$ with the following type:

$$\vdash \mathtt{g_2} : \mathtt{bind}^{\mathtt{C}}\ \mathcal{M}\ [\nu{:}\mathtt{int} \mid 0 \leq \nu]\ (n{:}\mathtt{int} \rightarrow \mathcal{M}\ \{\nu{:}\mathtt{int\ list} \mid \mathit{len}(\nu, n) \wedge \mathit{sorted}(\nu)\})$$

Here, $\mathtt{g_2}$ produces all possible non-negative list lengths, while imposing no specific coverage obligations over the list contents of a generated sorted list of a given length. This form is useful when the target property under test is sensitive to shape of the datatype (i.e., list length) but not necessarily to the elements it contains. Developers are then allowed to specialize the generated list of length $n$ as a fixed choice (e.g, $[1; 2; \ldots; n]$), rather than generating all possible sorted lists, which would require more effort. In both of these examples, the combined use of safety and coverage monads provides developers more flexibility to specify generators that are neither totally over-approximated nor totally under-approximated.

### 5.7.2 Monadic Combinator Examples

This section provides examples of the coverage guarantees our system can provide for several monadic combinators typically found in PBT frameworks like `QCheck`. We show the (monadic) coverage type of each example combinator first using the coverage monad, and then a version in which $\mathcal{M}$ and $\texttt{bind}^{\texttt{C}}$ are unfolded.

$$
\begin{aligned}
\texttt{option} : \mathcal{M}\ [\nu{:}\texttt{'a} \mid P(\nu)] \rightarrow\ & \\
& \texttt{bind}^{\texttt{C}}\ \mathcal{M}\ [\nu{:}\texttt{'a} \mid P(\nu)]\ \ (x{:}\texttt{'a} \rightarrow \mathcal{M}\ \ [\nu{:}\texttt{'a option} \mid \nu = \mathit{None} \vee \nu = \texttt{Some}(x)]) \\
& \equiv \mathcal{M}\ [\nu{:}\texttt{'a option} \mid \nu = \mathit{None} \vee \exists x.P(x) \wedge \nu = \texttt{Some}(x)] \quad \text{(after unfolding)} \\
\texttt{pair} : \mathcal{M}\ [\nu{:}\texttt{'a} \mid P_1(\nu)] \rightarrow \mathcal{M}\ [\nu{:}\texttt{'b} \mid P_2(\nu)] \rightarrow\ & \\
& \texttt{bind}^{\texttt{C}}\ \mathcal{M}\ [\nu{:}\texttt{'a} \mid P_1(\nu)] \\
& \quad (x{:}\texttt{'a} \rightarrow \texttt{bind}^{\texttt{C}}\ \mathcal{M}\ [\nu{:}\texttt{'b} \mid P_2(\nu)]\ (y{:}\texttt{'b} \rightarrow \mathcal{M}\ \ [\nu{:}\texttt{'a} \times \texttt{'b} \mid \nu = (x, y)])) \\
& \equiv \mathcal{M}\ [\nu{:}\texttt{'a} \times \texttt{'b} \mid P_1(\mathit{fst}(\nu)) \wedge P_2(\mathit{snd}(\nu))] \quad \text{(after unfolding)}
\end{aligned}
$$

The `option` combinator lifts a generator into a version for option types that covers both the `Some`() and `None` cases ($\nu = \mathit{None} \vee \nu = \texttt{Some}(x)$). Similarly, the `pair` combinator returns a pair containing the results of two input test generators ($\nu = (x, y)$).

$$
\begin{aligned}
\texttt{list_repeat} : s{:}\{\nu{:}\texttt{int} \mid 0 \leq \nu\} \rightarrow g{:}(\mathcal{M}\ [\nu{:}\texttt{'a} \mid P(\nu)]) \rightarrow & \\
& \mathcal{M}\ [\nu{:}\texttt{'a list} \mid \mathit{len}(\nu, s) \wedge \forall x.\mathit{mem}(\nu, x) \Longrightarrow P(x)]
\end{aligned}
$$

The `list_repeat` combinator uses an input generator ($g$) to construct a list of $s$ elements. Its coverage type guarantees that it produces all lists with of the stipulated length $\mathit{len}(\nu, s)$ that contain elements covered by the input generators (i.e., $\forall x.\mathit{mem}(\nu, x) \Longrightarrow P_2(x)$). As a simple example, the following generator produces all lists with three positive numbers:

```
1  let pos_list_gen = list_repeat 3 pos_gen
```

The `oneof` combinator is a more powerful version of the `union` combinator that combines an arbitrary number of test generators, instead of only two. This combinator takes two

$$
\begin{aligned}
\texttt{oneof} : {} & s{:}\{\nu{:}\texttt{int} \mid 0 < \nu\} \to (i{:}\{\nu{:}\texttt{int} \mid P_1(\nu)\} \to \mathcal{M}\ [\nu{:}\texttt{'a} \mid P_2(i,\nu)]) \to \\
& \texttt{bind}^{\mathsf{C}}\ (\mathcal{M}\ \ [\nu{:}\texttt{int} \mid 0 \le \nu < s])\ (i{:}\{\nu{:}\texttt{int} \mid P_1(\nu)\} \to \mathcal{M}\ [\nu{:}\texttt{'a} \mid P_2(i,\nu)]) \\
& \equiv \mathcal{M}\ [\nu{:}\texttt{'a} \mid \exists i.0 \le i < s \wedge P_1(i) \wedge P_2(i,\nu)] \qquad \text{(after unfolding)} \\
\texttt{frequency} : {} & s{:}\{\nu{:}\texttt{int} \mid 0 < \nu\} \to (i{:}\{\nu{:}\texttt{int} \mid P_1(\nu)\} \to \mathcal{M}\ [\nu{:}\texttt{int} \times \texttt{'a} \mid P_2(i,\nu)]) \to \\
& \texttt{bind}^{\mathsf{C}}\ (\texttt{bind}^{\mathsf{C}}\ (\mathcal{M}\ \ [\nu{:}\texttt{nat} \mid 0 \le \nu < s]) \\
& \qquad (i{:}\{\nu{:}\texttt{int} \mid P_1(\nu)\} \to \mathcal{M}\ [\nu{:}\texttt{int} \times \texttt{'a} \mid P_2(i,\nu)])) \\
& \qquad (p{:}\texttt{nat} \times \texttt{'a} \to \mathcal{M}\ \ [\nu{:}\texttt{'a} \mid \mathit{fst}(p) > 0 \wedge \mathit{snd}(p) = \nu]) \\
& \equiv \mathcal{M}\ [\nu{:}\texttt{'a} \mid \exists i.0 \le i < s \wedge P_1(i) \wedge \exists w.w > 0 \wedge P_2(i,(w,\nu))]
\end{aligned}
$$

arguments: the total number of input generators ($s$) and an indexed function ($f$) where $f\ i$ represents the $i^{\text{th}}$ input generator.[11] As captured by its type signature, the number of input generators is constrained by predicate variable $P_1$ and the coverage guarantee of each input generator is constrained by $P_2$. The coverage type of `union` guarantees that it builds a generator that covers everything that the input generators do (i.e., $\exists\ i.0 \le i < s$). As an example, the following (redundant) generator for integers uses `oneof` to combine 3 generators: a negative number generator (`neg_gen`), a constant generator that only produce 0 (`return 0`), and a positive number generator (`pos_gen`).

```
let int_gen = oneof 3 (function
                        | 0 -> neg_gen
                        | 1 -> return 0
                        | _ -> pos_gen)
```

The `frequency` combinator is similar to `oneof`, but it additionally allows the developer to provide a weight for each input generator that governs their sampling distribution. The type of this combinator requires that each of these weights are positive ($w > 0$).

The `fix` combinator constructs a test generator that produces larger test inputs from a function that generators smaller test inputs; the first parameter of this function is a natural

[11] ↑The `oneof` and `frequency` combinators in QCheck use lists instead of a function to hold its input generators. Our formulation of both combinators allows them to straightforwardly combine generators with different coverage guarantees, without having to deal with heterogeneous lists.

$$\texttt{fix} : (m{:}\texttt{nat} \to (n{:}\{\nu{:}\texttt{nat} \mid \nu < m\} \to \mathcal{M}\ [\nu{:}\texttt{'a} \mid P(n,\nu)]) \to \mathcal{M}\ [\nu{:}\texttt{'a} \mid P(m,\nu)]) \to (m{:}\texttt{nat} \to \mathcal{M}\ [\nu{:}\texttt{'a} \mid P(m,\nu)])$$

number that limits the number of recursive calls. Unsurprisingly, this combinator has the same premises as TFix in Fig. 5.7.

```
type 'a tree = Leaf of 'a | Node of 'a tree * 'a tree
let tree_gen : int -> int tree Gen.t = fix
  (fun n self -> match n with
    | 0 -> return err
    | 1 -> bind nat_gen (fun x -> return (Leaf x))
    | n ->
      frequency 2
        (function
        | 0 -> (1, bind nat_gen (fun x -> return (Leaf x)))
        | _ -> (2, bind (self n/2) (fun tr1 ->
                     bind (self n/2) (fun tr2 ->
                       return (Node (tr1, tr2))))))
  )
```

**Figure 5.22.** Defining a tree generator using `frequency` and `fix`.

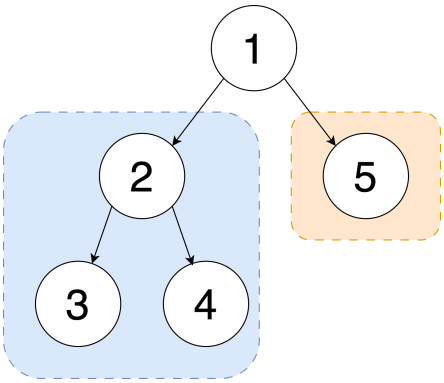


**Figure 5.23.** A 5-node tree that cannot be produced by `tree_gen 5`.

**Example 5.7.3.** We demonstrate how these combinators work by reimplementing the tree generator example shown in Fig. 5.2 using the `fix` and `frequency` combinators. The new

implementation is given in Fig. 5.22. Since every `tree` contains at least one element, the base case returns an error. Otherwise, if the target number of elements is 1 (line 5), we generate a leaf node by `bind`ing the natural number generator with the `Leaf` constructor. In the other cases, we use the `frequency` combinator to combine two test generators with weight 1 and 2 respectively (line 9 - 12). The first generator is a singleton tree generator, while the second generator calls `self` recursively to generate left and right subtrees containing half the nodes. Note that each subtree has at most half the target number of nodes, to ensure that the final tree contains the expected number of elements.

Although this tree generator accepts a bound $n$ as input, it cannot be assigned the coverage type $n{:}\texttt{nat} \to \mathcal{M}\ [\nu{:}\texttt{nat tree} \mid \mathit{numElem}(\nu) \leq n]$, which would require it to generate all trees containing at most $n$ elements. This is because the implementation divides the input bound evenly during recursive calls (lines 10 and 11), making it unable to construct trees that are highly unbalanced. For instance, with input 5, the generator cannot produce the tree illustrated in Fig. 5.23, where the left subtree (highlighted in blue) contains 3 elements which is exceeding the divided bound 5/2. Instead, we can assign it the coverage type $n{:}\texttt{nat} \to \mathcal{M}\ [\nu{:}\texttt{nat tree} \mid 2^{\mathit{depth}(\nu)} \leq n]$, which captures the idea that all trees of depth $m$ can be covered when the input bound is at least $2^m$. According to this type, any tree can be generated by this tree generator given a sufficiently large input bound. This alternative interpretation of the input bound as "depth" is coverage-complete but less intuitive than the "number of nodes" interpretation, highlighting the intricate nature involved in underapproximate-style reasoning.

## 5.8 Implementation and Evaluation

### Implementation

We have implemented a coverage type checker, called Poirot, based on the above approach. Poirot targets functional, non-concurrent OCaml programs that rely on libraries to manipulate

algebraic data types; it consists of approximately 11K lines of OCaml and uses Z3 [8] as its backend solver.[12]

Poirot takes as input an Ocaml program representing a test input generator and a user-supplied coverage type for that generator. After basic type-checking and translation into MNF, Poirot applies bidirectional type inference and checking to validate that the program satisfies the requirements specified by the type. Our implementation provides built-in coverage types for a number of OCaml primitives, including constants, various arithmetic operators, and data constructors for a range of datatypes. Refinements defined in coverage types can also use predefined (polymorphic) method predicates that capture non-trivial datatype shape properties. For example, the method predicate $mem(\mathtt{l}, \mathtt{u})$ indicates the element `u:'a` is contained in the data type instance `l:'a list`; the method predicate $len(\mathtt{l}, 3)$ indicates the list `l` has length 3, or the tree `l` has depth 3. The semantics of these method predicates are defined as a set of FOL-encoded lemmas and axioms to facilitate automated verification; e.g., the lemma $len(\mathtt{l}, 0) \Longrightarrow \forall \mathtt{u}, \neg mem(\mathtt{l}, \mathtt{u})$ indicates that the empty datatype instance contains no element.

In order to improve efficiency, verification conditions generated by Poirot are simplified before being solved by the underlying SMT solver. For example, the `Ex` function generates queries of the form $\exists x.x = v \wedge \phi$, which can be simplified to $\phi[x \mapsto v]$. This transformation can significantly reduce both the number of quantifiers and the overall size of queries.

### 5.8.1 Completeness of Hand-Written Generators

We have evaluated Poirot on a corpus of hand-written, non-trivial test input generators drawn from a variety of sources (see Table 5.3). These benchmarks provide test input generators over a diverse range of datatypes, including various kinds of lists, trees, queues, streams, heaps, and sets. For each datatype implementation, Poirot type checks the provided implementation against its supplied coverage type to verify that the generator is able to generate all possible datatype instances consistent with this type. Our method predicates

[12]↑An artifact containing this tool, our benchmark suite, results and corresponding Coq proofs is publicly available on Zenodo[88]. The benchmark suite of new case study is available on GitHub: https://github.com/zhezhouzz/CoverageType/tree/jfp

**Table 5.3.** Experimental results on hand-written generators. Existing benchmarks are annotated with their source: QuickChick [23] (*), QuickCheck [52] (°), and bespoke test input generators from [80] (⋆) and [72] (◇).

| | #Branch | Rec | #LVar | #MP | #Query | (max. #∀,#∃) | total(avg. time) (ms) |
|---|---|---|---|---|---|---|---|
| SizedList* | 3 | ✓ | 9 | 3 | 8 | (2, 5) | 10.45(1.31) |
| SortedList* | 3 | ✓ | 13 | 4 | 11 | (4, 7) | 31.21(2.84) |
| UniqueList◇ | 3 | ✓ | 10 | 5 | 9 | (2, 5) | 12.28(1.36) |
| SizedTree* | 3 | ✓ | 12 | 4 | 11 | (2, 7) | 15.00(1.36) |
| CompleteTree⋆ | 2 | ✓ | 9 | 5 | 8 | (2, 5) | 12.01(1.50) |
| RedBlackTree* | 6 | ✓ | 44 | 7 | 41 | (5, 20) | 83.28(2.03) |
| SizedBST⋆ | 4 | ✓ | 28 | 6 | 25 | (6, 12) | 42.69(1.71) |
| BatchedQueue◇ | 1 | | 5 | 1 | 5 | (2, 4) | 6.58(1.65) |
| BankersQueue◇ | 1 | | 5 | 1 | 5 | (2, 4) | 7.53(1.51) |
| Stream◇ | 3 | ✓ | 10 | 4 | 9 | (2, 6) | 11.84(1.32) |
| SizedHeap° | 4 | ✓ | 17 | 5 | 15 | (5, 9) | 23.10(1.54) |
| LeftistHeap◇ | 2 | ✓ | 13 | 5 | 12 | (2, 7) | 47.06(3.92) |
| SizedSet° | 3 | ✓ | 26 | 6 | 23 | (5, 11) | 45.17(1.96) |
| UnbalanceSet◇ | 4 | ✓ | 35 | 6 | 31 | (6, 14) | 67.22(2.17) |

allow us capture non-trivial structural properties. For example, to verify a red-black tree generator, we use the predicate $black_height(\nu, n)$ to indicate that all branches of the tree $\nu$ have exactly $n$ black nodes, the predicate $no_red_red(\nu)$ to indicates $\nu$ contains no red node with red children, and the predicate $root_color(\nu, b)$ to indicate the root of the tree $\nu$ has the red (black) color when the boolean value $b$ is true (false).[13]

Given this rich set of predicates, it is straightforward to express interesting coverage types. For example, given size `s` and lower bound `lo`, we can express the property that a sorted list generator `sorted_list_gen` *must* generate all possible sorted lists with the length `s` and in which all elements are greater than or equal to `lo`, as the following type:

$$\texttt{s}{:}\{\nu{:}\texttt{int} \mid \nu \leq 0\} \rightarrow \texttt{lo}{:}\{\nu{:}\texttt{int} \mid \top\}[\nu{:}\texttt{int list} \mid len(\nu, \texttt{s}) \wedge sorted(\nu) \wedge \forall u, mem(\nu, u) \Longrightarrow \texttt{lo} \leq u]$$

Notice that this type is remarkably similar to a normal refinement type:

$$\texttt{s}{:}\{\nu{:}\texttt{int} \mid \nu \leq 0\} \rightarrow \texttt{lo}{:}\{\nu{:}\texttt{int} \mid \top\}\{\nu{:}\texttt{int list} \mid len(\nu, \texttt{s}) \wedge sorted(\nu) \wedge \forall u, mem(\nu, u) \Longrightarrow \texttt{lo} \leq u\}$$

[13]↑These method predicates can be found in the implementation of the red-black tree generator given in [23].

albeit with the return type marked as a coverage type to capture our desired must-property.

The first group of columns in Table 5.3 describes the salient features of our benchmarks. Each benchmark exhibits non-trivial control-flow, containing anywhere from 1 to 6 nested branches; the majority of our benchmarks are also recursive (column Rec). The number of local (i.e., let-bound) variables (column #LVar) is a proxy for path lengths that must be encoded within the types inferred by our type-checker; column #MP indicates the number of method predicates found in the benchmark's type specification.

The second group of columns presents type checking results. The column #Query indicates the number of SMT queries that are triggered during type checking. The column $\#(\forall, \exists)$ indicates the maximum number of universal and existential quantifiers in these queries, respectively. The $\exists$ column is a direct reflection of control-flow (path) complexity — complex generators with deeply nested match-expressions like RedBlackTree result in queries with over 20 existential quantifiers. These numbers broadly track with the values in columns #Branch and #LVar. Despite the complexity of some of these queries, as evidenced by the number of their quantifiers, overall verification time (average verification time per query, resp.), reported in the last column, is quite reasonable, with times ranging from 6.58 to 83.28 milliseconds, i.e., all benchmarks finishing in less than 0.1 second.

### 5.8.2 Case Study: Well-Typed STLC Terms

```
type ty = Ty_nat | Ty_arr of ty * ty

type term =
  | Const of int
  | Var of int
  | Abs of ty * term
  | App of term * term

type tyctx = ty list
```

**Figure 5.24.** Datatypes from the STLC case study.

We have also applied Poirot to a more substantial example: a generator for well-typed simply typed lambda calculus (STLC) terms adapted from [92, 93]. Such a generator can be

used to test that the typing relation guarantees the expected runtime behaviors of programs, e.g. progress and preservation. In addition to the complexity of the coverage property itself (well-typedness), this case study features multiple inductive datatypes (for types, terms, and typing contexts, as shown in Fig. 5.24), and 13 auxiliary functions. The coverage type of `gen_term_size`, the top-level generator, stipulates that it can generate all terms of a desired type, up to a user-provided size bound:

$$\texttt{gen_term_size} : \underbrace{\texttt{n}{:}\{\nu{:}\texttt{int} \mid 0 \leq \nu\}}_{\texttt{maximum term size}} \rightarrow \underbrace{\texttt{t}{:}\{\nu{:}\texttt{ty} \mid \top\}}_{\texttt{type of term}} \rightarrow \underbrace{\Gamma{:}\{\nu{:}\texttt{tyctx} \mid \top\}}_{\texttt{typing context}} \rightarrow$$
$$\underbrace{[\nu{:}\texttt{term} \mid \mathit{has_ty}(\Gamma, \nu, \texttt{t}) \wedge \mathit{max_app_num}(\nu, \texttt{n})]}_{\texttt{result is well-typed and has at most n applications}}$$

The results of using Poirot to verify that `gen_term_size` meets the above specification are shown in Table 5.4. Each function is implemented as a wrapper around a subsidiary function that takes an additional strictly decreasing argument to ensure termination (the original QuickChick implementations uses Coq's Program command for this purpose). These subsidiary functions are responsible for the bulk of the computation, so we report the results for those functions here. The table also reports the results for the most interesting auxiliary functions used by the function. The last column shows that Poirot is able to verify these functions within a reasonable time, ranging from 0.01 to 4.74 seconds. Although more complex functions (as indicated by the column labeled #Branch) require more time to verify, total verification time is nonetheless reasonable: less than 5 seconds in total. Taken together, these results highlight the compositionality of Poirot's type-based approach: each of the 13 auxiliary functions used by `gen_term_size` is individually type-checked against its signature; these signatures are then used to verify any procedures that call the function.

**Table 5.4.** Experimental results from the STLC case study.

| | #Branch | Rec | #LVar | #MP | #Query | (max. #∀,#∃) | total (avg. time) (ms) |
|---|---|---|---|---|---|---|---|
| type_eq | 5 | ✓ | 6 | 4 | 4 | $(3, 12)$ | 57.15(14.29) |
| gen_type | 2 | ✓ | 9 | 3 | 8 | $(2, 5)$ | 9.47(1.18) |
| vars_with_type | 4 | ✓ | 7 | 5 | 5 | $(3, 6)$ | 8.38(1.68) |
| gen_term_no_app | 3 | ✓ | 8 | 12 | 6 | $(3, 7)$ | 13.88(2.31) |
| gen_term_size | 4 | ✓ | 36 | 11 | 32 | $(5, 16)$ | 4735.37(147.98) |

Interestingly, the function `type_eq` has a longer average query time than most of other functions, despite having fewer local variables and method predicates. This function implements a deterministic equality test, returning `true` when two types are the same and `false` otherwise. Thus, the coverage type of this function degenerates into a singleton type for each of the branches, resulting in stricter queries to the SMT solver that take longer to find a valid witness.

#### Discussion

To handle the complexity of this benchmark, Poirot requires 16 method predicates and 24 axioms, the large majority of which correspond to helper definitions and lemmas from the original development. The predicates that encode typing and the bounds on the number of applications in a term (*has_ty* and *max_num_app*, resp.) come directly from the QuickCheck version, for example. The following axiom encodes the semantic relationship of these predicates

$$\forall \Gamma{:}\texttt{tyctx}.\ \forall \texttt{t}{:}\texttt{ty}.\ \forall \texttt{e}{:}\texttt{term}.$$
$$\mathit{has_ty}(\Gamma, \texttt{e}, \texttt{t}) \iff \exists \texttt{n}{:}\texttt{nat}.\ \mathit{max_num_app}(\texttt{e}, \texttt{n}) \wedge \mathit{has_ty}(\Gamma, \texttt{e}, \texttt{t})$$

and is analogous to the helper lemma `has_ty_max_tau_correct` in the Coq development. In addition, some predicates and axioms are independent of this particular case study: the typing context is implemented as a list of STLC types, and thus we were able to reuse generic predicates and axioms about polymorphic lists.

### 5.8.3 Completeness of Synthesized Generators

An underlying hypothesis motivating our work is that writing sound and *complete* test input generators can be subtle and tricky, as demonstrated by our motivating example (Figure 5.1). To justify this hypothesis, we repurposed an existing deductive component-based program synthesizer [94] to automatically synthesize correct (albeit possibly incomplete) generators that satisfy a specification given as an overapproximate refinement type; these

**Table 5.5.** Quantifying the space of safe and complete test input generators constructed by an automated program synthesis tool.

| Benchmark | #Total | #Complete |
|---|---|---|
| UniqueList | 284 | 10 |
| SizedList | 126 | 28 |
| SortedList | 30 | 8 |
| SizedTree | 103 | 2 |
| SizedBST | 229 | 54 |
| RedBlackTree | 234 | 2 |

generators are then fed to Poirot to validate their completeness. We provided the synthesizer with a datatype definition and a set of specifications describing constraints on that datatype the synthesized generator should use, along with a library of functions, including primitive generators such as `nat_gen`, available to the synthesizer for construction. A refinement type-guided enumeration is performed to find all correct programs consistent with the specification. Since the space of these programs is potentially quite large (possibly infinite), we constrain the synthesizer to only generate programs with bounded function call depths; in our experiments, this bound was set to three. The generator outputs all programs that are safe with respect to the specification. Table 5.5 shows results of this experiment for five of the benchmarks given in Table 5.3; results for the other benchmarks are similar. We report the total number of synthesized generators (#Total) constructed and the number of those that are *correct and complete* as verified by Poirot (#Complete). The table confirms our hypothesis that the space of complete generators with respect to the supplied coverage type is significantly smaller than the space of safe generators, as defined by an overapproximate refinement type specification.

More concretely, Figure 5.25 shows three synthesized generators that satisfy the following specification of a list generator that is meant to construct *all* lists no longer than some provided bound:

$$\texttt{size}{:}\{\nu{:}\texttt{int} \mid \nu \leq 0\} \rightarrow [\nu{:}\texttt{int list} \mid \forall u, len(\nu, u) \Longrightarrow (0 \leq u \wedge u \leq \texttt{size})]$$

Fig. 5.25b is incomplete because it never generates an empty list when the size parameter `size` is greater than 0. On the other hand, while Fig. 5.25c does generate empty lists, the

```
1 let rec sized_list_gen
2     (size : int) : (int list) =
3   if (size == 0) then []
4   else
5     if bool_gen () then sized_list_gen (size - 1)
6     else (int_gen ()) :: (sized_list_gen (size - 1))
```

(a) A sound and complete generator.

```
let rec sized_list_gen
    (size : int) : (int list) =
  if (size == 0) then []
  else (int_gen ()) :: (sized_list_gen (size - 1))
```

(b) A sound but incomplete generator.

```
let rec sized_list_gen
    (size : int) : (int list) =
  if (size == 0) then []
  else
    if bool_gen () then sized_list_gen (size - 1)
    else size :: (sized_list_gen (size - 1))
```

(c) Another sound but incomplete generator.

**Figure 5.25.** Three example generators that generate size-bounded lists.

else branch of its second conditional has a fixed first element and will therefore never generate lists with distinct elements. The complete generator shown in Fig. 5.25a incorporates a control-flow path (line 5) that can non-deterministically choose to make a recursive call to `sized_list_gen` with a smaller size, thereby allowing it to generate lists of variable size up to the `size` bound, including the empty list; another conditional branch uses `int_gen()` to generate a new randomly selected list element, thereby allowing the implementation to generate lists containing distinct elements. We again emphasize that Poirot was able to verify the correct generator and discard the two incorrect generators *automatically*, without any user involvement.

**Table 5.6.** Experimental results that use monadic combinators from the `Gen` library of the `QCheck` framework.

| | #Branch | Rec | #LVar | #'a | #P | #MP | #Query | (max. #∀,#∃) | total (avg. time) (ms) |
|---|---|---|---|---|---|---|---|---|---|
| return | 1 | | 2 | 1 | 0 | 0 | 1 | $(0, 0)$ | 0.14(0.14) |
| bind | 1 | | 5 | 2 | 2 | 0 | 3 | $(2, 1)$ | 28.74(9.58) |
| fmap | 1 | | 4 | 2 | 2 | 0 | 2 | $(2, 1)$ | 2.23(1.11) |
| pair | 1 | | 5 | 2 | 2 | 2 | 3 | $(1, 2)$ | 2.61(0.87) |
| option | 2 | | 7 | 1 | 1 | 3 | 14 | $(2, 3)$ | 5.91(0.42) |
| union | 2 | | 4 | 1 | 2 | 0 | 12 | $(1, 1)$ | 2.48(0.21) |
| oneof | 1 | | 4 | 1 | 1 | 0 | 3 | $(2, 1)$ | 4.35(1.45) |
| frequency | 1 | | 5 | 1 | 1 | 2 | 3 | $(3, 2)$ | 3.04(1.01) |
| list_repeat | 2 | ✓ | 12 | 1 | 1 | 4 | 12 | $(3, 5)$ | 11.48(0.96) |
| fix | 1 | | 5 | 1 | 1 | 0 | 15 | $(2, 0)$ | 11.23(0.75) |

### 5.8.4 Case Study: Checking Coverage in Practice

In this section, we focus on test input generators from real-world applications as well as the underlying combinators provided by mainstream PBT frameworks. The experimental result of verifying the monadic combinators for Sec. 5.7 is shown in Table 5.6. The columns are the same as in Table 5.3, with an additional column showing the number of type variables (#`'a`) and predicate variables ($\#P$) for these polymorphic combinators. A number of combinator types do not require method predicates since they do not make use of datatypes; list_repeat (`list`), option (`option`), pair (`pair`), and frequency (`pair`), do however, employ such predicates. These combinators are relatively simple and Poirot is able to verify them within 0.03 seconds with small variance.

We also apply Poirot to identify coverage completeness violations across five open-source repositories that use the `QCheck` framework to test their implementation. Each project deals with a different real-world application domain: blockchain [95], virtualization [96], compiler verification [97], weak memory model testing [98], and automated theorem proving [99]. The details of these projects are as follows:

1. **Tezos**. Tezos [95] is a blockchain platform implemented in OCaml that supports decentralized assets and applications. Tezos writes its own generators to construct block trees, i.e., trees of block nodes representing the shape of a blockchain, where blocks are expected to have unique hash codes.

2. **Xen API**. Xen API [96] is a management API for controlling the virtualization environments of then Xen Project Hypervisor [100]. We selected the file descriptor generator of the API, which produces random file descriptors with different attributes, e.g., file size, kind, read and write delay time. The generated test inputs are expected to be a subset of valid Unix file descriptors (e.g., not a symbolic link).

3. **Vellvm**. Vellvm [97] is a formal verification framework for the LLVM IR, implemented in both OCaml and Coq. We selected the generator for LLVM values used to provide as arguments of various LLVM functions under test. Given a specific LLVM type, this generator is expected to generate all well-typed LLVM values.

4. **Herdtools7**. Herdtools7 [98] is a collection of tools for modeling and analyzing concurrent programs with weak memory models. The generator `Carpenter` in Herdtools7 produces random ASTs of the Arm Architecture Specification Language [101], where the generator is expected to generate every AST of a specified size.

5. **Zipperposition**. Zipperposition [99] is an automated theorem prover that supports polymorphic types and higher-order unification. Test generators are used to produce valid first-order logic (e.g., $\forall x{:}\alpha.\exists y{:}\alpha.x \neq y$) and lambda-free higher-order logic formulae.

**Table 5.7.** Experimental results for test generators of open-source repositories.

| | #Branch | Rec | #LVar | #MP | #C | #Query | (max. #∀,#∃) | total (avg. time) (s) |
|---|---|---|---|---|---|---|---|---|
| Tezos | 7 | ✓ | 40 | 15 | 3 | 73 | $(3, 35)$ | 10.75(0.15) |
| Xen API | 1 | | 21 | 5 | 3 | 24 | $(1, 16)$ | 6.51(0.27) |
| Vellvm | 10 | ✓ | 41 | 19 | 4 | 84 | $(4, 19)$ | 2.05(0.02) |
| Herdtools7 | 1 | | 32 | 10 | 3 | 90 | $(2, 8)$ | 2.08(0.02) |
| Zipperposition | 7 | ✓ | 71 | 16 | 4 | 134 | $(2, 49)$ | 15.81(0.12) |

We used our polymorphic version of Poirot to check that these test generators can cover all test inputs that satisfy the precondition of the program under test. Thus, we type check generators against coverage types that encode specifications extracted from README files, documentation, and comments in the source code. The experimental results are shown in Table 5.7, where the columns follow the same format as in Table 5.3, with an additional

column showing the number of combinators (#C) used by these generators. The verification time in the last column is given in seconds. As shown in Table 5.7, these test generators are complex, including 1-10 control flow branches, 21-71 local variables, and 5-19 method predicates. Most method predicates are directly converted from datatype constructors (e.g., `cons` for list datatype). Notably, Poirot reports that *none* of these generators are coverage complete with respect to the preconditions we expected, with verification times ranging from 2.08 to 15.81 seconds. Due to the sophisticated nature of realistic test generators, verification takes longer compared to previous benchmarks, though it remains within acceptable limits. One observation is that the result of type checking does not significantly affect the verification time, which differs from standard refinement type checkers that can return early when detecting safety violations in one control flow. This is because Poirot cannot determine the coverage completeness of a program without analyzing *all* corresponding control flows, as discussed in Sec. 5.3.

In the rest of section, we highlight three representative categories of coverage incompleteness detected by Poirot for these benchmarks, examine the underlying causes, and suggest candidate fixes to address these coverage missing.

### Limited sampling range

Manually written test input generators often rely on restricted sampling ranges, resulting in coverage completeness violations. For example, Vellvm [97] generates `LLVM int32` and `LLVM int64` values only within the range 0–10000, and the generators in the herdtools7 framework [98] also restrict integer literals appearing in theorems to this interval. These generators typically use QCheck's built-in `Gen.nat` combinator, which only produces natural numbers below 10000. While developers adopt this restriction under the assumption that smaller values are more likely to detect bugs, such approaches (theoretically) compromise completeness. One solution is to run two tests one with smaller values and one with the full range of values. On the other hand, we also propose a patched version of `Gen.nat` that retains a sampling bias toward small values but guarantees completeness by combining the restricted

generator with a complete one using QCheck's `frequency` combinator. An example of this updated generator is shown below.

```
let nat = frequency 2 (function
                        | 0 -> (10, Gen.nat)
                        | _ -> (1, full_nat_gen))
```

### Fixed values

Zipperposition [99] uses the `oneof` combinator to restrict variable names to one of three fixed options: `X`, `Y`, or `Z`. Similarly, the Xen hypervisor [96] constrains file sizes to 9 predetermined lengths and only uses one of 4 fixed float number (e.g., 0.001, 0.01, 0.1, and 0.4) as file read and write delay times. While such choices may reflect the developer's intuition about likely corner cases, they also prevent the exploration of errors outside these considerations. The solution proposed earlier can also be used to repair incompleteness here as well.

```
1        type 'a tree =
2        | Leaf of 'a
3        | Node1 of ('a * 'a tree)
4        | Node2 of ('a * 'a tree * 'a tree)
5
6        let rec tree_gen (blocks: block list): block tree gen =
7            match block with
8            | [x] -> return (Leaf x)
9            | x :: xs -> (
10             ... (* the case producing Leaf *)
11             ... (* the case producing Node1 *)
12             int_bound (List.length xs - 1) >>= fun n ->
13             let left, right = List.split_n n xs in
14             tree_gen left >>= fun left ->
15             tree_gen right >>= fun right ->
16             return (Node2 (x, left, right))
17
18       tree_gen (Block.set_to_list unique_nonempty_block_gen)
```

**Figure 5.26.** A tree-like block chain generator.

### Datatype Transformation

Poirot identified an interesting coverage completeness violation in the Tezos blockchain project [95], where test input generators construct block trees, i.e., trees of block nodes representing the shape of the block chain. Instead of generating such trees directly, Tezos implements a helper function `tree_gen` (shown in Fig. 5.26) that builds trees from lists of blocks. Specifically, `tree_gen` accepts a non-empty, duplicate-free list of blocks and returns a generator that produces a tree with non-duplicate blocks. The code comments state that this approach allows better control over sampling distributions and uniqueness. Thus, the expected coverage refinement type of `tree_gen` should be:

$$\texttt{tree_gen} : \texttt{l}{:}\{\nu{:}\texttt{block list} \mid \mathit{uniqList}(\nu)\} \rightarrow$$
$$\mathcal{M}\ [\nu{:}\texttt{block tree} \mid \mathit{uniqTree}(\nu) \wedge \forall u.\mathit{memTree}(\nu, u) \iff \mathit{mem}(\texttt{l}, u)]$$

However, the actual implementation in Tezos fails to satisfy this specification and is rejected by Poirot. The root of every generated tree (line 16) is deterministically selected as the head of the input list. Moreover, the use of `List.split_n` partitions the input list into subtlists that preserve element ordering, leading to the left subtree always containing earlier elements in the list. Since Tezos applies `tree_gen` to sorted lists (line 18), the resulting block trees consistently place the smallest element at the root, with left subtree elements always being smaller than those in the right subtree. The root cause of this error lies in the complexity of maintaining coverage completeness through transformations between inductive datatypes (e.g., from lists to trees).

One possible solution is to randomly permute the input list of blocks. Another patch is to replace the list input with a `set`:

$$\texttt{tree_gen} : \texttt{s}{:}\{\nu{:}\texttt{block set} \mid \top\} \rightarrow$$
$$\mathcal{M}\ [\nu{:}\texttt{block tree} \mid \mathit{uniqTree}(\nu) \wedge \forall u.\mathit{memTree}(\nu, u) \iff \mathit{memSet}(\texttt{s}, u)]$$

The implementation should randomly select the root node from the set and partition the set into two disjoint subsets using a randomized split instead of relying on list indexing. We have submitted this patch to the Tezos codebase.

# 6. RELATED WORK

## Refinement Types

Prior work on refinement types has also focused on incomplete specification. Refinement reflection has been introduced to enable SMT-based reasoning about arbitrary user-defined functions by embedding their semantics directly into the logic [102]. However, this still requires users to provide instantiations for lemmas and induction hypotheses. Our data-driven technique in Chapter 3 avoids the need to reflect the implementation into the solver's logic, which is critical in our setting where the implementation is often unavailable.

There has been significant prior work on the type-based verification of functional programs with effects. Ynot [63] and F* [62] are two well-known examples that allow fine-grained effect tracking using type specifications based on abstractions like the Hoare monad [66] or the Dijkstra monad [64, 65, 103–105]. Specifications in these systems are typically written as pre- and post-conditions over the concrete program heap.

Our approach in Chapter 4 is fundamentally different because we assume the implementation of effectful operations is hidden, our verification methodology must reason about stateful actions indirectly, in terms of the *history* of calls made by the client. We observe that verification in this setting benefits from specifying pre- and post-conditions on a computation in terms of such histories, and that Symbolic Finite Automata (SFAs)—and their embodiment as Hoare Automata Types (HATs)—are well-suited to serve as their underlying representation. Unlike the predicate-transformer semantics of the Dijkstra monad, our history-based approach allows us to reuse a standard subtyping relation (suitably adapted), enabling a high degree of automation.

## Data-Driven and Learning-Based Specification Inference

Several recent data-driven approaches have used learning techniques to infer specifications of library functions. For example, some methods automatically infer specifications that use a fixed set of features to identify relationships between the inputs and outputs of a function [47]. Others use program synthesis to learn these features on demand when inferring preconditions

for data-structure-manipulating library functions [48]. This line of work has been extended to synthesize representation invariants sufficient to verify abstract datatype operations [49, 106].

In contrast to approaches that infer specifications for libraries in isolation, our work in Chapter 3 focuses on the complementary problem of discovering specifications to verify specific library clients. This change in perspective means we cannot always appeal to a source of ground truth (e.g., SMT verification as in [48] or bounded model checking as in [49]). This distinction necessitates the symbolic weakening procedure introduced in this dissertation, which facilitates generalization and prevents overfitting to client-specific contexts. However, techniques like those for representation invariant synthesis could be used alongside ours, for example, by leveraging abduced specifications to infer and verify library-side invariants.

### Abductive Inference

Logical formulation of specification inference can be understood as an instance of abductive inference. This observation has been previously exploited to develop inference algorithms for loop invariants [44] and specifications of functions within a client program [46]. For a given program, these algorithms typically rely on an abduction procedure to iteratively strengthen invariants or specifications until they are sufficient to prove a user-provided post-condition. While fully automated, these approaches critically depend on an abduction procedure for the underlying logic, such as the first-order theory of linear integer arithmetic.

However, no general abduction procedure exists for the theory of equalities with uninterpreted functions, which is commonly used to specify recursive functions over algebraic datatypes. Our work in Chapter 3 provides an alternative solution that combines data-driven methods with SMT-based, counterexample-guided refinement to discover library method specifications that aid client-side verification. While Bastani et al. [107] consider a similar abductive problem, their focus is on CFL reachability for alias and taint specifications, whereas our hypothesis space involves rich shape properties over algebraic datatypes. Furthermore, our approach guarantees consistency with black-box library implementations and incorporates an explicit weakening procedure to generalize candidate solutions.

### Specification Mining

While recent work has focused on mining specifications from client code [108, 109] under the assumption that the client is correct, we leverage abductive methods to derive guarantees that any library implementation must satisfy to ensure client safety, without assuming the client is already safe. Another distinguishing feature is our focus on generating maximally weak specifications to prevent overfitting. While some work learns specifications from natural language comments [110], it does not provide the formal safety and consistency guarantees of our method. Qin et al. [111] also infer specifications for unknown procedures in a rich domain, but their technique does not query library implementations and requires users to provide interpretations of predicates, which can lead to overfitted specifications. Our algorithmic weakening procedure in Chapter 3 directly addresses this challenge. Finally, while it is possible to frame our problem as an instance of data-driven CHC solving [112], this would not address the crucial challenge of generating maximally weak specifications, a concern not typically handled by Horn-clause semantics [46].

### Effect Verification and Monadic Approaches

Monadic techniques for reasoning about effectful programs have also been well-studied in the context of interactive verification. A notable example is Interaction Trees (ITrees) [113], a coinductive abstraction for sequences of visible events. Analogous to HATs introduced in Chapter 4, context and effect traces can be expressed as pre- and post-ITrees [114]. In contrast to ITrees, HATs' use of SFAs provides a decidable inclusion check, which enables fully automated verification and a lighter-weight specification language based on $\mathrm{LTL}_f$.

### Type and Effect Systems

Type and effect systems provide guarantees about both *what* values a program computes and *how* it computes them. Many such systems are agnostic to the order in which effects are produced; for instance, Java's exception system only tracks whether a computation *may* raise

an exception [115]. In contrast, *sequential* effect systems provide finer-grained information about the order of effects [116].

More closely related to our work in Chapter 4 are type and effect systems targeting temporal properties of event sequences. One such system uses regular expressions to specify properties of event histories, but cannot capture data-dependent properties [117]. Another supports infinite traces specified as Büchi automata, but does not allow events to refer to program variables [118]. This restriction was later lifted to support value-dependent predicates in a first-order fixpoint logic [67], and further extended to support richer control-flow structures [68]. To our knowledge, all of these prior effect systems lack an analogue to the precondition automaton of HATs, forcing them to rely on function arguments to constrain output traces, rather than the context of previously seen events.

### Type-Based Protocol Enforcement

Because HATs in Chapter 4 express a history of interaction between a client and a stateful library, they naturally serve as a form of protocol specification. In this sense, they are related to typestate systems in object-oriented languages [119–121], which augment object interfaces with coarse-grained state information to constrain the set of available methods. HATs differ by capturing much finer-grained, data-dependent interactions necessary to specify useful representation invariants. A more natural point of comparison is session types [122, 123], which specify fine-grained communication protocols. Unlike session types, which often model symmetric interactions, the protocols captured by HATs are specified from the client's perspective, observing the results of a sequence of method calls. By restricting protocols to those encodable as symbolic regular languages, our type system achieves a high degree of automation and a reduced specification burden compared to more expressive session type systems [124–126].

### Test Generator Quality and Completeness

Although PBT can automatically derive simple test generators, handwritten generators remain crucial for complex datatypes. A recent empirical study found that writing generators

with complex preconditions (e.g., for valid red-black trees) remains a significant challenge for developers, who find it "tedious" and "high-effort" [127]. Our work in Chapter 5 aims to alleviate this burden by providing an automated validation mechanism to ensure generators satisfy both safety and completeness constraints. Other recent work has sought to reduce user effort by visualizing generator quality [128], tuning generator weights [129], or using combinatorial testing to improve input-space partitioning [130].

A general limitation of PBT is its reduced effectiveness when properties have strict preconditions, as purely random generation strategies discard most inputs [131]. Consequently, much recent interest has focused on improving generator coverage. Proposed solutions range from fuzzing-inspired mutation [80, 81, 132, 133] to focusing on particular input classes (e.g., well-typed programs) [93, 134, 135] or building complete-by-construction generators [82, 92, 136].

Our approach differs by framing coverage in purely type-theoretic terms. This allows us to statically and compositionally verify coverage properties without runtime instrumentation. Unlike mechanized proofs of generator coverage [37, 137, 138], our type-based framing is highly automated and agnostic to how generators are constructed.

### Under-approximate Reasoning

A number of logics have been proposed for under-approximating program behavior, including Incorrectness Logic (IL) [39, 85], Reverse Hoare Logic (RHL) [139], and Dynamic Logic (DL) [140]. Both IL and RHL are formalisms similar to Hoare logic but for asserting under-approximate postconditions, with IL adding special assertions for error states. Coverage type in Chapter 5 provides the first development that interprets these "must-style" reasoning principles in the context of a refinement type system for a rich functional language. While our ideas are formulated for verifying test generator coverage, we believe our framework is equally adept at expressing type-based analyses for bug finding or compiler optimizations. Our focus distinguishes our approach from other refinement-type-based testing solutions like TARGET [141], which do not reason about coverage.

Finally, a related line of work has explored reasoning about the distribution of data produced by a function, with a focus on fairness properties in machine learning applications [142, 143]. These works typically encode properties as probabilistic formulas in real arithmetic. In contrast, our coverage types can only verify that a generator has a non-zero probability of producing a particular output. Extending our system to provide stronger guarantees about the distribution of discrete data is an exciting direction for future work.

# 7. CONCLUSION

This dissertation has addressed several critical challenges at the intersection of automated program verification, testing, and compositional reasoning. Modern software development increasingly relies on complex libraries with rich semantics, but the verification of client applications is often hindered by missing API specifications and opaque internal library states. Furthermore, ensuring the coverage completeness of testing methodologies like Property-Based Testing (PBT) introduces its own set of verification challenges. To overcome these obstacles, this work has introduced a suite of novel techniques that extend refinement type systems, enabling more powerful and automated reasoning about real-world software.

To combat the problem of missing or incomplete specifications, Chapter 3 presented a novel, data-driven abductive inference procedure. This technique addresses the challenge of verifying client code when library behavior is unknown. It combines a learning framework that postulates candidate specifications from observed examples with a counterexample-guided refinement loop that uses a Satisfiability Modulo Theories (SMT) solver to systematically improve them. A key innovation of this work is a symbolic weakening procedure that ensures the inferred specifications are general and reusable, preventing the common pitfall of overfitting to a specific client context. The result is a system that can automatically synthesize weakest-precondition specifications sufficient for verification.

To enable reasoning about clients of stateful libraries with hidden internal state, Chapter 4 introduced Hoare Automata Types (HATs). This formalism provides a new way to verify programs whose correctness depends on a history of interactions with an opaque component. HATs abstract the hidden state as a trace of events and extend refinement types with pre- and post-conditions expressed as Symbolic Finite Automata (SFA). By leveraging the decidability of SFA inclusion and the expressiveness of Linear Temporal Logic on finite traces ($\mathrm{LTL}_f$) as a user-friendly specification language, HATs enable the fully automated verification of complex, history-based invariants that were previously beyond the reach of automated tools.

Finally, to bring formal guarantees to the domain of automated testing, Chapter 5 introduced coverage types, a novel type system founded on the principles of under-approximate, ”must-style” reasoning. Where traditional refinement types prove the absence of bad behaviors

(safety), coverage types prove the presence of good ones. This allows, for the first time, a static and compositional proof that a test input generator can produce all valid inputs satisfying a given precondition, thus guaranteeing coverage completeness. To make this approach practical, the system is extended with type and qualifier polymorphism, enabling it to seamlessly handle the monadic combinator patterns prevalent in modern PBT frameworks.

# REFERENCES


[1] R. W. Floyd, "Assigning meanings to programs," in *Program Verification: Fundamental Issues in Computer Science*, T. R. Colburn, J. H. Fetzer, and T. L. Rankin, Eds. Dordrecht: Springer Netherlands, 1993, pp. 65–81, ISBN: 978-94-011-1793-7. DOI: 10.1007/978-94-011-1793-7_4. [Online]. Available: https://doi.org/10.1007/978-94-011-1793-7_4.

[2] C. A. R. Hoare, "An axiomatic basis for computer programming," *Commun. ACM*, vol. 12, no. 10, pp. 576–580, Oct. 1969, ISSN: 0001-0782. DOI: 10.1145/363235.363259. [Online]. Available: https://doi.org/10.1145/363235.363259.

[3] X. Leroy, "Formal verification of a realistic compiler," *Commun. ACM*, vol. 52, no. 7, pp. 107–115, Jul. 2009, ISSN: 0001-0782. DOI: 10.1145/1538788.1538814. [Online]. Available: https://doi.org/10.1145/1538788.1538814.

[4] D. Kästner *et al.*, "Compcert: Practical experience on integrating and qualifying a formally verified optimizing compiler," in *ERTS2 2018-9th European Congress Embedded Real-Time Software and Systems*, 2018, pp. 1–9.

[5] G. Klein *et al.*, "Comprehensive formal verification of an os microkernel," *ACM Trans. Comput. Syst.*, vol. 32, no. 1, Feb. 2014, ISSN: 0734-2071. DOI: 10.1145/2560537. [Online]. Available: https://doi.org/10.1145/2560537.

[6] C. Hawblitzel *et al.*, "Ironfleet: Proving practical distributed systems correct," in *Proceedings of the 25th Symposium on Operating Systems Principles*, ser. SOSP '15, Monterey, California: Association for Computing Machinery, 2015, pp. 1–17, ISBN: 9781450338349. DOI: 10.1145/2815400.2815428. [Online]. Available: https://doi.org/10.1145/2815400.2815428.

[7] J.-K. Zinzindohoué, K. Bhargavan, J. Protzenko, and B. Beurdouche, "Hacl*: A verified modern cryptographic library," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '17, Dallas, Texas, USA: Association for Computing Machinery, 2017, pp. 1789–1806, ISBN: 9781450349468. DOI: 10.1145/3133956.3134043. [Online]. Available: https://doi.org/10.1145/3133956.3134043.

[8] L. de Moura and N. Bjørner, "Z3: An efficient smt solver," in *Tools and Algorithms for the Construction and Analysis of Systems*, C. R. Ramakrishnan and J. Rehof, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 337–340, ISBN: 978-3-540-78800-3. DOI: 10.1007/978-3-540-78800-3_24.

[9] H. Wang, S. Yu, C. Chen, B. Turhan, and X. Zhu, "Beyond accuracy: An empirical study on unit testing in open-source deep learning projects," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 4, Apr. 2024, ISSN: 1049-331X. DOI: 10.1145/3638245. [Online]. Available: https://doi.org/10.1145/3638245.

[10] E. Daka and G. Fraser, "A survey on unit testing practices and problems," in *Proceedings of the 2014 IEEE 25th International Symposium on Software Reliability Engineering*, ser. ISSRE '14, USA: IEEE Computer Society, 2014, pp. 201–211, ISBN: 9781479960330. DOI: 10.1109/ISSRE.2014.11. [Online]. Available: https://doi.org/10.1109/ISSRE.2014.11.

[11] F. Trautsch and J. Grabowski, "Are there any unit tests? an empirical study on unit testing in open source python projects," in *2017 IEEE International Conference on Software Testing, Verification and Validation (ICST)*, 2017, pp. 207–218. DOI: 10.1109/ICST.2017.26.

[12] H. Goldstein, J. W. Cutler, D. Dickstein, B. C. Pierce, and A. Head, "Property-based testing in practice," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24, <conf-loc>, <city>Lisbon</city>, <country>Portugal</country>, </conf-loc>: Association for Computing Machinery, 2024, ISBN: 9798400702174. DOI: 10.1145/3597503.3639581. [Online]. Available: https://doi.org/10.1145/3597503.3639581.

[13] N. Vazou, E. L. Seidel, R. Jhala, D. Vytiniotis, and S. Peyton-Jones, "Refinement types for haskell," *SIGPLAN Not.*, vol. 49, no. 9, pp. 269–282, Aug. 2014, ISSN: 0362-1340. DOI: 10.1145/2692915.2628161. [Online]. Available: https://doi.org/10.1145/2692915.2628161.

[14] F. Kawamata, H. Unno, T. Sekiyama, and T. Terauchi, "Answer refinement modification: Refinement type system for algebraic effects and handlers," *Proc. ACM Program. Lang.*, vol. 8, no. POPL, Jan. 2024. DOI: 10.1145/3633280. [Online]. Available: https://doi.org/10.1145/3633280.

[15] P. Vekris, B. Cosman, and R. Jhala, "Refinement types for typescript," in *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '16, Santa Barbara, CA, USA: Association for Computing Machinery, 2016, pp. 310–325, ISBN: 9781450342612. DOI: 10.1145/2908080.2908110. [Online]. Available: https://doi.org/10.1145/2908080.2908110.

[16] N. Lehmann, A. T. Geller, N. Vazou, and R. Jhala, "Flux: Liquid Types for Rust," vol. 7, no. PLDI, 2023. DOI: 10.1145/3591283. [Online]. Available: https://doi.org/10.1145/3591283.

[17] M. Kazerounian, N. Vazou, A. Bourgerie, J. S. Foster, and E. Torlak, "Refinement types for ruby," in *Verification, Model Checking, and Abstract Interpretation*, I. Dillig and J. Palsberg, Eds., Cham: Springer International Publishing, 2018, pp. 269–290, ISBN: 978-3-319-73721-8.

[18] P. M. Rondon, M. Kawaguchi, and R. Jhala, "Low-level liquid types," *SIGPLAN Not.*, vol. 45, no. 1, pp. 131–144, Jan. 2010, ISSN: 0362-1340. DOI: 10.1145/1707801.1706316. [Online]. Available: https://doi.org/10.1145/1707801.1706316.

[19] M. Sammler, R. Lepigre, R. Krebbers, K. Memarian, D. Dreyer, and D. Garg, "Refinedc: Automating the foundational verification of c code with refined ownership types," in *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, ser. PLDI 2021, Virtual, Canada: Association for Computing Machinery, 2021, pp. 158–174, ISBN: 9781450383912. DOI: 10.1145/3453483.3454036. [Online]. Available: https://doi.org/10.1145/3453483.3454036.

[20] N. Vazou, E. L. Seidel, and R. Jhala, "Liquidhaskell: Experience with refinement types in the real world," *SIGPLAN Not.*, vol. 49, no. 12, pp. 39–51, Sep. 2014, ISSN: 0362-1340. DOI: 10.1145/2775050.2633366. [Online]. Available: https://doi.org/10.1145/2775050.2633366.

[21] E. L. Seidel, N. Vazou, and R. Jhala, "Type targeted testing," in *Programming Languages and Systems*, J. Vitek, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 2015, pp. 812–836, ISBN: 978-3-662-46669-8.

[22] R. Jhala and N. Vazou, "Refinement Types: A Tutorial," *Found. Trends Program. Lang.*, vol. 6, no. 3-4, pp. 159–317, 2021. DOI: 10.1561/2500000032. [Online]. Available: https://doi.org/10.1561/2500000032.

[23] L. Lampropoulos and B. C. Pierce, *QuickChick: Property-Based Testing in Coq* (Software Foundations). Electronic textbook, 2022, vol. 4, Version 1.3.1, https://softwarefoundations.cis.upenn.edu.

[24] Z. Zhou, Q. Ye, B. Delaware, and S. Jagannathan, *A hat trick: Automatically verifying representation invariants using symbolic finite automata*, 2024. arXiv: 2404.01484 [cs.PL].

[25] Z. Zhou, R. Dickerson, B. Delaware, and S. Jagannathan, "Data-driven abductive inference of library specifications," *Proc. ACM Program. Lang.*, vol. 5, no. OOPSLA, Oct. 2021. DOI: 10.1145/3485493. [Online]. Available: https://doi.org/10.1145/3485493.

[26] Z. Zhou, A. Mishra, B. Delaware, and S. Jagannathan, "Covering all the bases: Type-based verification of test input generators," *Proc. ACM Program. Lang.*, vol. 7, no. PLDI, Jun. 2023. DOI: 10.1145/3591271. [Online]. Available: https://doi.org/10.1145/3591271.

[27] A. Tondwalkar, M. Kolosick, and R. Jhala, "Refinements of Futures Past: Higher-Order Specification with Implicit Refinement Types," in *35th European Conference on Object-Oriented Programming (ECOOP 2021)*, A. Møller and M. Sridharan, Eds., ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 194, Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, 18:1–18:29, ISBN: 978-3-95977-190-0. DOI: 10.4230/LIPIcs.ECOOP.2021.18. [Online]. Available: https://drops-dev.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2021.18.

[28] P. M. Rondon, M. Kawaguci, and R. Jhala, "Liquid types," in *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '08, Tucson, AZ, USA: Association for Computing Machinery, 2008, pp. 159–169, ISBN: 9781595938602. DOI: 10.1145/1375581.1375602. [Online]. Available: https://doi.org/10.1145/1375581.1375602.

[29] S. Weirich, A. Voizard, P. H. A. de Amorim, and R. A. Eisenberg, "A specification for dependent types in haskell," *Proc. ACM Program. Lang.*, vol. 1, no. ICFP, Aug. 2017. DOI: 10.1145/3110275. [Online]. Available: https://doi.org/10.1145/3110275.

[30] C. Pulte, D. C. Makwana, T. Sewell, K. Memarian, P. Sewell, and N. Krishnaswami, "Cn: Verifying systems c code with separation-logic refinement types," *Proc. ACM Program. Lang.*, vol. 7, no. POPL, Jan. 2023. DOI: 10.1145/3571194. [Online]. Available: https://doi.org/10.1145/3571194.

[31] J. Bengtson, K. Bhargavan, C. Fournet, A. D. Gordon, and S. Maffeis, "Refinement types for secure implementations," *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 33, no. 2, pp. 1–45, 2011. DOI: 10.1145/1890028.1890031.

[32] Y. Gao, Z. Li, H. Lin, H. Zhang, M. Wu, and M. Yang, "Refty: Refinement types for valid deep learning models," in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22, Pittsburgh, Pennsylvania: Association for Computing Machinery, 2022, pp. 1843–1855, ISBN: 9781450392211. DOI: 10.1145/3510003.3510077. [Online]. Available: https://doi.org/10.1145/3510003.3510077.

[33] E. Vasilenko, N. Vazou, and G. Barthe, "Safe couplings: Coupled refinement types," *Proc. ACM Program. Lang.*, vol. 6, no. ICFP, Aug. 2022. DOI: 10.1145/3547643. [Online]. Available: https://doi.org/10.1145/3547643.

[34] J. Dunfield and N. Krishnaswami, "Bidirectional Typing," *ACM Comput. Surv.*, vol. 54, no. 5, May 2021, ISSN: 0360-0300. DOI: 10.1145/3450952. [Online]. Available: https://doi.org/10.1145/3450952.

[35] W. A. Howard *et al.*, "The formulae-as-types notion of construction," *To HB Curry: essays on combinatory logic, lambda calculus and formalism*, vol. 44, pp. 479–490, 1980.

[36] M. Dénès, C. Hritcu, L. Lampropoulos, Z. Paraskevopoulou, and B. C. Pierce, "Quickchick: Property-based testing for coq," in *The Coq Workshop*, 2014.

[37] Z. Paraskevopoulou, C. HriŢcu, M. Dénès, L. Lampropoulos, and B. C. Pierce, "Foundational Property-Based Testing," in *Interactive Theorem Proving*, C. Urban and X. Zhang, Eds., Cham: Springer International Publishing, 2015, pp. 325–343. DOI: 10.1007/978-3-319-22102-1_22.

[38] P. Godefroid, A. V. Nori, S. K. Rajamani, and S. D. Tetali, "Compositional may-must program analysis: Unleashing the power of alternation," in *Proceedings of the 37th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ser. POPL '10, Madrid, Spain: Association for Computing Machinery, 2010, pp. 43–56, ISBN: 9781605584799. DOI: 10.1145/1706299.1706307. [Online]. Available: https://doi.org/10.1145/1706299.1706307.

[39] P. W. O'Hearn, "Incorrectness logic," *Proc. ACM Program. Lang.*, vol. 4, no. POPL, Dec. 2019. DOI: 10.1145/3371078. [Online]. Available: https://doi.org/10.1145/3371078.

[40] A. Pnueli, "The temporal logic of programs," in *Proceedings of the 18th Annual Symposium on Foundations of Computer Science*, ser. SFCS '77, USA: IEEE Computer Society, 1977, pp. 46–57. DOI: 10.1109/SFCS.1977.32. [Online]. Available: https://doi.org/10.1109/SFCS.1977.32.

[41] G. De Giacomo and M. Y. Vardi, "Linear temporal logic and linear dynamic logic on finite traces," in *Proceedings of the Twenty-Third International Joint Conference on Artificial Intelligence*, ser. IJCAI '13, Beijing, China: AAAI Press, 2013, pp. 854–860, ISBN: 9781577356332. DOI: 10.5555/2540128.2540252.

[42] L. D'Antoni and M. Veanes, "Minimization of Symbolic Automata," *SIGPLAN Not.*, vol. 49, no. 1, pp. 541–553, Jan. 2014, ISSN: 0362-1340. DOI: 10.1145/2578855.2535849. [Online]. Available: https://doi.org/10.1145/2578855.2535849.

[43] L. D'Antoni and M. Veanes, "The Power of Symbolic Automata and Transducers," in *Computer Aided Verification: 29th International Conference, CAV 2017, Heidelberg, Germany, July 24-28, 2017, Proceedings, Part I 30*, Springer, 2017, pp. 47–67. DOI: 10.1007/978-3-319-63387-9_3.

[44] I. Dillig, T. Dillig, B. Li, and K. McMillan, "Inductive invariant generation via abductive inference," in *Proceedings of the 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages and Applications*, ser. OOPSLA '13, Indianapolis, Indiana, USA: Association for Computing Machinery, 2013, pp. 443–456, ISBN: 9781450323741. DOI: 10.1145/2509136.2509511. [Online]. Available: https://doi.org/10.1145/2509136.2509511.

[45] S. Itzhaky, A. Banerjee, N. Immerman, O. Lahav, A. Nanevski, and M. Sagiv, "Modular reasoning about heap paths via effectively propositional formulas," *SIGPLAN Not.*, vol. 49, no. 1, pp. 385–396, Jan. 2014, ISSN: 0362-1340. DOI: 10.1145/2578855.2535854. [Online]. Available: https://doi.org/10.1145/2578855.2535854.

[46] A. Albarghouthi, I. Dillig, and A. Gurfinkel, "Maximal specification synthesis," in *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ser. POPL '16, St. Petersburg, FL, USA: Association for Computing Machinery, 2016, pp. 789–801, ISBN: 9781450335492. DOI: 10.1145/2837614.2837628. [Online]. Available: https://doi.org/10.1145/2837614.2837628.

[47] H. Zhu, G. Petri, and S. Jagannathan, "Automatically learning shape specifications," in *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '16, Santa Barbara, CA, USA: Association for Computing Machinery, 2016, pp. 491–507, ISBN: 9781450342612. DOI: 10.1145/2908080.2908125. [Online]. Available: https://doi.org/10.1145/2908080.2908125.

[48] S. Padhi, R. Sharma, and T. Millstein, "Data-driven precondition inference with learned features," in *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '16, Santa Barbara, CA, USA: Association for Computing Machinery, 2016, pp. 42–56, ISBN: 9781450342612. DOI: 10.1145/2908080.2908099. [Online]. Available: https://doi.org/10.1145/2908080.2908099.

[49] A. Miltner, S. Padhi, T. Millstein, and D. Walker, "Data-driven inference of representation invariants," in *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI 2020, London, UK: Association for Computing Machinery, 2020, pp. 1–15, ISBN: 9781450376136. DOI: 10.1145/3385412.3385967. [Online]. Available: https://doi.org/10.1145/3385412.3385967.

[50] C. Okasaki, *Purely Functional Data Structures*. USA: Cambridge University Press, 1999, ISBN: 0521663504.

[51] Z. Zhou, R. Dickerson, B. Delaware, and S. Jagannathan, *OOPSLA2021 Artifact: Data-Driven Abductive Inference of Library Specifications*, Jul. 2021. DOI: 10.5281/zenodo.5130646. [Online]. Available: https://doi.org/10.5281/zenodo.5130646.

[52] K. Claessen and J. Hughes, "Quickcheck: A lightweight tool for random testing of haskell programs," in *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*, ser. ICFP '00, New York, NY, USA: Association for Computing Machinery, 2000, pp. 268–279, ISBN: 1581132026. DOI: 10.1145/351240.351266. [Online]. Available: https://doi.org/10.1145/351240.351266.

[53] S. Ruggieri, "Efficient c4.5 [classification algorithm]," *IEEE Transactions on Knowledge and Data Engineering*, vol. 14, no. 2, pp. 438–444, 2002. DOI: 10.1109/69.991727.

[54] X. Leroy, D. Doligez, A. Frisch, J. Garrigue, D. Rémy, and J. Vouillon, "The ocaml system release 4.02," *Institut National de Recherche en Informatique et en Automatique*, vol. 54, 2014.

[55] A. Appel, *Software foundations volume 3: Verified functional algorithms*, 2018.

[56] B. C. Pierce *et al.*, "Software foundations," *Webpage: http://www. cis. upenn. edu/bcpierce/s-f/current/index. html*, 2010.

[57] A. Charguéraud, J.-C. Filliâtre, M. Pereira, and F. Pottier, *VOCAL – A Verified OCaml Library*, ML Family Workshop, Sep. 2017. [Online]. Available: https://hal.inria.fr/hal-01561094.

[58] M. Z. Malik, A. Pervaiz, and S. Khurshid, "Generating Representation Invariants of Structurally Complex Data," in *Tools and Algorithms for the Construction and Analysis of Systems, 13th International Conference, TACAS 2007, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2007 Braga, Portugal, March 24 - April 1, 2007, Proceedings*, O. Grumberg and M. Huth, Eds., ser. Lecture Notes in Computer Science, vol. 4424, Springer, 2007, pp. 34–49. DOI: 10.1007/978-3-540-71209-1_5. [Online]. Available: https://doi.org/10.1007/978-3-540-71209-1%5C_5.

[59] A. Miltner, S. Padhi, T. D. Millstein, and D. Walker, "Data-driven Inference of Representation Invariants," in *Proceedings of the 41st ACM SIGPLAN International Conference on Programming Language Design and Implementation, PLDI 2020, London, UK, June 15-20, 2020*, A. F. Donaldson and E. Torlak, Eds., ACM, 2020, pp. 1–15. DOI: 10.1145/3385412.3385967. [Online]. Available: https://doi.org/10.1145/3385412.3385967.

[60] M. Veanes, N. Bjørner, and L. de Moura, "Symbolic Automata Constraint Solving," in *Logic for Programming, Artificial Intelligence, and Reasoning*, A. Fermüller Christian G.and Voronkov, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 640–654. DOI: https://doi.org/10.1007/978-3-642-16242-8_45.

[61] M. Veanes, "Applications of Symbolic Finite Automata," in *Implementation and Application of Automata*, S. Konstantinidis, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 16–23, ISBN: 978-3-642-39274-0. DOI: 10.1007/978-3-642-39274-0_3.

[62] N. Swamy, G. Martinez, and A. Rastogi, *Proof-Oriented Programming in F**. 2023. [Online]. Available: https://fstar-lang.org/tutorial/proof-oriented-programming-in-fstar.pdf.

[63] A. Nanevski, G. Morrisett, A. Shinnar, P. Govereau, and L. Birkedal, "Ynot: Dependent Types for Imperative Programs," in *Proceedings of the 13th ACM SIGPLAN International Conference on Functional Programming*, ser. ICFP '08, New York, NY, USA: Association for Computing Machinery, 2008, pp. 229–240, ISBN: 9781595939197. DOI: 10.1145/1411204.1411237. [Online]. Available: https://doi.org/10.1145/1411204.1411237.

[64] K. Maillard *et al.*, "Dijkstra Monads for All," *Proc. ACM Program. Lang.*, vol. 3, no. ICFP, Jul. 2019. DOI: 10.1145/3341708. [Online]. Available: https://doi.org/10.1145/3341708.

[65] N. Swamy, J. Weinberger, C. Schlesinger, J. Chen, and B. Livshits, "Verifying Higher-Order Programs with the Dijkstra Monad," in *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '13, Seattle, WA, USA, June 16-19, 2013*, H. Boehm and C. Flanagan, Eds., ACM, 2013, pp. 387–398. DOI: 10.1145/2491956.2491978. [Online]. Available: https://doi.org/10.1145/2491956.2491978.

[66] A. Nanevski, J. G. Morrisett, and L. Birkedal, "Hoare Type Theory, Polymorphism and Separation," *J. Funct. Program.*, vol. 18, no. 5-6, pp. 865–911, 2008. DOI: 10.1017/S0956796808006953. [Online]. Available: https://doi.org/10.1017/S0956796808006953.

[67] Y. Nanjo, H. Unno, E. Koskinen, and T. Terauchi, "A Fixpoint Logic and Dependent Effects for Temporal Property Verification," in *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science*, ser. LICS '18, Oxford, United Kingdom: Association for Computing Machinery, 2018, pp. 759–768, ISBN: 9781450355834. DOI: 10.1145/3209108.3209204. [Online]. Available: https://doi.org/10.1145/3209108.3209204.

[68] T. Sekiyama and H. Unno, "Temporal Verification with Answer-Effect Modification: Dependent Temporal Type-and-Effect System with Delimited Continuations," *Proc. ACM Program. Lang.*, vol. 7, no. POPL, Jan. 2023. DOI: 10.1145/3571264. [Online]. Available: https://doi.org/10.1145/3571264.

[69] J. Hatcliff and O. Danvy, "A Generic Account of Continuation-Passing Styles," in *Proceedings of the 21st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ser. POPL '94, Portland, Oregon, USA: Association for Computing Machinery, 1994, pp. 458–471, ISBN: 0897916360. DOI: 10.1145/174675.178053. [Online]. Available: https://doi.org/10.1145/174675.178053.

[70] C. Flanagan, A. Sabry, B. F. Duba, and M. Felleisen, "The Essence of Compiling with Continuations," in *Proceedings of the ACM SIGPLAN 1993 Conference on Programming Language Design and Implementation*, ser. PLDI '93, Albuquerque, New Mexico, USA: Association for Computing Machinery, 1993, pp. 237–247, ISBN: 0897915984. DOI: 10.1145/155090.155113. [Online]. Available: https://doi.org/10.1145/155090.155113.

[71] F. P. Ramsey, "On a problem of formal logic," in *Classic Papers in Combinatorics*, I. Gessel and G.-C. Rota, Eds. Boston, MA: Birkhäuser Boston, 1987, pp. 1–24, ISBN: 978-0-8176-4842-8. DOI: 10.1007/978-0-8176-4842-8_1. [Online]. Available: https://doi.org/10.1007/978-0-8176-4842-8_1.

[72] Z. Zhou, R. Dickerson, B. Delaware, and S. Jagannathan, "Data-driven Abductive Inference of Library Specifications," *Proc. ACM Program. Lang.*, vol. 5, no. OOPSLA, pp. 1–29, 2021. DOI: 10.1145/3485493. [Online]. Available: https://doi.org/10.1145/3485493.

[73] Z. Zhou, Q. Ye, B. Delaware, and S. Jagannathan, *PLDI2024 Artifact: A HAT Trick: Automatically Verifying Representation Invariants Using Symbolic Finite Automata*, version 1.1, Mar. 2024. DOI: 10.5281/zenodo.10806686. [Online]. Available: https://doi.org/10.5281/zenodo.10806686.

[74] K. Claessen, *QuickCheck*, 2020. [Online]. Available: https://hackage.haskell.org/package/QuickCheck.

[75] S. Cruanes, *Qcheck*, 2013. [Online]. Available: https://github.com/c-cube/qcheck.

[76] *Fast-check: Property based testing for JavaScript and TypeScript*, 2022. [Online]. Available: https://dubzzz.github.io/fast-check.github.com/.

[77] *Crate for pbt in rust*, 2021. [Online]. Available: https://github.com/BurntSushi/quickcheck.

[78] *Hypothesis*, 2022. [Online]. Available: https://github.com/HypothesisWorks/hypothesis/tree/master/hypothesis-python.

[79] *Scalacheck*, 2021. [Online]. Available: https://scalacheck.org/.

[80] L. Lampropoulos, M. Hicks, and B. C. Pierce, "Coverage Guided, Property Based Testing," *Proc. ACM Program. Lang.*, vol. 3, no. OOPSLA, Oct. 2019. DOI: 10.1145/3360607. [Online]. Available: https://doi.org/10.1145/3360607.

[81] S. Dolan, *Crowbar*, 2022. [Online]. Available: https://github.com/stedolan/crowbar.

[82] L. Lampropoulos, Z. Paraskevopoulou, and B. C. Pierce, "Generating Good Generators for Inductive Relations," *Proc. ACM Program. Lang.*, vol. 2, no. POPL, 45:1–45:30, 2018. DOI: 10.1145/3158133. [Online]. Available: https://doi.org/10.1145/3158133.

[83] K. Germane and J. McCarthy, "Newly-single and Loving It: Improving Higher-Order Must-Alias Analysis with Heap Fragments," *Proc. ACM Program. Lang.*, vol. 5, no. ICFP, pp. 1–28, 2021. DOI: 10.1145/3473601. [Online]. Available: https://doi.org/10.1145/3473601.

[84] S. Jagannathan, P. Thiemann, S. Weeks, and A. K. Wright, "Single and Loving It: Must-Alias Analysis for Higher-Order Languages," in *POPL '98, Proceedings of the 25th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, San Diego, CA, USA, January 19-21, 1998*, D. B. MacQueen and L. Cardelli, Eds., ACM, 1998, pp. 329–341. DOI: 10.1145/268946.268973. [Online]. Available: https://doi.org/10.1145/268946.268973.

[85] A. Raad, J. Berdine, H.-H. Dang, D. Dreyer, P. O'Hearn, and J. Villard, "Local reasoning about the presence of bugs: Incorrectness separation logic," in *Computer Aided Verification: 32nd International Conference, CAV 2020, Los Angeles, CA, USA, July 21–24, 2020, Proceedings, Part II*, Los Angeles, CA, USA: Springer-Verlag, 2020, pp. 225–252, ISBN: 978-3-030-53290-1. DOI: 10.1007/978-3-030-53291-8_14. [Online]. Available: https://doi.org/10.1007/978-3-030-53291-8_14.

[86] Q. L. Le, A. Raad, J. Villard, J. Berdine, D. Dreyer, and P. W. O'Hearn, "Finding Real Bugs in Big Programs with Incorrectness Logic," *Proc. ACM Program. Lang.*, vol. 6, no. OOPSLA, pp. 1–27, 2022. DOI: 10.1145/3527325. [Online]. Available: https://doi.org/10.1145/3527325.

[87] C. A. R. Hoare, "An axiomatic basis for computer programming," *Commun. ACM*, vol. 12, no. 10, pp. 576–580, Oct. 1969, ISSN: 0001-0782. DOI: 10.1145/363235.363259. [Online]. Available: https://doi.org/10.1145/363235.363259.

[88] Z. Zhou, A. Mishra, B. Delaware, and S. Jagannathan, *PLDI2023 Artifact: Covering All the Bases: Type-Based Verification of Test Input Generators*, version v1.4, Mar. 2023. DOI: 10.5281/zenodo.7811004. [Online]. Available: https://doi.org/10.5281/zenodo.7811004.

[89] G. Kaki and S. Jagannathan, "A relational framework for higher-order shape analysis," in *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming*, ser. ICFP '14, Gothenburg, Sweden: Association for Computing Machinery, 2014, pp. 311–324, ISBN: 9781450328739. DOI: 10.1145/2628136.2628159. [Online]. Available: https://doi.org/10.1145/2628136.2628159.

[90] N. Vazou, P. M. Rondon, and R. Jhala, "Abstract refinement types," in *Proceedings of the 22nd European Conference on Programming Languages and Systems*, ser. ESOP'13, Rome, Italy: Springer-Verlag, 2013, pp. 209–228, ISBN: 9783642370359. DOI: 10.1007/978-3-642-37036-6_13. [Online]. Available: https://doi.org/10.1007/978-3-642-37036-6_13.

[91] B. C. Pierce, *Types and Programming Languages*, 1st. The MIT Press, 2002, ISBN: 0262162091.

[92] L. Lampropoulos, D. Gallois-Wong, C. Hriţcu, J. Hughes, B. C. Pierce, and L.-y. Xia, "Beginner's Luck: A Language for Property-Based Generators," *SIGPLAN Not.*, vol. 52, no. 1, pp. 114–129, Jan. 2017, ISSN: 0362-1340. DOI: 10.1145/3093333.3009868. [Online]. Available: https://doi.org/10.1145/3093333.3009868.

[93] M. H. Pałka, K. Claessen, A. Russo, and J. Hughes, "Testing an optimising compiler by generating random lambda terms," in *Proceedings of the 6th International Workshop on Automation of Software Test*, ser. AST '11, Waikiki, Honolulu, HI, USA: Association for Computing Machinery, 2011, pp. 91–97, ISBN: 9781450305921. DOI: 10.1145/1982595.1982615. [Online]. Available: https://doi.org/10.1145/1982595.1982615.

[94] A. Mishra and S. Jagannathan, "Specification-guided component-based synthesis from effectful libraries," *Proc. ACM Program. Lang.*, vol. 6, no. OOPSLA2, Oct. 2022. DOI: 10.1145/3563310. [Online]. Available: https://doi.org/10.1145/3563310.

[95] Tezos, *Tezos*, 2019. [Online]. Available: https://gitlab.com/tezos/tezos.

[96] Xen-Project, *Xen api*, 2010. [Online]. Available: https://github.com/xapi-project/xen-api.

[97] S. Zdancewic, Y. Zakowsky, C. Beck, I. Yoon, and H. Chen, *Vellvm*, 2012. [Online]. Available: https://github.com/vellvm/vellvm.

[98] L. Maranget, *Herdtools7*, 2015. [Online]. Available: https://github.com/herd/herdtools7.

[99] S. Cruanes, *Zipperposition*, 2012. [Online]. Available: https://github.com/sneeuwballen/zipperposition.

[100] Xen-Project, *Xen project*, 2010. [Online]. Available: https://xenproject.org/.

[101] A. D. Reid, "Defining interfaces between hardware and software: Quality and performance," Ph.D. dissertation, University of Glasgow, 2019.

[102] N. Vazou *et al.*, "Refinement reflection: Complete verification with smt," *Proc. ACM Program. Lang.*, vol. 2, no. POPL, Dec. 2017. DOI: 10.1145/3158141. [Online]. Available: https://doi.org/10.1145/3158141.

[103] D. Ahman *et al.*, "Dijkstra Monads for Free," in *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, ser. POPL '17, Paris, France: Association for Computing Machinery, 2017, pp. 515–529, ISBN: 9781450346603. DOI: 10.1145/3009837.3009878. [Online]. Available: https://doi.org/10.1145/3009837.3009878.

[104] A. Fromherz *et al.*, "Steel: Proof-oriented programming in a dependently typed concurrent separation logic," *Proc. ACM Program. Lang.*, vol. 5, no. ICFP, Aug. 2021. DOI: 10.1145/3473590. [Online]. Available: https://doi.org/10.1145/3473590.

[105] N. Swamy, A. Rastogi, A. Fromherz, D. Merigoux, D. Ahman, and G. Martínez, "Steelcore: An extensible concurrent separation logic for effectful dependently typed programs," *Proc. ACM Program. Lang.*, vol. 4, no. ICFP, Aug. 2020. DOI: 10.1145/3409003. [Online]. Available: https://doi.org/10.1145/3409003.

[106] P.-M. Osera and S. Zdancewic, "Type-and-example-directed program synthesis," in *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '15, Portland, OR, USA: Association for Computing Machinery, 2015, pp. 619–630, ISBN: 9781450334686. DOI: 10.1145/2737924.2738007. [Online]. Available: https://doi.org/10.1145/2737924.2738007.

[107] O. Bastani, S. Anand, and A. Aiken, "Specification inference using context-free language reachability," in *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ser. POPL '15, Mumbai, India: Association for Computing Machinery, 2015, pp. 553–566, ISBN: 9781450333009. DOI: 10.1145/2676726.2676977. [Online]. Available: https://doi.org/10.1145/2676726.2676977.

[108] H. A. Nguyen, R. Dyer, T. N. Nguyen, and H. Rajan, "Mining preconditions of apis in large-scale code corpus," in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, ser. FSE 2014, Hong Kong, China: Association for Computing Machinery, 2014, pp. 166–177, ISBN: 9781450330565. DOI: 10.1145/2635868.2635924. [Online]. Available: https://doi.org/10.1145/2635868.2635924.

[109] J. Su, M. Arafat, and R. Dyer, "Using consensus to automatically infer post-conditions," in *Proceedings of the 40th International Conference on Software Engineering: Companion Proceeedings*, ser. ICSE '18, Gothenburg, Sweden: Association for Computing Machinery, 2018, pp. 202–203, ISBN: 9781450356633. DOI: 10.1145/3183440.3195096. [Online]. Available: https://doi.org/10.1145/3183440.3195096.

[110] R. Pandita, X. Xiao, H. Zhong, T. Xie, S. Oney, and A. Paradkar, "Inferring method specifications from natural language api descriptions," in *2012 34th International Conference on Software Engineering (ICSE)*, 2012, pp. 815–825. DOI: 10.1109/ICSE.2012.6227137.

[111] S. Qin, C. Luo, G. He, F. Craciun, and W.-N. Chin, "Verifying heap-manipulating programs with unknown procedure calls," in *Formal Methods and Software Engineering*, J. S. Dong and H. Zhu, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 171–187, ISBN: 978-3-642-16901-4. DOI: 10.1007/978-3-642-16901-4_13.

[112] H. Zhu, S. Magill, and S. Jagannathan, "A data-driven chc solver," in *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI 2018, Philadelphia, PA, USA: Association for Computing Machinery, 2018, pp. 707–721, ISBN: 9781450356985. DOI: 10.1145/3192366.3192416. [Online]. Available: https://doi.org/10.1145/3192366.3192416.

[113] L.-y. Xia *et al.*, "Interaction trees: Representing recursive and impure programs in coq," *Proc. ACM Program. Lang.*, vol. 4, no. POPL, Dec. 2019. DOI: 10.1145/3371119. [Online]. Available: https://doi.org/10.1145/3371119.

[114] H. Zhang *et al.*, "Verifying an HTTP Key-Value Server with Interaction Trees and VST," in *12th International Conference on Interactive Theorem Proving (ITP 2021)*, L. Cohen and C. Kaliszyk, Eds., ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 193, Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, 32:1–32:19, ISBN: 978-3-95977-188-7. DOI: 10.4230/LIPIcs.ITP.2021.32. [Online]. Available: https://drops-dev.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2021.32.

[115] *Java platform standard edition 7 documentation.* 2013. [Online]. Available: https://docs.oracle.com/javase/7/docs/.

[116] R. Tate, "The Sequential Semantics of Producer Effect Systems," in *Proceedings of the 40th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ser. POPL '13, Rome, Italy: Association for Computing Machinery, 2013, pp. 15–26, ISBN: 9781450318327. DOI: 10.1145/2429069.2429074. [Online]. Available: https://doi.org/10.1145/2429069.2429074.

[117] C. Skalka and S. Smith, "History Effects and Verification," in *Programming Languages and Systems*, W.-N. Chin, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 2004, pp. 107–128, ISBN: 978-3-540-30477-7. DOI: 10.1007/978-3-540-30477-7_8.

[118] E. Koskinen and T. Terauchi, "Local temporal reasoning," in *Proceedings of the Joint Meeting of the Twenty-Third EACSL Annual Conference on Computer Science Logic (CSL) and the Twenty-Ninth Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, ser. CSL-LICS '14, Vienna, Austria: Association for Computing Machinery, 2014, ISBN: 9781450328869. DOI: 10.1145/2603088.2603138. [Online]. Available: https://doi.org/10.1145/2603088.2603138.

[119] R. DeLine and M. Fähndrich, "Typestates for Objects," in *ECOOP 2004–Object-Oriented Programming: 18th European Conference, Oslo, Norway, June 14-18, 2004. Proceedings 18*, Springer, 2004, pp. 465–490. DOI: https://doi.org/10.1007/978-3-540-24851-4_21.

[120] E. Bodden and L. Hendren, "The Clara framework for hybrid typestate analysis," *International Journal on Software Tools for Technology Transfer*, vol. 14, pp. 307–326, 2012. DOI: https://doi.org/10.1007/s10009-010-0183-5.

[121] J. Sunshine, K. Naden, S. Stork, J. Aldrich, and É. Tanter, "First-class state change in plaid," *SIGPLAN Not.*, vol. 46, no. 10, pp. 713–732, Oct. 2011, ISSN: 0362-1340. DOI: 10.1145/2076021.2048122. [Online]. Available: https://doi.org/10.1145/2076021.2048122.

[122] K. Honda, "Types for Dyadic Interaction," in *CONCUR'93*, E. Best, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 1993, pp. 509–523, ISBN: 978-3-540-47968-0. DOI: 10.1007/3-540-57208-2_35.

[123] V. T. Vasconcelos, "Fundamentals of Session Types," in *Formal Methods for Web Services: 9th International School on Formal Methods for the Design of Computer, Communication, and Software Systems, SFM 2009, Bertinoro, Italy, June 1-6, 2009, Advanced Lectures*, M. Bernardo, L. Padovani, and G. Zavattaro, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 158–186, ISBN: 978-3-642-01918-0. DOI: 10.1007/978-3-642-01918-0_4. [Online]. Available: https://doi.org/10.1007/978-3-642-01918-0_4.

[124] B. Toninho, L. Caires, and F. Pfenning, "Dependent Session Types via Intuitionistic Linear Type Theory," in *Proceedings of the 13th International ACM SIGPLAN Symposium on Principles and Practices of Declarative Programming*, New York, NY, USA: Association for Computing Machinery, 2011, pp. 161–172, ISBN: 9781450307765. DOI: 10.1145/2003476.2003499. [Online]. Available: https://doi.org/10.1145/2003476.2003499.

[125] P. Baltazar, D. Mostrous, and V. T. Vasconcelos, "Linearly Refined Session Types," in *Proceedings 2nd International Workshop on Linearity, LINEARITY 2012, Tallinn, Estonia, 1 April 2012*, S. Alves and I. Mackie, Eds., ser. EPTCS, vol. 101, 2012, pp. 38–49. DOI: 10.4204/EPTCS.101.4. [Online]. Available: https://doi.org/10.4204/EPTCS.101.4.

[126] P. Thiemann and V. T. Vasconcelos, "Label-Dependent Session Types," *Proc. ACM Program. Lang.*, vol. 4, no. POPL, Dec. 2019. DOI: 10.1145/3371135. [Online]. Available: https://doi.org/10.1145/3371135.

[127] H. Goldstein, J. W. Cutler, D. Dickstein, B. C. Pierce, and A. Head, "Property-based testing in practice," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24, Lisbon, Portugal: Association for Computing Machinery, 2024, ISBN: 9798400702174. DOI: 10.1145/3597503.3639581. [Online]. Available: https://doi.org/10.1145/3597503.3639581.

[128] H. Goldstein, J. Tao, Z. Hatfield-Dodds, B. C. Pierce, and A. Head, "Tyche: Making sense of pbt effectiveness," in *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*, ser. UIST '24, Pittsburgh, PA, USA: Association for Computing Machinery, 2024, ISBN: 9798400706288. DOI: 10.1145/3654777.3676407. [Online]. Available: https://doi.org/10.1145/3654777.3676407.

[129] A. Mista and A. Russo, "Generating random structurally rich algebraic data type values," in *Proceedings of the 14th International Workshop on Automation of Software Test*, ser. AST '19, Montreal, Quebec, Canada: IEEE Press, 2019, pp. 48–54. DOI: 10.1109/AST.2019.00013. [Online]. Available: https://doi.org/10.1109/AST.2019.00013.

[130] S. Elazar Mittelman, A. Resnick, I. Perez, A. E. Goodloe, and L. Lampropoulos, "Don't go down the rabbit hole: Reprioritizing enumeration for property-based testing," in *Proceedings of the 16th ACM SIGPLAN International Haskell Symposium*, ser. Haskell 2023, Seattle, WA, USA: Association for Computing Machinery, 2023, pp. 59–71, ISBN: 9798400702983. DOI: 10.1145/3609026.3609730. [Online]. Available: https://doi.org/10.1145/3609026.3609730.

[131] L. Lampropoulos, "Random testing for language design," Ph.D. dissertation, University of Pennsylvania, 2018. [Online]. Available: https://repository.upenn.edu/edissertations/2879.

[132] M. Zalewski, *American fuzzy lop*, 2020. [Online]. Available: https://github.com/google/afl.

[133] R. Padhye, C. Lemieux, K. Sen, M. Papadakis, and Y. Le Traon, "Semantic Fuzzing with Zest," in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, New York, NY, USA: Association for Computing Machinery, 2019, pp. 329–340, ISBN: 9781450362245. DOI: 10.1145/3293882.3330576. [Online]. Available: https://doi.org/10.1145/3293882.3330576.

[134] B. Fetscher, K. Claessen, M. H. Palka, J. Hughes, and R. B. Findler, "Making random judgments: Automatically generating well-typed terms from the definition of a type-system," in *Programming Languages and Systems - 24th European Symposium on Programming, ESOP 2015, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2015, London, UK, April 11-18, 2015. Proceedings*, J. Vitek, Ed., ser. Lecture Notes in Computer Science, vol. 9032, Springer, 2015, pp. 383–405. DOI: 10.1007/978-3-662-46669-8_16. [Online]. Available: https://doi.org/10.1007/978-3-662-46669-8%5C_16.

[135] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in c compilers," in *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '11, San Jose, California, USA: Association for Computing Machinery, 2011, pp. 283–294, ISBN: 9781450306638. DOI: 10.1145/1993498.1993532. [Online]. Available: https://doi.org/10.1145/1993498.1993532.

[136] K. Claessen, J. Duregård, and M. H. Pałka, "Generating constrained random data with uniform distribution," vol. 25, Jun. 2014, ISBN: 978-3-319-07150-3. DOI: 10.1007/978-3-319-07151-0_2.

[137] P. Dybjer, Q. Haiyan, and M. Takeyama, "Combining Testing and Proving in Dependent Type Theory," in *Theorem Proving in Higher Order Logics, 16th International Conference, TPHOLs 2003, Rom, Italy, September 8-12, 2003, Proceedings*, D. A. Basin and B. Wolff, Eds., ser. Lecture Notes in Computer Science, vol. 2758, Springer, 2003, pp. 188–203. DOI: 10.1007/10930755_12. [Online]. Available: https://doi.org/10.1007/10930755%5C_12.

[138] P. Dybjer, Q. Haiyan, and M. Takeyama, "Random Generators for Dependent Types," in *Theoretical Aspects of Computing - ICTAC 2004, First International Colloquium, Guiyang, China, September 20-24, 2004, Revised Selected Papers*, Z. Liu and K. Araki, Eds., ser. Lecture Notes in Computer Science, vol. 3407, Springer, 2004, pp. 341–355. DOI: 10.1007/978-3-540-31862-0_25. [Online]. Available: https://doi.org/10.1007/978-3-540-31862-0%5C_25.

[139] E. de Vries and V. Koutavas, "Reverse hoare logic," in *Software Engineering and Formal Methods*, G. Barthe, A. Pardo, and G. Schneider, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 155–171, ISBN: 978-3-642-24690-6. DOI: 10.1007/978-3-642-24690-6_12.

[140] V. R. Pratt, "Semantical consideration on Floyd-Hoare logic," in *17th Annual Symposium on Foundations of Computer Science (sfcs 1976)*, IEEE, 1976, pp. 109–121.

[141] E. L. Seidel, N. Vazou, and R. Jhala, "Type Targeted Testing," in *Programming Languages and Systems*, Berlin, Heidelberg: Springer Berlin Heidelberg, 2015, pp. 812–836. DOI: 10.1007/978-3-662-46669-8_33.

[142] A. Albarghouthi, L. D'Antoni, S. Drews, and A. V. Nori, "Fairsquare: Probabilistic verification of program fairness," *Proc. ACM Program. Lang.*, vol. 1, no. OOPSLA, Oct. 2017. DOI: 10.1145/3133904. [Online]. Available: https://doi.org/10.1145/3133904.

[143] O. Bastani, X. Zhang, and A. Solar-Lezama, "Probabilistic verification of fairness properties via concentration," *Proc. ACM Program. Lang.*, vol. 3, no. OOPSLA, Oct. 2019. DOI: 10.1145/3360544. [Online]. Available: https://doi.org/10.1145/3360544.

# A. OMITTED RULES AND EVALUATION DETAILS FOR HAT

## A.1 Basic Typing Rules

**Basic Typing** $\boxed{\Gamma \vdash_{\mathsf{s}} e : s}$

$$\frac{}{\Gamma \vdash_{\mathsf{s}} c : \mathbf{Ty}(c)}\ \textsc{BtConst} \qquad \frac{\Gamma(x) = s}{\Gamma \vdash_{\mathsf{s}} x : s}\ \textsc{BtVar} \qquad \frac{\Gamma, x{:}\lfloor t_1 \rfloor \vdash_{\mathsf{s}} e : s_2}{\Gamma \vdash_{\mathsf{s}} \lambda x{:}t_1.e : \lfloor t_1 \rfloor \to s_2}\ \textsc{BtFun}$$

$$\frac{\Gamma, f{:}\lfloor t_1 \to t_2 \rfloor \vdash_{\mathsf{s}} \lambda x{:}t_1.e : \lfloor t_1 \to t_2 \rfloor}{\Gamma \vdash_{\mathsf{s}} \texttt{fix} f{:}t_1 \to t_2.\lambda x{:}t_1.e : \lfloor t_1 \to t_2 \rfloor}\ \textsc{BtFix}$$

$$\frac{\mathbf{Ty}(op) = \overline{s_i} \to s_x \quad \forall i.\Gamma \vdash_{\mathsf{s}} v_i : s_i \quad \Gamma, x{:}s_x \vdash_{\mathsf{s}} e : s}{\Gamma \vdash_{\mathsf{s}} \texttt{let}\ x = op\,\overline{v_i}\ \texttt{in}\ e : s}\ \textsc{BtPureOp}$$

$$\frac{\Gamma \vdash_{\mathsf{s}} v_1 : s_2 \to s_x \quad \Gamma \vdash_{\mathsf{s}} v_2 : s_2 \quad \Gamma, x{:}s_x \vdash_{\mathsf{s}} e : s}{\Gamma \vdash_{\mathsf{s}} \texttt{let}\ x = v_1\, v_2\ \texttt{in}\ e : s}\ \textsc{BtApp}$$

$$\frac{\mathbf{Ty}(\mathtt{op}) = \overline{s_i} \to s_x \quad \forall i.\Gamma \vdash_{\mathsf{s}} v_i : s_i \quad \Gamma, x{:}s_x \vdash_{\mathsf{s}} e : s}{\Gamma \vdash_{\mathsf{s}} \texttt{let}\ x = \mathtt{op}\,\overline{v_i}\ \texttt{in}\ e : s}\ \textsc{BtEffOp}$$

$$\frac{\emptyset \vdash_{\mathsf{s}} e_1 : s_x \quad \Gamma, x{:}s_x \vdash_{\mathsf{s}} e_2 : s}{\Gamma \vdash_{\mathsf{s}} \texttt{let}\ x = e_1\ \texttt{in}\ e_2 : s}\ \textsc{BtLetE} \qquad \frac{\Gamma \vdash_{\mathsf{s}} v : s_v \quad \forall i, \mathbf{Ty}(d_i) = \overline{s_j} \to s_v \quad \Gamma, \overline{y_j{:}s_j} \vdash_{\mathsf{s}} e_i : s}{\Gamma \vdash_{\mathsf{s}} \texttt{match}\ v\ \texttt{with}\ \overline{d_i\,\overline{y_j} \to e_i} : s}\ \textsc{BtMatch}$$

**Figure A.1.** Basic Typing Rules

**Basic Qualifier Typing** $\boxed{\Gamma \vdash_{\mathsf{s}} l : s \quad \Gamma \vdash_{\mathsf{s}} \phi : s}$

$$\frac{\mathbf{Ty}(c) = s}{\Gamma \vdash_{\mathsf{s}} c : s}\ \textsc{BtLitConst} \qquad \frac{\Gamma(x) = s}{\Gamma \vdash_{\mathsf{s}} x : s}\ \textsc{BtLitVar} \qquad \frac{}{\Gamma \vdash_{\mathsf{s}} \top : \texttt{bool}}\ \textsc{BtTop} \qquad \frac{}{\Gamma \vdash_{\mathsf{s}} \bot : \texttt{bool}}\ \textsc{BtBot}$$

$$\frac{\mathbf{Ty}(op) = \overline{s_i} \to s \quad \forall i.\Gamma \vdash_{\mathsf{s}} l_i : s_i}{\Gamma \vdash_{\mathsf{s}} op\,\overline{l_i} : s}\ \textsc{BtLitOp} \qquad \frac{\mathbf{Ty}(mp) = \overline{s_i} \to \texttt{bool} \quad \forall i.\Gamma \vdash_{\mathsf{s}} l_i : s_i}{\Gamma \vdash_{\mathsf{s}} mp\,\overline{l_i} : s}\ \textsc{BtLitMp}$$

$$\frac{\Gamma \vdash_{\mathsf{s}} \phi : \texttt{bool}}{\Gamma \vdash_{\mathsf{s}} \neg\phi : \texttt{bool}}\ \textsc{BtNeg} \qquad \frac{\Gamma \vdash_{\mathsf{s}} \phi_1 : \texttt{bool} \quad \Gamma \vdash_{\mathsf{s}} \phi_2 : \texttt{bool}}{\Gamma \vdash_{\mathsf{s}} \phi_1 \wedge \phi_2 : \texttt{bool}}\ \textsc{BtAnd}$$

$$\frac{\Gamma \vdash_{\mathsf{s}} \phi_1 : \texttt{bool} \quad \Gamma \vdash_{\mathsf{s}} \phi_2 : \texttt{bool}}{\Gamma \vdash_{\mathsf{s}} \phi_1 \vee \phi_2 : \texttt{bool}}\ \textsc{BtOr} \qquad \frac{\Gamma, x{:}b \vdash_{\mathsf{s}} \phi : \texttt{bool}}{\Gamma \vdash_{\mathsf{s}} \forall x{:}b.\phi : \texttt{bool}}\ \textsc{BtForall}$$

**Figure A.2.** Basic Qualifier Typing Rules

The basic typing rules of our core language and qualifiers are shown in Fig. A.1 and Fig. A.2. We use an auxiliary function **Ty** to provide a basic type for the primitives of our language, e.g., constants, built-in operators, and data constructors.

## A.2 Symbolic Regular Language

$$\textbf{Symbolic Regular Language} \quad A, B ::= \langle \mathtt{op}\, \overline{x} = \nu \mid \phi \rangle \mid \langle \phi \rangle \mid \bullet \mid \emptyset \mid \epsilon \mid A \wedge A \mid A \vee A \mid A; A \mid A \setminus A \mid A^*$$

**Figure A.3.** Symbolic Regular Language.

**SRL Language** $\boxed{\mathcal{L}(A) \in \mathcal{P}(\alpha)}$

$$\mathcal{L}(\langle \mathtt{op}\, \overline{x_i} = \nu \mid \phi \rangle) \doteq \{[\mathtt{op}\, \overline{v_i} = v] \in Tr^{\mathbf{WF}} \mid \phi[\overline{x_i \mapsto v_i}][\nu \mapsto v]\}$$
$$\mathcal{L}(\langle \phi \rangle) \doteq \{[\mathtt{op}\, \overline{v_i} = v] \in Tr^{\mathbf{WF}} \mid \phi\}$$
$$\mathcal{L}(\emptyset) \doteq \emptyset$$
$$\mathcal{L}(\epsilon) \doteq \{[\,]\}$$
$$\mathcal{L}(A; A') \doteq \{\alpha +\!\!+ \alpha' \mid \alpha \in \mathcal{L}(A), \alpha' \in \mathcal{L}(A')\}$$
$$\mathcal{L}(\bullet) \doteq \{\alpha \in Tr^{\mathbf{WF}} \mid len(\alpha) = 1\}$$
$$\mathcal{L}(A \vee A') \doteq \mathcal{L}(A) \cup \mathcal{L}(A')$$
$$\mathcal{L}(A \wedge A') \doteq \mathcal{L}(A) \cap \mathcal{L}(A')$$
$$\mathcal{L}(A^*) \doteq \bigcup_{i=0}^{\infty} \mathcal{L}(\underbrace{\epsilon A A \ldots A}_{\text{repeat } A \text{ for } i \text{ times}})$$
$$\mathcal{L}(A \setminus A') \doteq \mathcal{L}(A) \setminus \mathcal{L}(A')$$

**Figure A.4.** Symbolic Regular Language.

We post the syntax of Symbolic Regular Languages (SRL) and corresponding denotation in Fig. A.3 and Fig. A.4, which is strictly more expressive than $\text{LTL}_f$. Our desugaring procedure converts all SFAs encoded in $\text{LTL}_f$ formulae into SRL terms by adapting the standard $\text{LTL}_f$ translation algorithm [41].

## A.3 Evaluation Details

Table A.1 and Table A.2 list the information for individual ADT methods from Table 4.1. All our benchmarks suite, and corresponding Coq proofs are available on Zenodo https://doi.org/10.5281/zenodo.10806686

**Table A.1.** Full table of Table 4.1, first part.

| Datatype | Library | #Ghost | $s_I$ | Method | #Branch | #App | #SAT | #Inc | avg. $s_A$ | $t_{SAT}$ (s) | $t_{Inc}$ (s) |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Stack | LinkedList | 0 | 4 | is_empty | 3 | 2 | 60 | 3 | 59 | 0.29 | 0.03 |
| | LinkedList | 0 | 4 | empty | 2 | 3 | 68 | 2 | 47 | 0.37 | 0.01 |
| | LinkedList | 0 | 4 | concat_aux | 4 | 7 | 297 | 7 | 98 | 1.63 | 0.06 |
| | LinkedList | 0 | 4 | concat | 2 | 3 | 34 | 2 | 57 | 0.16 | 0.01 |
| | LinkedList | 0 | 4 | cons | 3 | 7 | 203 | 5 | 112 | 1.18 | 0.05 |
| | LinkedList | 0 | 4 | head | 2 | 2 | 65 | 3 | 55 | 0.36 | 0.03 |
| | LinkedList | 0 | 4 | tail | 2 | 2 | 159 | 4 | 69 | 0.79 | 0.04 |
| | KVStore | 1 | 9 | is_empty | 2 | 1 | 93 | 4 | 79 | 0.55 | 0.04 |
| | KVStore | 1 | 9 | empty | 3 | 5 | 213 | 6 | 176 | 1.29 | 0.06 |
| | KVStore | 1 | 9 | concat_aux | 3 | 6 | 709 | 16 | 167 | 3.84 | 0.19 |
| | KVStore | 1 | 9 | concat | 2 | 3 | 100 | 4 | 89 | 0.58 | 0.03 |
| | KVStore | 1 | 9 | cons | 4 | 10 | 874 | 17 | 226 | 5.78 | 0.23 |
| | KVStore | 1 | 9 | head | 2 | 2 | 183 | 5 | 98 | 0.97 | 0.06 |
| | KVStore | 1 | 9 | tail | 2 | 2 | 396 | 8 | 125 | 1.79 | 0.12 |
| Set | Tree | 0 | 12 | mem | 3 | 5 | 115 | 4 | 212 | 0.69 | 0.05 |
| | Tree | 0 | 12 | insert | 2 | 5 | 85 | 3 | 215 | 0.51 | 0.04 |
| | Tree | 0 | 12 | empty | 1 | 0 | 17 | 1 | 111 | 0.11 | 0.01 |
| | Tree | 0 | 12 | insert_aux | 5 | 12 | 1589 | 11 | 531 | 9.34 | 0.53 |
| | Tree | 0 | 12 | mem_aux | 5 | 10 | 1469 | 11 | 490 | 8.48 | 0.41 |
| | KVStore | 1 | 9 | insert | 3 | 5 | 245 | 6 | 160 | 1.51 | 0.06 |
| | KVStore | 1 | 9 | empty | 1 | 0 | 7 | 1 | 42 | 0.04 | 0.01 |
| | KVStore | 1 | 9 | mem | 1 | 1 | 97 | 4 | 73 | 0.60 | 0.03 |
| Queue | LinkedList | 0 | 4 | snoc | 2 | 6 | 171 | 4 | 110 | 0.84 | 0.03 |
| | LinkedList | 0 | 4 | head | 2 | 2 | 65 | 3 | 55 | 0.36 | 0.03 |
| | LinkedList | 0 | 4 | append | 4 | 9 | 190 | 4 | 96 | 1.08 | 0.06 |
| | LinkedList | 0 | 4 | empty | 2 | 3 | 68 | 2 | 47 | 0.37 | 0.01 |
| | LinkedList | 0 | 4 | is_empty | 3 | 2 | 60 | 3 | 59 | 0.29 | 0.03 |
| | LinkedList | 0 | 4 | tail | 2 | 2 | 167 | 4 | 93 | 0.84 | 0.04 |
| | Graph | 1 | 24 | snoc | 3 | 9 | 984 | 12 | 491 | 6.60 | 0.20 |
| | Graph | 1 | 24 | head | 2 | 2 | 291 | 5 | 251 | 1.57 | 0.24 |
| | Graph | 1 | 24 | append | 5 | 12 | 1212 | 14 | 525 | 8.19 | 0.86 |
| | Graph | 1 | 24 | empty | 2 | 3 | 202 | 4 | 318 | 1.27 | 0.08 |
| | Graph | 1 | 24 | is_empty | 2 | 1 | 165 | 4 | 234 | 1.03 | 0.10 |
| | Graph | 1 | 24 | tail | 3 | 4 | 980 | 12 | 409 | 6.58 | 0.59 |
| MinSet | Set | 1 | 28 | minset_mem | 1 | 1 | 199 | 6 | 241 | 1.43 | 0.24 |
| | Set | 1 | 28 | minimum | 2 | 2 | 181 | 5 | 251 | 0.89 | 0.37 |
| | Set | 1 | 28 | minset_insert | 3 | 6 | 612 | 17 | 294 | 4.19 | 1.58 |
| | Set | 1 | 28 | minset_singleton | 2 | 3 | 120 | 4 | 259 | 0.80 | 0.13 |
| | KVStore | 1 | 25 | minset_mem | 1 | 1 | 207 | 6 | 223 | 1.49 | 0.31 |
| | KVStore | 1 | 25 | minimum | 2 | 2 | 191 | 5 | 234 | 0.95 | 0.36 |
| | KVStore | 1 | 25 | minset_insert | 5 | 8 | 2227 | 23 | 519 | 10.93 | 9.34 |
| | KVStore | 1 | 25 | minset_singleton | 3 | 6 | 235 | 5 | 408 | 1.59 | 0.32 |
| LazySet | Tree | 0 | 12 | mem_aux | 5 | 10 | 1469 | 11 | 490 | 8.37 | 0.41 |
| | Tree | 0 | 12 | lazy_insert | 2 | 6 | 85 | 3 | 301 | 0.50 | 0.04 |
| | Tree | 0 | 12 | force | 1 | 1 | 17 | 1 | 171 | 0.11 | 0.01 |
| | Tree | 0 | 12 | insert_aux | 5 | 12 | 1589 | 11 | 531 | 9.33 | 0.57 |
| | Tree | 0 | 12 | lazy_mem | 3 | 6 | 115 | 4 | 298 | 0.68 | 0.07 |
| | Tree | 0 | 12 | new_thunk | 1 | 0 | 17 | 1 | 111 | 0.11 | 0.01 |
| | Set | 1 | 9 | lazy_mem | 1 | 2 | 98 | 4 | 105 | 0.59 | 0.04 |
| | Set | 1 | 9 | force | 1 | 1 | 7 | 1 | 66 | 0.03 | 0.01 |
| | Set | 1 | 9 | lazy_insert | 2 | 3 | 101 | 4 | 106 | 0.57 | 0.04 |
| | Set | 1 | 9 | new_thunk | 1 | 0 | 7 | 1 | 42 | 0.03 | 0.01 |
| | KVStore | 1 | 9 | lazy_mem | 1 | 2 | 98 | 4 | 105 | 0.59 | 0.04 |
| | KVStore | 1 | 9 | insert_aux | 3 | 5 | 245 | 6 | 160 | 1.49 | 0.06 |
| | KVStore | 1 | 9 | force | 1 | 1 | 7 | 1 | 66 | 0.03 | 0.01 |
| | KVStore | 1 | 9 | lazy_insert | 2 | 3 | 101 | 4 | 121 | 0.58 | 0.04 |
| | KVStore | 1 | 9 | new_thunk | 1 | 0 | 7 | 1 | 42 | 0.03 | 0.01 |

**Table A.2.** Full table of Table 4.1, second part.

| Datatype | Library | #Ghost | $s_I$ | Method | #Branch | #App | #SAT | #Inc | avg. $s_A$ | $t_{SAT}$ (s) | $t_{Inc}$ (s) |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Heap | Tree | 0 | 12 | minimum | 2 | 3 | 82 | 3 | 207 | 0.49 | 0.03 |
| | Tree | 0 | 12 | insert | 2 | 5 | 85 | 3 | 215 | 0.50 | 0.04 |
| | Tree | 0 | 12 | contains_aux | 5 | 10 | 871 | 11 | 311 | 4.87 | 0.17 |
| | Tree | 0 | 12 | contains | 3 | 5 | 115 | 4 | 212 | 0.68 | 0.06 |
| | Tree | 0 | 12 | empty | 1 | 0 | 17 | 1 | 111 | 0.11 | 0.01 |
| | Tree | 0 | 12 | insert_aux | 5 | 12 | 1589 | 11 | 531 | 9.33 | 0.52 |
| | Tree | 0 | 12 | minimum_aux | 2 | 3 | 408 | 5 | 334 | 2.11 | 0.09 |
| | LinkedList | 0 | 4 | insert_aux | 4 | 8 | 497 | 8 | 118 | 3.03 | 0.08 |
| | LinkedList | 0 | 4 | empty | 1 | 0 | 11 | 1 | 19 | 0.07 | 0.01 |
| | LinkedList | 0 | 4 | contains | 2 | 4 | 65 | 3 | 72 | 0.36 | 0.02 |
| | LinkedList | 0 | 4 | contains_aux | 3 | 5 | 359 | 6 | 129 | 1.88 | 0.05 |
| | LinkedList | 0 | 4 | insert | 2 | 6 | 137 | 5 | 93 | 0.88 | 0.05 |
| | LinkedList | 0 | 4 | minimum | 2 | 2 | 63 | 3 | 65 | 0.36 | 0.03 |
| File System | Tree | 1 | 20 | deleteChildren | 3 | 7 | 1055 | 13 | 470 | 6.39 | 1.79 |
| | Tree | 1 | 20 | del_path_in_dir | 2 | 5 | 452 | 5 | 284 | 1.76 | 0.17 |
| | Tree | 1 | 20 | add | 3 | 8 | 2085 | 17 | 652 | 14.15 | 2.21 |
| | Tree | 1 | 20 | add_path_in_dir | 2 | 5 | 837 | 9 | 424 | 4.90 | 7.25 |
| | Tree | 1 | 20 | delete | 4 | 16 | 1064 | 12 | 520 | 6.95 | 0.76 |
| | Tree | 1 | 20 | init | 1 | 5 | 431 | 7 | 323 | 3.22 | 0.49 |
| | KVStore | 1 | 17 | deleteChildren | 3 | 7 | 2244 | 19 | 392 | 14.00 | 1.38 |
| | KVStore | 1 | 17 | add | 4 | 10 | 8144 | 43 | 481 | 56.64 | 16.54 |
| | KVStore | 1 | 17 | delete | 4 | 20 | 5658 | 10 | 234 | 7.91 | 1.49 |
| | KVStore | 1 | 17 | init | 1 | 3 | 449 | 5 | 166 | 3.45 | 0.08 |
| DFA | KVStore | 2 | 18 | is_node | 1 | 0 | 19 | 1 | 79 | 0.11 | 0.01 |
| | KVStore | 2 | 18 | add_transition | 2 | 2 | 1417 | 14 | 176 | 9.68 | 0.45 |
| | KVStore | 2 | 18 | add_node | 1 | 0 | 19 | 1 | 79 | 0.11 | 0.01 |
| | KVStore | 2 | 18 | del_transition | 2 | 2 | 704 | 8 | 152 | 4.72 | 0.32 |
| | KVStore | 2 | 18 | is_transition | 3 | 3 | 3604 | 25 | 228 | 18.84 | 1.60 |
| | Graph | 2 | 11 | is_node | 1 | 1 | 115 | 4 | 95 | 0.74 | 0.04 |
| | Graph | 2 | 11 | add_transition | 4 | 4 | 2420 | 27 | 192 | 16.73 | 2.12 |
| | Graph | 2 | 11 | add_node | 2 | 2 | 118 | 4 | 96 | 0.71 | 0.04 |
| | Graph | 2 | 11 | del_transition | 4 | 4 | 3625 | 27 | 225 | 25.53 | 3.42 |
| | Graph | 2 | 11 | is_transition | 4 | 3 | 3622 | 27 | 208 | 25.86 | 2.72 |
| Connected Graph | Set | 2 | 9 | is_transition | 2 | 1 | 1503 | 30 | 77 | 10.03 | 0.25 |
| | Set | 2 | 9 | add_transition | 4 | 4 | 3889 | 50 | 357 | 27.39 | 12.80 |
| | Set | 2 | 9 | add_node | 3 | 3 | 2332 | 35 | 314 | 15.80 | 1.24 |
| | Set | 2 | 9 | is_node | 3 | 2 | 1441 | 30 | 197 | 9.59 | 0.45 |
| | Set | 2 | 9 | singleton | 2 | 2 | 362 | 10 | 160 | 2.30 | 0.09 |
| | Graph | 1 | 20 | is_transition | 4 | 3 | 1349 | 17 | 360 | 19.19 | 53.90 |
| | Graph | 1 | 20 | add_transition | 4 | 4 | 1351 | 17 | 361 | 19.23 | 52.38 |
| | Graph | 1 | 20 | add_node | 3 | 4 | 666 | 12 | 300 | 14.40 | 14.34 |
| | Graph | 1 | 20 | is_node | 1 | 1 | 139 | 4 | 237 | 0.90 | 0.34 |
| | Graph | 1 | 20 | singleton | 2 | 3 | 202 | 4 | 255 | 1.28 | 0.47 |

# B. OMITTED RULES FOR COVERAGE TYPE

## B.1 Operational Semantics

**Operational Semantics** $\boxed{e \hookrightarrow e}$

$$\frac{op\ \overline{v} \equiv v_y}{\texttt{let}\ y = op\ \overline{v}\ \texttt{in}\ e \hookrightarrow e[y \mapsto v_y]}\ \textsc{StAppOp}$$

$$\frac{e_1 \hookrightarrow e_1'}{\texttt{let}\ y = e_1\ \texttt{in}\ e_2 \hookrightarrow \texttt{let}\ y = e_1'\ \texttt{in}\ e_2}\ \textsc{StLetE1} \qquad \frac{}{\texttt{let}\ y = v\ \texttt{in}\ e \hookrightarrow e[y \mapsto v]}\ \textsc{StLetE2}$$

$$\frac{}{\texttt{let}\ y = \lambda x{:}t.e_1\ v_x\ \texttt{in}\ e_2 \hookrightarrow \texttt{let}\ y = e_1[x \mapsto v_x]\ \texttt{in}\ e_2}\ \textsc{StLetAppLam}$$

$$\frac{}{\begin{array}{c}\texttt{let}\ y = \texttt{fix}f{:}t.\lambda x{:}t_x.e_1\ v_x\ \texttt{in}\ e_2 \hookrightarrow \\ \texttt{let}\ y = (\lambda f{:}t.e_1[x \mapsto v_x])\ (\texttt{fix}f{:}t.\lambda x{:}t_x.e_1)\ \texttt{in}\ e_2\end{array}}\ \textsc{StLetAppFix}$$

$$\frac{}{\texttt{match}\ d_i\ \overline{v_j}\ \texttt{with}\ \overline{d_i\ \overline{y_j} \rightarrow e_i} \hookrightarrow e_i[\overline{y_j \mapsto v_j}]}\ \textsc{StMatch}$$

**Figure B.1.** Small Step Operational Semantics

The operational semantics of our core language is shown in Fig. B.1, which is a standard small step semantics.

## B.2 Basic Typing rules

The basic typing rules of our core language and qualifiers are shown in Fig. B.2 and Fig. B.3. We use an auxiliary function **Ty** to provide a basic type for the primitives of our language, e.g., constants, built-in operators, and data constructors.

**Basic Typing** $\boxed{\Gamma \vdash_{\mathsf{s}} e : t}$

$$\frac{}{\Gamma \vdash_{\mathsf{s}} \texttt{err} : t}\ \textsc{BtErr} \qquad \frac{}{\Gamma \vdash_{\mathsf{s}} c : \mathbf{Ty}(c)}\ \textsc{BtConst} \qquad \frac{}{\Gamma \vdash_{\mathsf{s}} op : \mathbf{Ty}(op)}\ \textsc{BtOp} \qquad \frac{\Gamma(x) = t}{\Gamma \vdash_{\mathsf{s}} x : t}\ \textsc{BtVar}$$

$$\frac{\Gamma, x{:}t_1 \vdash_{\mathsf{s}} e : t_2}{\Gamma \vdash_{\mathsf{s}} \lambda x{:}t_1.e : t_1 \to t_2}\ \textsc{BtFun} \qquad \frac{\Gamma, f{:}t_1 \to t_2 \vdash_{\mathsf{s}} \lambda x{:}t_1.e : t_1 \to t_2}{\Gamma \vdash_{\mathsf{s}} \texttt{fix} f{:}t_1 \to t_2.\lambda x{:}t_1.e : t_1 \to t_2}\ \textsc{BtFix}$$

$$\frac{\emptyset \vdash_{\mathsf{s}} e_1 : t_x \quad \Gamma, x{:}t_x \vdash_{\mathsf{s}} e_2 : t}{\Gamma \vdash_{\mathsf{s}} \texttt{let}\ x = e_1\ \texttt{in}\ e_2 : t}\ \textsc{BtLetE} \qquad \frac{\mathbf{Ty}(op) = \overline{t_i} \to t_x \quad \Gamma \vdash_{\mathsf{s}} v_i : t_i \quad \Gamma, x{:}t_x \vdash_{\mathsf{s}} e : t}{\Gamma \vdash_{\mathsf{s}} \texttt{let}\ x = op\ \overline{v_i}\ \texttt{in}\ e : t}\ \textsc{BtAppOp}$$

$$\frac{\Gamma \vdash_{\mathsf{s}} v_1 : t_2 \to t_x \quad \Gamma \vdash_{\mathsf{s}} v_2 : t_2 \quad \Gamma, x{:}t_x \vdash_{\mathsf{s}} e : t}{\Gamma \vdash_{\mathsf{s}} \texttt{let}\ x = v_1\ v_2\ \texttt{in}\ e : t}\ \textsc{BtApp}$$

$$\frac{\Gamma \vdash_{\mathsf{s}} v : t_v \quad \forall i, \mathbf{Ty}(d_i) = \overline{t_j} \to t_v \quad \Gamma, \overline{y_j{:}t_j} \vdash_{\mathsf{s}} e_i : t}{\Gamma \vdash_{\mathsf{s}} \texttt{match}\ v\ \texttt{with}\ \overline{d_i\ \overline{y_j} \to e_i} : t}\ \textsc{BtMatch}$$

**Figure B.2.** Basic Typing Rules

**Basic Qualifier Typing** $\boxed{\Gamma \vdash_{\mathsf{s}} l : t \quad \Gamma \vdash_{\mathsf{s}} \phi : t}$

$$\frac{\mathbf{Ty}(c) = t}{\Gamma \vdash_{\mathsf{s}} c : t}\ \textsc{BtLitConst} \qquad \frac{\Gamma(x) = t}{\Gamma \vdash_{\mathsf{s}} x : t}\ \textsc{BtLitVar} \qquad \frac{}{\Gamma \vdash_{\mathsf{s}} \top : \texttt{bool}}\ \textsc{BtTop}$$

$$\frac{}{\Gamma \vdash_{\mathsf{s}} \bot : \texttt{bool}}\ \textsc{BtBot}$$

$$\frac{\mathbf{Ty}(op) = \overline{t_i} \to t \quad \forall i.\Gamma \vdash_{\mathsf{s}} l_i : t_i}{\Gamma \vdash_{\mathsf{s}} op\,\overline{l_i} : t}\ \textsc{BtLitOp} \qquad \frac{\mathbf{Ty}(mp) = \overline{t_i} \to \texttt{bool} \quad \forall i.\Gamma \vdash_{\mathsf{s}} l_i : t_i}{\Gamma \vdash_{\mathsf{s}} mp\,\overline{l_i} : t}\ \textsc{BtLitMp}$$

$$\frac{\Gamma \vdash_{\mathsf{s}} \phi_1 : \texttt{bool} \quad \Gamma \vdash_{\mathsf{s}} \phi_2 : \texttt{bool}}{\Gamma \vdash_{\mathsf{s}} \phi_1 \wedge \phi_2 : \texttt{bool}}\ \textsc{BtAnd} \qquad \frac{\Gamma \vdash_{\mathsf{s}} \phi_1 : \texttt{bool} \quad \Gamma \vdash_{\mathsf{s}} \phi_2 : \texttt{bool}}{\Gamma \vdash_{\mathsf{s}} \phi_1 \vee \phi_2 : \texttt{bool}}\ \textsc{BtOr}$$

$$\frac{\Gamma, x{:}b \vdash_{\mathsf{s}} \phi : \texttt{bool}}{\Gamma \vdash_{\mathsf{s}} \forall x{:}b.\phi : \texttt{bool}}\ \textsc{BtForall} \qquad \frac{\Gamma, x{:}b \vdash_{\mathsf{s}} \phi : \texttt{bool}}{\Gamma \vdash_{\mathsf{s}} \exists x{:}b.\phi : \texttt{bool}}\ \textsc{BtExists} \qquad \frac{\Gamma \vdash_{\mathsf{s}} \phi : \texttt{bool}}{\Gamma \vdash_{\mathsf{s}} \neg\phi : \texttt{bool}}\ \textsc{BtNeg}$$

**Figure B.3.** Basic Qualifier Typing Rules